

Reinforcement Learning Algorithms Set 1

Table of Contents

Policy Gradient Methods	2
Actor-Critic Algorithm in Reinforcement Learning	19
High-Dimensional Continuous Control Using Generalized Advantage Estimation	26
Asynchronous Methods for Deep Reinforcement Learning	39
Sample Efficient Actor-Critic with Experience Replay	56
Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation	75
Learning to reinforcement learn	88
Proximal Policy Optimization Algorithms	103
Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor	114
Soft Actor-Critic Algorithms and Applications	126
CrossQ: Batch Normalization in Deep Reinforcement Learning for Greater Sample Efficiency and Simplicity	141
Stable-Baselines3: Reliable Reinforcement Learning Implementations	157
END	160

Chapter 13

Policy Gradient Methods

In this chapter we consider something new. So far in this book almost all the methods have been *action-value methods*; they learned the values of actions and then selected actions based on their estimated action values¹; their policies would not even exist without the action-value estimates. In this chapter we consider methods that instead learn a *parameterized policy* that can select actions without consulting a value function. A value function may still be used to *learn* the policy parameter, but is not required for action selection. We use the notation $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ for the policy's parameter vector. Thus we write $\pi(a|s, \boldsymbol{\theta}) = \Pr\{A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}\}$ for the probability that action a is taken at time t given that the environment is in state s at time t with parameter $\boldsymbol{\theta}$. If a method uses a learned value function as well, then the value function's weight vector is denoted $\mathbf{w} \in \mathbb{R}^d$ as usual, as in $\hat{v}(s, \mathbf{w})$.

In this chapter we consider methods for learning the policy parameter based on the gradient of some scalar performance measure $J(\boldsymbol{\theta})$ with respect to the policy parameter. These methods seek to *maximize* performance, so their updates approximate gradient *ascent* in J :

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)}, \quad (13.1)$$

where $\widehat{\nabla J(\boldsymbol{\theta}_t)} \in \mathbb{R}^d$ is a stochastic estimate whose expectation approximates the gradient of the performance measure with respect to its argument $\boldsymbol{\theta}_t$. All methods that follow this general schema we call *policy gradient methods*, whether or not they also learn an approximate value function. Methods that learn approximations to both policy and value functions are often called *actor-critic methods*, where ‘actor’ is a reference to the learned policy, and ‘critic’ refers to the learned value function, usually a state-value function. First we treat the episodic case, in which performance is defined as the value of the start state under the parameterized policy, before going on to consider the continuing case, in which performance is defined as the average reward rate, as in Section 10.3. In the end, we are able to express the algorithms for both cases in very similar terms.

¹The lone exception is the gradient bandit algorithms of Section 2.8. In fact, that section goes through many of the same steps, in the single-state bandit case, as we go through here for full MDPs. Reviewing that section would be good preparation for fully understanding this chapter.

13.1 Policy Approximation and its Advantages

In policy gradient methods, the policy can be parameterized in any way, as long as $\pi(a|s, \boldsymbol{\theta})$ is differentiable with respect to its parameters, that is, as long as $\nabla \pi(a|s, \boldsymbol{\theta})$ (the column vector of partial derivatives of $\pi(a|s, \boldsymbol{\theta})$ with respect to the components of $\boldsymbol{\theta}$) exists and is finite for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $\boldsymbol{\theta} \in \mathbb{R}^{d'}$. In practice, to ensure exploration we generally require that the policy never becomes deterministic (i.e., that $\pi(a|s, \boldsymbol{\theta}) \in (0, 1)$, for all $s, a, \boldsymbol{\theta}$). In this section we introduce the most common parameterization for discrete action spaces and point out the advantages it offers over action-value methods. Policy-based methods also offer useful ways of dealing with continuous action spaces, as we describe later in Section 13.7.

If the action space is discrete and not too large, then a natural and common kind of parameterization is to form parameterized numerical preferences $h(s, a, \boldsymbol{\theta}) \in \mathbb{R}$ for each state–action pair. The actions with the highest preferences in each state are given the highest probabilities of being selected, for example, according to an exponential soft-max distribution:

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{e^{h(s, a, \boldsymbol{\theta})}}{\sum_b e^{h(s, b, \boldsymbol{\theta})}}, \quad (13.2)$$

where $e \approx 2.71828$ is the base of the natural logarithm. Note that the denominator here is just what is required so that the action probabilities in each state sum to one. We call this kind of policy parameterization *soft-max in action preferences*.

The action preferences themselves can be parameterized arbitrarily. For example, they might be computed by a deep artificial neural network (ANN), where $\boldsymbol{\theta}$ is the vector of all the connection weights of the network (as in the AlphaGo system described in Section 16.6). Or the preferences could simply be linear in features,

$$h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}(s, a), \quad (13.3)$$

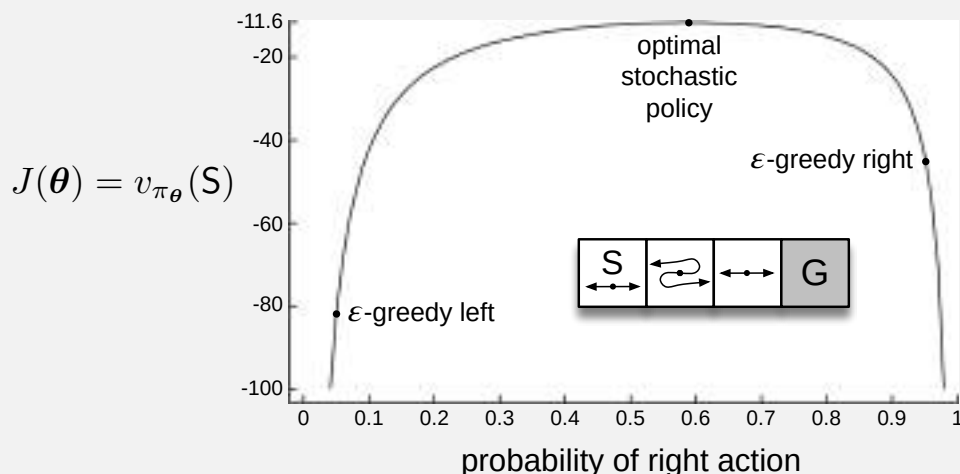
using feature vectors $\mathbf{x}(s, a) \in \mathbb{R}^{d'}$ constructed by any of the methods described in Chapter 9.

One advantage of parameterizing policies according to the soft-max in action preferences is that the approximate policy can approach a deterministic policy, whereas with ε -greedy action selection over action values there is always an ε probability of selecting a random action. Of course, one could select according to a soft-max distribution based on action values, but this alone would not allow the policy to approach a deterministic policy. Instead, the action-value estimates would converge to their corresponding true values, which would differ by a finite amount, translating to specific probabilities other than 0 and 1. If the soft-max distribution included a temperature parameter, then the temperature could be reduced over time to approach determinism, but in practice it would be difficult to choose the reduction schedule, or even the initial temperature, without more prior knowledge of the true action values than we would like to assume. Action preferences are different because they do not approach specific values; instead they are driven to produce the optimal stochastic policy. If the optimal policy is deterministic, then the preferences of the optimal actions will be driven infinitely higher than all suboptimal actions (if permitted by the parameterization).

A second advantage of parameterizing policies according to the soft-max in action preferences is that it enables the selection of actions with arbitrary probabilities. In problems with significant function approximation, the best approximate policy may be stochastic. For example, in card games with imperfect information the optimal play is often to do two different things with specific probabilities, such as when bluffing in Poker. Action-value methods have no natural way of finding stochastic optimal policies, whereas policy approximating methods can, as shown in Example 13.1.

Example 13.1 Short corridor with switched actions

Consider the small corridor gridworld shown inset in the graph below. The reward is -1 per step, as usual. In each of the three nonterminal states there are only two actions, *right* and *left*. These actions have their usual consequences in the first and third states (*left* causes no movement in the first state), but in the second state they are reversed, so that *right* moves to the left and *left* moves to the right. The problem is difficult because all the states appear identical under the function approximation. In particular, we define $\mathbf{x}(s, \text{right}) = [1, 0]^\top$ and $\mathbf{x}(s, \text{left}) = [0, 1]^\top$, for all s . An action-value method with ε -greedy action selection is forced to choose between just two policies: choosing *right* with high probability $1 - \varepsilon/2$ on all steps or choosing *left* with the same high probability on all time steps. If $\varepsilon = 0.1$, then these two policies achieve a value (at the start state) of less than -44 and -82 , respectively, as shown in the graph. A method can do significantly better if it can learn a specific probability with which to select *right*. The best probability is about 0.59 , which achieves a value of about -11.6 .



Perhaps the simplest advantage that policy parameterization may have over action-value parameterization is that the policy may be a simpler function to approximate. Problems vary in the complexity of their policies and action-value functions. For some, the action-value function is simpler and thus easier to approximate. For others, the policy is simpler. In the latter case a policy-based method will typically learn faster and yield a superior asymptotic policy (as in Tetris; see Şimşek, Algórta, and Kothiyal, 2016).

Finally, we note that the choice of policy parameterization is sometimes a good way of injecting prior knowledge about the desired form of the policy into the reinforcement learning system. This is often the most important reason for using a policy-based learning method.

Exercise 13.1 Use your knowledge of the gridworld and its dynamics to determine an *exact* symbolic expression for the optimal probability of selecting the **right** action in Example 13.1. $\square$

13.2 The Policy Gradient Theorem

In addition to the practical advantages of policy parameterization over ε -greedy action selection, there is also an important theoretical advantage. With continuous policy parameterization the action probabilities change smoothly as a function of the learned parameter, whereas in ε -greedy selection the action probabilities may change dramatically for an arbitrarily small change in the estimated action values, if that change results in a different action having the maximal value. Largely because of this stronger convergence guarantees are available for policy-gradient methods than for action-value methods. In particular, it is the continuity of the policy dependence on the parameters that enables policy-gradient methods to approximate gradient ascent (13.1).

The episodic and continuing cases define the performance measure, $J(\theta)$, differently and thus have to be treated separately to some extent. Nevertheless, we will try to present both cases uniformly, and we develop a notation so that the major theoretical results can be described with a single set of equations.

In this section we treat the episodic case, for which we define the performance measure as the value of the start state of the episode. We can simplify the notation without losing any meaningful generality by assuming that every episode starts in some particular (non-random) state s_0 . Then, in the episodic case we define performance as

$$J(\theta) \doteq v_{\pi_\theta}(s_0), \quad (13.4)$$

where v_{π_θ} is the true value function for π_θ , the policy determined by θ . From here on in our discussion we will assume no discounting ($\gamma = 1$) for the episodic case, although for completeness we do include the possibility of discounting in the boxed algorithms.

With function approximation, it may seem challenging to change the policy parameter in a way that ensures improvement. The problem is that performance depends on both the action selections and the distribution of states in which those selections are made, and that both of these are affected by the policy parameter. Given a state, the effect of the policy parameter on the actions, and thus on reward, can be computed in a relatively straightforward way from knowledge of the parameterization. But the effect of the policy on the state distribution is a function of the environment and is typically unknown. How can we estimate the performance gradient with respect to the policy parameter when the gradient depends on the unknown effect of policy changes on the state distribution?

Fortunately, there is an excellent theoretical answer to this challenge in the form of the *policy gradient theorem*, which provides an analytic expression for the gradient of

Proof of the Policy Gradient Theorem (episodic case)

With just elementary calculus and re-arranging of terms, we can prove the policy gradient theorem from first principles. To keep the notation simple, we leave it implicit in all cases that π is a function of θ , and all gradients are also implicitly with respect to θ . First note that the gradient of the state-value function can be written in terms of the action-value function as

$$\begin{aligned}
 \nabla v_\pi(s) &= \nabla \left[\sum_a \pi(a|s) q_\pi(s, a) \right], \quad \text{for all } s \in \mathcal{S} && \text{(Exercise 3.18)} \\
 &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \nabla q_\pi(s, a) \right] && \text{(product rule of calculus)} \\
 &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \nabla \sum_{s', r} p(s', r|s, a) (r + v_\pi(s')) \right] \\
 &&& \text{(Exercise 3.19 and Equation 3.2)} \\
 &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \sum_{s'} p(s'|s, a) \nabla v_\pi(s') \right] && \text{(Eq. 3.4)} \\
 &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \sum_{s'} p(s'|s, a) \right. && \text{(unrolling)} \\
 &\quad \left. \sum_{a'} [\nabla \pi(a'|s') q_\pi(s', a') + \pi(a'|s') \sum_{s''} p(s''|s', a') \nabla v_\pi(s'')] \right] \\
 &= \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \rightarrow x, k, \pi) \sum_a \nabla \pi(a|x) q_\pi(x, a),
 \end{aligned}$$

after repeated unrolling, where $\Pr(s \rightarrow x, k, \pi)$ is the probability of transitioning from state s to state x in k steps under policy π . It is then immediate that

$$\begin{aligned}
 \nabla J(\theta) &= \nabla v_\pi(s_0) \\
 &= \sum_s \left(\sum_{k=0}^{\infty} \Pr(s_0 \rightarrow s, k, \pi) \right) \sum_a \nabla \pi(a|s) q_\pi(s, a) \\
 &= \sum_s \eta(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) && \text{(box page 199)} \\
 &= \sum_{s'} \eta(s') \sum_s \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_a \nabla \pi(a|s) q_\pi(s, a) \\
 &= \sum_{s'} \eta(s') \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) && \text{(Eq. 9.3)} \\
 &\propto \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) && \text{(Q.E.D.)}
 \end{aligned}$$

performance with respect to the policy parameter (which is what we need to approximate for gradient ascent (13.1)) that does *not* involve the derivative of the state distribution. The policy gradient theorem for the episodic case establishes that

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \boldsymbol{\theta}), \quad (13.5)$$

where the gradients are column vectors of partial derivatives with respect to the components of $\boldsymbol{\theta}$, and π denotes the policy corresponding to parameter vector $\boldsymbol{\theta}$. The symbol $\propto$ here means “proportional to”. In the episodic case, the constant of proportionality is the average length of an episode, and in the continuing case it is 1, so that the relationship is actually an equality. The distribution μ here (as in Chapters 9 and 10) is the on-policy distribution under π (see page 199). The policy gradient theorem is proved for the episodic case in the box on the previous page.

13.3 REINFORCE: Monte Carlo Policy Gradient

We are now ready to derive our first policy-gradient learning algorithm. Recall our overall strategy of stochastic gradient ascent (13.1), which requires a way to obtain samples such that the expectation of the sample gradient is proportional to the actual gradient of the performance measure as a function of the parameter. The sample gradients need only be proportional to the gradient because any constant of proportionality can be absorbed into the step size α , which is otherwise arbitrary. The policy gradient theorem gives an exact expression proportional to the gradient; all that is needed is some way of sampling whose expectation equals or approximates this expression. Notice that the right-hand side of the policy gradient theorem is a sum over states weighted by how often the states occur under the target policy π ; if π is followed, then states will be encountered in these proportions. Thus

$$\begin{aligned} \nabla J(\boldsymbol{\theta}) &\propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \boldsymbol{\theta}) \\ &= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a|S_t, \boldsymbol{\theta}) \right]. \end{aligned} \quad (13.6)$$

We could stop here and instantiate our stochastic gradient-ascent algorithm (13.1) as

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \sum_a \hat{q}(S_t, a, \mathbf{w}) \nabla \pi(a|S_t, \boldsymbol{\theta}), \quad (13.7)$$

where $\hat{q}$ is some learned approximation to q_π . This algorithm, which has been called an *all-actions* method because its update involves all of the actions, is promising and deserving of further study, but our current interest is the classical REINFORCE algorithm (Williams, 1992) whose update at time t involves just A_t , the one action actually taken at time t .

We continue our derivation of REINFORCE by introducing A_t in the same way as we introduced S_t in (13.6)—by replacing a sum over the random variable’s possible values

by an expectation under π , and then sampling the expectation. Equation (13.6) involves an appropriate sum over actions, but each term is not weighted by $\pi(a|S_t, \theta)$ as is needed for an expectation under π . So we introduce such a weighting, without changing the equality, by multiplying and then dividing the summed terms by $\pi(a|S_t, \theta)$. Continuing from (13.6), we have

$$\begin{aligned}\nabla J(\theta) &= \mathbb{E}_\pi \left[\sum_a \pi(a|S_t, \theta) q_\pi(S_t, a) \frac{\nabla \pi(a|S_t, \theta)}{\pi(a|S_t, \theta)} \right] \\ &= \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \frac{\nabla \pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)} \right] && \text{(replacing } a \text{ by the sample } A_t \sim \pi) \\ &= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)} \right], && \text{(because } \mathbb{E}_\pi[G_t|S_t, A_t] = q_\pi(S_t, A_t))\end{aligned}$$

where G_t is the return as usual. The final expression in brackets is exactly what is needed, a quantity that can be sampled on each time step whose expectation is equal to the gradient. Using this sample to instantiate our generic stochastic gradient ascent algorithm (13.1) yields the REINFORCE update:

$$\theta_{t+1} \doteq \theta_t + \alpha G_t \frac{\nabla \pi(A_t|S_t, \theta_t)}{\pi(A_t|S_t, \theta_t)}. \quad (13.8)$$

This update has an intuitive appeal. Each increment is proportional to the product of a return G_t and a vector, the gradient of the probability of taking the action actually taken divided by the probability of taking that action. The vector is the direction in parameter space that most increases the probability of repeating the action A_t on future visits to state S_t . The update increases the parameter vector in this direction proportional to the return, and inversely proportional to the action probability. The former makes sense because it causes the parameter to move most in the directions that favor actions that yield the highest return. The latter makes sense because otherwise actions that are selected frequently are at an advantage (the updates will be more often in their direction) and might win out even if they do not yield the highest return.

Note that REINFORCE uses the complete return from time t , which includes all future rewards up until the end of the episode. In this sense REINFORCE is a Monte Carlo algorithm and is well defined only for the episodic case with all updates made in retrospect after the episode is completed (like the Monte Carlo algorithms in Chapter 5). This is shown explicitly in the boxed on the next page.

Notice that the update in the last line of pseudocode appears rather different from the REINFORCE update rule (13.8). One difference is that the pseudocode uses the compact expression $\nabla \ln \pi(A_t|S_t, \theta_t)$ for the fractional vector $\frac{\nabla \pi(A_t|S_t, \theta_t)}{\pi(A_t|S_t, \theta_t)}$ in (13.8). That these two expressions for the vector are equivalent follows from the identity $\nabla \ln x = \frac{\nabla x}{x}$. This vector has been given several names and notations in the literature; we will refer to it simply as the *eligibility vector*. Note that it is the only place that the policy parameterization appears in the algorithm.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T-1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

The second difference between the pseudocode update and the REINFORCE update equation (13.8) is that the former includes a factor of γ^t . This is because, as mentioned earlier, in the text we are treating the non-discounted case ($\gamma = 1$) while in the boxed algorithms we are giving the algorithms for the general discounted case. All of the ideas go through in the discounted case with appropriate adjustments (including to the box on page 199) but involve additional complexity that distracts from the main ideas.

**Exercise 13.2* Generalize the box on page 199, the policy gradient theorem (13.5), the proof of the policy gradient theorem (page 325), and the steps leading to the REINFORCE update equation (13.8), so that (13.8) ends up with a factor of γ^t and thus aligns with the general algorithm given in the pseudocode. $\square$

Figure 13.1 shows the performance of REINFORCE on the short-corridor gridworld from Example 13.1.

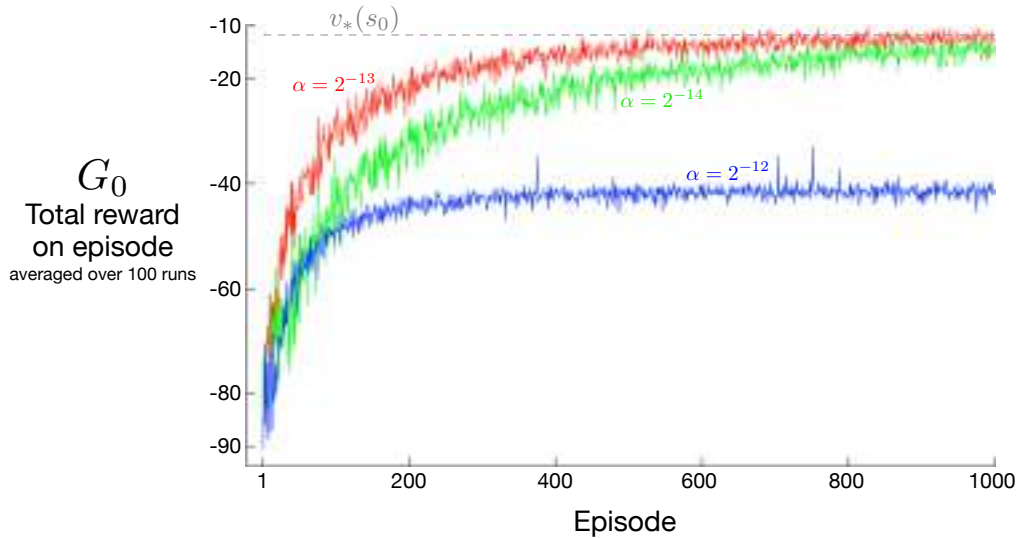


Figure 13.1: REINFORCE on the short-corridor gridworld (Example 13.1). With a good step size, the total reward per episode approaches the optimal value of the start state.

As a stochastic gradient method, REINFORCE has good theoretical convergence properties. By construction, the expected update over an episode is in the same direction as the performance gradient. This assures an improvement in expected performance for sufficiently small α , and convergence to a local optimum under standard stochastic approximation conditions for decreasing α . However, as a Monte Carlo method REINFORCE may be of high variance and thus produce slow learning.

Exercise 13.3 In Section 13.1 we considered policy parameterizations using the soft-max in action preferences (13.2) with linear action preferences (13.3). For this parameterization, prove that the eligibility vector is

$$\nabla \ln \pi(a|s, \boldsymbol{\theta}) = \mathbf{x}(s, a) - \sum_b \pi(b|s, \boldsymbol{\theta}) \mathbf{x}(s, b), \quad (13.9)$$

using the definitions and elementary calculus. $\square$

13.4 REINFORCE with Baseline

The policy gradient theorem (13.5) can be generalized to include a comparison of the action value to an arbitrary *baseline* $b(s)$:

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \sum_a \left(q_\pi(s, a) - b(s) \right) \nabla \pi(a|s, \boldsymbol{\theta}). \quad (13.10)$$

The baseline can be any function, even a random variable, as long as it does not vary with a ; the equation remains valid because the subtracted quantity is zero:

$$\sum_a b(s) \nabla \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla \sum_a \pi(a|s, \boldsymbol{\theta}) = b(s) \nabla 1 = 0.$$

The policy gradient theorem with baseline (13.10) can be used to derive an update rule using similar steps as in the previous section. The update rule that we end up with is a new version of REINFORCE that includes a general baseline:

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_t - b(S_t) \right) \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta}_t)}{\pi(A_t|S_t, \boldsymbol{\theta}_t)}. \quad (13.11)$$

Because the baseline could be uniformly zero, this update is a strict generalization of REINFORCE. In general, the baseline leaves the expected value of the update unchanged, but it can have a large effect on its variance. For example, we saw in Section 2.8 that an analogous baseline can significantly reduce the variance (and thus speed the learning) of gradient bandit algorithms. In the bandit algorithms the baseline was just a number (the average of the rewards seen so far), but for MDPs the baseline should vary with state. In some states all actions have high values and we need a high baseline to differentiate the higher valued actions from the less highly valued ones; in other states all actions will have low values and a low baseline is appropriate.

One natural choice for the baseline is an estimate of the state value, $\hat{v}(S_t, \mathbf{w})$, where $\mathbf{w} \in \mathbb{R}^m$ is a weight vector learned by one of the methods presented in previous chapters.

Because REINFORCE is a Monte Carlo method for learning the policy parameter, θ , it seems natural to also use a Monte Carlo method to learn the state-value weights, $\mathbf{w}$. A complete pseudocode algorithm for REINFORCE with baseline using such a learned state-value function as the baseline is given in the box below.

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Algorithm parameters: step sizes $\alpha^\theta > 0$, $\alpha^\mathbf{w} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$
 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:
 $G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$ (G_t)
 $\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^\mathbf{w} \delta \nabla \hat{v}(S_t, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^\theta \gamma^t \delta \nabla \ln \pi(A_t|S_t, \theta)$

This algorithm has two step sizes, denoted α^θ and $\alpha^\mathbf{w}$ (where α^θ is the α in (13.11)). Choosing the step size for values (here $\alpha^\mathbf{w}$) is relatively easy; in the linear case we have rules of thumb for setting it, such as $\alpha^\mathbf{w} = 0.1/\mathbb{E}[\|\nabla \hat{v}(S_t, \mathbf{w})\|_\mu^2]$ (see Section 9.6). It is much less clear how to set the step size for the policy parameters, α^θ , whose best value depends on the range of variation of the rewards and on the policy parameterization.

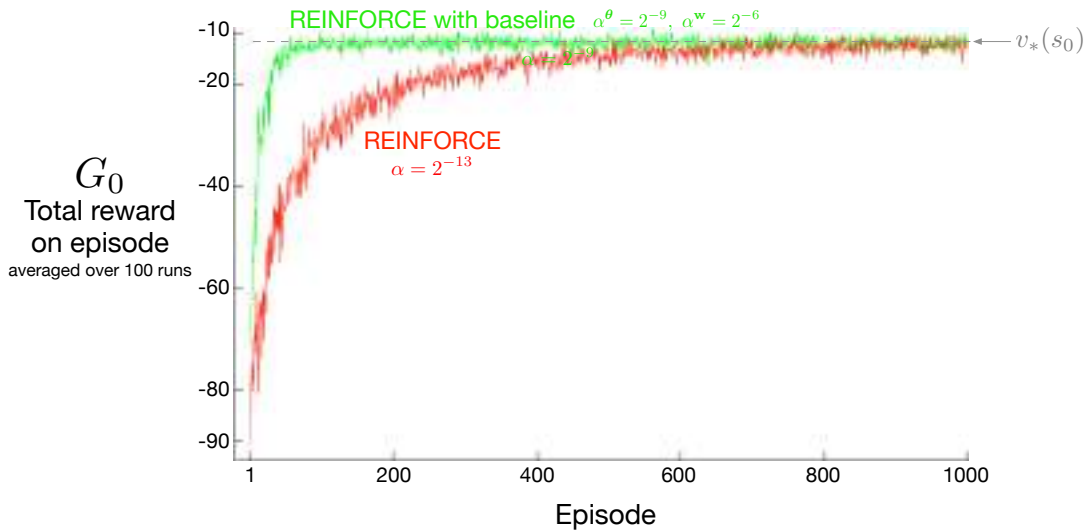


Figure 13.2: Adding a baseline to REINFORCE can make it learn much faster, as illustrated here on the short-corridor gridworld (Example 13.1). The step size used here for plain REINFORCE is that at which it performs best (to the nearest power of two; see Figure 13.1).

Figure 13.2 compares the behavior of REINFORCE with and without a baseline on the short-corridor gridworld (Example 13.1). Here the approximate state-value function used in the baseline is $\hat{v}(s, \mathbf{w}) = w$. That is, $\mathbf{w}$ is a single component, w .

13.5 Actor–Critic Methods

Although the REINFORCE-with-baseline method learns both a policy and a state-value function, we do not consider it to be an actor–critic method because its state-value function is used only as a baseline, not as a critic. That is, it is not used for bootstrapping (updating the value estimate for a state from the estimated values of subsequent states), but only as a baseline for the state whose estimate is being updated. This is a useful distinction, for only through bootstrapping do we introduce bias and an asymptotic dependence on the quality of the function approximation. As we have seen, the bias introduced through bootstrapping and reliance on the state representation is often beneficial because it reduces variance and accelerates learning. REINFORCE with baseline is unbiased and will converge asymptotically to a local minimum, but like all Monte Carlo methods it tends to learn slowly (produce estimates of high variance) and to be inconvenient to implement online or for continuing problems. As we have seen earlier in this book, with temporal-difference methods we can eliminate these inconveniences, and through multi-step methods we can flexibly choose the degree of bootstrapping. In order to gain these advantages in the case of policy gradient methods we use actor–critic methods with a bootstrapping critic.

First consider one-step actor–critic methods, the analog of the TD methods introduced in Chapter 6 such as TD(0), Sarsa(0), and Q-learning. The main appeal of one-step methods is that they are fully online and incremental, yet avoid the complexities of eligibility traces. They are a special case of the eligibility trace methods, and not as general, but easier to understand. One-step actor–critic methods replace the full return of REINFORCE (13.11) with the one-step return (and use a learned state-value function as the baseline) as follows:

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \left(G_{t:t+1} - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \quad (13.12)$$

$$= \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \right) \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)} \quad (13.13)$$

$$= \boldsymbol{\theta}_t + \alpha \delta_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}. \quad (13.14)$$

The natural state-value-function learning method to pair with this is semi-gradient TD(0). Pseudocode for the complete algorithm is given in the box at the top of the next page. Note that it is now a fully online, incremental algorithm, with states, actions, and rewards processed as they occur and then never revisited.

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$
 $\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

The generalizations to the forward view of n -step methods and then to a λ -return algorithm are straightforward. The one-step return in (13.12) is merely replaced by $G_{t:t+n}$ or G_t^{λ} respectively. The backward view of the λ -return algorithm is also straightforward, using separate eligibility traces for the actor and critic, each after the patterns in Chapter 12. Pseudocode for the complete algorithm is given in the box below.

Actor–Critic with Eligibility Traces (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Parameters: trace-decay rates $\lambda^{\theta} \in [0, 1]$, $\lambda^{\mathbf{w}} \in [0, 1]$; step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$
 Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
 Loop forever (for each episode):
 Initialize S (first state of episode)
 $\mathbf{z}^{\theta} \leftarrow \mathbf{0}$ (d' -component eligibility trace vector)
 $\mathbf{z}^{\mathbf{w}} \leftarrow \mathbf{0}$ (d -component eligibility trace vector)
 $I \leftarrow 1$
 Loop while S is not terminal (for each time step):
 $A \sim \pi(\cdot|S, \theta)$
 Take action A , observe S', R
 $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)
 $\mathbf{z}^{\mathbf{w}} \leftarrow \gamma \lambda^{\mathbf{w}} \mathbf{z}^{\mathbf{w}} + \nabla \hat{v}(S, \mathbf{w})$
 $\mathbf{z}^{\theta} \leftarrow \gamma \lambda^{\theta} \mathbf{z}^{\theta} + I \nabla \ln \pi(A|S, \theta)$
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \mathbf{z}^{\mathbf{w}}$
 $\theta \leftarrow \theta + \alpha^{\theta} \delta \mathbf{z}^{\theta}$
 $I \leftarrow \gamma I$
 $S \leftarrow S'$

13.6 Policy Gradient for Continuing Problems

As discussed in Section 10.3, for continuing problems without episode boundaries we need to define performance in terms of the average rate of reward per time step:

$$\begin{aligned}
 J(\boldsymbol{\theta}) &\doteq r(\pi) \doteq \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=1}^h \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
 &= \lim_{t \rightarrow \infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\
 &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) r,
 \end{aligned} \tag{13.15}$$

where μ is the steady-state distribution under π , $\mu(s) \doteq \lim_{t \rightarrow \infty} \Pr\{S_t = s \mid A_{0:t} \sim \pi\}$, which is assumed to exist and to be independent of S_0 (an ergodicity assumption). Remember that this is the special distribution under which, if you select actions according to π , you remain in the same distribution:

$$\sum_s \mu(s) \sum_a \pi(a|s, \boldsymbol{\theta}) p(s'|s, a) = \mu(s'), \text{ for all } s' \in \mathcal{S}. \tag{13.16}$$

Complete pseudocode for the actor–critic algorithm in the continuing case (backward view) is given in the box below.

Actor–Critic with Eligibility Traces (continuing), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
 Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$
 Algorithm parameters: $\lambda^{\mathbf{w}} \in [0, 1]$, $\lambda^{\boldsymbol{\theta}} \in [0, 1]$, $\alpha^{\mathbf{w}} > 0$, $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\bar{R}} > 0$
 Initialize $\bar{R} \in \mathbb{R}$ (e.g., to 0)
 Initialize state-value weights $\mathbf{w} \in \mathbb{R}^d$ and policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)
 Initialize $S \in \mathcal{S}$ (e.g., to s_0)

$\mathbf{z}^{\mathbf{w}} \leftarrow \mathbf{0}$ (d -component eligibility trace vector)
 $\mathbf{z}^{\boldsymbol{\theta}} \leftarrow \mathbf{0}$ (d' -component eligibility trace vector)

Loop forever (for each time step):
 $A \sim \pi(\cdot|S, \boldsymbol{\theta})$
 Take action A , observe S', R
 $\delta \leftarrow R - \bar{R} + \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$
 $\bar{R} \leftarrow \bar{R} + \alpha^{\bar{R}} \delta$
 $\mathbf{z}^{\mathbf{w}} \leftarrow \lambda^{\mathbf{w}} \mathbf{z}^{\mathbf{w}} + \nabla \hat{v}(S, \mathbf{w})$
 $\mathbf{z}^{\boldsymbol{\theta}} \leftarrow \lambda^{\boldsymbol{\theta}} \mathbf{z}^{\boldsymbol{\theta}} + \nabla \ln \pi(A|S, \boldsymbol{\theta})$
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \mathbf{z}^{\mathbf{w}}$
 $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \delta \mathbf{z}^{\boldsymbol{\theta}}$
 $S \leftarrow S'$

Naturally, in the continuing case, we define values, $v_\pi(s) \doteq \mathbb{E}_\pi[G_t|S_t=s]$ and $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t|S_t=s, A_t=a]$, with respect to the differential return:

$$G_t \doteq R_{t+1} - r(\pi) + R_{t+2} - r(\pi) + R_{t+3} - r(\pi) + \dots \quad (13.17)$$

With these alternate definitions, the policy gradient theorem as given for the episodic case (13.5) remains true for the continuing case. A proof is given in the box on the next page. The forward and backward view equations also remain the same.

Proof of the Policy Gradient Theorem (continuing case)

The proof of the policy gradient theorem for the continuing case begins similarly to the episodic case. Again we leave it implicit in all cases that π is a function of θ and that the gradients are with respect to θ . Recall that in the continuing case $J(\theta) = r(\pi)$ (13.15) and that v_π and q_π denote values with respect to the differential return (13.17). The gradient of the state-value function can be written, for any $s \in \mathcal{S}$, as

$$\begin{aligned} \nabla v_\pi(s) &= \nabla \left[\sum_a \pi(a|s) q_\pi(s, a) \right], \quad \text{for all } s \in \mathcal{S} && \text{(Exercise 3.18)} \\ &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \nabla q_\pi(s, a) \right] && \text{(product rule of calculus)} \\ &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \nabla \sum_{s', r} p(s', r|s, a) (r - r(\theta) + v_\pi(s')) \right] \\ &= \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \left[-\nabla r(\theta) + \sum_{s'} p(s'|s, a) \nabla v_\pi(s') \right] \right]. \end{aligned}$$

After re-arranging terms, we obtain

$$\nabla r(\theta) = \sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \sum_{s'} p(s'|s, a) \nabla v_\pi(s') \right] - \nabla v_\pi(s).$$

Notice that the left-hand side can be written $\nabla J(\theta)$, and that it does not depend on s . Thus the right-hand side does not depend on s either, and we can safely sum it over all $s \in \mathcal{S}$, weighted by $\mu(s)$, without changing it (because $\sum_s \mu(s) = 1$):

$$\begin{aligned} \nabla J(\theta) &= \sum_s \mu(s) \left(\sum_a \left[\nabla \pi(a|s) q_\pi(s, a) + \pi(a|s) \sum_{s'} p(s'|s, a) \nabla v_\pi(s') \right] - \nabla v_\pi(s) \right) \\ &= \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) \\ &\quad + \sum_s \mu(s) \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) \nabla v_\pi(s') - \sum_s \mu(s) \nabla v_\pi(s) \end{aligned}$$

$$\begin{aligned}
&= \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) \\
&\quad + \underbrace{\sum_{s'} \sum_s \mu(s) \sum_a \pi(a|s) p(s'|s, a) \nabla v_\pi(s')}_{\mu(s') \text{ (13.16)}} - \sum_s \mu(s) \nabla v_\pi(s) \\
&= \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a) + \sum_{s'} \mu(s') \nabla v_\pi(s') - \sum_s \mu(s) \nabla v_\pi(s) \\
&= \sum_s \mu(s) \sum_a \nabla \pi(a|s) q_\pi(s, a). \quad \text{Q.E.D.}
\end{aligned}$$

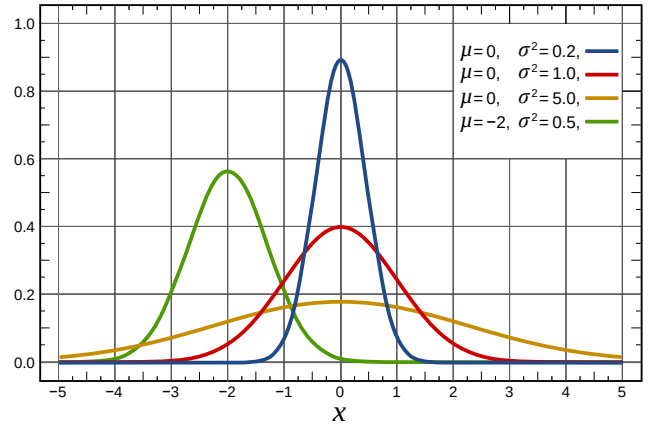
13.7 Policy Parameterization for Continuous Actions

Policy-based methods offer practical ways of dealing with large actions spaces, even continuous spaces with an infinite number of actions. Instead of computing learned probabilities for each of the many actions, we instead learn statistics of the probability distribution. For example, the action set might be the real numbers, with actions chosen from a normal (Gaussian) distribution.

The probability density function for the normal distribution is conventionally written

$$p(x) \doteq \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad (13.18)$$

where μ and σ here are the mean and standard deviation of the normal distribution, and of course π here is just the number $\pi \approx 3.14159$. The probability density functions for several different means and standard deviations are shown to the right. The value $p(x)$ is the *density* of the probability at x , not the probability. It can be greater than 1; it is the total area under $p(x)$ that must sum to 1. In general, one can take the integral under $p(x)$ for any range of x values to get the probability of x falling within that range.



To produce a policy parameterization, the policy can be defined as the normal probability density over a real-valued scalar action, with mean and standard deviation given by parametric function approximators that depend on the state. That is,

$$\pi(a|s, \boldsymbol{\theta}) \doteq \frac{1}{\sigma(s, \boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a - \mu(s, \boldsymbol{\theta}))^2}{2\sigma(s, \boldsymbol{\theta})^2}\right), \quad (13.19)$$

where $\mu : \mathcal{S} \times \mathbb{R}^{d'} \rightarrow \mathbb{R}$ and $\sigma : \mathcal{S} \times \mathbb{R}^{d'} \rightarrow \mathbb{R}^+$ are two parameterized function approximators.

To complete the example we need only give a form for these approximators. For this we divide the policy's parameter vector into two parts, $\boldsymbol{\theta} = [\boldsymbol{\theta}_\mu, \boldsymbol{\theta}_\sigma]^\top$, one part to be used for the approximation of the mean and one part for the approximation of the standard deviation. The mean can be approximated as a linear function. The standard deviation must always be positive and is better approximated as the exponential of a linear function. Thus

$$\mu(s, \boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu^\top \mathbf{x}_\mu(s) \quad \text{and} \quad \sigma(s, \boldsymbol{\theta}) \doteq \exp\left(\boldsymbol{\theta}_\sigma^\top \mathbf{x}_\sigma(s)\right), \quad (13.20)$$

where $\mathbf{x}_\mu(s)$ and $\mathbf{x}_\sigma(s)$ are state feature vectors perhaps constructed by one of the methods described in Chapter 9. With these definitions, all the algorithms described in the rest of this chapter can be applied to learn to select real-valued actions.

Exercise 13.4 Show that for the gaussian policy parameterization (13.19) the eligibility vector has the following two parts:

$$\begin{aligned} \nabla \ln \pi(a|s, \boldsymbol{\theta}_\mu) &= \frac{\nabla \pi(a|s, \boldsymbol{\theta}_\mu)}{\pi(a|s, \boldsymbol{\theta})} = \frac{1}{\sigma(s, \boldsymbol{\theta})^2} (a - \mu(s, \boldsymbol{\theta})) \mathbf{x}_\mu(s), \text{ and} \\ \nabla \ln \pi(a|s, \boldsymbol{\theta}_\sigma) &= \frac{\nabla \pi(a|s, \boldsymbol{\theta}_\sigma)}{\pi(a|s, \boldsymbol{\theta})} = \left(\frac{(a - \mu(s, \boldsymbol{\theta}))^2}{\sigma(s, \boldsymbol{\theta})^2} - 1 \right) \mathbf{x}_\sigma(s). \end{aligned} \quad \square$$

Exercise 13.5 A *Bernoulli-logistic unit* is a stochastic neuron-like unit used in some ANNs (Section 9.6). Its input at time t is a feature vector $\mathbf{x}(S_t)$; its output, A_t , is a random variable having two values, 0 and 1, with $\Pr\{A_t = 1\} = P_t$ and $\Pr\{A_t = 0\} = 1 - P_t$ (the Bernoulli distribution). Let $h(s, 0, \boldsymbol{\theta})$ and $h(s, 1, \boldsymbol{\theta})$ be the preferences in state s for the unit's two actions given policy parameter $\boldsymbol{\theta}$. Assume that the difference between the action preferences is given by a weighted sum of the unit's input vector, that is, assume that $h(s, 1, \boldsymbol{\theta}) - h(s, 0, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}(s)$, where $\boldsymbol{\theta}$ is the unit's weight vector.

- Show that if the exponential soft-max distribution (13.2) is used to convert action preferences to policies, then $P_t = \pi(1|S_t, \boldsymbol{\theta}_t) = 1/(1 + \exp(-\boldsymbol{\theta}_t^\top \mathbf{x}(S_t)))$ (the logistic function).
- What is the Monte-Carlo REINFORCE update of $\boldsymbol{\theta}_t$ to $\boldsymbol{\theta}_{t+1}$ upon receipt of return G_t ?
- Express the eligibility $\nabla \ln \pi(a|s, \boldsymbol{\theta})$ for a Bernoulli-logistic unit, in terms of a , $\mathbf{x}(s)$, and $\pi(a|s, \boldsymbol{\theta})$ by calculating the gradient.

Hint: separately for each action compute the derivative of the logarithm first with respect to $P_t = \pi(a|s, \boldsymbol{\theta}_t)$, combine the two results into one expression that depends on a and P_t , and then use the chain rule, noting that the derivative of the logistic function $f(x)$ is $f(x)(1 - f(x))$. $\square$

13.8 Summary

Prior to this chapter, this book focused on *action-value methods*—meaning methods that learn action values and then use them to determine action selections. In this chapter, on the other hand, we considered methods that learn a parameterized policy that enables actions to be taken without consulting action-value estimates. In particular, we have considered *policy-gradient methods*—meaning methods that update the policy parameter on each step in the direction of an estimate of the gradient of performance with respect to the policy parameter.

Methods that learn and store a policy parameter have many advantages. They can learn specific probabilities for taking the actions. They can learn appropriate levels of exploration and approach deterministic policies asymptotically. They can naturally handle continuous action spaces. All these things are easy for policy-based methods but awkward or impossible for ε -greedy methods and for action-value methods in general. In addition, on some problems the policy is just simpler to represent parametrically than the value function; these problems are more suited to parameterized policy methods.

Parameterized policy methods also have an important theoretical advantage over action-value methods in the form of the *policy gradient theorem*, which gives an exact formula for how performance is affected by the policy parameter that does not involve derivatives of the state distribution. This theorem provides a theoretical foundation for all policy gradient methods.

The REINFORCE method follows directly from the policy gradient theorem. Adding a state-value function as a *baseline* reduces REINFORCE's variance without introducing bias. Using the state-value function for bootstrapping introduces bias but is often desirable for the same reason that bootstrapping TD methods are often superior to Monte Carlo methods (substantially reduced variance). The state-value function assigns credit to—critiques—the policy's action selections, and accordingly the former is termed the *critic* and the latter the *actor*, and these overall methods are termed *actor-critic* methods.

Overall, policy-gradient methods provide a significantly different set of strengths and weaknesses than action-value methods. Today they are less well understood in some respects, but a subject of excitement and ongoing research.

Bibliographical and Historical Remarks

Methods that we now see as related to policy gradients were actually some of the earliest to be studied in reinforcement learning (Witten, 1977; Barto, Sutton, and Anderson, 1983; Sutton, 1984; Williams, 1987, 1992) and in predecessor fields (Phansalkar and Thathachar, 1995). They were largely supplanted in the 1990s by the action-value methods that are the focus of the other chapters of this book. In recent years, however, attention has returned to actor-critic methods and to policy-gradient methods in general. Among the further developments beyond what we cover here are natural-gradient methods (Amari, 1998; Kakade, 2002; Peters, Vijayakumar and Schaal, 2005; Peters and Schall, 2008; Park, Kim and Kang, 2005; Bhatnagar, Sutton, Ghavamzadeh and Lee, 2009; see Grondman, Busoniu, Lopes and Babuska, 2012), deterministic policy gradient methods (Silver et al.,

Actor-Critic Algorithm in Reinforcement Learning

Last Updated : 26 Feb, 2025

Actor-Critic Algorithm is a type of reinforcement learning algorithm that combines aspects of both **policy-based methods (Actor)** and **value-based methods (Critic)**. This hybrid approach is designed to address the limitations of each method when used individually.

In the actor-critic framework, an agent (the "actor") learns a policy to make decisions, and a value function (the "Critic") evaluates the actions taken by the Actor.

Simultaneously, the critic evaluates these actions by estimating their value or quality. This dual role allows the method to strike a balance between exploration and exploitation, leveraging the strengths of both policy and value functions.

Roles of Actor and Critic

- **Actor:** The actor makes decisions by selecting actions based on the current policy. Its responsibility lies in exploring the action space to maximize expected cumulative rewards. By continuously refining the policy, the actor adapts to the dynamic nature of the environment.
- **Critic:** The critic evaluates the actions taken by the actor. It estimates the value or quality of these actions by providing feedback on their performance. The critic's role is pivotal in guiding the actor towards actions that lead to higher expected returns, contributing to the overall improvement of the learning process.

Key Terms in Actor Critic Algorithm

There are two key terms:

- **Policy (Actor) :**

- The actor seeks to maximize the expected return by optimizing this policy.
- The policy is modeled by the actor network, and its parameters are denoted by θ
- **Value Function (Critic) :**
 - The value function, denoted as $V(s)$, estimates the expected cumulative reward starting from state s .
 - The value function is modeled by the critic network, and its parameters are denoted by w .

How Actor-Critic algorithm works?

Actor Critic Algorithm Objective Function

- The objective function for the Actor-Critic algorithm is a combination of the policy gradient (for the actor) and the value function (for the critic).
- The overall objective function is typically expressed as the sum of two components:

Policy Gradient (Actor)

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \nabla_{\theta} \log \pi_{\theta}(a_i | s_i) \cdot A(s_i, a_i)$$

Here,

- $J(\theta)$ represents the expected return under the policy parameterized by θ
- $\pi_{\theta}(a | s)$ is the policy function
- N is the number of sampled experiences.
- $A(s, a)$ is the advantage function representing the advantage of taking action a in state s .
- i represents the index of the sample

Value Function Update (Critic)

$$\nabla_w J(w) \approx \frac{1}{N} \sum_{i=1}^N \nabla_w (V_w(s_i) - Q_w(s_i, a_i))^2$$

Here,

- $\nabla_w J(w)$ is the gradient of the loss function with respect to the critic's

- $V_w(s_i)$ is the critic's estimate of value of state s with parameter w
- $Q_w(s_i, a_i)$ is the critic's estimate of the action-value of taking action a
- i represents the index of the sample

Update Rules

The update rules for the actor and critic involve adjusting their respective parameters using gradient ascent (for the actor) and gradient descent (for the critic).

Actor Update

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t)$$

Here,

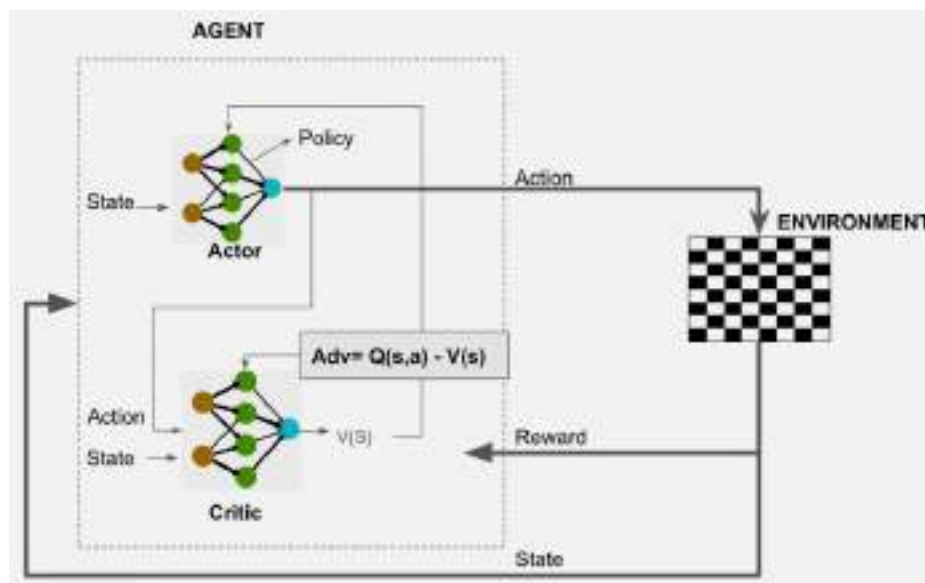
- α : learning rate for the actor
- t is the time step within an episode

Critic Update

$$w_t = w_t - \beta \nabla_w J(w_t)$$

Here,

- w represents the parameters of the critic network
- β is the learning rate for the critic



Advantage Function

state s over the expected value of the state under the current policy.

$$A(s, a) = Q(s, a) - V(s)$$

The advantage function, then, provides a measure of how much better or worse an action is compared to the average action.

These mathematical expressions highlight the essential computations involved in the Actor-Critic method. The actor is updated based on the policy gradient, encouraging actions with higher advantages, while the critic is updated to minimize the difference between the estimated value and the action-value.

Training Agent: Actor-Critic Algorithm

Let's understand how the Actor-Critic algorithm works in practice. Below is an implementation of a simple Actor-Critic algorithm using [TensorFlow](#) and OpenAI Gym to train an agent in the **CartPole environment**.

Step 1: Import Libraries

```
import numpy as np
import tensorflow as tf
import gym
```



Step 2: Creating CartPole Environment

Create the CartPole environment using the **`gym.make()`** function from the Gym library because it provides a standardized and convenient way to interact with various reinforcement learning tasks.

```
# Create the CartPole Environment
env = gym.make('CartPole-v1')
```



Step 3: Defining Actor and Critic Networks

- Actor and the Critic are implemented as neural networks using **TensorFlow's Keras API**.
- Actor network maps the state to a probability distribution over actions.

```
# Define the actor and critic networks
actor = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(env.action_space.n, activation='softmax')
])

critic = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])
```

Step 4: Defining Optimizers and Loss Functions

We use Adam optimizer for both networks.

```
# Define optimizer and loss functions
actor_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
critic_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

Step 5: Training Loop

The training loop runs for 1000 episodes, with the agent interacting with the environment, calculating advantages, and updating both the actor and critic.

```
# Main training loop
num_episodes = 1000
gamma = 0.99

for episode in range(num_episodes):
    state = env.reset()
    episode_reward = 0

    with tf.GradientTape(persistent=True) as tape:
        for t in range(1, 10000): # Limit the number of time steps
            # Choose an action using the actor
            action_probs = actor(np.array([state]))
            action = np.random.choice(env.action_space.n,
p=action_probs.numpy()[0])

            # Take the chosen action and observe the next state and reward
            next_state, reward, done, _ = env.step(action)

            # Compute the advantage
            state_value = critic(np.array([state]))[0, 0]
            next_state_value = critic(np.array([next_state]))[0, 0]
            advantage = reward + gamma * next_state_value - state_value

            # Compute actor and critic losses
            actor_loss = -tf.math.log(action_probs[0, action]) * advantage
```

```

        # Update actor and critic
        actor_gradients = tape.gradient(actor_loss,
actor.trainable_variables)
        critic_gradients = tape.gradient(critic_loss,
critic.trainable_variables)
        actor_optimizer.apply_gradients(zip(actor_gradients,
actor.trainable_variables))
        critic_optimizer.apply_gradients(zip(critic_gradients,
critic.trainable_variables))

    if done:
        break

if episode % 10 == 0:
    print(f'Episode {episode}, Reward: {episode_reward}')

env.close()

```

Output:

```

Episode 0, Reward: 13.0
Episode 10, Reward: 19.0
Episode 20, Reward: 25.0
Episode 30, Reward: 42.0
Episode 40, Reward: 12.0
Episode 50, Reward: 20.0
Episode 60, Reward: 16.0
Episode 70, Reward: 20.0
Episode 80, Reward: 11.0
Episode 90, Reward: 20.0
Episode 100, Reward: 16.0
Episode 110, Reward: 23.0
Episode 120, Reward: 26.0
Episode 130, Reward: 25.0
Episode 140, Reward: 15.0
Episode 150, Reward: 20.0
Episode 160, Reward: 23.0
Episode 170, Reward: 16.0
Episode 180, Reward: 24.0

```

Advantages of Actor Critic Algorithm

The Actor-Critic method offer several advantages:

- **Improved Sample Efficiency:** The hybrid nature of Actor-Critic algorithms often leads to improved sample efficiency, requiring fewer interactions with the environment to achieve optimal performance.
- **Faster Convergence:** The method's ability to update both the policy and value function concurrently contributes to faster convergence during training, enabling quicker adaptation to the learning task.
- **Versatility Across Action Spaces:** Actor-Critic architectures can seamlessly handle both discrete and continuous action spaces, offering flexibility in addressing a wide range of RL problems.

Variants of Actor-Critic Algorithms

Several variants of the Actor-Critic algorithm have been developed to address specific challenges or improve performance in certain types of environments:

- **Advantage Actor-Critic (A2C):** A2C modifies the critic's value function to estimate the **advantage function**, which measures how much better or worse an action is compared to the average action. The advantage function is defined as:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

A2C helps reduce the variance of the policy gradient, leading to better learning performance.

- **Asynchronous Advantage Actor-Critic (A3C):** A3C is an extension of A2C that uses multiple agents (threads) running in parallel to update the policy asynchronously. This allows for more stable and faster learning by reducing correlations between updates.

Comment

More info

Advertise with us

Next Article

How Interaction Design Enhances UI/UX?

Similar Reads

Dyna Algorithm in Reinforcement Learning

The Dyna algorithm introduces a hybrid approach that leverages both real-world and simulated experiences, enhancing the agent's learning efficiency. This article delves into the key concepts, architecture, and benefits of...

5 min read

Epsilon-Greedy Algorithm in Reinforcement Learning

In Reinforcement Learning, the agent or decision-maker learns what to do—how to map situations to actions—so as to maximize a numerical reward signal. The agent is not explicitly told which actions to take,...

4 min read

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel

Department of Electrical Engineering and Computer Science

University of California, Berkeley

{joschu, pcmoritz, levine, jordan, pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experience required for the learning tasks on 3D bipeds corresponds to 1-2 weeks of real time.

1 INTRODUCTION

The typical problem formulation in reinforcement learning is to maximize the expected total reward of a policy. A key source of difficulty is the long time delay between actions and their positive or negative effect on rewards; this issue is called the *credit assignment problem* in the reinforcement learning literature (Minsky, 1961; Sutton & Barto, 1998), and the *distal reward problem* in the behavioral literature (Hull, 1943). Value functions offer an elegant solution to the credit assignment problem—they allow us to estimate the goodness of an action before the delayed reward arrives. Reinforcement learning algorithms make use of value functions in a variety of different ways; this paper considers algorithms that optimize a parameterized policy and use value functions to help estimate how the policy should be improved.

When using a parameterized *stochastic* policy, it is possible to obtain an unbiased estimate of the gradient of the expected total returns (Williams, 1992; Sutton et al., 1999; Baxter & Bartlett, 2000); these noisy gradient estimates can be used in a stochastic gradient ascent algorithm. Unfortunately, the variance of the gradient estimator scales unfavorably with the time horizon, since the effect of an action is confounded with the effects of past and future actions. Another class of policy gradient algorithms, called actor-critic methods, use a value function rather than the empirical returns, obtaining an estimator with lower variance at the cost of introducing bias (Konda & Tsitsiklis, 2003; Hafner & Riedmiller, 2011). But while high variance necessitates using more samples, bias is more pernicious—even with an unlimited number of samples, bias can cause the algorithm to fail to converge, or to converge to a poor solution that is not even a local optimum.

We propose a family of policy gradient estimators that significantly reduce variance while maintaining a tolerable level of bias. We call this estimation scheme, parameterized by $\gamma \in [0, 1]$ and

$\lambda \in [0, 1]$, the generalized advantage estimator (GAE). Related methods have been proposed in the context of online actor-critic methods (Kimura & Kobayashi, 1998; Wawrzyński, 2009). We provide a more general analysis, which is applicable in both the online and batch settings, and discuss an interpretation of our method as an instance of reward shaping (Ng et al., 1999), where the approximate value function is used to shape the reward.

We present experimental results on a number of highly challenging 3D locomotion tasks, where we show that our approach can learn complex gaits using high-dimensional, general purpose neural network function approximators for both the policy and the value function, each with over 10^4 parameters. The policies perform torque-level control of simulated 3D robots with up to 33 state dimensions and 10 actuators.

The contributions of this paper are summarized as follows:

1. We provide justification and intuition for an effective variance reduction scheme for policy gradients, which we call generalized advantage estimation (GAE). While the formula has been proposed in prior work (Kimura & Kobayashi, 1998; Wawrzyński, 2009), our analysis is novel and enables GAE to be applied with a more general set of algorithms, including the batch trust-region algorithm we use for our experiments.
2. We propose the use of a trust region optimization method for the value function, which we find is a robust and efficient way to train neural network value functions with thousands of parameters.
3. By combining (1) and (2) above, we obtain an algorithm that empirically is effective at learning neural network policies for challenging control tasks. The results extend the state of the art in using reinforcement learning for high-dimensional continuous control. Videos are available at <https://sites.google.com/site/gaepapersupp>.

2 PRELIMINARIES

We consider an undiscounted formulation of the policy optimization problem. The initial state s_0 is sampled from distribution ρ_0 . A trajectory $(s_0, a_0, s_1, a_1, \dots)$ is generated by sampling actions according to the policy $a_t \sim \pi(a_t | s_t)$ and sampling the states according to the dynamics $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$, until a terminal (absorbing) state is reached. A reward $r_t = r(s_t, a_t, s_{t+1})$ is received at each timestep. The goal is to maximize the expected total reward $\sum_{t=0}^{\infty} r_t$, which is assumed to be finite for all policies. Note that we are not using a discount as part of the problem specification; it will appear below as an algorithm parameter that adjusts a bias-variance tradeoff. But the discounted problem (maximizing $\sum_{t=0}^{\infty} \gamma^t r_t$) can be handled as an instance of the undiscounted problem in which we absorb the discount factor into the reward function, making it time-dependent.

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_{\theta} \mathbb{E} [\sum_{t=0}^{\infty} r_t]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (1)$$

where Ψ_t may be one of the following:

- | | |
|--|---|
| 1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory. | 4. $Q^{\pi}(s_t, a_t)$: state-action value function. |
| 2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t . | 5. $A^{\pi}(s_t, a_t)$: advantage function. |
| 3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula. | 6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual. |

The latter formulas use the definitions

$$V^{\pi}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad Q^{\pi}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad (2)$$

$$A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (\text{Advantage function}). \quad (3)$$

Here, the subscript of $\mathbb{E}$ enumerates the variables being integrated over, where states and actions are sampled sequentially from the dynamics model $P(s_{t+1} | s_t, a_t)$ and policy $\pi(a_t | s_t)$, respectively. The colon notation $a : b$ refers to the inclusive range $(a, a + 1, \dots, b)$. These formulas are well known and straightforward to obtain; they follow directly from Proposition 1, which will be stated shortly.

The choice $\Psi_t = A^\pi(s_t, a_t)$ yields almost the lowest possible variance, though in practice, the advantage function is not known and must be estimated. This statement can be intuitively justified by the following interpretation of the policy gradient: that a step in the policy gradient direction should increase the probability of better-than-average actions and decrease the probability of worse-than-average actions. The advantage function, by its definition $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$, measures whether or not the action is better or worse than the policy’s default behavior. Hence, we should choose Ψ_t to be the advantage function $A^\pi(s_t, a_t)$, so that the gradient term $\Psi_t \nabla_\theta \log \pi_\theta(a_t | s_t)$ points in the direction of increased $\pi_\theta(a_t | s_t)$ if and only if $A^\pi(s_t, a_t) > 0$. See Greensmith et al. (2004) for a more rigorous analysis of the variance of policy gradient estimators and the effect of using a baseline.

We will introduce a parameter γ that allows us to reduce variance by downweighting rewards corresponding to delayed effects, at the cost of introducing bias. This parameter corresponds to the discount factor used in discounted formulations of MDPs, but we treat it as a variance reduction parameter in an undiscounted problem; this technique was analyzed theoretically by Marbach & Tsitsiklis (2003); Kakade (2001b); Thomas (2014). The discounted value functions are given by:

$$V^{\pi, \gamma}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} \gamma^l r_{t+l} \right] \quad Q^{\pi, \gamma}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} \gamma^l r_{t+l} \right] \quad (4)$$

$$A^{\pi, \gamma}(s_t, a_t) := Q^{\pi, \gamma}(s_t, a_t) - V^{\pi, \gamma}(s_t). \quad (5)$$

The discounted approximation to the policy gradient is defined as follows:

$$g^\gamma := \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\sum_{t=0}^{\infty} A^{\pi, \gamma}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \quad (6)$$

The following section discusses how to obtain biased (but not too biased) estimators for $A^{\pi, \gamma}$, giving us noisy estimates of the discounted policy gradient in Equation (6).

Before proceeding, we will introduce the notion of a γ -just estimator of the advantage function, which is an estimator that does not introduce bias when we use it in place of $A^{\pi, \gamma}$ (which is not known and must be estimated) in Equation (6) to estimate g^γ .¹ Consider an advantage estimator $\hat{A}_t(s_{0:\infty}, a_{0:\infty})$, which may in general be a function of the entire trajectory.

Definition 1. The estimator $\hat{A}_t$ is γ -just if

$$\mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\hat{A}_t(s_{0:\infty}, a_{0:\infty}) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[A^{\pi, \gamma}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \quad (7)$$

It follows immediately that if $\hat{A}_t$ is γ -just for all t , then

$$\mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\sum_{t=0}^{\infty} \hat{A}_t(s_{0:\infty}, a_{0:\infty}) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = g^\gamma \quad (8)$$

One sufficient condition for $\hat{A}_t$ to be γ -just is that $\hat{A}_t$ decomposes as the difference between two functions Q_t and b_t , where Q_t can depend on any trajectory variables but gives an unbiased estimator of the γ -discounted Q -function, and b_t is an arbitrary function of the states and actions sampled before a_t .

Proposition 1. Suppose that $\hat{A}_t$ can be written in the form $\hat{A}_t(s_{0:\infty}, a_{0:\infty}) = Q_t(s_{t:\infty}, a_{t:\infty}) - b_t(s_{0:t}, a_{0:t-1})$ such that for all (s_t, a_t) , $\mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty} | s_t, a_t} [Q_t(s_{t:\infty}, a_{t:\infty})] = Q^{\pi, \gamma}(s_t, a_t)$. Then $\hat{A}$ is γ -just.

¹Note, that we have already introduced bias by using $A^{\pi, \gamma}$ in place of A^π ; here we are concerned with obtaining an unbiased estimate of g^γ , which is a biased estimate of the policy gradient of the undiscounted MDP.

The proof is provided in Appendix B. It is easy to verify that the following expressions are γ -just advantage estimators for $\hat{A}_t$:

- $\sum_{l=0}^{\infty} \gamma^l r_{t+l}$
- $Q^{\pi, \gamma}(s_t, a_t)$
- $A^{\pi, \gamma}(s_t, a_t)$
- $r_t + \gamma V^{\pi, \gamma}(s_{t+1}) - V^{\pi, \gamma}(s_t)$.

3 ADVANTAGE FUNCTION ESTIMATION

This section will be concerned with producing an accurate estimate $\hat{A}_t$ of the discounted advantage function $A^{\pi, \gamma}(s_t, a_t)$, which will then be used to construct a policy gradient estimator of the following form:

$$\hat{g} = \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^{\infty} \hat{A}_t^n \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n) \quad (9)$$

where n indexes over a batch of episodes.

Let V be an approximate value function. Define $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$, i.e., the TD residual of V with discount γ (Sutton & Barto, 1998). Note that δ_t^V can be considered as an estimate of the advantage of the action a_t . In fact, if we have the correct value function $V = V^{\pi, \gamma}$, then it is a γ -just advantage estimator, and in fact, an unbiased estimator of $A^{\pi, \gamma}$:

$$\begin{aligned} \mathbb{E}_{s_{t+1}} [\delta_t^{V^{\pi, \gamma}}] &= \mathbb{E}_{s_{t+1}} [r_t + \gamma V^{\pi, \gamma}(s_{t+1}) - V^{\pi, \gamma}(s_t)] \\ &= \mathbb{E}_{s_{t+1}} [Q^{\pi, \gamma}(s_t, a_t) - V^{\pi, \gamma}(s_t)] = A^{\pi, \gamma}(s_t, a_t). \end{aligned} \quad (10)$$

However, this estimator is only γ -just for $V = V^{\pi, \gamma}$, otherwise it will yield biased policy gradient estimates.

Next, let us consider taking the sum of k of these δ terms, which we will denote by $\hat{A}_t^{(k)}$

$$\hat{A}_t^{(1)} := \delta_t^V = -V(s_t) + r_t + \gamma V(s_{t+1}) \quad (11)$$

$$\hat{A}_t^{(2)} := \delta_t^V + \gamma \delta_{t+1}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \quad (12)$$

$$\hat{A}_t^{(3)} := \delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3}) \quad (13)$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}) \quad (14)$$

These equations result from a telescoping sum, and we see that $\hat{A}_t^{(k)}$ involves a k -step estimate of the returns, minus a baseline term $-V(s_t)$. Analogously to the case of $\delta_t^V = \hat{A}_t^{(1)}$, we can consider $\hat{A}_t^{(k)}$ to be an estimator of the advantage function, which is only γ -just when $V = V^{\pi, \gamma}$. However, note that the bias generally becomes smaller as $k \rightarrow \infty$, since the term $\gamma^k V(s_{t+k})$ becomes more heavily discounted, and the term $-V(s_t)$ does not affect the bias. Taking $k \rightarrow \infty$, we get

$$\hat{A}_t^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^V = -V(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l}, \quad (15)$$

which is simply the empirical returns minus the value function baseline.

The generalized advantage estimator $\text{GAE}(\gamma, \lambda)$ is defined as the exponentially-weighted average of these k -step estimators:

$$\begin{aligned}
\hat{A}_t^{\text{GAE}(\gamma, \lambda)} &:= (1 - \lambda) \left(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V + \lambda(\delta_t^V + \gamma \delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V) + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V (1 + \lambda + \lambda^2 + \dots) + \gamma \delta_{t+1}^V (\lambda + \lambda^2 + \lambda^3 + \dots) \right. \\
&\quad \left. + \gamma^2 \delta_{t+2}^V (\lambda^2 + \lambda^3 + \lambda^4 + \dots) + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V \left(\frac{1}{1 - \lambda} \right) + \gamma \delta_{t+1}^V \left(\frac{\lambda}{1 - \lambda} \right) + \gamma^2 \delta_{t+2}^V \left(\frac{\lambda^2}{1 - \lambda} \right) + \dots \right) \\
&= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V
\end{aligned} \tag{16}$$

From Equation (16), we see that the advantage estimator has a remarkably simple formula involving a discounted sum of Bellman residual terms. Section 4 discusses an interpretation of this formula as the returns in an MDP with a modified reward function. The construction we used above is closely analogous to the one used to define $\text{TD}(\lambda)$ (Sutton & Barto, 1998), however $\text{TD}(\lambda)$ is an estimator of the value function, whereas here we are estimating the advantage function.

There are two notable special cases of this formula, obtained by setting $\lambda = 0$ and $\lambda = 1$.

$$\text{GAE}(\gamma, 0) : \hat{A}_t := \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{17}$$

$$\text{GAE}(\gamma, 1) : \hat{A}_t := \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t) \tag{18}$$

$\text{GAE}(\gamma, 1)$ is γ -just regardless of the accuracy of V , but it has high variance due to the sum of terms. $\text{GAE}(\gamma, 0)$ is γ -just for $V = V^{\pi, \gamma}$ and otherwise induces bias, but it typically has much lower variance. The generalized advantage estimator for $0 < \lambda < 1$ makes a compromise between bias and variance, controlled by parameter λ .

We’ve described an advantage estimator with two separate parameters γ and λ , both of which contribute to the bias-variance tradeoff when using an approximate value function. However, they serve different purposes and work best with different ranges of values. γ most importantly determines the scale of the value function $V^{\pi, \gamma}$, which does not depend on λ . Taking $\gamma < 1$ introduces bias into the policy gradient estimate, regardless of the value function’s accuracy. On the other hand, $\lambda < 1$ introduces bias only when the value function is inaccurate. Empirically, we find that the best value of λ is much lower than the best value of γ , likely because λ introduces far less bias than γ for a reasonably accurate value function.

Using the generalized advantage estimator, we can construct a biased estimator of g^γ , the discounted policy gradient from Equation (6):

$$g^\gamma \approx \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{\text{GAE}(\gamma, \lambda)} \right] = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \right], \tag{19}$$

where equality holds when $\lambda = 1$.

4 INTERPRETATION AS REWARD SHAPING

In this section, we discuss how one can interpret λ as an extra discount factor applied after performing a reward shaping transformation on the MDP. We also introduce the notion of a response function to help understand the bias introduced by γ and λ .

Reward shaping (Ng et al., 1999) refers to the following transformation of the reward function of an MDP: let $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ be an arbitrary scalar-valued function on state space, and define the transformed reward function $\tilde{r}$ by

$$\tilde{r}(s, a, s') = r(s, a, s') + \gamma \Phi(s') - \Phi(s), \tag{20}$$

which in turn defines a transformed MDP. This transformation leaves the discounted advantage function $A^{\pi,\gamma}$ unchanged for any policy π . To see this, consider the discounted sum of rewards of a trajectory starting with state s_t :

$$\sum_{l=0}^{\infty} \gamma^l \tilde{r}(s_{t+l}, a_t, s_{t+l+1}) = \sum_{l=0}^{\infty} \gamma^l r(s_{t+l}, a_{t+l}, s_{t+l+1}) - \Phi(s_t). \quad (21)$$

Letting $\tilde{Q}^{\pi,\gamma}$, $\tilde{V}^{\pi,\gamma}$, $\tilde{A}^{\pi,\gamma}$ be the value and advantage functions of the transformed MDP, one obtains from the definitions of these quantities that

$$\tilde{Q}^{\pi,\gamma}(s, a) = Q^{\pi,\gamma}(s, a) - \Phi(s) \quad (22)$$

$$\tilde{V}^{\pi,\gamma}(s, a) = V^{\pi,\gamma}(s) - \Phi(s) \quad (23)$$

$$\tilde{A}^{\pi,\gamma}(s, a) = (Q^{\pi,\gamma}(s, a) - \Phi(s)) - (V^{\pi,\gamma}(s) - \Phi(s)) = A^{\pi,\gamma}(s, a). \quad (24)$$

Note that if Φ happens to be the state-value function $V^{\pi,\gamma}$ from the original MDP, then the transformed MDP has the interesting property that $\tilde{V}^{\pi,\gamma}(s)$ is zero at every state.

Note that (Ng et al., 1999) showed that the reward shaping transformation leaves the policy gradient and optimal policy unchanged when our objective is to maximize the discounted sum of rewards $\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1})$. In contrast, this paper is concerned with maximizing the undiscounted sum of rewards, where the discount γ is used as a variance-reduction parameter.

Having reviewed the idea of reward shaping, let us consider how we could use it to get a policy gradient estimate. The most natural approach is to construct policy gradient estimators that use discounted sums of shaped rewards $\tilde{r}$. However, Equation (21) shows that we obtain the discounted sum of the original MDP’s rewards r minus a baseline term. Next, let’s consider using a “steeper” discount $\gamma\lambda$, where $0 \leq \lambda \leq 1$. It’s easy to see that the shaped reward $\tilde{r}$ equals the Bellman residual term δ^V , introduced in Section 3, where we set $\Phi = V$. Letting $\Phi = V$, we see that

$$\sum_{l=0}^{\infty} (\gamma\lambda)^l \tilde{r}(s_{t+l}, a_t, s_{t+l+1}) = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V = \hat{A}_t^{\text{GAE}(\gamma,\lambda)}. \quad (25)$$

Hence, by considering the $\gamma\lambda$ -discounted sum of shaped rewards, we exactly obtain the generalized advantage estimators from Section 3. As shown previously, $\lambda = 1$ gives an unbiased estimate of g^γ , whereas $\lambda < 1$ gives a biased estimate.

To further analyze the effect of this shaping transformation and parameters γ and λ , it will be useful to introduce the notion of a response function χ , which we define as follows:

$$\chi(l; s_t, a_t) = \mathbb{E}[r_{t+l} | s_t, a_t] - \mathbb{E}[r_{t+l} | s_t]. \quad (26)$$

Note that $A^{\pi,\gamma}(s, a) = \sum_{l=0}^{\infty} \gamma^l \chi(l; s, a)$, hence the response function decomposes the advantage function across timesteps. The response function lets us quantify the temporal credit assignment problem: long range dependencies between actions and rewards correspond to nonzero values of the response function for $l \gg 0$.

Next, let us revisit the discount factor γ and the approximation we are making by using $A^{\pi,\gamma}$ rather than $A^{\pi,1}$. The discounted policy gradient estimator from Equation (6) has a sum of terms of the form

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi,\gamma}(s_t, a_t) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{l=0}^{\infty} \gamma^l \chi(l; s_t, a_t). \quad (27)$$

Using a discount $\gamma < 1$ corresponds to dropping the terms with $l \gg 1/(1 - \gamma)$. Thus, the error introduced by this approximation will be small if χ rapidly decays as l increases, i.e., if the effect of an action on rewards is “forgotten” after $\approx 1/(1 - \gamma)$ timesteps.

If the reward function $\tilde{r}$ were obtained using $\Phi = V^{\pi,\gamma}$, we would have $\mathbb{E}[\tilde{r}_{t+l} | s_t, a_t] = \mathbb{E}[\tilde{r}_{t+l} | s_t] = 0$ for $l > 0$, i.e., the response function would only be nonzero at $l = 0$. Therefore, this shaping transformation would turn temporally extended response into an immediate response. Given that $V^{\pi,\gamma}$ completely reduces the temporal spread of the response function, we can hope that a good approximation $V \approx V^{\pi,\gamma}$ partially reduces it. This observation suggests an interpretation of Equation (16): reshape the rewards using V to shrink the temporal extent of the response function, and then introduce a “steeper” discount $\gamma\lambda$ to cut off the noise arising from long delays, i.e., ignore terms $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \delta_{t+l}^V$ where $l \gg 1/(1 - \gamma\lambda)$.

5 VALUE FUNCTION ESTIMATION

A variety of different methods can be used to estimate the value function (see, e.g., Bertsekas (2012)). When using a nonlinear function approximator to represent the value function, the simplest approach is to solve a nonlinear regression problem:

$$\underset{\phi}{\text{minimize}} \sum_{n=1}^N \|V_{\phi}(s_n) - \hat{V}_n\|^2, \quad (28)$$

where $\hat{V}_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l}$ is the discounted sum of rewards, and n indexes over all timesteps in a batch of trajectories. This is sometimes called the Monte Carlo or TD(1) approach for estimating the value function (Sutton & Barto, 1998).²

For the experiments in this work, we used a trust region method to optimize the value function in each iteration of a batch optimization procedure. The trust region helps us to avoid overfitting to the most recent batch of data. To formulate the trust region problem, we first compute $\sigma^2 = \frac{1}{N} \sum_{n=1}^N \|V_{\phi_{\text{old}}}(s_n) - \hat{V}_n\|^2$, where ϕ_{old} is the parameter vector before optimization. Then we solve the following constrained optimization problem:

$$\begin{aligned} &\underset{\phi}{\text{minimize}} \quad \sum_{n=1}^N \|V_{\phi}(s_n) - \hat{V}_n\|^2 \\ &\text{subject to} \quad \frac{1}{N} \sum_{n=1}^N \frac{\|V_{\phi}(s_n) - V_{\phi_{\text{old}}}(s_n)\|^2}{2\sigma^2} \leq \epsilon. \end{aligned} \quad (29)$$

This constraint is equivalent to constraining the average KL divergence between the previous value function and the new value function to be smaller than ϵ , where the value function is taken to parameterize a conditional Gaussian distribution with mean $V_{\phi}(s)$ and variance σ^2 .

We compute an approximate solution to the trust region problem using the conjugate gradient algorithm (Wright & Nocedal, 1999). Specifically, we are solving the quadratic program

$$\begin{aligned} &\underset{\phi}{\text{minimize}} \quad g^T(\phi - \phi_{\text{old}}) \\ &\text{subject to} \quad \frac{1}{N} \sum_{n=1}^N (\phi - \phi_{\text{old}})^T H(\phi - \phi_{\text{old}}) \leq \epsilon. \end{aligned} \quad (30)$$

where g is the gradient of the objective, and $H = \frac{1}{N} \sum_n j_n j_n^T$, where $j_n = \nabla_{\phi} V_{\phi}(s_n)$. Note that H is the ‘‘Gauss-Newton’’ approximation of the Hessian of the objective, and it is (up to a σ^2 factor) the Fisher information matrix when interpreting the value function as a conditional probability distribution. Using matrix-vector products $v \rightarrow H v$ to implement the conjugate gradient algorithm, we compute a step direction $s \approx -H^{-1}g$. Then we rescale $s \rightarrow \alpha s$ such that $\frac{1}{2}(\alpha s)^T H(\alpha s) = \epsilon$ and take $\phi = \phi_{\text{old}} + \alpha s$. This procedure is analogous to the procedure we use for updating the policy, which is described further in Section 6 and based on Schulman et al. (2015).

6 EXPERIMENTS

We designed a set of experiments to investigate the following questions:

1. What is the empirical effect of varying $\lambda \in [0, 1]$ and $\gamma \in [0, 1]$ when optimizing episodic total reward using generalized advantage estimation?
2. Can generalized advantage estimation, along with trust region algorithms for policy and value function optimization, be used to optimize large neural network policies for challenging control problems?

²Another natural choice is to compute target values with an estimator based on the TD(λ) backup (Bertsekas, 2012; Sutton & Barto, 1998), mirroring the expression we use for policy gradient estimation: $\hat{V}_t^{\lambda} = V_{\phi_{\text{old}}}(s_t) + \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$. While we experimented with this choice, we did not notice a difference in performance from the $\lambda = 1$ estimator in Equation (28).

6.1 POLICY OPTIMIZATION ALGORITHM

While generalized advantage estimation can be used along with a variety of different policy gradient methods, for these experiments, we performed the policy updates using trust region policy optimization (TRPO) (Schulman et al., 2015). TRPO updates the policy by approximately solving the following constrained optimization problem each iteration:

$$\begin{aligned}
& \underset{\theta}{\text{minimize}} \quad L_{\theta_{old}}(\theta) \\
& \text{subject to} \quad \overline{D}_{KL}^{\theta_{old}}(\pi_{\theta_{old}}, \pi_{\theta}) \leq \epsilon \\
& \text{where} \quad L_{\theta_{old}}(\theta) = \frac{1}{N} \sum_{n=1}^N \frac{\pi_{\theta}(a_n | s_n)}{\pi_{\theta_{old}}(a_n | s_n)} \hat{A}_n \\
& \quad \overline{D}_{KL}^{\theta_{old}}(\pi_{\theta_{old}}, \pi_{\theta}) = \frac{1}{N} \sum_{n=1}^N D_{KL}(\pi_{\theta_{old}}(\cdot | s_n) \parallel \pi_{\theta}(\cdot | s_n))
\end{aligned} \tag{31}$$

As described in (Schulman et al., 2015), we approximately solve this problem by linearizing the objective and quadraticizing the constraint, which yields a step in the direction $\theta - \theta_{old} \propto -F^{-1}g$, where F is the average Fisher information matrix, and g is a policy gradient estimate. This policy update yields the same step direction as the natural policy gradient (Kakade, 2001a) and natural actor-critic (Peters & Schaal, 2008), however it uses a different stepsize determination scheme and numerical procedure for computing the step.

Since prior work (Schulman et al., 2015) compared TRPO to a variety of different policy optimization algorithms, we will not repeat these comparisons; rather, we will focus on varying the γ, λ parameters of policy gradient estimator while keeping the underlying algorithm fixed.

For completeness, the whole algorithm for iteratively updating policy and value function is given below:

```

Initialize policy parameter  $\theta_0$  and value function parameter  $\phi_0$ .
for  $i = 0, 1, 2, \dots$  do
    Simulate current policy  $\pi_{\theta_i}$  until  $N$  timesteps are obtained.
    Compute  $\delta_t^V$  at all timesteps  $t \in \{1, 2, \dots, N\}$ , using  $V = V_{\phi_i}$ .
    Compute  $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$  at all timesteps.
    Compute  $\theta_{i+1}$  with TRPO update, Equation (31).
    Compute  $\phi_{i+1}$  with Equation (30).
end for

```

Note that the policy update $\theta_i \rightarrow \theta_{i+1}$ is performed using the value function V_{ϕ_i} for advantage estimation, not $V_{\phi_{i+1}}$. Additional bias would have been introduced if we updated the value function first. To see this, consider the extreme case where we overfit the value function, and the Bellman residual $r_t + \gamma V(s_{t+1}) - V(s_t)$ becomes zero at all timesteps—the policy gradient estimate would be zero.

6.2 EXPERIMENTAL SETUP

We evaluated our approach on the classic cart-pole balancing problem, as well as several challenging 3D locomotion tasks: (1) bipedal locomotion; (2) quadrupedal locomotion; (3) dynamically standing up, for the biped, which starts off laying on its back. The models are shown in Figure 1.

6.2.1 ARCHITECTURE

We used the same neural network architecture for all of the 3D robot tasks, which was a feedforward network with three hidden layers, with 100, 50 and 25 tanh units respectively. The same architecture was used for the policy and value function. The final output layer had linear activation. The value function estimator used the same architecture, but with only one scalar output. For the simpler cart-pole task, we used a linear policy, and a neural network with one 20-unit hidden layer as the value function.

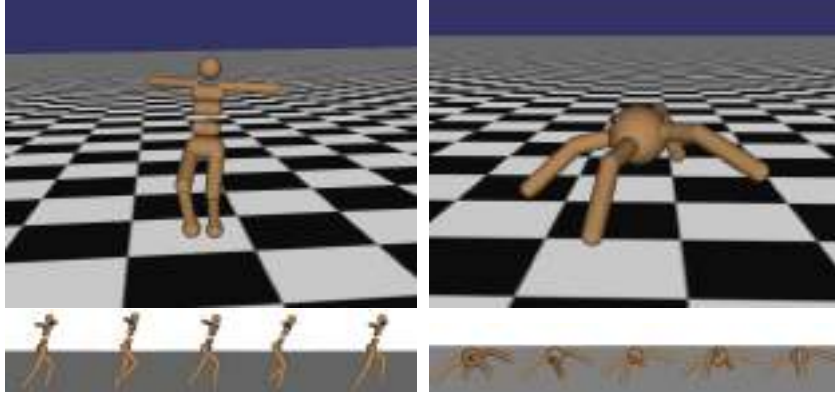


Figure 1: Top figures: robot models used for 3D locomotion. Bottom figures: a sequence of frames from the learned gaits. Videos are available at <https://sites.google.com/site/gaepapersupp>.

6.2.2 TASK DETAILS

For the cart-pole balancing task, we collected 20 trajectories per batch, with a maximum length of 1000 timesteps, using the physical parameters from Barto et al. (1983).

The simulated robot tasks were simulated using the MuJoCo physics engine (Todorov et al., 2012). The humanoid model has 33 state dimensions and 10 actuated degrees of freedom, while the quadruped model has 29 state dimensions and 8 actuated degrees of freedom. The initial state for these tasks consisted of a uniform distribution centered on a reference configuration. We used 50000 timesteps per batch for bipedal locomotion, and 200000 timesteps per batch for quadrupedal locomotion and bipedal standing. Each episode was terminated after 2000 timesteps if the robot had not reached a terminal state beforehand. The timestep was 0.01 seconds.

The reward functions are provided in the table below.

Task	Reward
3D biped locomotion	$v_{\text{fwd}} - 10^{-5}\ u\ ^2 - 10^{-5}\ f_{\text{impact}}\ ^2 + 0.2$
Quadruped locomotion	$v_{\text{fwd}} - 10^{-6}\ u\ ^2 - 10^{-3}\ f_{\text{impact}}\ ^2 + 0.05$
Biped getting up	$-(h_{\text{head}} - 1.5)^2 - 10^{-5}\ u\ ^2$

Here, $v_{\text{fwd}} :=$ forward velocity, $u :=$ vector of joint torques, $f_{\text{impact}} :=$ impact forces, $h_{\text{head}} :=$ height of the head.

In the locomotion tasks, the episode is terminated if the center of mass of the actor falls below a predefined height: .8 m for the biped, and .2 m for the quadruped. The constant offset in the reward function encourages longer episodes; otherwise the quadratic reward terms might lead to a policy that ends the episodes as quickly as possible.

6.3 EXPERIMENTAL RESULTS

All results are presented in terms of the cost, which is defined as negative reward and is minimized. Videos of the learned policies are available at <https://sites.google.com/site/gaepapersupp>. In plots, “No VF” means that we used a time-dependent baseline that did not depend on the state, rather than an estimate of the state value function. The time-dependent baseline was computed by averaging the return at each timestep over the trajectories in the batch.

6.3.1 CART-POLE

The results are averaged across 21 experiments with different random seeds. Results are shown in Figure 2, and indicate that the best results are obtained at intermediate values of the parameters: $\gamma \in [0.96, 0.99]$ and $\lambda \in [0.92, 0.99]$.

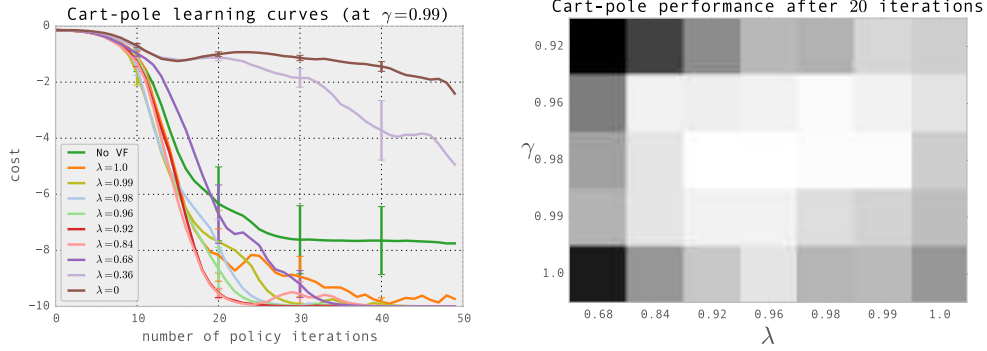


Figure 2: Left: learning curves for cart-pole task, using generalized advantage estimation with varying values of λ at $\gamma = 0.99$. The fastest policy improvement is obtained by intermediate values of λ in the range $[0.92, 0.98]$. Right: performance after 20 iterations of policy optimization, as γ and λ are varied. White means higher reward. The best results are obtained at intermediate values of both.

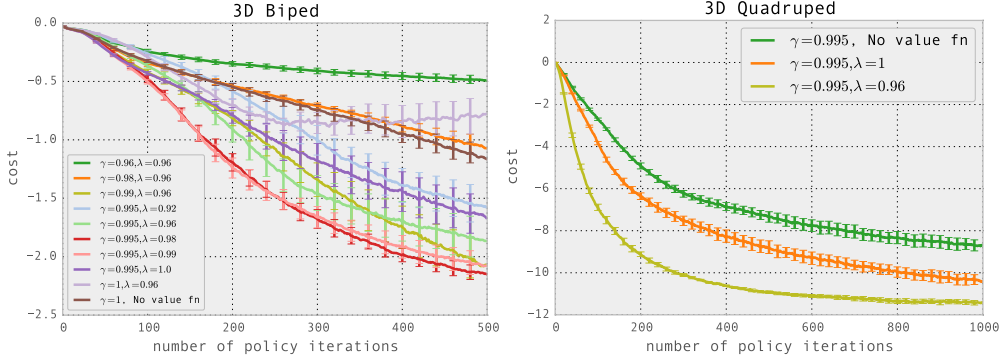


Figure 3: Left: Learning curves for 3D bipedal locomotion, averaged across nine runs of the algorithm. Right: learning curves for 3D quadrupedal locomotion, averaged across five runs.

6.3.2 3D BIPEDAL LOCOMOTION

Each trial took about 2 hours to run on a 16-core machine, where the simulation rollouts were parallelized, as were the function, gradient, and matrix-vector-product evaluations used when optimizing the policy and value function. Here, the results are averaged across 9 trials with different random seeds. The best performance is again obtained using intermediate values of $\gamma \in [0.99, 0.995]$, $\lambda \in [0.96, 0.99]$. The result after 1000 iterations is a fast, smooth, and stable gait that is effectively completely stable. We can compute how much “real time” was used for this learning process: $0.01 \text{ seconds/timestep} \times 50000 \text{ timesteps/batch} \times 1000 \text{ batches/3600} \cdot 24 \text{ seconds/day} = 5.8 \text{ days}$. Hence, it is plausible that this algorithm could be run on a real robot, or multiple real robots learning in parallel, if there were a way to reset the state of the robot and ensure that it doesn’t damage itself.

6.3.3 OTHER 3D ROBOT TASKS

The other two motor behaviors considered are quadrupedal locomotion and getting up off the ground for the 3D biped. Again, we performed 5 trials per experimental condition, with different random seeds (and initializations). The experiments took about 4 hours per trial on a 32-core machine. We performed a more limited comparison on these domains (due to the substantial computational resources required to run these experiments), fixing $\gamma = 0.995$ but varying $\lambda = \{0, 0.96\}$, as well as an experimental condition with no value function. For quadrupedal locomotion, the best results are obtained using a value function with $\lambda = 0.96$ Section 6.3.2. For 3D standing, the value function always helped, but the results are roughly the same for $\lambda = 0.96$ and $\lambda = 1$.

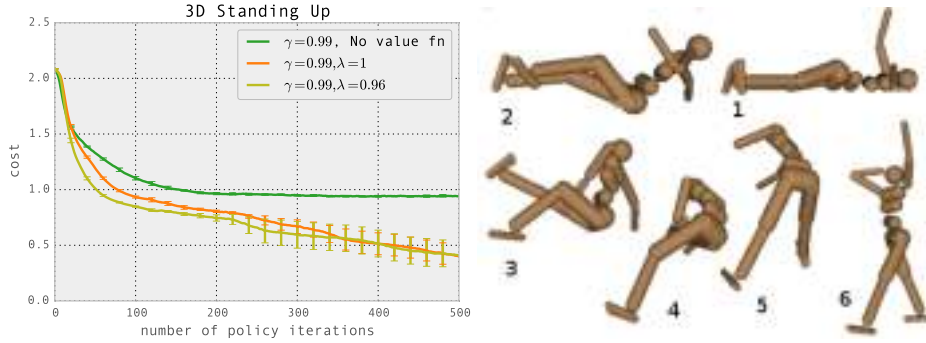


Figure 4: (a) Learning curve from quadrupedal walking, (b) learning curve for 3D standing up, (c) clips from 3D standing up.

7 DISCUSSION

Policy gradient methods provide a way to reduce reinforcement learning to stochastic gradient descent, by providing unbiased gradient estimates. However, so far their success at solving difficult control problems has been limited, largely due to their high sample complexity. We have argued that the key to variance reduction is to obtain good estimates of the advantage function.

We have provided an intuitive but informal analysis of the problem of advantage function estimation, and justified the generalized advantage estimator, which has two parameters γ, λ which adjust the bias-variance tradeoff. We described how to combine this idea with trust region policy optimization and a trust region algorithm that optimizes a value function, both represented by neural networks. Combining these techniques, we are able to learn to solve difficult control tasks that have previously been out of reach for generic reinforcement learning methods.

Our main experimental validation of generalized advantage estimation is in the domain of simulated robotic locomotion. As shown in our experiments, choosing an appropriate intermediate value of λ in the range $[0.9, 0.99]$ usually results in the best performance. A possible topic for future work is how to adjust the estimator parameters γ, λ in an adaptive or automatic way.

One question that merits future investigation is the relationship between value function estimation error and policy gradient estimation error. If this relationship were known, we could choose an error metric for value function fitting that is well-matched to the quantity of interest, which is typically the accuracy of the policy gradient estimation. Some candidates for such an error metric might include the Bellman error or projected Bellman error, as described in Bhatnagar et al. (2009).

Another enticing possibility is to use a shared function approximation architecture for the policy and the value function, while optimizing the policy using generalized advantage estimation. While formulating this problem in a way that is suitable for numerical optimization and provides convergence guarantees remains an open question, such an approach could allow the value function and policy representations to share useful features of the input, resulting in even faster learning.

In concurrent work, researchers have been developing policy gradient methods that involve differentiation with respect to the continuous-valued action (Lillicrap et al., 2015; Heess et al., 2015). While we found empirically that the one-step return ($\lambda = 0$) leads to excessive bias and poor performance, these papers show that such methods can work when tuned appropriately. However, note that those papers consider control problems with substantially lower-dimensional state and action spaces than the ones considered here. A comparison between both classes of approach would be useful for future work.

ACKNOWLEDGEMENTS

We thank Emo Todorov for providing the simulator as well as insightful discussions, and we thank Greg Wayne, Yuval Tassa, Dave Silver, Carlos Florensa Campo, and Greg Brockman for insightful discussions. This research was funded in part by the Office of Naval Research through a Young

Investigator Award and under grant number N00014-11-1-0688, DARPA through a Young Faculty Award, by the Army Research Office through the MAST program.

A FREQUENTLY ASKED QUESTIONS

A.1 WHAT’S THE RELATIONSHIP WITH COMPATIBLE FEATURES?

Compatible features are often mentioned in relation to policy gradient algorithms that make use of a value function, and the idea was proposed in the paper *On Actor-Critic Methods* by Konda & Tsitsiklis (2003). These authors pointed out that due to the limited representation power of the policy, the policy gradient only depends on a certain subspace of the space of advantage functions. This subspace is spanned by the compatible features $\nabla_{\theta_i} \log \pi_{\theta}(a_t | s_t)$, where $i \in \{1, 2, \dots, \dim \theta\}$. This theory of compatible features provides no guidance on how to exploit the temporal structure of the problem to obtain better estimates of the advantage function, making it mostly orthogonal to the ideas in this paper.

The idea of compatible features motivates an elegant method for computing the natural policy gradient (Kakade, 2001a; Peters & Schaal, 2008). Given an empirical estimate of the advantage function $\hat{A}_t$ at each timestep, we can project it onto the subspace of compatible features by solving the following least squares problem:

$$\underset{\mathbf{r}}{\text{minimize}} \sum_t \|\mathbf{r} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) - \hat{A}_t\|^2. \quad (32)$$

If $\hat{A}$ is γ -just, the least squares solution is the natural policy gradient (Kakade, 2001a). Note that any estimator of the advantage function can be substituted into this formula, including the ones we derive in this paper. For our experiments, we also compute natural policy gradient steps, but we use the more computationally efficient numerical procedure from Schulman et al. (2015), as discussed in Section 6.

A.2 WHY DON’T YOU JUST USE A Q -FUNCTION?

Previous actor critic methods, e.g. in Konda & Tsitsiklis (2003), use a Q -function to obtain potentially low-variance policy gradient estimates. Recent papers, including Heess et al. (2015); Lillicrap et al. (2015), have shown that a neural network Q -function approximator can be used effectively in a policy gradient method. However, there are several advantages to using a state-value function in the manner of this paper. First, the state-value function has a lower-dimensional input and is thus easier to learn than a state-action value function. Second, the method of this paper allows us to smoothly interpolate between the high-bias estimator ($\lambda = 0$) and the low-bias estimator ($\lambda = 1$). On the other hand, using a parameterized Q -function only allows us to use a high-bias estimator. We have found that the bias is prohibitively large when using a one-step estimate of the returns, i.e., the $\lambda = 0$ estimator, $\hat{A}_t = \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$. We expect that similar difficulty would be encountered when using an advantage estimator involving a parameterized Q -function, $\hat{A}_t = Q(s, a) - V(s)$. There is an interesting space of possible algorithms that would use a parameterized Q -function and attempt to reduce bias, however, an exploration of these possibilities is beyond the scope of this work.

B PROOFS

Proof of Proposition 1: First we can split the expectation into terms involving Q and b ,

$$\begin{aligned} & \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_t(s_{0:\infty}, a_{0:\infty}) - b_t(s_{0:t}, a_{0:t-1}))] \\ &= \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_t(s_{0:\infty}, a_{0:\infty}))] \\ &\quad - \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (b_t(s_{0:t}, a_{0:t-1}))] \end{aligned} \quad (33)$$

We'll consider the terms with Q and b in turn.

$$\begin{aligned}
& \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_t(s_{0:\infty}, a_{0:\infty})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t}} [\mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_t(s_{0:\infty}, a_{0:\infty})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} [Q_t(s_{0:\infty}, a_{0:\infty})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t)]
\end{aligned}$$

Next,

$$\begin{aligned}
& \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b_t(s_{0:t}, a_{0:t-1})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b_t(s_{0:t}, a_{0:t-1})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] b_t(s_{0:t}, a_{0:t-1})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [0 \cdot b_t(s_{0:t}, a_{0:t-1})] \\
&= 0.
\end{aligned}$$

REFERENCES

- Barto, Andrew G, Sutton, Richard S, and Anderson, Charles W. Neuronlike adaptive elements that can solve difficult learning control problems. *Systems, Man and Cybernetics, IEEE Transactions on*, (5):834–846, 1983.
- Baxter, Jonathan and Bartlett, Peter L. Reinforcement learning in POMDPs via direct gradient ascent. In *ICML*, pp. 41–48, 2000.
- Bertsekas, Dimitri P. *Dynamic programming and optimal control*, volume 2. Athena Scientific, 2012.
- Bhatnagar, Shalabh, Precup, Doina, Silver, David, Sutton, Richard S, Maei, Hamid R, and Szepesvári, Csaba. Convergent temporal-difference learning with arbitrary smooth function approximation. In *Advances in Neural Information Processing Systems*, pp. 1204–1212, 2009.
- Greensmith, Evan, Bartlett, Peter L, and Baxter, Jonathan. Variance reduction techniques for gradient estimates in reinforcement learning. *The Journal of Machine Learning Research*, 5:1471–1530, 2004.
- Hafner, Roland and Riedmiller, Martin. Reinforcement learning in feedback control. *Machine learning*, 84 (1-2):137–169, 2011.
- Heess, Nicolas, Wayne, Greg, Silver, David, Lillicrap, Timothy, Tassa, Yuval, and Erez, Tom. Learning continuous control policies by stochastic value gradients. *arXiv preprint arXiv:1510.09142*, 2015.
- Hull, Clark. Principles of behavior. 1943.
- Kakade, Sham. A natural policy gradient. In *NIPS*, volume 14, pp. 1531–1538, 2001a.
- Kakade, Sham. Optimizing average reward using discounted rewards. In *Computational Learning Theory*, pp. 605–615. Springer, 2001b.
- Kimura, Hajime and Kobayashi, Shigenobu. An analysis of actor/critic algorithms using eligibility traces: Reinforcement learning with imperfect value function. In *ICML*, pp. 278–286, 1998.
- Konda, Vijay R and Tsitsiklis, John N. On actor-critic algorithms. *SIAM journal on Control and Optimization*, 42(4):1143–1166, 2003.
- Lillicrap, Timothy P, Hunt, Jonathan J, Pritzel, Alexander, Heess, Nicolas, Erez, Tom, Tassa, Yuval, Silver, David, and Wierstra, Daan. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Marbach, Peter and Tsitsiklis, John N. Approximate gradient methods in policy-space optimization of markov reward processes. *Discrete Event Dynamic Systems*, 13(1-2):111–148, 2003.
- Minsky, Marvin. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1):8–30, 1961.
- Ng, Andrew Y, Harada, Daishi, and Russell, Stuart. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, volume 99, pp. 278–287, 1999.
- Peters, Jan and Schaal, Stefan. Natural actor-critic. *Neurocomputing*, 71(7):1180–1190, 2008.

Asynchronous Methods for Deep Reinforcement Learning

Volodymyr Mnih¹

Adrià Puigdomènech Badia¹

Mehdi Mirza^{1,2}

Alex Graves¹

Tim Harley¹

Timothy P. Lillicrap¹

David Silver¹

Koray Kavukcuoglu¹

¹ Google DeepMind

² Montreal Institute for Learning Algorithms (MILA), University of Montreal

VMNIH@GOOGLE.COM

ADRIAP@GOOGLE.COM

MIRZAMOM@IRO.UMONTREAL.CA

GRAVES@GOOGLE.COM

THARLEY@GOOGLE.COM

COUNTZERO@GOOGLE.COM

DAVIDSILVER@GOOGLE.COM

KORAYK@GOOGLE.COM

Abstract

We propose a conceptually simple and lightweight framework for deep reinforcement learning that uses asynchronous gradient descent for optimization of deep neural network controllers. We present asynchronous variants of four standard reinforcement learning algorithms and show that parallel actor-learners have a stabilizing effect on training allowing all four methods to successfully train neural network controllers. The best performing method, an asynchronous variant of actor-critic, surpasses the current state-of-the-art on the Atari domain while training for half the time on a single multi-core CPU instead of a GPU. Furthermore, we show that asynchronous actor-critic succeeds on a wide variety of continuous motor control problems as well as on a new task of navigating random 3D mazes using a visual input.

1. Introduction

Deep neural networks provide rich representations that can enable reinforcement learning (RL) algorithms to perform effectively. However, it was previously thought that the combination of simple online RL algorithms with deep neural networks was fundamentally unstable. Instead, a variety of solutions have been proposed to stabilize the algorithm (Riedmiller, 2005; Mnih et al., 2013; 2015; Van Hasselt et al., 2015; Schulman et al., 2015a). These approaches share a common idea: the sequence of observed data encountered by an online RL agent is non-stationary, and on-

line RL updates are strongly correlated. By storing the agent's data in an experience replay memory, the data can be batched (Riedmiller, 2005; Schulman et al., 2015a) or randomly sampled (Mnih et al., 2013; 2015; Van Hasselt et al., 2015) from different time-steps. Aggregating over memory in this way reduces non-stationarity and decorrelates updates, but at the same time limits the methods to off-policy reinforcement learning algorithms.

Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks: it uses more memory and computation per real interaction; and it requires off-policy learning algorithms that can update from data generated by an older policy.

In this paper we provide a very different paradigm for deep reinforcement learning. Instead of experience replay, we asynchronously execute multiple agents in parallel, on multiple instances of the environment. This parallelism also decorrelates the agents' data into a more stationary process, since at any given time-step the parallel agents will be experiencing a variety of different states. This simple idea enables a much larger spectrum of fundamental on-policy RL algorithms, such as Sarsa, n-step methods, and actor-critic methods, as well as off-policy RL algorithms such as Q-learning, to be applied robustly and effectively using deep neural networks.

Our parallel reinforcement learning paradigm also offers practical benefits. Whereas previous approaches to deep reinforcement learning rely heavily on specialized hardware such as GPUs (Mnih et al., 2015; Van Hasselt et al., 2015; Schaul et al., 2015) or massively distributed architectures (Nair et al., 2015), our experiments run on a single machine with a standard multi-core CPU. When applied to a variety of Atari 2600 domains, on many games asynchronous reinforcement learning achieves better results, in far less

time than previous GPU-based algorithms, using far less resource than massively distributed approaches. The best of the proposed methods, asynchronous advantage actor-critic (A3C), also mastered a variety of continuous motor control tasks as well as learned general strategies for exploring 3D mazes purely from visual inputs. We believe that the success of A3C on both 2D and 3D games, discrete and continuous action spaces, as well as its ability to train feedforward and recurrent agents makes it the most general and successful reinforcement learning agent to date.

2. Related Work

The General Reinforcement Learning Architecture (Gorila) of (Nair et al., 2015) performs asynchronous training of reinforcement learning agents in a distributed setting. In Gorila, each process contains an actor that acts in its own copy of the environment, a separate replay memory, and a learner that samples data from the replay memory and computes gradients of the DQN loss (Mnih et al., 2015) with respect to the policy parameters. The gradients are asynchronously sent to a central parameter server which updates a central copy of the model. The updated policy parameters are sent to the actor-learners at fixed intervals. By using 100 separate actor-learner processes and 30 parameter server instances, a total of 130 machines, Gorila was able to significantly outperform DQN over 49 Atari games. On many games Gorila reached the score achieved by DQN over 20 times faster than DQN. We also note that a similar way of parallelizing DQN was proposed by (Chavez et al., 2015).

In earlier work, (Li & Schuurmans, 2011) applied the Map Reduce framework to parallelizing batch reinforcement learning methods with linear function approximation. Parallelism was used to speed up large matrix operations but not to parallelize the collection of experience or stabilize learning. (Grounds & Kudenko, 2008) proposed a parallel version of the Sarsa algorithm that uses multiple separate actor-learners to accelerate training. Each actor-learner learns separately and periodically sends updates to weights that have changed significantly to the other learners using peer-to-peer communication.

(Tsitsiklis, 1994) studied convergence properties of Q-learning in the asynchronous optimization setting. These results show that Q-learning is still guaranteed to converge when some of the information is outdated as long as outdated information is always eventually discarded and several other technical assumptions are satisfied. Even earlier, (Bertsekas, 1982) studied the related problem of distributed dynamic programming.

Another related area of work is in evolutionary methods, which are often straightforward to parallelize by distributing fitness evaluations over multiple machines or threads (Tomassini, 1999). Such parallel evolutionary ap-

proaches have recently been applied to some visual reinforcement learning tasks. In one example, (Koutník et al., 2014) evolved convolutional neural network controllers for the TORCS driving simulator by performing fitness evaluations on 8 CPU cores in parallel.

3. Reinforcement Learning Background

We consider the standard reinforcement learning setting where an agent interacts with an environment $\mathcal{E}$ over a number of discrete time steps. At each time step t , the agent receives a state s_t and selects an action a_t from some set of possible actions $\mathcal{A}$ according to its policy π , where π is a mapping from states s_t to actions a_t . In return, the agent receives the next state s_{t+1} and receives a scalar reward r_t . The process continues until the agent reaches a terminal state after which the process restarts. The return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ is the total accumulated return from time step t with discount factor $\gamma \in (0, 1]$. The goal of the agent is to maximize the expected return from each state s_t .

The action value $Q^\pi(s, a) = \mathbb{E}[R_t | s_t = s, a]$ is the expected return for selecting action a in state s and following policy π . The optimal value function $Q^*(s, a) = \max_{\pi} Q^\pi(s, a)$ gives the maximum action value for state s and action a achievable by any policy. Similarly, the value of state s under policy π is defined as $V^\pi(s) = \mathbb{E}[R_t | s_t = s]$ and is simply the expected return for following policy π from state s .

In value-based model-free reinforcement learning methods, the action value function is represented using a function approximator, such as a neural network. Let $Q(s, a; \theta)$ be an approximate action-value function with parameters θ . The updates to θ can be derived from a variety of reinforcement learning algorithms. One example of such an algorithm is Q-learning, which aims to directly approximate the optimal action value function: $Q^*(s, a) \approx Q(s, a; \theta)$. In one-step Q-learning, the parameters θ of the action value function $Q(s, a; \theta)$ are learned by iteratively minimizing a sequence of loss functions, where the i th loss function defined as

$$L_i(\theta_i) = \mathbb{E} \left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right)^2$$

where s' is the state encountered after state s .

We refer to the above method as one-step Q-learning because it updates the action value $Q(s, a)$ toward the one-step return $r + \gamma \max_{a'} Q(s', a'; \theta)$. One drawback of using one-step methods is that obtaining a reward r only directly affects the value of the state action pair s, a that led to the reward. The values of other state action pairs are affected only indirectly through the updated value $Q(s, a)$. This can make the learning process slow since many updates are required to propagate a reward to the relevant preceding states and actions.

One way of propagating rewards faster is by using n -step returns (Watkins, 1989; Peng & Williams, 1996). In n -step Q-learning, $Q(s, a)$ is updated toward the n -step return defined as $r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a)$. This results in a single reward r directly affecting the values of n preceding state action pairs. This makes the process of propagating rewards to relevant state-action pairs potentially much more efficient.

In contrast to value-based methods, policy-based model-free methods directly parameterize the policy $\pi(a|s; \theta)$ and update the parameters θ by performing, typically approximate, gradient ascent on $\mathbb{E}[R_t]$. One example of such a method is the REINFORCE family of algorithms due to Williams (1992). Standard REINFORCE updates the policy parameters θ in the direction $\nabla_{\theta} \log \pi(a_t|s_t; \theta) R_t$, which is an unbiased estimate of $\nabla_{\theta} \mathbb{E}[R_t]$. It is possible to reduce the variance of this estimate while keeping it unbiased by subtracting a learned function of the state $b_t(s_t)$, known as a baseline (Williams, 1992), from the return. The resulting gradient is $\nabla_{\theta} \log \pi(a_t|s_t; \theta) (R_t - b_t(s_t))$.

A learned estimate of the value function is commonly used as the baseline $b_t(s_t) \approx V^{\pi}(s_t)$ leading to a much lower variance estimate of the policy gradient. When an approximate value function is used as the baseline, the quantity $R_t - b_t$ used to scale the policy gradient can be seen as an estimate of the *advantage* of action a_t in state s_t , or $A(a_t, s_t) = Q(a_t, s_t) - V(s_t)$, because R_t is an estimate of $Q^{\pi}(a_t, s_t)$ and b_t is an estimate of $V^{\pi}(s_t)$. This approach can be viewed as an actor-critic architecture where the policy π is the actor and the baseline b_t is the critic (Sutton & Barto, 1998; Degris et al., 2012).

4. Asynchronous RL Framework

We now present multi-threaded asynchronous variants of one-step Sarsa, one-step Q-learning, n -step Q-learning, and advantage actor-critic. The aim in designing these methods was to find RL algorithms that can train deep neural network policies reliably and without large resource requirements. While the underlying RL methods are quite different, with actor-critic being an on-policy policy search method and Q-learning being an off-policy value-based method, we use two main ideas to make all four algorithms practical given our design goal.

First, we use asynchronous actor-learners, similarly to the Gorila framework (Nair et al., 2015), but instead of using separate machines and a parameter server, we use multiple CPU threads on a single machine. Keeping the learners on a single machine removes the communication costs of sending gradients and parameters and enables us to use Hogwild! (Recht et al., 2011) style updates for training.

Second, we make the observation that multiple actors-

Algorithm 1 Asynchronous one-step Q-learning - pseudocode for each actor-learner thread.

```
// Assume global shared  $\theta, \theta^-$ , and counter  $T = 0$ .
Initialize thread step counter  $t \leftarrow 0$ 
Initialize target network weights  $\theta^- \leftarrow \theta$ 
Initialize network gradients  $d\theta \leftarrow 0$ 
Get initial state  $s$ 
repeat
  Take action  $a$  with  $\epsilon$ -greedy policy based on  $Q(s, a; \theta)$ 
  Receive new state  $s'$  and reward  $r$ 
   $y = \begin{cases} r & \text{for terminal } s' \\ r + \gamma \max_{a'} Q(s', a'; \theta^-) & \text{for non-terminal } s' \end{cases}$ 
  Accumulate gradients wrt  $\theta$ :  $d\theta \leftarrow d\theta + \frac{\partial(y - Q(s, a; \theta))^2}{\partial \theta}$ 
   $s = s'$ 
   $T \leftarrow T + 1$  and  $t \leftarrow t + 1$ 
  if  $T \bmod I_{target} == 0$  then
    Update the target network  $\theta^- \leftarrow \theta$ 
  end if
  if  $t \bmod I_{AsyncUpdate} == 0$  or  $s$  is terminal then
    Perform asynchronous update of  $\theta$  using  $d\theta$ .
    Clear gradients  $d\theta \leftarrow 0$ .
  end if
until  $T > T_{max}$ 
```

learners running in parallel are likely to be exploring different parts of the environment. Moreover, one can explicitly use different exploration policies in each actor-learner to maximize this diversity. By running different exploration policies in different threads, the overall changes being made to the parameters by multiple actor-learners applying online updates in parallel are likely to be less correlated in time than a single agent applying online updates. Hence, we do not use a replay memory and rely on parallel actors employing different exploration policies to perform the stabilizing role undertaken by experience replay in the DQN training algorithm.

In addition to stabilizing learning, using multiple parallel actor-learners has multiple practical benefits. First, we obtain a reduction in training time that is roughly linear in the number of parallel actor-learners. Second, since we no longer rely on experience replay for stabilizing learning we are able to use on-policy reinforcement learning methods such as Sarsa and actor-critic to train neural networks in a stable way. We now describe our variants of one-step Q-learning, one-step Sarsa, n -step Q-learning and advantage actor-critic.

Asynchronous one-step Q-learning: Pseudocode for our variant of Q-learning, which we call Asynchronous one-step Q-learning, is shown in Algorithm 1. Each thread interacts with its own copy of the environment and at each step computes a gradient of the Q-learning loss. We use a shared and slowly changing target network in computing the Q-learning loss, as was proposed in the DQN training method. We also accumulate gradients over multiple timesteps before they are applied, which is similar to us-

ing minibatches. This reduces the chances of multiple actor learners overwriting each other’s updates. Accumulating updates over several steps also provides some ability to trade off computational efficiency for data efficiency.

Finally, we found that giving each thread a different exploration policy helps improve robustness. Adding diversity to exploration in this manner also generally improves performance through better exploration. While there are many possible ways of making the exploration policies differ we experiment with using ϵ -greedy exploration with ϵ periodically sampled from some distribution by each thread.

Asynchronous one-step Sarsa: The asynchronous one-step Sarsa algorithm is the same as asynchronous one-step Q-learning as given in Algorithm 1 except that it uses a different target value for $Q(s, a)$. The target value used by one-step Sarsa is $r + \gamma Q(s', a'; \theta^-)$ where a' is the action taken in state s' (Rummery & Niranjan, 1994; Sutton & Barto, 1998). We again use a target network and updates accumulated over multiple timesteps to stabilize learning.

Asynchronous n-step Q-learning: Pseudocode for our variant of multi-step Q-learning is shown in Supplementary Algorithm S2. The algorithm is somewhat unusual because it operates in the forward view by explicitly computing n-step returns, as opposed to the more common backward view used by techniques like eligibility traces (Sutton & Barto, 1998). We found that using the forward view is easier when training neural networks with momentum-based methods and backpropagation through time. In order to compute a single update, the algorithm first selects actions using its exploration policy for up to t_{max} steps or until a terminal state is reached. This process results in the agent receiving up to t_{max} rewards from the environment since its last update. The algorithm then computes gradients for n-step Q-learning updates for each of the state-action pairs encountered since the last update. Each n-step update uses the longest possible n-step return resulting in a one-step update for the last state, a two-step update for the second last state, and so on for a total of up to t_{max} updates. The accumulated updates are applied in a single gradient step.

Asynchronous advantage actor-critic: The algorithm, which we call asynchronous advantage actor-critic (A3C), maintains a policy $\pi(a_t|s_t; \theta)$ and an estimate of the value function $V(s_t; \theta_v)$. Like our variant of n-step Q-learning, our variant of actor-critic also operates in the forward view and uses the same mix of n-step returns to update both the policy and the value-function. The policy and the value function are updated after every t_{max} actions or when a terminal state is reached. The update performed by the algorithm can be seen as $\nabla_{\theta'} \log \pi(a_t|s_t; \theta') A(s_t, a_t; \theta, \theta_v)$ where $A(s_t, a_t; \theta, \theta_v)$ is an estimate of the advantage function given by $\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$, where k can vary from state to state and is upper-bounded

by t_{max} . The pseudocode for the algorithm is presented in Supplementary Algorithm S3.

As with the value-based methods we rely on parallel actor-learners and accumulated updates for improving training stability. Note that while the parameters θ of the policy and θ_v of the value function are shown as being separate for generality, we always share some of the parameters in practice. We typically use a convolutional neural network that has one softmax output for the policy $\pi(a_t|s_t; \theta)$ and one linear output for the value function $V(s_t; \theta_v)$, with all non-output layers shared.

We also found that adding the entropy of the policy π to the objective function improved exploration by discouraging premature convergence to suboptimal deterministic policies. This technique was originally proposed by (Williams & Peng, 1991), who found that it was particularly helpful on tasks requiring hierarchical behavior. The gradient of the full objective function including the entropy regularization term with respect to the policy parameters takes the form $\nabla_{\theta'} \log \pi(a_t|s_t; \theta') (R_t - V(s_t; \theta_v)) + \beta \nabla_{\theta'} H(\pi(s_t; \theta'))$, where H is the entropy. The hyperparameter β controls the strength of the entropy regularization term.

Optimization: We investigated three different optimization algorithms in our asynchronous framework – SGD with momentum, RMSProp (Tieleman & Hinton, 2012) without shared statistics, and RMSProp with shared statistics. We used the standard non-centered RMSProp update given by

$$g = \alpha g + (1 - \alpha) \Delta \theta^2 \text{ and } \theta \leftarrow \theta - \eta \frac{\Delta \theta}{\sqrt{g + \epsilon}}, \quad (1)$$

where all operations are performed elementwise. A comparison on a subset of Atari 2600 games showed that a variant of RMSProp where statistics g are shared across threads is considerably more robust than the other two methods. Full details of the methods and comparisons are included in Supplementary Section 7.

5. Experiments

We use four different platforms for assessing the properties of the proposed framework. We perform most of our experiments using the Arcade Learning Environment (Bellemare et al., 2012), which provides a simulator for Atari 2600 games. This is one of the most commonly used benchmark environments for RL algorithms. We use the Atari domain to compare against state of the art results (Van Hasselt et al., 2015; Wang et al., 2015; Schaul et al., 2015; Nair et al., 2015; Mnih et al., 2015), as well as to carry out a detailed stability and scalability analysis of the proposed methods. We performed further comparisons using the TORCS 3D car racing simulator (Wymann et al., 2013). We also use

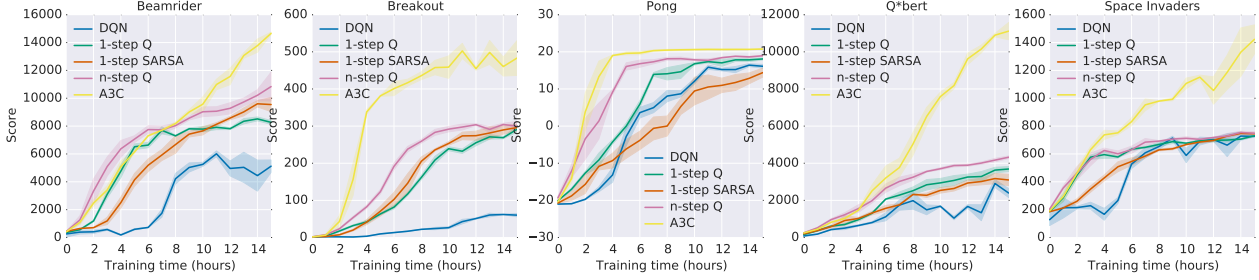


Figure 1. Learning speed comparison for DQN and the new asynchronous algorithms on five Atari 2600 games. DQN was trained on a single Nvidia K40 GPU while the asynchronous methods were trained using 16 CPU cores. The plots are averaged over 5 runs. In the case of DQN the runs were for different seeds with fixed hyperparameters. For asynchronous methods we average over the best 5 models from 50 experiments with learning rates sampled from $\text{LogUniform}(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

two additional domains to evaluate only the A3C algorithm – Mujoco and Labyrinth. MuJoCo (Todorov, 2015) is a physics simulator for evaluating agents on continuous motor control tasks with contact dynamics. Labyrinth is a new 3D environment where the agent must learn to find rewards in randomly generated mazes from a visual input. The precise details of our experimental setup can be found in Supplementary Section 8.

5.1. Atari 2600 Games

We first present results on a subset of Atari 2600 games to demonstrate the training speed of the new methods. Figure 1 compares the learning speed of the DQN algorithm trained on an Nvidia K40 GPU with the asynchronous methods trained using 16 CPU cores on five Atari 2600 games. The results show that all four asynchronous methods we presented can successfully train neural network controllers on the Atari domain. The asynchronous methods tend to learn faster than DQN, with significantly faster learning on some games, while training on only 16 CPU cores. Additionally, the results suggest that n-step methods learn faster than one-step methods on some games. Overall, the policy-based advantage actor-critic method significantly outperforms all three value-based methods.

We then evaluated asynchronous advantage actor-critic on 57 Atari games. In order to compare with the state of the art in Atari game playing, we largely followed the training and evaluation protocol of (Van Hasselt et al., 2015). Specifically, we tuned hyperparameters (learning rate and amount of gradient norm clipping) using a search on six Atari games (Beamrider, Breakout, Pong, Q*bert, Seaquest and Space Invaders) and then fixed all hyperparameters for all 57 games. We trained both a feedforward agent with the same architecture as (Mnih et al., 2015; Nair et al., 2015; Van Hasselt et al., 2015) as well as a recurrent agent with an additional 256 LSTM cells after the final hidden layer. We additionally used the final network weights for evaluation to make the results more comparable to the original results

Method	Training Time	Mean	Median
DQN	8 days on GPU	121.9%	47.5%
Gorila	4 days, 100 machines	215.2%	71.3%
D-DQN	8 days on GPU	332.9%	110.9%
Dueling D-DQN	8 days on GPU	343.8%	117.1%
Prioritized DQN	8 days on GPU	463.6%	127.6%
A3C, FF	1 day on CPU	344.1%	68.2%
A3C, FF	4 days on CPU	496.8%	116.6%
A3C, LSTM	4 days on CPU	623.0%	112.6%

Table 1. Mean and median human-normalized scores on 57 Atari games using the human starts evaluation metric. Supplementary Table S3 shows the raw scores for all games.

from (Bellemare et al., 2012). We trained our agents for four days using 16 CPU cores, while the other agents were trained for 8 to 10 days on Nvidia K40 GPUs. Table 1 shows the average and median human-normalized scores obtained by our agents trained by asynchronous advantage actor-critic (A3C) as well as the current state-of-the-art. Supplementary Table S3 shows the scores on all games. A3C significantly improves on state-of-the-art the average score over 57 games in half the training time of the other methods while using only 16 CPU cores and no GPU. Furthermore, after just one day of training, A3C matches the average human normalized score of Dueling Double DQN and almost reaches the median human normalized score of Gorila. We note that many of the improvements that are presented in Double DQN (Van Hasselt et al., 2015) and Dueling Double DQN (Wang et al., 2015) can be incorporated to 1-step Q and n-step Q methods presented in this work with similar potential improvements.

5.2. TORCS Car Racing Simulator

We also compared the four asynchronous methods on the TORCS 3D car racing game (Wymann et al., 2013). TORCS not only has more realistic graphics than Atari 2600 games, but also requires the agent to learn the dynamics of the car it is controlling. At each step, an agent received only a visual input in the form of an RGB image

of the current frame as well as a reward proportional to the agent’s velocity along the center of the track at the agent’s current position. We used the same neural network architecture as the one used in the Atari experiments specified in Supplementary Section 8. We performed experiments using four different settings – the agent controlling a slow car with and without opponent bots, and the agent controlling a fast car with and without opponent bots. Full results can be found in Supplementary Figure S6. A3C was the best performing agent, reaching between roughly 75% and 90% of the score obtained by a human tester on all four game configurations in about 12 hours of training. A video showing the learned driving behavior of the A3C agent can be found at <https://youtu.be/0xo1Ldx3L5Q>.

5.3. Continuous Action Control Using the MuJoCo Physics Simulator

We also examined a set of tasks where the action space is continuous. In particular, we looked at a set of rigid body physics domains with contact dynamics where the tasks include many examples of manipulation and locomotion. These tasks were simulated using the Mujoco physics engine. We evaluated only the asynchronous advantage actor-critic algorithm since, unlike the value-based methods, it is easily extended to continuous actions. In all problems, using either the physical state or pixels as input, Asynchronous Advantage-Critic found good solutions in less than 24 hours of training and typically in under a few hours. Some successful policies learned by our agent can be seen in the following video <https://youtu.be/Ajjc08-iPx8>. Further details about this experiment can be found in Supplementary Section 9.

5.4. Labyrinth

We performed an additional set of experiments with A3C on a new 3D environment called Labyrinth. The specific task we considered involved the agent learning to find rewards in randomly generated mazes. At the beginning of each episode the agent was placed in a new randomly generated maze consisting of rooms and corridors. Each maze contained two types of objects that the agent was rewarded for finding – apples and portals. Picking up an apple led to a reward of 1. Entering a portal led to a reward of 10 after which the agent was respawned in a new random location in the maze and all previously collected apples were regenerated. An episode terminated after 60 seconds after which a new episode would begin. The aim of the agent is to collect as many points as possible in the time limit and the optimal strategy involves first finding the portal and then repeatedly going back to it after each respawn. This task is much more challenging than the TORCS driving domain because the agent is faced with a new maze in each episode and must learn a general strategy for exploring random mazes.

Method	Number of threads				
	1	2	4	8	16
1-step Q	1.0	3.0	6.3	13.3	24.1
1-step SARSA	1.0	2.8	5.9	13.1	22.1
n-step Q	1.0	2.7	5.9	10.7	17.2
A3C	1.0	2.1	3.7	6.9	12.5

Table 2. The average training speedup for each method and number of threads averaged over seven Atari games. To compute the training speed-up on a single game we measured the time to required reach a fixed reference score using each method and number of threads. The speedup from using n threads on a game was defined as the time required to reach a fixed reference score using one thread divided the time required to reach the reference score using n threads. The table shows the speedups averaged over seven Atari games (Beamrider, Breakout, Enduro, Pong, Q*bert, Seaquest, and Space Invaders).

We trained an A3C LSTM agent on this task using only 84×84 RGB images as input. The final average score of around 50 indicates that the agent learned a reasonable strategy for exploring random 3D maxes using only a visual input. A video showing one of the agents exploring previously unseen mazes is included at <https://youtu.be/nMR5mjCFZCw>.

5.5. Scalability and Data Efficiency

We analyzed the effectiveness of our proposed framework by looking at how the training time and data efficiency changes with the number of parallel actor-learners. When using multiple workers in parallel and updating a shared model, one would expect that in an ideal case, for a given task and algorithm, the number of training steps to achieve a certain score would remain the same with varying numbers of workers. Therefore, the advantage would be solely due to the ability of the system to consume more data in the same amount of wall clock time and possibly improved exploration. Table 2 shows the training speed-up achieved by using increasing numbers of parallel actor-learners averaged over seven Atari games. These results show that all four methods achieve substantial speedups from using multiple worker threads, with 16 threads leading to at least an order of magnitude speedup. This confirms that our proposed framework scales well with the number of parallel workers, making efficient use of resources.

Somewhat surprisingly, asynchronous one-step Q-learning and Sarsa algorithms exhibit superlinear speedups that cannot be explained by purely computational gains. We observe that one-step methods (one-step Q and one-step Sarsa) often require less data to achieve a particular score when using more parallel actor-learners. We believe this is due to positive effect of multiple threads to reduce the bias in one-step methods. These effects are shown more clearly in Figure 3, which shows plots of the average score against the total number of training frames for different

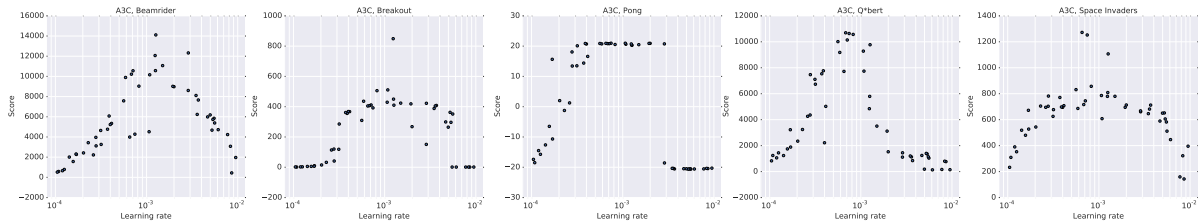


Figure 2. Scatter plots of scores obtained by asynchronous advantage actor-critic on five games (Beamrider, Breakout, Pong, Q*bert, Space Invaders) for 50 different learning rates and random initializations. On each game, there is a wide range of learning rates for which all random initializations achieve good scores. This shows that A3C is quite robust to learning rates and initial random weights.

numbers of actor-learners and training methods on five Atari games, and Figure 4, which shows plots of the average score against wall-clock time.

5.6. Robustness and Stability

Finally, we analyzed the stability and robustness of the four proposed asynchronous algorithms. For each of the four algorithms we trained models on five games (Breakout, Beamrider, Pong, Q*bert, Space Invaders) using 50 different learning rates and random initializations. Figure 2 shows scatter plots of the resulting scores for A3C, while Supplementary Figure S11 shows plots for the other three methods. There is usually a range of learning rates for each method and game combination that leads to good scores, indicating that all methods are quite robust to the choice of learning rate and random initialization. The fact that there are virtually no points with scores of 0 in regions with good learning rates indicates that the methods are stable and do not collapse or diverge once they are learning.

6. Conclusions and Discussion

We have presented asynchronous versions of four standard reinforcement learning algorithms and showed that they are able to train neural network controllers on a variety of domains in a stable manner. Our results show that in our proposed framework stable training of neural networks through reinforcement learning is possible with both value-based and policy-based methods, off-policy as well as on-policy methods, and in discrete as well as continuous domains. When trained on the Atari domain using 16 CPU cores, the proposed asynchronous algorithms train faster than DQN trained on an Nvidia K40 GPU, with A3C surpassing the current state-of-the-art in half the training time.

One of our main findings is that using parallel actor-learners to update a shared model had a stabilizing effect on the learning process of the three value-based methods we considered. While this shows that stable online Q-learning is possible without experience replay, which was used for this purpose in DQN, it does not mean that experience replay is not useful. Incorporating experience replay into the asynchronous reinforcement learning framework could

substantially improve the data efficiency of these methods by reusing old data. This could in turn lead to much faster training times in domains like TORCS where interacting with the environment is more expensive than updating the model for the architecture we used.

Combining other existing reinforcement learning methods or recent advances in deep reinforcement learning with our asynchronous framework presents many possibilities for immediate improvements to the methods we presented. While our *n*-step methods operate in the *forward view* (Sutton & Barto, 1998) by using corrected *n*-step returns directly as targets, it has been more common to use the *backward view* to implicitly combine different returns through eligibility traces (Watkins, 1989; Sutton & Barto, 1998; Peng & Williams, 1996). The asynchronous advantage actor-critic method could be potentially improved by using other ways of estimating the advantage function, such as generalized advantage estimation of (Schulman et al., 2015b). All of the value-based methods we investigated could benefit from different ways of reducing overestimation bias of Q-values (Van Hasselt et al., 2015; Belle-mare et al., 2016). Yet another, more speculative, direction is to try and combine the recent work on true online temporal difference methods (van Seijen et al., 2015) with non-linear function approximation.

In addition to these algorithmic improvements, a number of complementary improvements to the neural network architecture are possible. The dueling architecture of (Wang et al., 2015) has been shown to produce more accurate estimates of Q-values by including separate streams for the state value and advantage in the network. The spatial softmax proposed by (Levine et al., 2015) could improve both value-based and policy-based methods by making it easier for the network to represent feature coordinates.

ACKNOWLEDGMENTS

We thank Thomas Degris, Remi Munos, Marc Lanctot, Sasha Vezhnevets and Joseph Modayil for many helpful discussions, suggestions and comments on the paper. We also thank the DeepMind evaluation team for setting up the environments used to evaluate the agents in the paper.

Asynchronous Methods for Deep Reinforcement Learning

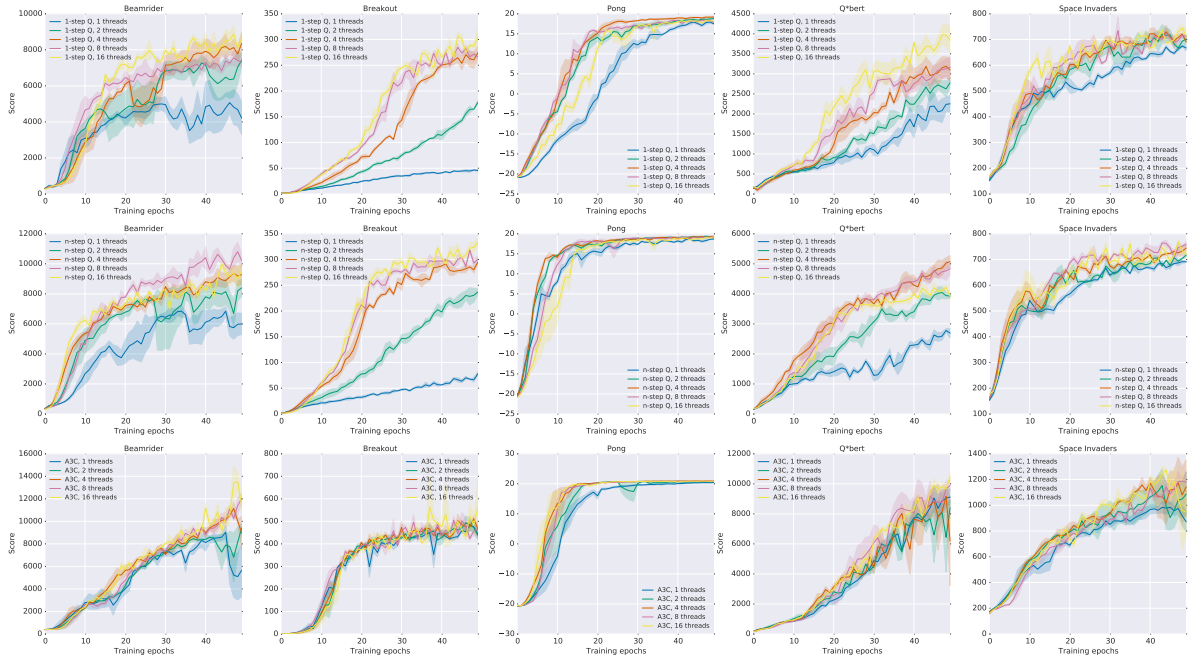


Figure 3. Data efficiency comparison of different numbers of actor-learners for three asynchronous methods on five Atari games. The x-axis shows the total number of training epochs where an epoch corresponds to four million frames (across all threads). The y-axis shows the average score. Each curve shows the average over the three best learning rates. Single step methods show increased data efficiency from more parallel workers. Results for Sarsa are shown in Supplementary Figure S9.

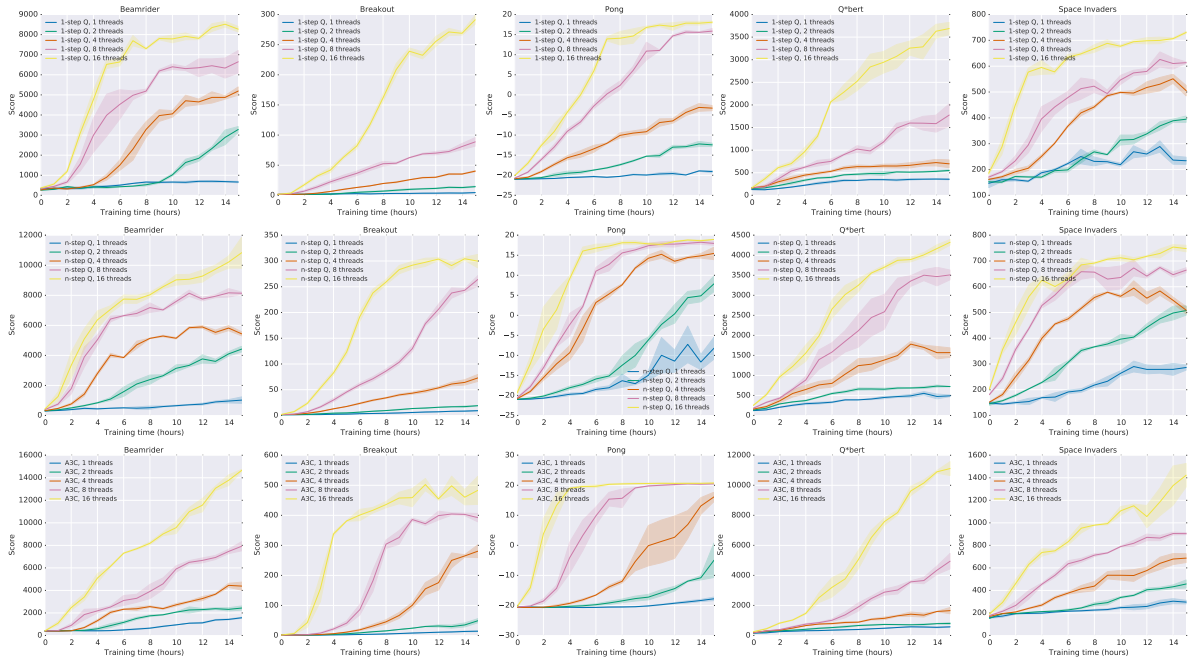


Figure 4. Training speed comparison of different numbers of actor-learners on five Atari games. The x-axis shows training time in hours while the y-axis shows the average score. Each curve shows the average over the three best learning rates. All asynchronous methods show significant speedups from using greater numbers of parallel actor-learners. Results for Sarsa are shown in Supplementary Figure S10.

Supplementary Material for "Asynchronous Methods for Deep Reinforcement Learning"

June 17, 2016

7. Optimization Details

We investigated two different optimization algorithms with our asynchronous framework – stochastic gradient descent and RMSProp. Our implementations of these algorithms do not use any locking in order to maximize throughput when using a large number of threads.

Momentum SGD: The implementation of SGD in an asynchronous setting is relatively straightforward and well studied (Recht et al., 2011). Let θ be the parameter vector that is shared across all threads and let $\Delta\theta_i$ be the accumulated gradients of the loss with respect to parameters θ computed by thread number i . Each thread i independently applies the standard momentum SGD update $m_i = \alpha m_i + (1 - \alpha)\Delta\theta_i$ followed by $\theta \leftarrow \theta - \eta m_i$ with learning rate η , momentum α and without any locks. Note that in this setting, each thread maintains its own separate gradient and momentum vector.

RMSProp: While RMSProp (Tieleman & Hinton, 2012) has been widely used in the deep learning literature, it has not been extensively studied in the asynchronous optimization setting. The standard non-centered RMSProp update is given by

$$g = \alpha g + (1 - \alpha)\Delta\theta^2 \tag{S2}$$

$$\theta \leftarrow \theta - \eta \frac{\Delta\theta}{\sqrt{g + \epsilon}}, \tag{S3}$$

where all operations are performed elementwise. In order to apply RMSProp in the asynchronous optimization setting one must decide whether the moving average of elementwise squared gradients g is shared or per-thread. We experimented with two versions of the algorithm. In one version, which we refer to as RMSProp, each thread maintains its own g shown in Equation S2. In the other version, which we call Shared RMSProp, the vector g is shared among threads and is updated asynchronously and without locking. Sharing statistics among threads also reduces memory requirements by using one fewer copy of the parameter vector per thread.

We compared these three asynchronous optimization algorithms in terms of their sensitivity to different learning rates and random network initializations. Figure S5 shows a comparison of the methods for two different reinforcement learning methods (Async n -step Q and Async Advantage Actor-Critic) on four different games (Breakout, Beamrider, Seaquest and Space Invaders). Each curve shows the scores for 50 experiments that correspond to 50 different random learning rates and initializations. The x-axis shows the rank of the model after sorting in descending order by final average score and the y-axis shows the final average score achieved by the corresponding model. In this representation, the algorithm that performs better would achieve higher maximum rewards on the y-axis and the algorithm that is most robust would have its slope closest to horizontal, thus maximizing the area under the curve. RMSProp with shared statistics tends to be more robust than RMSProp with per-thread statistics, which is in turn more robust than Momentum SGD.

8. Experimental Setup

The experiments performed on a subset of Atari games (Figures 1, 3, 4 and Table 2) as well as the TORCS experiments (Figure S6) used the following setup. Each experiment used 16 actor-learner threads running on a single machine and no GPUs. All methods performed updates after every 5 actions ($t_{max} = 5$ and $I_{Update} = 5$) and shared RMSProp was used for optimization. The three asynchronous value-based methods used a shared target network that was updated every 40000 frames. The Atari experiments used the same input preprocessing as (Mnih et al., 2015) and an action repeat of 4. The agents used the network architecture from (Mnih et al., 2013). The network used a convolutional layer with 16 filters of size 8×8 with stride 4, followed by a convolutional layer with 32 filters of size 4×4 with stride 2, followed by a fully connected layer with 256 hidden units. All three hidden layers were followed by a rectifier nonlinearity. The value-based methods had a single linear output unit for each action representing the action-value. The model used by actor-critic agents had two set of outputs – a softmax output with one entry per action representing the probability of selecting the action, and a single linear output representing the value function. All experiments used a discount of $\gamma = 0.99$ and an RMSProp decay factor of $\alpha = 0.99$.

The value based methods sampled the exploration rate ϵ from a distribution taking three values $\epsilon_1, \epsilon_2, \epsilon_3$ with probabilities 0.4, 0.3, 0.3. The values of $\epsilon_1, \epsilon_2, \epsilon_3$ were annealed from 1 to 0.1, 0.01, 0.5 respectively over the first four million frames. Advantage actor-critic used entropy regularization with a weight $\beta = 0.01$ for all Atari and TORCS experiments. We performed a set of 50 experiments for five Atari games and every TORCS level, each using a different random initialization and initial learning rate. The initial learning rate was sampled from a $LogUniform(10^{-4}, 10^{-2})$ distribution and annealed to 0 over the course of training. Note that in comparisons to prior work (Tables 1 and S3) we followed standard evaluation protocol and used fixed hyperparameters.

9. Continuous Action Control Using the MuJoCo Physics Simulator

To apply the asynchronous advantage actor-critic algorithm to the Mujoco tasks the necessary setup is nearly identical to that used in the discrete action domains, so here we enumerate only the differences required for the continuous action domains. The essential elements for many of the tasks (i.e. the physics models and task objectives) are near identical to the tasks examined in (Lillicrap et al., 2015). However, the rewards and thus performance are not comparable for most of the tasks due to changes made by the developers of Mujoco which altered the contact model.

For all the domains we attempted to learn the task using the physical state as input. The physical state consisted of the joint positions and velocities as well as the target position if the task required a target. In addition, for three of the tasks (pendulum, pointmass2D, and gripper) we also examined training directly from RGB pixel inputs. In the low dimensional physical state case, the inputs are mapped to a hidden state using one hidden layer with 200 ReLU units. In the cases where we used pixels, the input was passed through two layers of spatial convolutions without any non-linearity or pooling. In either case, the output of the encoder layers were fed to a single layer of 128 LSTM cells. The most important difference in the architecture is in the the output layer of the policy network. Unlike the discrete action domain where the action output is a Softmax, here the two outputs of the policy network are two real number vectors which we treat as the mean vector μ and scalar variance σ^2 of a multidimensional normal distribution with a spherical covariance. To act, the input is passed through the model to the output layer where we sample from the normal distribution determined by μ and σ^2 . In practice, μ is modeled by a linear layer and σ^2 by a SoftPlus operation, $\log(1 + \exp(x))$, as the activation computed as a function of the output of a linear layer. In our experiments with continuous control problems the networks for policy network and value network do not share any parameters, though this detail is unlikely to be crucial. Finally, since the episodes were typically at most several hundred time steps long, we did not use any bootstrapping in the policy or value function updates and batched each episode into a single update.

As in the discrete action case, we included an entropy cost which encouraged exploration. In the continuous

case the we used a cost on the differential entropy of the normal distribution defined by the output of the actor network, $-\frac{1}{2}(\log(2\pi\sigma^2) + 1)$, we used a constant multiplier of 10^{-4} for this cost across all of the tasks examined. The asynchronous advantage actor-critic algorithm finds solutions for all the domains. Figure S8 shows learning curves against wall-clock time, and demonstrates that most of the domains from states can be solved within a few hours. All of the experiments, including those done from pixel based observations, were run on CPU. Even in the case of solving the domains directly from pixel inputs we found that it was possible to reliably discover solutions within 24 hours. Figure S7 shows scatter plots of the top scores against the sampled learning rates. In most of the domains there is large range of learning rates that consistently achieve good performance on the task.

Algorithm S2 Asynchronous n-step Q-learning - pseudocode for each actor-learner thread.

```

// Assume global shared parameter vector  $\theta$ .
// Assume global shared target parameter vector  $\theta^-$ .
// Assume global shared counter  $T = 0$ .
Initialize thread step counter  $t \leftarrow 1$ 
Initialize target network parameters  $\theta^- \leftarrow \theta$ 
Initialize thread-specific parameters  $\theta' = \theta$ 
Initialize network gradients  $d\theta \leftarrow 0$ 
repeat
    Clear gradients  $d\theta \leftarrow 0$ 
    Synchronize thread-specific parameters  $\theta' = \theta$ 
     $t_{start} = t$ 
    Get state  $s_t$ 
    repeat
        Take action  $a_t$  according to the  $\epsilon$ -greedy policy based on  $Q(s_t, a; \theta')$ 
        Receive reward  $r_t$  and new state  $s_{t+1}$ 
         $t \leftarrow t + 1$ 
         $T \leftarrow T + 1$ 
    until terminal  $s_t$  or  $t - t_{start} == t_{max}$ 
     $R = \begin{cases} 0 & \text{for terminal } s_t \\ \max_a Q(s_t, a; \theta^-) & \text{for non-terminal } s_t \end{cases}$ 
    for  $i \in \{t - 1, \dots, t_{start}\}$  do
         $R \leftarrow r_i + \gamma R$ 
        Accumulate gradients wrt  $\theta'$ :  $d\theta \leftarrow d\theta + \frac{\partial(R - Q(s_i, a_i; \theta'))^2}{\partial \theta'}$ 
    end for
    Perform asynchronous update of  $\theta$  using  $d\theta$ .
    if  $T \bmod I_{target} == 0$  then
         $\theta^- \leftarrow \theta$ 
    end if
until  $T > T_{max}$ 
    
```

Algorithm S3 Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

// Assume global shared parameter vectors θ and θ_v and global shared counter $T = 0$

// Assume thread-specific parameter vectors θ' and θ'_v

Initialize thread step counter $t \leftarrow 1$

repeat

Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.

Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$

$t_{start} = t$

Get state s_t

repeat

Perform a_t according to policy $\pi(a_t|s_t; \theta')$

Receive reward r_t and new state s_{t+1}

$t \leftarrow t + 1$

$T \leftarrow T + 1$

until terminal s_t **or** $t - t_{start} == t_{max}$

$R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t // \text{ Bootstrap from last state} \end{cases}$

for $i \in \{t - 1, \dots, t_{start}\}$ **do**

$R \leftarrow r_i + \gamma R$

Accumulate gradients wrt θ' : $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i; \theta')(R - V(s_i; \theta'_v))$

Accumulate gradients wrt θ'_v : $d\theta_v \leftarrow d\theta_v + \partial (R - V(s_i; \theta'_v))^2 / \partial \theta'_v$

end for

Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.

until $T > T_{max}$

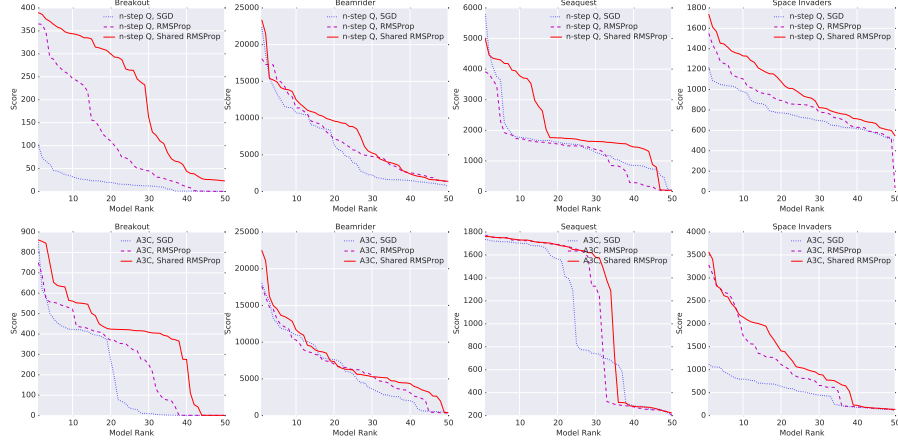


Figure S5. Comparison of three different optimization methods (Momentum SGD, RMSProp, Shared RMSProp) tested using two different algorithms (Async n -step Q and Async Advantage Actor-Critic) on four different Atari games (Breakout, Beamrider, Seaquest and Space Invaders). Each curve shows the final scores for 50 experiments sorted in descending order that covers a search over 50 random initializations and learning rates. The top row shows results using Async n -step Q algorithm and bottom row shows results with Async Advantage Actor-Critic. Each individual graph shows results for one of the four games and three different optimization methods. Shared RMSProp tends to be more robust to different learning rates and random initializations than Momentum SGD and RMSProp without sharing.

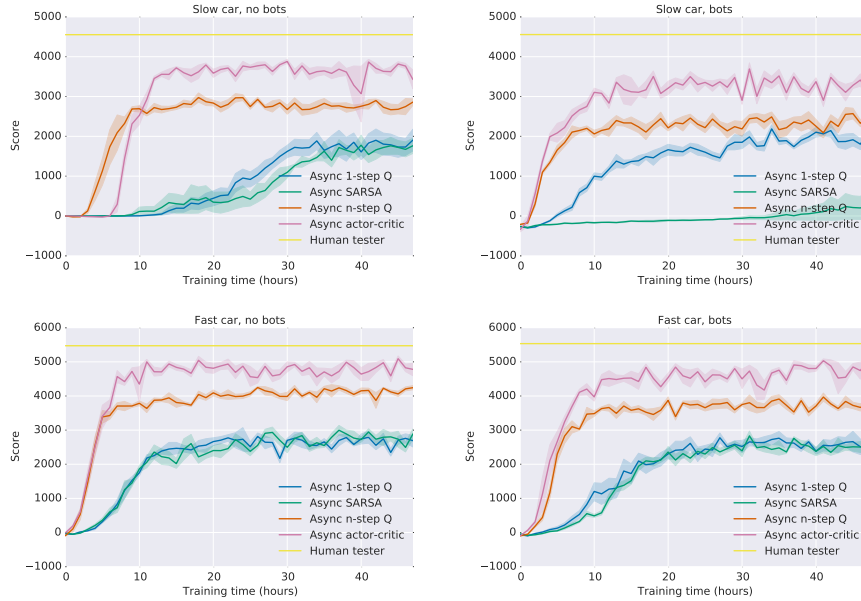


Figure S6. Comparison of algorithms on the TORCS car racing simulator. Four different configurations of car speed and opponent presence or absence are shown. In each plot, all four algorithms (one-step Q, one-step Sarsa, n -step Q and Advantage Actor-Critic) are compared on score vs training time in wall clock hours. Multi-step algorithms achieve better policies much faster than one-step algorithms on all four levels. The curves show averages over the 5 best runs from 50 experiments with learning rates sampled from $\text{LogUniform}(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

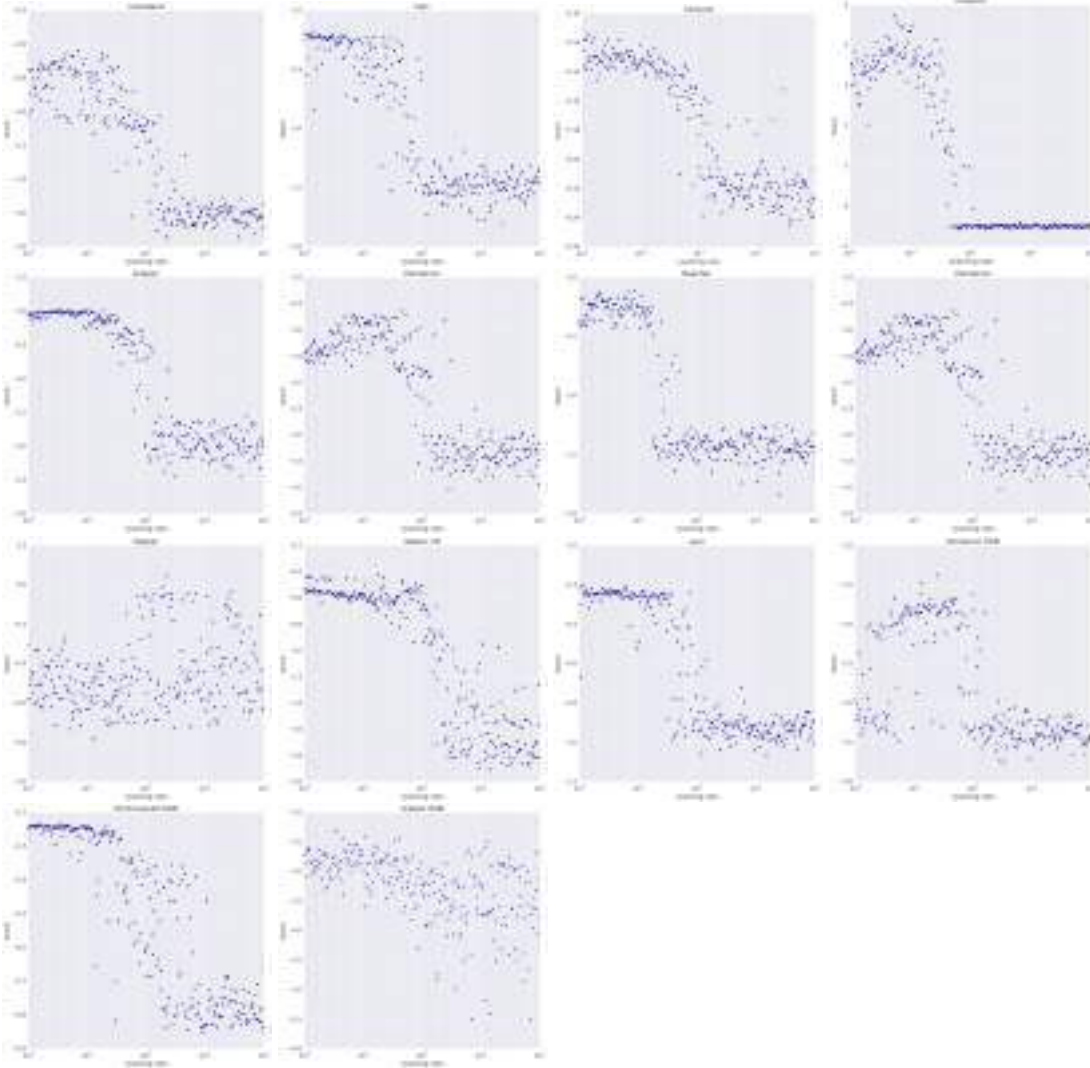


Figure S7. Performance for the Mujoco continuous action domains. Scatter plot of the best score obtained against learning rates sampled from $\text{LogUniform}(10^{-5}, 10^{-1})$. For nearly all of the tasks there is a wide range of learning rates that lead to good performance on the task.

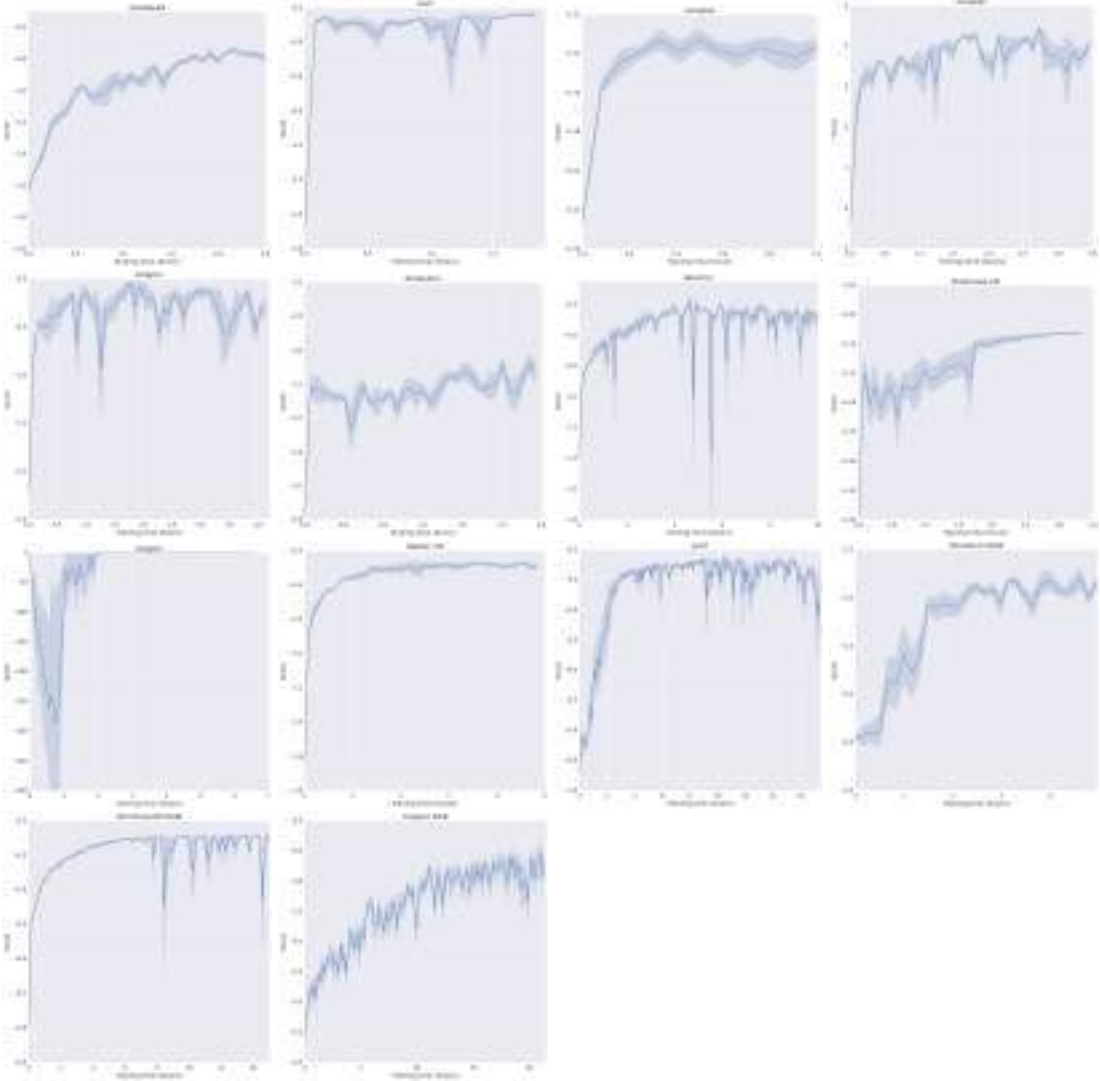


Figure S8. Score per episode vs wall-clock time plots for the Mujoco domains. Each plot shows error bars for the top 5 experiments.

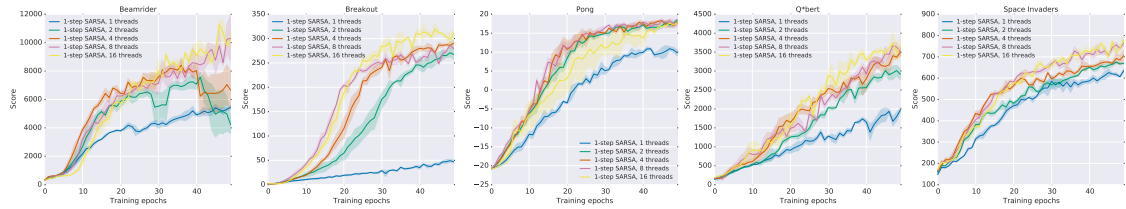


Figure S9. Data efficiency comparison of different numbers of actor-learners one-step Sarsa on five Atari games. The x-axis shows the total number of training epochs where an epoch corresponds to four million frames (across all threads). The y-axis shows the average score. Each curve shows the average of the three best performing agents from a search over 50 random learning rates. Sarsa shows increased data efficiency with increased numbers of parallel workers.

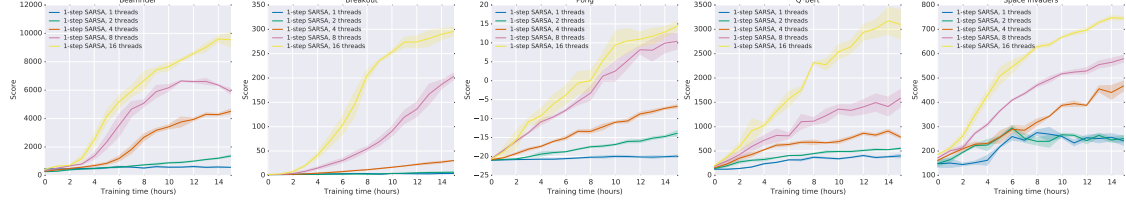


Figure S10. Training speed comparison of different numbers of actor-learners for all one-step Sarsa on five Atari games. The x-axis shows training time in hours while the y-axis shows the average score. Each curve shows the average of the three best performing agents from a search over 50 random learning rates. Sarsa shows significant speedups from using greater numbers of parallel actor-learners.

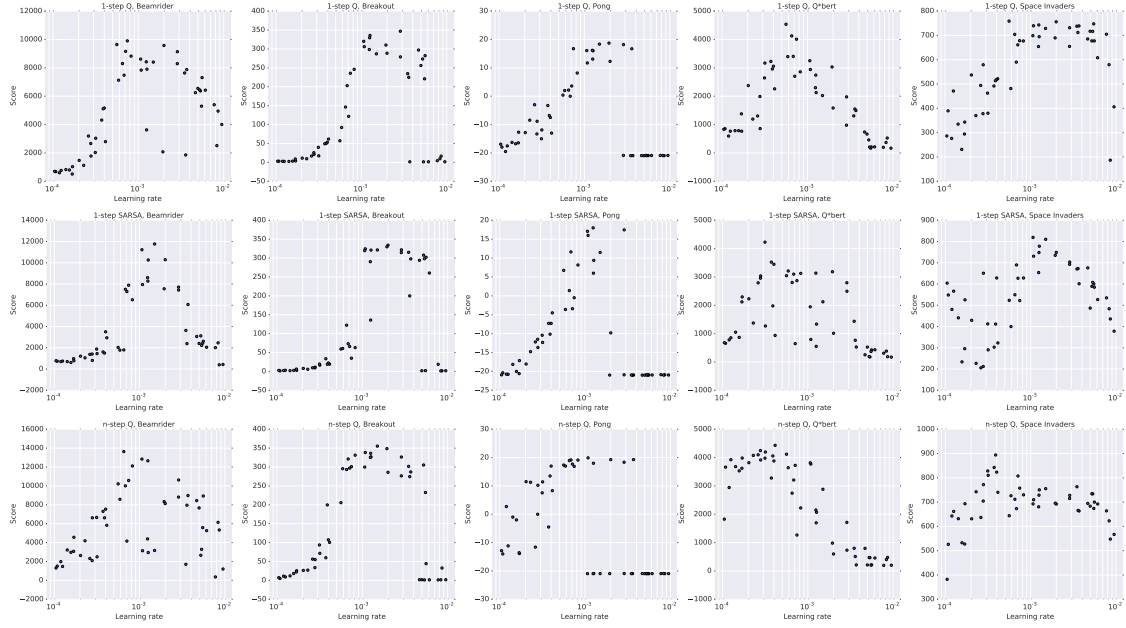


Figure S11. Scatter plots of scores obtained by one-step Q, one-step Sarsa, and n -step Q on five games (Beamrider, Breakout, Pong, Q*bert, Space Invaders) for 50 different learning rates and random initializations. All algorithms exhibit some level of robustness to the choice of learning rate.

Asynchronous Methods for Deep Reinforcement Learning

Game	DQN	Gorila	Double	Dueling	Prioritized	A3C FF, 1 day	A3C FF	A3C LSTM
Alien	570.2	813.5	1033.4	1486.5	900.5	182.1	518.4	945.3
Amidar	133.4	189.2	169.1	172.7	218.4	283.9	263.9	173.0
Assault	3332.3	1195.8	6060.8	3994.8	7748.5	3746.1	5474.9	14497.9
Asterix	124.5	3324.7	16837.0	15840.0	31907.5	6723.0	22140.5	17244.5
Asteroids	697.1	933.6	1193.2	2035.4	1654.0	3009.4	4474.5	5093.1
Atlantis	76108.0	629166.5	319688.0	445360.0	593642.0	772392.0	911091.0	875822.0
Bank Heist	176.3	399.4	886.0	1129.3	816.8	946.0	970.1	932.8
Battle Zone	17560.0	19938.0	24740.0	31320.0	29100.0	11340.0	12950.0	20760.0
Beam Rider	8672.4	3822.1	17417.2	14591.3	26172.7	13235.9	22707.9	24622.2
Berzerk			1011.1	910.6	1165.6	1433.4	817.9	862.2
Bowling	41.2	54.0	69.6	65.7	65.8	36.2	35.1	41.8
Boxing	25.8	74.2	73.5	77.3	68.6	33.7	59.8	37.3
Breakout	303.9	313.0	368.9	411.6	371.6	551.6	681.9	766.8
Centipede	3773.1	6296.9	3853.5	4881.0	3421.9	3306.5	3755.8	1997.0
Chopper Comman	3046.0	3191.8	3495.0	3784.0	6604.0	4669.0	7021.0	10150.0
Crazy Climber	50992.0	65451.0	113782.0	124566.0	131086.0	101624.0	112646.0	138518.0
Defender			27510.0	33996.0	21093.5	36242.5	56533.0	233021.5
Demon Attack	12835.2	14880.1	69803.4	56322.8	73185.8	84997.5	113308.4	115201.9
Double Dunk	-21.6	-11.3	-0.3	-0.8	2.7	0.1	-0.1	0.1
Enduro	475.6	71.0	1216.6	2077.4	1884.4	-82.2	-82.5	-82.5
Fishing Derby	-2.3	4.6	3.2	-4.1	9.2	13.6	18.8	22.6
Freeway	25.8	10.2	28.8	0.2	27.9	0.1	0.1	0.1
Frostbite	157.4	426.6	1448.1	2332.4	2930.2	180.1	190.5	197.6
Gopher	2731.8	4373.0	15253.0	20051.4	57783.8	8442.8	10022.8	17106.8
Gravitar	216.5	538.4	200.5	297.0	218.0	269.5	303.5	320.0
H.E.R.O.	12952.5	8963.4	14892.5	15207.9	20506.4	28765.8	32464.1	28889.5
Ice Hockey	-3.8	-1.7	-2.5	-1.3	-1.0	-4.7	-2.8	-1.7
James Bond	348.5	444.0	573.0	835.5	3511.5	351.5	541.0	613.0
Kangaroo	2696.0	1431.0	11204.0	10334.0	10241.0	106.0	94.0	125.0
Krull	3864.0	6363.1	6796.1	8051.6	7406.5	8066.6	5560.0	5911.4
Kung-Fu Master	11875.0	20620.0	30207.0	24288.0	31244.0	3046.0	28819.0	40835.0
Montezuma's Revenge	50.0	84.0	42.0	22.0	13.0	53.0	67.0	41.0
Ms. Pacman	763.5	1263.0	1241.3	2250.6	1824.6	594.4	653.7	850.7
Name This Game	5439.9	9238.5	8960.3	11185.1	11836.1	5614.0	10476.1	12093.7
Phoenix			12366.5	20410.5	27430.1	28181.8	52894.1	74786.7
Pit Fall			-186.7	-46.9	-14.8	-123.0	-78.5	-135.7
Pong	16.2	16.7	19.1	18.8	18.9	11.4	5.6	10.7
Private Eye	298.2	2598.6	-575.5	292.6	179.0	194.4	206.9	421.1
Q*Bert	4589.8	7089.8	11020.8	14175.8	11277.0	13752.3	15148.8	21307.5
River Raid	4065.3	5310.3	10838.4	16569.4	18184.4	10001.2	12201.8	6591.9
Road Runner	9264.0	43079.8	43156.0	58549.0	56990.0	31769.0	34216.0	73949.0
Robotank	58.5	61.8	59.1	62.0	55.4	2.3	32.8	2.6
Seaquest	2793.9	10145.9	14498.0	37361.6	39096.7	2300.2	2355.4	1326.1
Skiing			-11490.4	-11928.0	-10852.8	-13700.0	-10911.1	-14863.8
Solaris			810.0	1768.4	2238.2	1884.8	1956.0	1936.4
Space Invaders	1449.7	1183.3	2628.7	5993.1	9063.0	2214.7	15730.5	23846.0
Star Gunner	34081.0	14919.2	58365.0	90804.0	51959.0	64393.0	138218.0	164766.0
Surround			1.9	4.0	-0.9	-9.6	-9.7	-8.3
Tennis	-2.3	-0.7	-7.8	4.4	-2.0	-10.2	-6.3	-6.4
Time Pilot	5640.0	8267.8	6608.0	6601.0	7448.0	5825.0	12679.0	27202.0
Tutankham	32.4	118.5	92.2	48.0	33.6	26.1	156.3	144.2
Up and Down	3311.3	8747.7	19086.9	24759.2	29443.7	54525.4	74705.7	105728.7
Venture	54.0	523.4	21.0	200.0	244.0	19.0	23.0	25.0
Video Pinball	20228.1	112093.4	367823.7	110976.2	374886.9	185852.6	331628.1	470310.5
Wizard of Wor	246.0	10431.0	6201.0	7054.0	7451.0	5278.0	17244.0	18082.0
Yars Revenge			6270.6	25976.5	5965.1	7270.8	7157.5	5615.5
Zaxxon	831.0	6159.4	8593.0	10164.0	9501.0	2659.0	24622.0	23519.0

Table S3. Raw scores for the human start condition (30 minutes emulator time). DQN scores taken from (Nair et al., 2015). Double DQN scores taken from (Van Hasselt et al., 2015), Dueling scores from (Wang et al., 2015) and Prioritized scores taken from (Schaul et al., 2015)

SAMPLE EFFICIENT ACTOR-CRITIC WITH EXPERIENCE REPLAY

Ziyu Wang

DeepMind

ziyu@google.com

Victor Bapst

DeepMind

vbapst@google.com

Nicolas Heess

DeepMind

heess@google.com

Volodymyr Mnih

DeepMind

vmnih@google.com

Remi Munos

DeepMind

Munos@google.com

Koray Kavukcuoglu

DeepMind

korayk@google.com

Nando de Freitas

DeepMind, CIFAR, Oxford University

nandodefrees@google.com

ABSTRACT

This paper presents an actor-critic deep reinforcement learning agent with experience replay that is stable, sample efficient, and performs remarkably well on challenging environments, including the discrete 57-game Atari domain and several continuous control problems. To achieve this, the paper introduces several innovations, including truncated importance sampling with bias correction, stochastic dueling network architectures, and a new trust region policy optimization method.

1 INTRODUCTION

Realistic simulated environments, where agents can be trained to learn a large repertoire of cognitive skills, are at the core of recent breakthroughs in AI (Bellemare et al., 2013; Mnih et al., 2015; Schulman et al., 2015a; Narasimhan et al., 2015; Mnih et al., 2016; Brockman et al., 2016; Oh et al., 2016). With richer realistic environments, the capabilities of our agents have increased and improved. Unfortunately, these advances have been accompanied by a substantial increase in the cost of simulation. In particular, every time an agent acts upon the environment, an expensive simulation step is conducted. Thus to reduce the cost of simulation, we need to reduce the number of simulation steps (i.e. samples of the environment). This need for sample efficiency is even more compelling when agents are deployed in the real world.

Experience replay (Lin, 1992) has gained popularity in deep Q -learning (Mnih et al., 2015; Schaul et al., 2016; Wang et al., 2016; Narasimhan et al., 2015), where it is often motivated as a technique for reducing sample correlation. Replay is actually a valuable tool for improving sample efficiency and, as we will see in our experiments, state-of-the-art deep Q -learning methods (Schaul et al., 2016; Wang et al., 2016) have been up to this point the most sample efficient techniques on Atari by a significant margin. However, we need to do better than deep Q -learning, because it has two important limitations. First, the deterministic nature of the optimal policy limits its use in adversarial domains. Second, finding the greedy action with respect to the Q function is costly for large action spaces.

Policy gradient methods have been at the heart of significant advances in AI and robotics (Silver et al., 2014; Lillicrap et al., 2015; Silver et al., 2016; Levine et al., 2015; Mnih et al., 2016; Schulman et al., 2015a; Heess et al., 2015). Many of these methods are restricted to continuous domains or to very specific tasks such as playing Go. The existing variants applicable to both continuous and discrete domains, such as the on-policy asynchronous advantage actor critic (A3C) of (Mnih et al., 2016), are sample inefficient.

The design of stable, sample efficient actor critic methods that apply to both continuous and discrete action spaces has been a long-standing hurdle of reinforcement learning (RL). We believe this paper

is the first to address this challenge successfully at scale. More specifically, we introduce an actor critic with experience replay (ACER) that nearly matches the state-of-the-art performance of deep Q -networks with prioritized replay on Atari, and substantially outperforms A3C in terms of sample efficiency on both Atari and continuous control domains.

ACER capitalizes on recent advances in deep neural networks, variance reduction techniques, the off-policy Retrace algorithm (Munos et al., 2016) and parallel training of RL agents (Mnih et al., 2016). Yet, crucially, its success hinges on innovations advanced in this paper: truncated importance sampling with bias correction, stochastic dueling network architectures, and efficient trust region policy optimization.

On the theoretical front, the paper proves that the Retrace operator can be rewritten from our proposed truncated importance sampling with bias correction technique.

2 BACKGROUND AND PROBLEM SETUP

Consider an agent interacting with its environment over discrete time steps. At time step t , the agent observes the n_x -dimensional state vector $x_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$, chooses an action a_t according to a policy $\pi(a|x_t)$ and observes a reward signal $r_t \in \mathbb{R}$ produced by the environment. We will consider discrete actions $a_t \in \{1, 2, \dots, N_a\}$ in Sections 3 and 4, and continuous actions $a_t \in \mathcal{A} \subseteq \mathbb{R}^{n_a}$ in Section 5.

The goal of the agent is to maximize the discounted return $R_t = \sum_{i \geq 0} \gamma^i r_{t+i}$ in expectation. The discount factor $\gamma \in [0, 1)$ trades-off the importance of immediate and future rewards. For an agent following policy π , we use the standard definitions of the state-action and state only value functions:

$$Q^\pi(x_t, a_t) = \mathbb{E}_{x_{t+1:\infty}, a_{t+1:\infty}} [R_t | x_t, a_t] \quad \text{and} \quad V^\pi(x_t) = \mathbb{E}_{a_t} [Q^\pi(x_t, a_t) | x_t]$$

Here, the expectations are with respect to the observed environment states x_t and the actions generated by the policy π , where $x_{t+1:\infty}$ denotes a state trajectory starting at time $t+1$.

We also need to define the advantage function $A^\pi(x_t, a_t) = Q^\pi(x_t, a_t) - V^\pi(x_t)$, which provides a relative measure of value of each action since $\mathbb{E}_{a_t} [A^\pi(x_t, a_t)] = 0$.

The parameters θ of the differentiable policy $\pi_\theta(a_t|x_t)$ can be updated using the discounted approximation to the policy gradient (Sutton et al., 2000), which borrowing notation from Schulman et al. (2015b), is defined as:

$$g = \mathbb{E}_{x_{0:\infty}, a_{0:\infty}} \left[\sum_{t \geq 0} A^\pi(x_t, a_t) \nabla_\theta \log \pi_\theta(a_t | x_t) \right] \quad (1)$$

Following Proposition 1 of Schulman et al. (2015b), we can replace $A^\pi(x_t, a_t)$ in the above expression with the state-action value $Q^\pi(x_t, a_t)$, the discounted return R_t , or the temporal difference residual $r_t + \gamma V^\pi(x_{t+1}) - V^\pi(x_t)$, without introducing bias. These choices will however have different variance. Moreover, in practice we will approximate these quantities with neural networks thus introducing additional approximation errors and biases. Typically, the policy gradient estimator using R_t will have higher variance and lower bias whereas the estimators using function approximation will have higher bias and lower variance. Combining R_t with the current value function approximation to minimize bias while maintaining bounded variance is one of the central design principles behind ACER.

To trade-off bias and variance, the asynchronous advantage actor critic (A3C) of Mnih et al. (2016) uses a single trajectory sample to obtain the following gradient approximation:

$$\hat{g}^{\text{a3c}} = \sum_{t \geq 0} \left(\left(\sum_{i=0}^{k-1} \gamma^i r_{t+i} \right) + \gamma^k V_{\theta_v}^\pi(x_{t+k}) - V_{\theta_v}^\pi(x_t) \right) \nabla_\theta \log \pi_\theta(a_t | x_t) \quad (2)$$

A3C combines both k -step returns and function approximation to trade-off variance and bias. We may think of $V_{\theta_v}^\pi(x_t)$ as a policy gradient baseline used to reduce variance.

In the following section, we will introduce the discrete-action version of ACER. ACER may be understood as the off-policy counterpart of the A3C method of Mnih et al. (2016). As such, ACER builds on all the engineering innovations of A3C, including efficient parallel CPU computation.

ACER uses a single deep neural network to estimate the policy $\pi_\theta(a_t|x_t)$ and the value function $V_{\theta_v}(x_t)$. (For clarity and generality, we are using two different symbols to denote the parameters of the policy and value function, θ and θ_v , but most of these parameters are shared in the single neural network.) Our neural networks, though building on the networks used in A3C, will introduce several modifications and new modules.

3 DISCRETE ACTOR CRITIC WITH EXPERIENCE REPLAY

Off-policy learning with experience replay may appear to be an obvious strategy for improving the sample efficiency of actor-critics. However, controlling the variance and stability of off-policy estimators is notoriously hard. Importance sampling is one of the most popular approaches for off-policy learning (Meuleau et al., 2000; Jie & Abbeel, 2010; Levine & Koltun, 2013). In our context, it proceeds as follows. Suppose we retrieve a trajectory $\{x_0 \ a_0 \ r_0 \ \mu(\cdot|x_0) \ \cdots \ x_k \ a_k \ r_k \ \mu(\cdot|x_k)\}$, where the actions have been sampled according to the behavior policy μ , from our memory of experiences. Then, the importance weighted policy gradient is given by:

$$\hat{g}^{\text{imp}} = \left(\prod_{t=0}^k \rho_t \right) \sum_{t=0}^k \left(\sum_{i=0}^k \gamma^i r_{t+i} \right) \nabla_\theta \log \pi_\theta(a_t|x_t) \quad (3)$$

where $\rho_t = \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)}$ denotes the importance weight. This estimator is unbiased, but it suffers from very high variance as it involves a product of many potentially unbounded importance weights. To prevent the product of importance weights from exploding, Wawrzyński (2009) truncates this product. Truncated importance sampling over entire trajectories, although bounded in variance, could suffer from significant bias.

Recently, Degris et al. (2012) attacked this problem by using marginal value functions over the limiting distribution of the process to yield the following approximation of the gradient:

$$g^{\text{marg}} = \mathbb{E}_{x_t \sim \beta \ a_t \sim \mu} [\rho_t \nabla_\theta \log \pi_\theta(a_t|x_t) Q^\pi(x_t \ a_t)] \quad (4)$$

where $\mathbb{E}_{x_t \sim \beta \ a_t \sim \mu}[\cdot]$ is the expectation with respect to the limiting distribution $\beta(x) = \lim_{t \rightarrow \infty} P(x_t = x | x_0 \ \mu)$ with behavior policy μ . To keep the notation succinct, we will replace $\mathbb{E}_{x_t \sim \beta \ a_t \sim \mu}[\cdot]$ with $\mathbb{E}_{x_t a_t}[\cdot]$ and ensure we remind readers of this when necessary.

Two important facts about equation (4) must be highlighted. First, note that it depends on Q^π and not on Q^μ , consequently we must be able to estimate Q^π . Second, we no longer have a product of importance weights, but instead only need to estimate the marginal importance weight ρ_t . Importance sampling in this lower dimensional space (over marginals as opposed to trajectories) is expected to exhibit lower variance.

Degris et al. (2012) estimate Q^π in equation (4) using *lambda returns*: $R_t^\lambda = r_t + (1 - \lambda)\gamma V(x_{t+1}) + \lambda\gamma\rho_{t+1}R_{t+1}^\lambda$. This estimator requires that we know how to choose λ ahead of time to trade off bias and variance. Moreover, when using small values of λ to reduce variance, occasional large importance weights can still cause instability.

In the following subsection, we adopt the Retrace algorithm of Munos et al. (2016) to estimate Q^π . Subsequently, we propose an importance weight truncation technique to improve the stability of the off-policy actor critic of Degris et al. (2012), and introduce a computationally efficient trust region scheme for policy optimization. The formulation of ACER for continuous action spaces will require further innovations that are advanced in Section 5.

3.1 MULTI-STEP ESTIMATION OF THE STATE-ACTION VALUE FUNCTION

In this paper, we estimate $Q^\pi(x_t \ a_t)$ using Retrace (Munos et al., 2016). (We also experimented with the related tree backup method of Precup et al. (2000) but found Retrace to perform better in practice.) Given a trajectory generated under the behavior policy μ , the Retrace estimator can be expressed recursively as follows¹:

$$Q^{\text{ret}}(x_t \ a_t) = r_t + \gamma \bar{\rho}_{t+1} [Q^{\text{ret}}(x_{t+1} \ a_{t+1}) - Q(x_{t+1} \ a_{t+1})] + \gamma V(x_{t+1}) \quad (5)$$

¹For ease of presentation, we consider only $\lambda = 1$ for Retrace.

where $\bar{\rho}_t$ is the truncated importance weight, $\bar{\rho}_t = \min \{c, \rho_t\}$ with $\rho_t = \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)}$, Q is the current value estimate of Q^π , and $V(x) = \mathbb{E}_{a \sim \pi} Q(x, a)$. Retrace is an off-policy, return-based algorithm which has low variance and is proven to converge (in the tabular case) to the value function of the target policy for any behavior policy, see [Munos et al. \(2016\)](#).

The recursive Retrace equation depends on the estimate Q . To compute it, in discrete action spaces, we adopt a convolutional neural network with “two heads” that outputs the estimate $Q_{\theta_v}(x_t, a_t)$, as well as the policy $\pi_\theta(a_t|x_t)$. This neural representation is the same as in [\(Mnih et al., 2016\)](#), with the exception that we output the vector $Q_{\theta_v}(x_t, a_t)$ instead of the scalar $V_{\theta_v}(x_t)$. The estimate $V_{\theta_v}(x_t)$ can be easily derived by taking the expectation of Q_{θ_v} under π_θ .

To approximate the policy gradient g^{marg} , ACER uses Q^{ret} to estimate Q^π . As Retrace uses multi-step returns, it can significantly reduce bias in the estimation of the policy gradient².

To learn the critic $Q_{\theta_v}(x_t, a_t)$, we again use $Q^{\text{ret}}(x_t, a_t)$ as a target in a mean squared error loss and update its parameters θ_v with the following standard gradient:

$$(Q^{\text{ret}}(x_t, a_t) - Q_{\theta_v}(x_t, a_t)) \nabla_{\theta_v} Q_{\theta_v}(x_t, a_t) \quad (6)$$

Because Retrace is return-based, it also enables faster learning of the critic. Thus the purpose of the multi-step estimator Q^{ret} in our setting is twofold: to reduce bias in the policy gradient, and to enable faster learning of the critic, hence further reducing bias.

3.2 IMPORTANCE WEIGHT TRUNCATION WITH BIAS CORRECTION

The marginal importance weights in Equation (4) can become large, thus causing instability. To safe-guard against high variance, we propose to truncate the importance weights and introduce a correction term via the following decomposition of g^{marg} :

$$\begin{aligned} g^{\text{marg}} &= \mathbb{E}_{x_t, a_t} [\rho_t \nabla_{\theta} \log \pi_\theta(a_t|x_t) Q^\pi(x_t, a_t)] \\ &= \mathbb{E}_{x_t} \left[\mathbb{E}_{a_t} [\bar{\rho}_t \nabla_{\theta} \log \pi_\theta(a_t|x_t) Q^\pi(x_t, a_t)] + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+ \nabla_{\theta} \log \pi_\theta(a|x_t) Q^\pi(x_t, a) \right) \right] \end{aligned} \quad (7)$$

where $\bar{\rho}_t = \min \{c, \rho_t\}$ with $\rho_t = \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)}$ as before. We have also introduced the notation $\rho_t(a) = \frac{\pi(a|x_t)}{\mu(a|x_t)}$, and $[x]_+ = x$ if $x > 0$ and it is zero otherwise. We remind readers that the above expectations are with respect to the limiting state distribution under the behavior policy: $x_t \sim \beta$ and $a_t \sim \mu$.

The clipping of the importance weight in the first term of equation (7) ensures that the variance of the gradient estimate is bounded. The correction term (second term in equation (7)) ensures that our estimate is *unbiased*. Note that the correction term is only active for actions such that $\rho_t(a) > c$. In particular, if we choose a large value for c , the correction term only comes into effect when the variance of the original off-policy estimator of equation (4) is very high. When this happens, our decomposition has the nice property that the truncated weight in the first term is at most c while the correction weight $\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+$ in the second term is at most 1.

We model $Q^\pi(x_t, a)$ in the correction term with our neural network approximation $Q_{\theta_v}(x_t, a_t)$. This modification results in what we call the *truncation with bias correction trick*, in this case applied to the function $\nabla_{\theta} \log \pi_\theta(a_t|x_t) Q^\pi(x_t, a_t)$:

$$\hat{g}^{\text{marg}} = \mathbb{E}_{x_t} \left[\mathbb{E}_{a_t} [\bar{\rho}_t \nabla_{\theta} \log \pi_\theta(a_t|x_t) Q^{\text{ret}}(x_t, a_t)] + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+ \nabla_{\theta} \log \pi_\theta(a|x_t) Q_{\theta_v}(x_t, a) \right) \right] \quad (8)$$

Equation (8) involves an expectation over the stationary distribution of the Markov process. We can however approximate it by sampling trajectories $\{x_0, a_0, r_0, \mu(\cdot|x_0), \dots, x_k, a_k, r_k, \mu(\cdot|x_k)\}$

²An alternative to Retrace here is $Q(\lambda)$ with off-policy corrections [\(Harutyunyan et al., 2016\)](#) which we discuss in more detail in Appendix B.

generated from the behavior policy μ . Here the terms $\mu(\cdot|x_t)$ are the policy vectors. Given these trajectories, we can compute the off-policy ACER gradient:

$$\begin{aligned}\hat{g}_t^{\text{acer}} &= \bar{\rho}_t \nabla_{\theta} \log \pi_{\theta}(a_t|x_t) [Q^{\text{ret}}(x_t, a_t) - V_{\theta_v}(x_t)] \\ &\quad + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+ \nabla_{\theta} \log \pi_{\theta}(a|x_t) [Q_{\theta_v}(x_t, a) - V_{\theta_v}(x_t)] \right)\end{aligned}\quad (9)$$

In the above expression, we have subtracted the classical baseline $V_{\theta_v}(x_t)$ to reduce variance.

It is interesting to note that, when $c = \infty$, (9) recovers (off-policy) policy gradient up to the use of Retrace. When $c = 0$, (9) recovers an actor critic update that depends entirely on Q estimates. In the continuous control domain, (9) also generalizes Stochastic Value Gradients if $c = 0$ and the reparametrization trick is used to estimate its second term (Heess et al., 2015).

3.3 EFFICIENT TRUST REGION POLICY OPTIMIZATION

The policy updates of actor-critic methods do often exhibit high variance. Hence, to ensure stability, we must limit the per-step changes to the policy. Simply using smaller learning rates is insufficient as they cannot guard against the occasional large updates while maintaining a desired learning speed. Trust Region Policy Optimization (TRPO) (Schulman et al., 2015a) provides a more adequate solution.

Schulman et al. (2015a) approximately limit the difference between the updated policy and the current policy to ensure safety. Despite the effectiveness of their TRPO method, it requires repeated computation of Fisher-vector products for each update. This can prove to be prohibitively expensive in large domains.

In this section we introduce a new trust region policy optimization method that scales well to large problems. Instead of constraining the updated policy to be close to the current policy (as in TRPO), we propose to maintain an *average policy network* that represents a running average of past policies and forces the updated policy to not deviate far from this average.

We decompose our policy network in two parts: a distribution f , and a deep neural network that generates the statistics $\phi_{\theta}(x)$ of this distribution. That is, given f , the policy is completely characterized by the network ϕ_{θ} : $\pi(\cdot|x) = f(\cdot|\phi_{\theta}(x))$. For example, in the discrete domain, we choose f to be the categorical distribution with a probability vector $\phi_{\theta}(x)$ as its statistics. The probability vector is of course parameterised by θ .

We denote the average policy network as ϕ_{θ_a} and update its parameters θ_a “softly” after each update to the policy parameter θ : $\theta_a \leftarrow \alpha \theta_a + (1 - \alpha) \theta$

Consider, for example, the ACER policy gradient as defined in Equation (9), but with respect to ϕ :

$$\begin{aligned}\hat{g}_t^{\text{acer}} &= \bar{\rho}_t \nabla_{\phi_{\theta}(x_t)} \log f(a_t|\phi_{\theta}(x_t)) [Q^{\text{ret}}(x_t, a_t) - V_{\theta_v}(x_t)] \\ &\quad + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+ \nabla_{\phi_{\theta}(x_t)} \log f(a_t|\phi_{\theta}(x_t)) [Q_{\theta_v}(x_t, a) - V_{\theta_v}(x_t)] \right)\end{aligned}\quad (10)$$

Given the averaged policy network, our proposed trust region update involves two stages. In the first stage, we solve the following optimization problem with a linearized KL divergence constraint:

$$\begin{aligned}\underset{z}{\text{minimize}} \quad & \frac{1}{2} \|g_t^{\text{acer}} - z\|_2^2 \\ \text{subject to} \quad & \nabla_{\phi_{\theta}(x_t)} D_{KL} [f(\cdot|\phi_{\theta_a}(x_t)) \| f(\cdot|\phi_{\theta}(x_t))]^T z \leq \delta\end{aligned}\quad (11)$$

Since the constraint is linear, the overall optimization problem reduces to a simple quadratic programming problem, the solution of which can be easily derived in closed form using the KKT conditions. Letting $k = \nabla_{\phi_{\theta}(x_t)} D_{KL} [f(\cdot|\phi_{\theta_a}(x_t)) \| f(\cdot|\phi_{\theta}(x_t))]$, the solution is:

$$z^* = g_t^{\text{acer}} - \max \left\{ 0, \frac{k^T g_t^{\text{acer}} - \delta}{\|k\|_2^2} \right\} k \quad (12)$$

This transformation of the gradient has a very natural form. If the constraint is satisfied, there is no change to the gradient with respect to $\phi_{\theta}(x_t)$. Otherwise, the update is scaled down in the direction

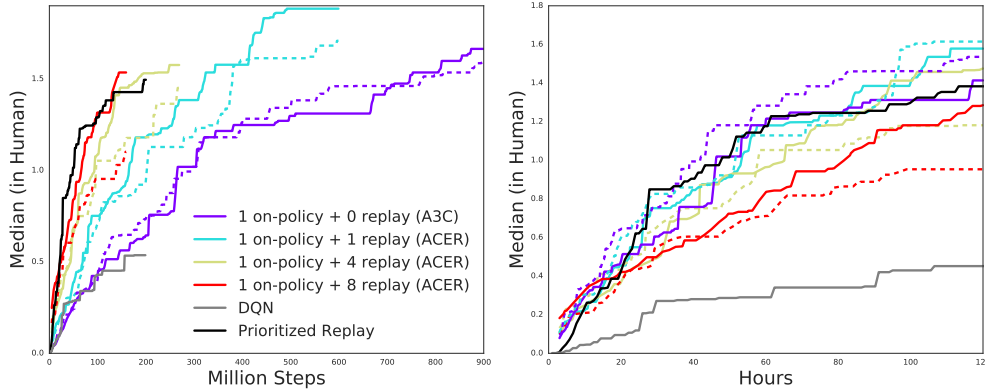


Figure 1: ACER improvements in sample (**LEFT**) and computation (**RIGHT**) complexity on Atari. On each plot, the median of the human-normalized score across all 57 Atari games is presented for 4 ratios of replay with 0 replay corresponding to on-policy A3C. The colored solid and dashed lines represent ACER with and without trust region updating respectively. The environment steps are counted over all threads. The gray curve is the original DQN agent (Mnih et al., 2015) and the black curve is one of the Prioritized Double DQN agents from Schaul et al. (2016).

of k , thus effectively lowering rate of change between the activations of the current policy and the average policy network.

In the second stage, we take advantage of back-propagation. Specifically, the updated gradient with respect to ϕ_θ , that is z^* , is back-propagated through the network to compute the derivatives with respect to the parameters. The parameter updates for the policy network follow from the chain rule: $-\frac{\phi_\theta(x)}{\theta} z^*$.

The trust region step is carried out in the space of the statistics of the distribution f , and not in the space of the policy parameters. This is done deliberately so as to avoid an additional back-propagation step through the policy network.

We would like to remark that the algorithm advanced in this section can be thought of as a general strategy for modifying the backward messages in back-propagation so as to stabilize the activations.

Instead of a trust region update, one could alternatively add an appropriately scaled KL cost to the objective function as proposed by Heess et al. (2015). This approach, however, is less robust to the choice of hyper-parameters in our experience.

The ACER algorithm results from a combination of the above ideas, with the precise pseudo-code appearing in Appendix A. A master algorithm (Algorithm 1) calls ACER on-policy to perform updates and propose trajectories. It then calls ACER off-policy component to conduct several replay steps. When on-policy, ACER effectively becomes a modified version of A3C where Q instead of V baselines are employed and trust region optimization is used.

4 RESULTS ON ATARI

We use the Arcade Learning Environment of Bellemare et al. (2013) to conduct an extensive evaluation. We deploy one single algorithm and network architecture, with fixed hyper-parameters, to learn to play 57 Atari games given only raw pixel observations and game rewards. This task is highly demanding because of the diversity of games, and high-dimensional pixel-level observations.

Our experimental setup uses 16 actor-learner threads running on a single machine with no GPUs. We adopt the same input pre-processing and network architecture as Mnih et al. (2015). Specifically, the network consists of a convolutional layer with $32 \ 8 \times 8$ filters with stride 4 followed by another convolutional layer with $64 \ 4 \times 4$ filters with stride 2, followed by a final convolutional layer with $64 \ 3 \times 3$ filters with stride 1, followed by a fully-connected layer of size 512. Each of the hidden layers is followed by a rectifier nonlinearity. The network outputs a softmax policy and Q values.

When using replay, we add to each thread a replay memory that is up to 50 000 frames in size. The total amount of memory used across all threads is thus similar in size to that of DQN (Mnih et al., 2015). For all Atari experiments, we use a single learning rate adopted from an earlier implementation of A3C without further tuning. We do not anneal the learning rates over the course of training as in Mnih et al. (2016). We otherwise adopt the same optimization procedure as in Mnih et al. (2016). Specifically, we adopt entropy regularization with weight 0.001, discount the rewards with $\gamma = 0.99$, and perform updates every 20 steps ($k = 20$ in the notation of Section 2). In all our experiments with experience replay, we use importance weight truncation with $c = 10$. We consider training ACER both with and without trust region updating as described in Section 3.3. When trust region updating is used, we use $\delta = 1$ and $\alpha = 0.99$ for all experiments.

To compare different agents, we adopt as our metric the median of the human normalized score over all 57 games. The normalization is calculated such that, for each game, human scores and random scores are evaluated to 1, and 0 respectively. The normalized score for a given game at time t is computed as the average normalized score over the past 1 million consecutive frames encountered until time t . For each agent, we plot its cumulative maximum median score over time. The result is summarized in Figure 1.

The four colors in Figure 1 correspond to four replay ratios (0, 1, 4 and 8) with a ratio of 4 meaning that we use the off-policy component of ACER 4 times after using the on-policy component (A3C). That is, a replay ratio of 0 means that we are using A3C. The solid and dashed lines represent ACER with and without trust region updating respectively. The gray and black curves are the original DQN (Mnih et al., 2015) and Prioritized Replay agent of Schaul et al. (2016) agents respectively.

As shown on the left panel of Figure 1, replay significantly increases data efficiency. We observe that when using the trust region optimizer, the average reward as a function of the number of environmental steps increases with the ratio of replay. This increase has diminishing returns, but with enough replay, ACER can match the performance of the best DQN agents. Moreover, it is clear that the off-policy actor critics (ACER) are much more sample efficient than their on-policy counterpart (A3C).

The right panel of Figure 1 shows that ACER agents perform similarly to A3C when measured by wall clock time. Thus, in this case, it is possible to achieve better data-efficiency without necessarily compromising on computation time. In particular, ACER with a replay ratio of 4 is an appealing alternative to either the prioritized DQN agent or A3C.

5 CONTINUOUS ACTOR CRITIC WITH EXPERIENCE REPLAY

Retrace requires estimates of both Q and V , but we cannot easily integrate over Q to derive V in continuous action spaces. In this section, we propose a solution to this problem in the form of a novel representation for RL, as well as modifications necessary for trust region updating.

5.1 POLICY EVALUATION

Retrace provides a target for learning Q_{θ_v} , but not for learning V_{θ_v} . We could use importance sampling to compute V_{θ_v} given Q_{θ_v} , but this estimator has high variance.

We propose a new architecture which we call Stochastic Dueling Networks (SDNs), inspired by the Dueling networks of Wang et al. (2016), which is designed to estimate both V^π and Q^π off-policy while maintaining consistency between the two estimates. At each time step, an SDN outputs a stochastic estimate $\tilde{Q}_{\theta_v}$ of Q^π and a deterministic estimate V_{θ_v} of V^π , such that

$$\tilde{Q}_{\theta_v}(x_t, a_t) \sim V_{\theta_v}(x_t) + A_{\theta_v}(x_t, a_t) - \frac{1}{n} \sum_{i=1}^n A_{\theta_v}(x_t, u_i) \quad \text{and} \quad u_i \sim \pi_\theta(\cdot | x_t) \quad (13)$$

where n is a parameter, see Figure 2. The two estimates are consistent in the sense that $\mathbb{E}_{a \sim \pi(\cdot | x_t)} [\mathbb{E}_{u_{1:n} \sim \pi(\cdot | x_t)} (\tilde{Q}_{\theta_v}(x_t, a))] = V_{\theta_v}(x_t)$. Furthermore, we can learn about V^π by learning $\tilde{Q}_{\theta_v}$. To see this, assume we have learned Q^π perfectly such that $\mathbb{E}_{u_{1:n} \sim \pi(\cdot | x_t)} (\tilde{Q}_{\theta_v}(x_t, a_t)) = Q^\pi(x_t, a_t)$, then $V_{\theta_v}(x_t) = \mathbb{E}_{a \sim \pi(\cdot | x_t)} [\mathbb{E}_{u_{1:n} \sim \pi(\cdot | x_t)} (\tilde{Q}_{\theta_v}(x_t, a))] = \mathbb{E}_{a \sim \pi(\cdot | x_t)} [Q^\pi(x_t, a)] = V^\pi(x_t)$. Therefore, a target on $\tilde{Q}_{\theta_v}(x_t, a_t)$ also provides an error signal for updating V_{θ_v} .

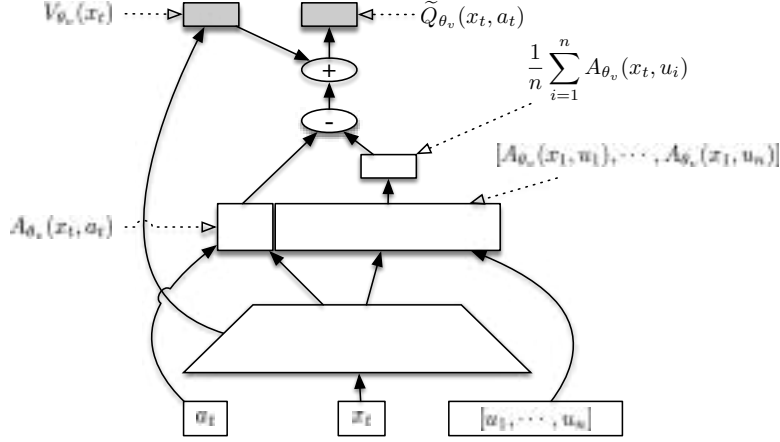


Figure 2: A schematic of the Stochastic Dueling Network. In the drawing, $[u_1 \dots u_n]$ are assumed to be samples from $\pi_\theta(\cdot|x_t)$. This schematic illustrates the concept of SDNs but does not reflect the real sizes of the networks used.

In addition to SDNs, however, we also construct the following novel target for estimating V^π :

$$V^{target}(x_t) = \min \left\{ 1 \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right\} \left(Q^{\text{ret}}(x_t, a_t) - Q_{\theta_v}(x_t, a_t) \right) + V_{\theta_v}(x_t) \quad (14)$$

The above target is also derived via the truncation and bias correction trick; for more details, see Appendix D.

Finally, when estimating Q^{ret} in continuous domains, we implement a slightly different formulation of the truncated importance weights $\bar{\rho}_t = \min \left\{ 1 \left(\frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right)^{\frac{1}{d}} \right\}$, where d is the dimensionality of the action space. Although not essential, we have found this formulation to lead to faster learning.

5.2 TRUST REGION UPDATING

To adopt the trust region updating scheme (Section 3.3) in the continuous control domain, one simply has to choose a distribution f and a gradient specification g_t^{acer} suitable for continuous action spaces.

For the distribution f , we choose Gaussian distributions with fixed diagonal covariance and mean $\phi_\theta(x)$.

To derive g_t^{acer} in continuous action spaces, consider the ACER policy gradient for the stochastic dueling network, but with respect to ϕ :

$$g_t^{\text{acer}} = \mathbb{E}_{x_t} \left[\mathbb{E}_{a_t} \left[\bar{\rho}_t \nabla_{\phi_\theta(x_t)} \log f(a_t|\phi_\theta(x_t)) (Q^{\text{opc}}(x_t, a_t) - V_{\theta_v}(x_t)) \right] + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - c}{\rho_t(a)} \right]_+ (\tilde{Q}_{\theta_v}(x_t, a) - V_{\theta_v}(x_t)) \nabla_{\phi_\theta(x_t)} \log f(a|\phi_\theta(x_t)) \right) \right] \quad (15)$$

In the above definition, we are using Q^{opc} instead of Q^{ret} . Here, $Q^{\text{opc}}(x_t, a_t)$ is the same as Retrace with the exception that the truncated importance ratio is replaced with 1 (Harutyunyan et al., 2016). Please refer to Appendix B an expanded discussion on this design choice. Given an observation x_t , we can sample $a'_t \sim \pi_\theta(\cdot|x_t)$ to obtain the following Monte Carlo approximation

$$g_t^{\text{acer}} = \bar{\rho}_t \nabla_{\phi_\theta(x_t)} \log f(a_t|\phi_\theta(x_t)) (Q^{\text{opc}}(x_t, a_t) - V_{\theta_v}(x_t)) + \left[\frac{\rho_t(a'_t) - c}{\rho_t(a'_t)} \right]_+ (\tilde{Q}_{\theta_v}(x_t, a'_t) - V_{\theta_v}(x_t)) \nabla_{\phi_\theta(x_t)} \log f(a'_t|\phi_\theta(x_t)) \quad (16)$$

Given f and g_t^{acer} , we apply the same steps as detailed in Section 3.3 to complete the update.

The precise pseudo-code of ACER algorithm for continuous spaces results is presented in Appendix A.

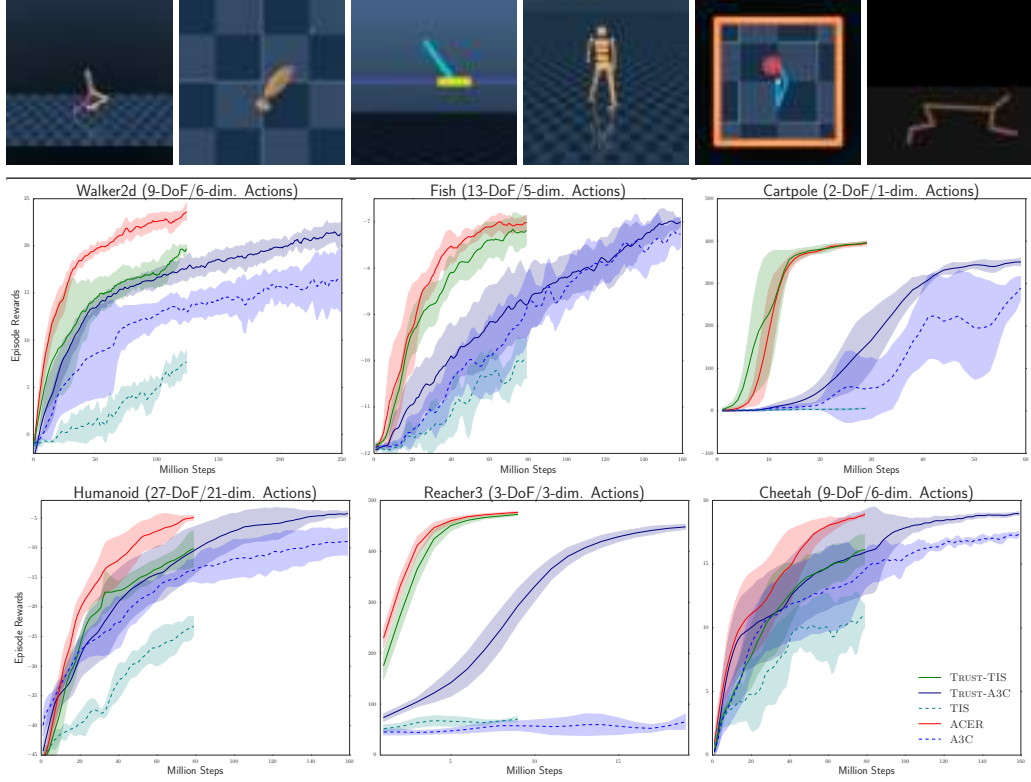


Figure 3: [TOP] Screen shots of the continuous control tasks. [BOTTOM] Performance of different methods on these tasks. ACER outperforms all other methods and shows clear gains for the higher-dimensionality tasks (humanoid, cheetah, walker and fish). The proposed trust region method by itself improves the two baselines (truncated importance sampling and A3C) significantly.

6 RESULTS ON MUJoCo

We evaluate our algorithms on 6 continuous control tasks, all of which are simulated using the MuJoCo physics engine (Todorov et al., 2012). For descriptions of the tasks, please refer to Appendix E.1. Briefly, the tasks with action dimensionality in brackets are: cartpole (1D), reacher (3D), cheetah (6D), fish (5D), walker (6D) and humanoid (21D). These tasks are illustrated in Figure 3.

To benchmark ACER for continuous control, we compare it to its on-policy counterpart both with and without trust region updating. We refer to these two baselines as A3C and Trust-A3C. Additionally, we also compare to a baseline with replay where we truncate the importance weights over trajectories as in (Wawrzynski, 2009). For a detailed description of this baseline, please refer to Appendix E. Again, we run this baseline both with and without trust region updating, and refer to these choices as Trust-TIS and TIS respectively. Last but not least, we refer to our proposed approach with SDN and trust region updating as simply ACER. All five setups are implemented in the asynchronous A3C framework.

All the aforementioned setups share the same network architecture that computes the policy and state values. We maintain an additional small network that computes the stochastic A values in the case of ACER. We use $n = 5$ (using the notation in Equation (13)) in all SDNs. Instead of mixing on-policy and replay learning as done in the Atari domain, ACER for continuous actions is entirely off-policy, with experiences generated from the simulator (4 times on average). When using replay, we add to each thread a replay memory that is 5 000 frames in size and perform updates every 50 steps ($k = 50$ in the notation of Section 2). The rate of the soft updating (α as in Section 3.3) is set to 0.995 in all setups involving trust region updating. The truncation threshold c is set to 5 for ACER.

We use diagonal Gaussian policies with fixed diagonal covariances where the diagonal standard deviation is set to 0.3. For all setups, we sample the learning rates log-uniformly in the range $[10^{-4} \ 10^{-3}]$. For setups involving trust region updating, we also sample δ uniformly in the range $[0 \ 1 \ 2]$. With all setups, we use 30 sampled hyper-parameter settings.

The empirical results for all continuous control tasks are shown Figure 3, where we show the mean and standard deviation of the best 5 out of 30 hyper-parameter settings over which we searched 3. For sensitivity analyses with respect to the hyper-parameters, please refer to Figures 5 and 6 in the Appendix.

In continuous control, ACER outperforms the A3C and truncated importance sampling baselines by a very significant margin.

Here, we also find that the proposed trust region optimization method can result in huge improvements over the baselines. The high-dimensional continuous action policies are much harder to optimize than the small discrete action policies in Atari, and hence we observe much higher gains for trust region optimization in the continuous control domains. In spite of the improvements brought in by trust region optimization, ACER still outperforms all other methods, specially in higher dimensions.

6.1 ABLATIONS

To further tease apart the contributions of the different components of ACER, we conduct an ablation analysis where we individually remove Retrace / $Q(\lambda)$ off-policy correction, SDNs, trust region, and truncation with bias correction from the algorithm. As shown in Figure 4, Retrace and off-policy correction, SDNs, and trust region are critical: removing any one of them leads to a clear deterioration of the performance. Truncation with bias correction did not alter the results in the Fish and Walker2d tasks. However, in Humanoid, where the dimensionality of the action space is much higher, including truncation and bias correction brings a significant boost which makes the originally kneeling humanoid stand. Presumably, the high dimensionality of the action space increases the variance of the importance weights which makes truncation with bias correction important. For more details on the experimental setup please see Appendix E.4

7 THEORETICAL ANALYSIS

Retrace is a very recent development in reinforcement learning. In fact, this work is the first to consider Retrace in the policy gradients setting. For this reason, and given the core role that Retrace plays in ACER, it is valuable to shed more light on this technique. In this section, we will prove that Retrace can be interpreted as an application of the importance weight truncation and bias correction trick advanced in this paper.

Consider the following equation:

$$Q^\pi(x_t, a_t) = \mathbb{E}_{x_{t+1}, a_{t+1}} [r_t + \gamma \rho_{t+1} Q^\pi(x_{t+1}, a_{t+1})] \quad (17)$$

If we apply the weight truncation and bias correction trick to the above equation we obtain

$$Q^\pi(x_t, a_t) = \mathbb{E}_{x_{t+1}, a_{t+1}} \left[r_t + \gamma \bar{\rho}_{t+1} Q^\pi(x_{t+1}, a_{t+1}) + \gamma \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_{t+1}(a) - c}{\rho_{t+1}(a)} \right]_+ Q^\pi(x_{t+1}, a) \right) \right] \quad (18)$$

By recursively expanding Q^π as in Equation 18, we can represent $Q^\pi(x, a)$ as:

$$Q^\pi(x, a) = \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q^\pi(x_{t+1}, b) \right) \right) \right] \quad (19)$$

The expectation $\mathbb{E}_\mu$ is taken over trajectories starting from x with actions generated with respect to μ . When Q^π is not available, we can replace it with our current estimate Q to get a return-based

³ For videos of the policies learned with ACER, please see: <https://www.youtube.com/watch?v=NmbeQYoVv5g&list=PLkmHIkh1FjiTlvwxEnsJMs3v7seR5HSP->

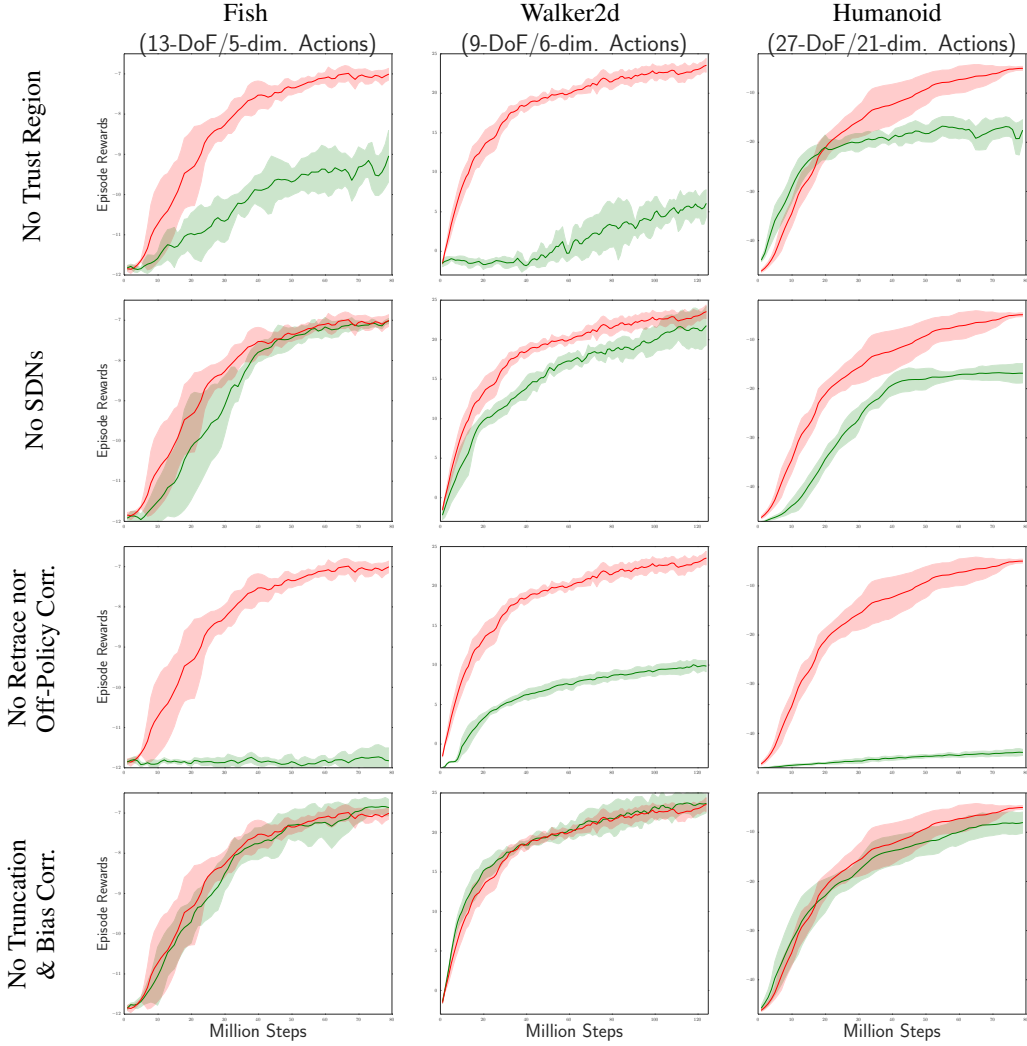


Figure 4: Ablation analysis evaluating the effect of different components of ACER. Each row compares ACER with and without one component. The columns represents three control tasks. Red lines, in all plots, represent ACER whereas green lines ACER with missing components. This study indicates that all 4 components studied improve performance where 3 are critical to success. Note that the ACER curve is of course the same in all rows.

estimate of Q^π . This operation also defines an operator:

$$\mathcal{B}Q(x, a) = \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q(x_{t+1}, b) \right) \right) \right] \quad (20)$$

In the following proposition, we show that $\mathcal{B}$ is a contraction operator with a unique fixed point Q^π and that it is equivalent to the Retrace operator.

Proposition 1. *The operator $\mathcal{B}$ is a contraction operator such that $\|\mathcal{B}Q - Q^\pi\|_\infty \leq \gamma \|Q - Q^\pi\|_\infty$ and $\mathcal{B}$ is equivalent to Retrace.*

The above proposition not only shows an alternative way of arriving at the same operator, but also provides a different proof of contraction for Retrace. Please refer to Appendix C for the regularization conditions and proof of the above proposition.

Finally, $\mathcal{B}$, and therefore Retrace, generalizes both the Bellman operator $\mathcal{T}^\pi$ and importance sampling. Specifically, when $c = 0$, $\mathcal{B} = \mathcal{T}^\pi$ and when $c = \infty$, $\mathcal{B}$ recovers importance sampling; see Appendix C.

8 CONCLUDING REMARKS

We have introduced a stable off-policy actor critic that scales to both continuous and discrete action spaces. This approach integrates several recent advances in RL in a principle manner. In addition, it integrates three innovations advanced in this paper: truncated importance sampling with bias correction, stochastic dueling networks and an efficient trust region policy optimization method.

We showed that the method not only matches the performance of the best known methods on Atari, but that it also outperforms popular techniques on several continuous control problems.

The efficient trust region optimization method advanced in this paper performs remarkably well in continuous domains. It could prove very useful in other deep learning domains, where it is hard to stabilize the training process.

ACKNOWLEDGMENTS

We are very thankful to Marc Bellemare, Jascha Sohl-Dickstein, and Sébastien Racaniere for proof-reading and valuable suggestions.

REFERENCES

- M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *JAIR*, 47:253–279, 2013.
- G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. *arXiv preprint 1606.01540*, 2016.
- T. Degris, M. White, and R. S. Sutton. Off-policy actor-critic. In *ICML*, pp. 457–464, 2012.
- Anna Harutyunyan, Marc G Bellemare, Tom Stepleton, and Remi Munos. $Q(\lambda)$ with off-policy corrections. *arXiv preprint arXiv:1602.04951*, 2016.
- N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa. Learning continuous control policies by stochastic value gradients. In *NIPS*, 2015.
- T. Jie and P. Abbeel. On a connection between importance sampling and the likelihood ratio policy gradient. In *NIPS*, pp. 1000–1008, 2010.
- S. Levine and V. Koltun. Guided policy search. In *ICML*, 2013.
- S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *arXiv preprint arXiv:1504.00702*, 2015.
- T. Lillicrap, J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv:1509.02971*, 2015.
- L.J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3):293–321, 1992.
- N. Meuleau, L. Peshkin, L. P. Kaelbling, and K. Kim. Off-policy policy search. Technical report, MIT AI Lab, 2000.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- V. Mnih, A. Puigdomènech Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *arXiv:1602.01783*, 2016.
- R. Munos, T. Stepleton, A. Harutyunyan, and M. G. Bellemare. Safe and efficient off-policy reinforcement learning. *arXiv preprint arXiv:1606.02647*, 2016.
- K. Narasimhan, T. Kulkarni, and R. Barzilay. Language understanding for text-based games using deep reinforcement learning. In *EMNLP*, 2015.

A ACER PSEUDO-CODE FOR DISCRETE ACTIONS

Algorithm 1 ACER for discrete actions (master algorithm)

// Assume global shared parameter vectors θ and θ_v .
 // Assume ratio of replay τ .

repeat
 Call ACER on-policy, Algorithm 2
 $n \leftarrow \text{Poisson}(\tau)$
 for $i \in \{1, \dots, n\}$ **do**
 Call ACER off-policy, Algorithm 2
 end for
until Max iteration or time reached.

Algorithm 2 ACER for discrete actions

Reset gradients $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.

Initialize parameters $\theta' \leftarrow \theta$ and $\theta'_v \leftarrow \theta_v$.

if not On-Policy **then**

 Sample the trajectory $\{x_0, a_0, r_0, \mu(\cdot|x_0), \dots, x_k, a_k, r_k, \mu(\cdot|x_k)\}$ from the replay memory.

else

 Get state x_0

end if

for $i \in \{0, \dots, k\}$ **do**

 Compute $f(\cdot|\phi_{\theta'}(x_i))$, $Q_{\theta'_v}(x_i, \cdot)$ and $f(\cdot|\phi_{\theta_a}(x_i))$.

if On-Policy **then**

 Perform a_i according to $f(\cdot|\phi_{\theta'}(x_i))$

 Receive reward r_i and new state x_{i+1}

$\mu(\cdot|x_i) \leftarrow f(\cdot|\phi_{\theta'}(x_i))$

end if

$\bar{\rho}_i \leftarrow \min \left\{ 1, \frac{f(a_i|\phi_{\theta'}(x_i))}{\mu(a_i|x_i)} \right\}$.

end for

$Q^{ret} \leftarrow \begin{cases} 0 & \text{for terminal } x_k \\ \sum_a Q_{\theta'_v}(x_k, a) f(a|\phi_{\theta'}(x_k)) & \text{otherwise} \end{cases}$

for $i \in \{k-1, \dots, 0\}$ **do**

$Q^{ret} \leftarrow r_i + \gamma Q^{ret}$

$V_i \leftarrow \sum_a Q_{\theta'_v}(x_i, a) f(a|\phi_{\theta'}(x_i))$

 Computing quantities needed for trust region updating:

$$\begin{aligned} g &\leftarrow \min \{c, \rho_i(a_i)\} \nabla_{\phi_{\theta'}(x_i)} \log f(a_i|\phi_{\theta'}(x_i)) (Q^{ret} - V_i) \\ &\quad + \sum_a \left[1 - \frac{c}{\rho_i(a)} \right]_+ f(a|\phi_{\theta'}(x_i)) \nabla_{\phi_{\theta'}(x_i)} \log f(a|\phi_{\theta'}(x_i)) (Q_{\theta'_v}(x_i, a) - V_i) \\ k &\leftarrow \nabla_{\phi_{\theta'}(x_i)} D_{KL} [f(\cdot|\phi_{\theta_a}(x_i)) \| f(\cdot|\phi_{\theta'}(x_i))] \end{aligned}$$

 Accumulate gradients wrt θ' : $d\theta' \leftarrow d\theta' + \frac{\partial \phi_{\theta'}(x_i)}{\partial \theta'} \left(g - \max \left\{ 0, \frac{k^T g - \delta}{\|k\|_2^2} \right\} k \right)$

 Accumulate gradients wrt θ'_v : $d\theta_v \leftarrow d\theta_v + \nabla_{\theta'_v} (Q^{ret} - Q_{\theta'_v}(x_i, a))^2$

 Update Retrace target: $Q^{ret} \leftarrow \bar{\rho}_i (Q^{ret} - Q_{\theta'_v}(x_i, a_i)) + V_i$

end for

Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.

Updating the average policy network: $\theta_a \leftarrow \alpha \theta_a + (1 - \alpha) \theta$

B $Q(\lambda)$ WITH OFF-POLICY CORRECTIONS

Given a trajectory generated under the behavior policy μ , the $Q(\lambda)$ with off-policy corrections estimator (Harutyunyan et al., 2016) can be expressed recursively as follows:

$$Q^{\text{opc}}(x_t, a_t) = r_t + \gamma [Q^{\text{opc}}(x_{t+1}, a_{t+1}) - Q(x_{t+1}, a_{t+1})] + \gamma V(x_{t+1}) \quad (21)$$

Notice that $Q^{\text{opc}}(x_t, a_t)$ is the same as Retrace with the exception that the truncated importance ratio is replaced with 1.

Algorithm 3 ACER for Continuous Actions

Reset gradients $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
Initialize parameters $\theta' \leftarrow \theta$ and $\theta'_v \leftarrow \theta_v$.
Sample the trajectory $\{x_0, a_0, r_0, \mu(\cdot|x_0), \dots, x_k, a_k, r_k, \mu(\cdot|x_k)\}$ from the replay memory.
for $i \in \{0, \dots, k\}$ **do**
 Compute $f(\cdot|\phi_{\theta'}(x_i))$, $V_{\theta'_v}(x_i)$, $\tilde{Q}_{\theta'_v}(x_i, a_i)$, and $f(\cdot|\phi_{\theta_a}(x_i))$.
 Sample $a'_i \sim f(\cdot|\phi_{\theta'}(x_i))$
 $\rho_i \leftarrow \frac{f(a_i|\phi_{\theta'}(x_i))}{\mu(a_i|x_i)}$ and $\rho'_i \leftarrow \frac{f(a'_i|\phi_{\theta'}(x_i))}{\mu(a'_i|x_i)}$
 $c_i \leftarrow \min \left\{ 1, (\rho_i)^{\frac{1}{d}} \right\}$.
end for
 $Q^{ret} \leftarrow \begin{cases} 0 & \text{for terminal } x_k \\ V_{\theta'_v}(x_k) & \text{otherwise} \end{cases}$
 $Q^{opc} \leftarrow Q^{ret}$
for $i \in \{k-1, \dots, 0\}$ **do**
 $Q^{ret} \leftarrow r_i + \gamma Q^{ret}$
 $Q^{opc} \leftarrow r_i + \gamma Q^{opc}$
 Computing quantities needed for trust region updating:

$$g \leftarrow \min \{c, \rho_i\} \nabla_{\phi_{\theta'}(x_i)} \log f(a_i|\phi_{\theta'}(x_i)) (Q^{opc}(x_i, a_i) - V_{\theta'_v}(x_i))$$

$$+ \left[1 - \frac{c}{\rho'_i} \right]_+ (\tilde{Q}_{\theta'_v}(x_i, a'_i) - V_{\theta'_v}(x_i)) \nabla_{\phi_{\theta'}(x_i)} \log f(a'_i|\phi_{\theta'}(x_i))$$

$$k \leftarrow \nabla_{\phi_{\theta'}(x_i)} D_{KL} [f(\cdot|\phi_{\theta_a}(x_i)) \| f(\cdot|\phi_{\theta'}(x_i))]$$

 Accumulate gradients wrt θ : $d\theta \leftarrow d\theta + \frac{\partial \phi_{\theta'}(x_i)}{\partial \theta'} \left(g - \max \left\{ 0, \frac{k^T g - \delta}{\|k\|_2^2} \right\} k \right)$
 Accumulate gradients wrt θ'_v : $d\theta_v \leftarrow d\theta_v + (Q^{ret} - \tilde{Q}_{\theta'_v}(x_i, a_i)) \nabla_{\theta'_v} \tilde{Q}_{\theta'_v}(x_i, a_i)$

$$d\theta_v \leftarrow d\theta_v + \min \{1, \rho_i\} \left(Q^{ret}(x_i, a_i) - \tilde{Q}_{\theta'_v}(x_i, a_i) \right) \nabla_{\theta'_v} V_{\theta'_v}(x_i)$$

 Update Retrace target: $Q^{ret} \leftarrow c_i \left(Q^{ret} - \tilde{Q}_{\theta'_v}(x_i, a_i) \right) + V_{\theta'_v}(x_i)$
 Update Retrace target: $Q^{opc} \leftarrow \left(Q^{opc} - \tilde{Q}_{\theta'_v}(x_i, a_i) \right) + V_{\theta'_v}(x_i)$
end for
Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.
Updating the average policy network: $\theta_a \leftarrow \alpha \theta_a + (1 - \alpha) \theta$

Because of the lack of the truncated importance ratio, the operator defined by Q^{opc} is only a contraction if the target and behavior policies are close to each other (Harutyunyan et al., 2016). $Q(\lambda)$ with off-policy corrections is therefore less stable compared to Retrace and unsafe for policy evaluation.

Q^{opc} , however, could better utilize the returns as the traces are not cut by the truncated importance weights. As a result, Q^{opc} could be used efficiently to estimate Q^π in policy gradient (e.g. in Equation (16)). In our continuous control experiments, we have found that Q^{opc} leads to faster learning.

C RETRACE AS TRUNCATED IMPORTANCE SAMPLING WITH BIAS CORRECTION

For the purpose of proving proposition 1, we assume our environment to be a Markov Decision Process $(\mathcal{X} \mathcal{A} \gamma P r)$. We restrict $\mathcal{X}$ to be a finite state space. For notational simplicity, we also restrict $\mathcal{A}$ to be a finite action space. $P : \mathcal{X} \mathcal{A} \rightarrow \mathcal{X}$ defines the state transition probabilities and $r : \mathcal{X} \mathcal{A} \rightarrow [-R_{MAX} R_{MAX}]$ defines a reward function. Finally, $\gamma \in [0, 1)$ is the discount factor.

Proof of proposition 1. First we show that $\mathcal{B}$ is a contraction operator.

$$\begin{aligned}
& |\mathcal{B}Q(x \ a) - Q^\pi(x \ a)| \\
&= \left| \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(\gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ (Q(x_{t+1} \ b) - Q^\pi(x_{t+1} \ b)) \right) \right) \right] \right| \\
&\leq \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left[\gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ |Q(x_{t+1} \ b) - Q^\pi(x_{t+1} \ b)| \right) \right] \right] \\
&\leq \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(\gamma(1 - \bar{P}_{t+1}) \sup_b |Q(x_{t+1} \ b) - Q^\pi(x_{t+1} \ b)| \right) \right] \quad (22)
\end{aligned}$$

Where $\bar{P}_{t+1} = 1 - \mathbb{E}_{b \sim \pi} \left[\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ \right] = \mathbb{E}_{b \sim \mu} [\bar{\rho}_{t+1}(b)]$. The last inequality in the above equation is due to Hölder's inequality.

$$\begin{aligned}
(22) &\leq \sup_{x \ b} |Q(x \ b) - Q^\pi(x \ b)| \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) (\gamma(1 - \bar{P}_{t+1})) \right] \\
&= \sup_{x \ b} |Q(x \ b) - Q^\pi(x \ b)| \mathbb{E}_\mu \left[\gamma \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) - \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) (\gamma \bar{P}_{t+1}) \right] \\
&= \sup_{x \ b} |Q(x \ b) - Q^\pi(x \ b)| \mathbb{E}_\mu \left[\gamma \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) - \sum_{t \geq 0} \gamma^{t+1} \left(\prod_{i=1}^{t+1} \bar{\rho}_i \right) \right] \\
&= \sup_{x \ b} |Q(x \ b) - Q^\pi(x \ b)| (\gamma C - (C - 1))
\end{aligned}$$

where $C = \sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right)$. Since $C \geq \sum_{t=0}^0 \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) = 1$, we have that $\gamma C - (C - 1) \leq \gamma$. Therefore, we have shown that $\mathcal{B}$ is a contraction operator.

Now we show that $\mathcal{B}$ is the same as Retrace. By apply the truncation and bias correction trick, we have

$$\mathbb{E}_{b \sim \pi} [Q(x_{t+1} \ b)] = \mathbb{E}_{b \sim \mu} [\bar{\rho}_{t+1}(b) Q(x_{t+1} \ b)] + \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q(x_{t+1} \ b) \right) \quad (23)$$

By adding and subtracting the two sides of Equation (23) inside the summand of Equation (20), we have

$$\begin{aligned}
\mathcal{B}Q(x \ a) &= \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left[r_t + \gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q(x_{t+1} \ b) \right) + \gamma \mathbb{E}_{b \sim \pi} [Q(x_{t+1} \ b)] \right. \right. \\
&\quad \left. \left. - \gamma \mathbb{E}_{b \sim \mu} [\bar{\rho}_{t+1}(b) Q(x_{t+1} \ b)] - \gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q(x_{t+1} \ b) \right) \right] \right] \\
&= \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} [Q(x_{t+1} \ b)] - \gamma \mathbb{E}_{b \sim \mu} [\bar{\rho}_{t+1}(b) Q(x_{t+1} \ b)] \right) \right] \\
&= \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} [Q(x_{t+1} \ b)] - \gamma \bar{\rho}_{t+1} Q(x_{t+1} \ a_{t+1}) \right) \right] \\
&= \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} [Q(x_{t+1} \ b)] - Q(x_t \ a_t) \right) \right] + Q(x \ a) = \mathcal{R}Q(x \ a)
\end{aligned}$$

□

In the remainder of this appendix, we show that $\mathcal{B}$ generalizes both the Bellman operator and importance sampling. First, we reproduce the definition of $\mathcal{B}$:

$$\mathcal{B}Q(x|a) = \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \bar{\rho}_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} \left(\left[\frac{\rho_{t+1}(b) - c}{\rho_{t+1}(b)} \right]_+ Q(x_{t+1}|b) \right) \right) \right]$$

When $c = 0$, we have that $\bar{\rho}_i = 0 \forall i$. Therefore only the first summand of the sum remains:

$$\mathcal{B}Q(x|a) = \mathbb{E}_\mu \left[r_t + \gamma \mathbb{E}_{b \sim \pi} (Q(x_{t+1}|b)) \right]$$

In this case $\mathcal{B} = \mathcal{T}$. When $c = \infty$, the compensation term disappears and $\bar{\rho}_i = \rho_i \forall i$:

$$\mathcal{B}Q(x|a) = \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \rho_i \right) \left(r_t + \gamma \mathbb{E}_{b \sim \pi} (0 \times Q(x_{t+1}|b)) \right) \right] = \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t \rho_i \right) r_t \right]$$

In this case $\mathcal{B}$ is the same operator defined by importance sampling.

D DERIVATION OF V^{target}

By using the truncation and bias correction trick, we can derive the following:

$$V^\pi(x_t) = \mathbb{E}_{a \sim \mu} \left[\min \left\{ 1, \frac{\pi(a|x_t)}{\mu(a|x_t)} \right\} Q^\pi(x_t|a) \right] + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - 1}{\rho_t(a)} \right]_+ Q^\pi(x_{t+1}|a) \right)$$

We, however, cannot use the above equation as a target as we do not have access to Q^π . To derive a target, we can take a Monte Carlo approximation of the first expectation in the RHS of the above equation and replace the first occurrence of Q^π with Q^{ret} and the second with our current neural net approximation $Q_{\theta_v}(x_t|a)$:

$$V_{pre}^{target}(x_t) := \min \left\{ 1, \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right\} Q^{\text{ret}}(x_t|a_t) + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - 1}{\rho_t(a)} \right]_+ Q_{\theta_v}(x_{t+1}|a) \right) \quad (24)$$

Through the truncation and bias correction trick again, we have the following identity:

$$\mathbb{E}_{a \sim \pi} [Q_{\theta_v}(x_t|a)] = \mathbb{E}_{a \sim \mu} \left[\min \left\{ 1, \frac{\pi(a|x_t)}{\mu(a|x_t)} \right\} Q_{\theta_v}(x_t|a) \right] + \mathbb{E}_{a \sim \pi} \left(\left[\frac{\rho_t(a) - 1}{\rho_t(a)} \right]_+ Q_{\theta_v}(x_{t+1}|a) \right) \quad (25)$$

Adding and subtracting both sides of Equation (25) to the RHS of (24) while taking a Monte Carlo approximation, we arrive at $V^{target}(x_t)$:

$$V^{target}(x_t) := \min \left\{ 1, \frac{\pi(a_t|x_t)}{\mu(a_t|x_t)} \right\} \left(Q^{\text{ret}}(x_t|a_t) - Q_{\theta_v}(x_t|a_t) \right) + V_{\theta_v}(x_t)$$

E CONTINUOUS CONTROL EXPERIMENTS

E.1 DESCRIPTION OF THE CONTINUOUS CONTROL PROBLEMS

Our continuous control tasks were simulated using the MuJoCo physics engine (Todorov et al., 2012). For all experiments we considered an episodic setup with an episode length of $T = 500$ steps and a discount factor of 0.99.

Cartpole swingup This is an instance of the classic cart-pole swing-up task. It consists of a pole attached to a cart running on a finite track. The agent is required to balance the pole near the center of the track by applying a force to the cart only. An episode starts with the pole at a random angle and zero velocity. A reward zero is given except when the pole is approximately upright (within ± 5 deg) and the track approximately in the center of the track (± 0.05) for a track length of 2.4. The observations include position and velocity of the cart, angle and angular velocity of the pole, a sine/cosine of the angle, the position of the tip of the pole, and Cartesian velocities of the pole. The dimension of the action space is 1.

Reacher3 The agent needs to control a planar 3-link robotic arm in order to minimize the distance between the end effector of the arm and a target. Both arm and target position are chosen randomly at the beginning of each episode. The reward is zero except when the tip of the arm is within 0.05 of the target, where it is one. The 8-dimensional observation consists of the angles and angular velocity of all joints as well as the displacement between target and the end effector of the arm. The 3-dimensional action are the torques applied to the joints.

Cheetah The Half-Cheetah (Wawrzynski (2009); Heess et al. (2015)) is a planar locomotion task where the agent is required to control a 9-DoF cheetah-like body (in the vertical plane) to move in the direction of the x-axis as quickly as possible. The reward is given by the velocity along the x-axis and a control cost: $r = v_x + 0.1 \|a\|^2$. The observation vector consists of the z-position of the torso and its x z velocities as well as the joint angles and angular velocities. The action dimension is 6.

Fish The goal of this task is to control a 13-DoF fish-like body to swim to a random target in 3D space. The reward is given by the distance between the head of the fish and the target, a small penalty for the body not being upright, and a control cost. At the beginning of an episode the fish is initialized facing in a random direction relative to the target. The 24-dimensional observation is given by the displacement between the fish and the target projected onto the torso coordinate frame, the joint angles and velocities, the cosine of the angle between the z-axis of the torso and the world z-axis, and the velocities of the torso in the torso coordinate frame. The 5-dimensional actions control the position of the side fins and the tail.

Walker The 9-DoF planar walker is inspired by (Schulman et al. (2015a)) and is required to move forward along the x-axis as quickly as possible without falling. The reward consists of the x-velocity of the torso, a quadratic control cost, and terms that penalize deviations of the torso from the preferred height and orientation (i.e. terms that encourage the walker to stay standing and upright). The 24-dimensional observation includes the torso height, velocities of all DoFs, as well as sines and cosines of all body orientations in the x-z plane. The 6-dimensional action controls the torques applied at the joints. Episodes are terminated early with a negative reward when the torso exceeds upper and lower limits on its height and orientation.

Humanoid The humanoid is a 27 degrees-of-freedom body with 21 actuators (21 action dimensions). It is initialized lying on the ground in a random configuration and the task requires it to achieve a standing position. The reward function penalizes deviations from the height of the head when standing, and includes additional terms that encourage upright standing, as well as a quadratic action penalty. The 94 dimensional observation contains information about joint angles and velocities and several derived features reflecting the body’s pose.

E.2 UPDATE EQUATIONS OF THE BASELINE TIS

The baseline TIS follows the following update equations,

$$\begin{aligned} \text{updates to the policy: } & \min \left\{ 5 \left(\prod_{i=0}^{k-1} \rho_{t+i} \right) \right\} \left[\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V_{\theta_v}(x_{k+t}) - V_{\theta_v}(x_t) \right] \nabla_{\theta} \log \pi_{\theta}(a_t | x_t) \\ \text{updates to the value: } & \min \left\{ 5 \left(\prod_{i=0}^{k-1} \rho_{t+i} \right) \right\} \left[\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V_{\theta_v}(x_{k+t}) - V_{\theta_v}(x_t) \right] \nabla_{\theta_v} V_{\theta_v}(x_t) \end{aligned}$$

The baseline Trust-TIS is appropriately modified according to the trust region update described in Section 3.3

E.3 SENSITIVITY ANALYSIS

In this section, we assess the sensitivity of ACER to hyper-parameters. In Figures 5 and 6 we show, for each game, the final performance of our ACER agent versus the choice of learning rates, and the trust region constraint δ respectively.

Note, as we are doing random hyper-parameter search, each learning rate is associated with a random δ and vice versa. It is therefore difficult to tease out the effect of either hyper-parameter independently.

We observe, however, that ACER is not very sensitive to the hyper-parameters overall. In addition, smaller δ 's do not seem to adversely affect the final performance while larger δ 's do in domains of higher action dimensionality. Similarly, smaller learning rates perform well while bigger learning rates tend to hurt final performance in domains of higher action dimensionality.

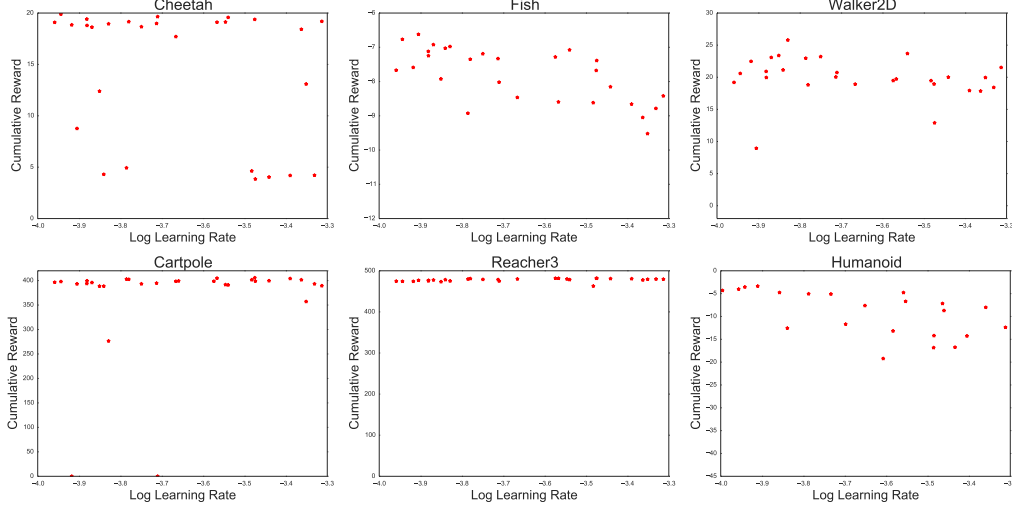


Figure 5: Log learning rate vs. cumulative rewards in all the continuous control tasks for ACER. The plots show the final performance after training for all 30 log learning rates considered. Note that each learning rate is associated with a different δ as a consequence of random search over hyper-parameters.

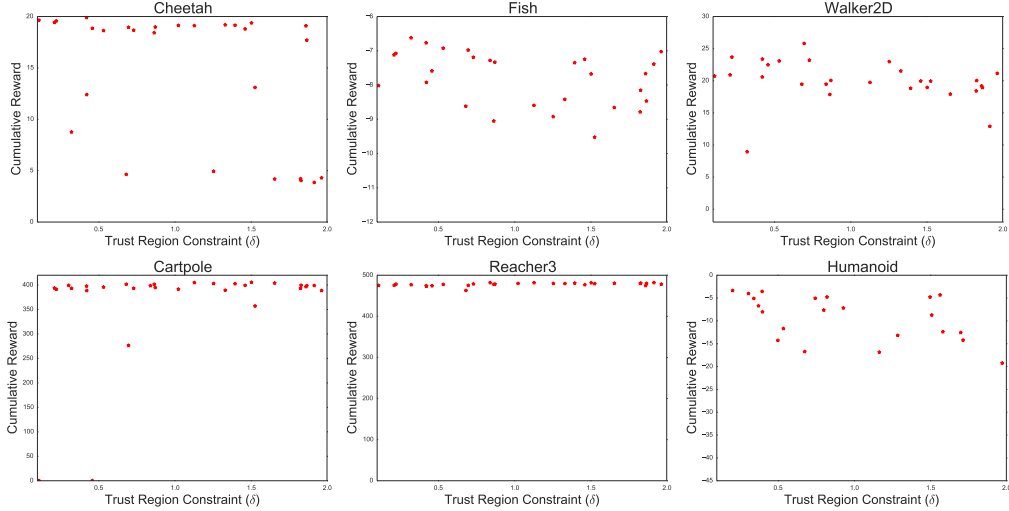


Figure 6: Trust region constraint (δ) vs. cumulative rewards in all the continuous control tasks for ACER. The plots show the final performance after training for all 30 trust region constraints (δ) searched over. Note that each δ is associated with a different learning rate as a consequence of random search over hyper-parameters.

E.4 EXPERIMENTAL SETUP OF ABLATION ANALYSIS

For the ablation analysis, we use the same experimental setup as in the continuous control experiments while removing one component at a time.

To evaluate the effectiveness of Retrace/ $Q(\lambda)$ with off-policy correction, we replace both with importance sampling based estimates (following [Degrís et al. \(2012\)](#)) which can be expressed recursively: $R_t = r_t + \rho_{t+1} R_{t+1}$.

To evaluate the Stochastic Dueling Networks, we replace it with two separate networks: one computing the state values and the other Q values. Given $Q^{ret}(x_t, a_t)$, the naive way of estimating the state values is to use the following update rule:

$$(\rho_t Q^{ret}(x_t, a_t) - V_{\theta_v}(x_t)) \nabla_{\theta_v} V_{\theta_v}(x_t)$$

The above update rule, however, suffers from high variance. We consider instead the following update rule:

$$\rho_t (Q^{ret}(x_t, a_t) - V_{\theta_v}(x_t)) \nabla_{\theta_v} V_{\theta_v}(x_t)$$

which has markedly lower variance. We update our Q estimates as before.

To evaluate the effects of the truncation and bias correction trick, we change our c parameter (see Equation [\(I6\)](#)) to ∞ so as to use pure importance sampling.

Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation

Yuhuai Wu*
University of Toronto
Vector Institute
ywu@cs.toronto.edu

Elman Mansimov*
New York University
mansimov@cs.nyu.edu

Shun Liao
University of Toronto
Vector Institute
sliao3@cs.toronto.edu

Roger Grosse
University of Toronto
Vector Institute
rgrosse@cs.toronto.edu

Jimmy Ba
University of Toronto
Vector Institute
jimmy@psi.utoronto.ca

Abstract

In this work, we propose to apply trust region optimization to deep reinforcement learning using a recently proposed Kronecker-factored approximation to the curvature. We extend the framework of natural policy gradient and propose to optimize both the actor and the critic using Kronecker-factored approximate curvature (K-FAC) with trust region; hence we call our method Actor Critic using Kronecker-Factored Trust Region (ACKTR). To the best of our knowledge, this is the first scalable trust region natural gradient method for actor-critic methods. It is also a method that learns non-trivial tasks in continuous control as well as discrete control policies directly from raw pixel inputs. We tested our approach across discrete domains in Atari games as well as continuous domains in the MuJoCo environment. With the proposed methods, we are able to achieve higher rewards and a 2- to 3-fold improvement in sample efficiency on average, compared to previous state-of-the-art on-policy actor-critic methods. Code is available at <https://github.com/openai/baselines>.

1 Introduction

Agents using deep reinforcement learning (deep RL) methods have shown tremendous success in learning complex behaviour skills and solving challenging control tasks in high-dimensional raw sensory state-space [25, 18, 13]. Deep RL methods make use of deep neural networks to represent control policies. Despite the impressive results, these neural networks are still trained using simple variants of stochastic gradient descent (SGD). SGD and related first-order methods explore weight space inefficiently. It often takes days for the current deep RL methods to master various continuous and discrete control tasks. Previously, a distributed approach was proposed [18] to reduce training time by executing multiple agents to interact with the environment simultaneously, but this leads to rapidly diminishing returns of sample efficiency as the degree of parallelism increases.

Sample efficiency is a dominant concern in RL; robotic interaction with the real world is typically scarcer than computation time, and even in simulated environments the cost of simulation often dominates that of the algorithm itself. One way to effectively reduce the sample size is to use more advanced optimization techniques for gradient updates. Natural policy gradient [11] uses the technique of natural gradient descent [1] to perform gradient updates. Natural gradient methods

*Equal contribution.

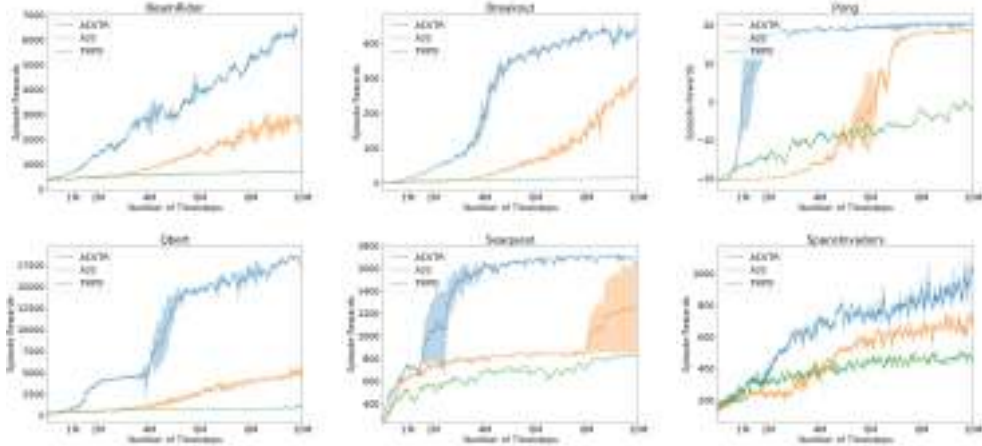


Figure 1: Performance comparisons on six standard Atari games trained for 10 million timesteps (1 timestep equals 4 frames). The shaded region denotes the standard deviation over 2 random seeds.

follow the steepest descent direction that uses the Fisher metric as the underlying metric, a metric that is based not on the choice of coordinates but rather on the manifold (i.e., the surface).

However, the exact computation of the natural gradient is intractable because it requires inverting the Fisher information matrix. Trust-region policy optimization (TRPO) [22] avoids explicitly storing and inverting the Fisher matrix by using Fisher-vector products [21]. However, it typically requires many steps of conjugate gradient to obtain a single parameter update, and accurately estimating the curvature requires a large number of samples in each batch; hence TRPO is impractical for large models and suffers from sample inefficiency.

Kronecker-factored approximated curvature (K-FAC) [16, 7] is a scalable approximation to natural gradient. It has been shown to speed up training of various state-of-the-art large-scale neural networks [2] in supervised learning by using larger mini-batches. Unlike TRPO, each update is comparable in cost to an SGD update, and it keeps a running average of curvature information, allowing it to use small batches. This suggests that applying K-FAC to policy optimization could improve the sample efficiency of the current deep RL methods.

In this paper, we introduce the actor-critic using Kronecker-factored trust region (ACKTR; pronounced “actor”) method, a scalable trust-region optimization algorithm for actor-critic methods. The proposed algorithm uses a Kronecker-factored approximation to natural policy gradient that allows the covariance matrix of the gradient to be inverted efficiently. To best of our knowledge, we are also the first to extend the natural policy gradient algorithm to optimize value functions via Gauss-Newton approximation. In practice, the per-update computation cost of ACKTR is only 10% to 25% higher than SGD-based methods. Empirically, we show that ACKTR substantially improves both sample efficiency and the final performance of the agent in the Atari environments [4] and the MuJoCo [27] tasks compared to the state-of-the-art on-policy actor-critic method A2C [18] and the famous trust region optimizer TRPO [22].

We make our source code available online at <https://github.com/openai/baselines>.

2 Background

2.1 Reinforcement learning and actor-critic methods

We consider an agent interacting with an infinite-horizon, discounted Markov Decision Process $(\mathcal{X}, \mathcal{A}, \gamma, P, r)$. At time t , the agent chooses an action $a_t \in \mathcal{A}$ according to its policy $\pi_\theta(a|s_t)$ given its current state $s_t \in \mathcal{X}$. The environment in turn produces a reward $r(s_t, a_t)$ and transitions to the next state s_{t+1} according to the transition probability $P(s_{t+1}|s_t, a_t)$. The goal of the agent is to maximize the expected γ -discounted cumulative return $\mathcal{J}(\theta) = \mathbb{E}_\pi[R_t] = \mathbb{E}_\pi[\sum_{i=0}^{\infty} \gamma^i r(s_{t+i}, a_{t+i})]$ with respect to the policy parameters θ . Policy gradient methods [30, 26] directly parameterize a policy $\pi_\theta(a|s_t)$ and update parameter θ so as to maximize the objective $\mathcal{J}(\theta)$. In its general form,

the policy gradient is defined as [23],

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \Psi^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right],$$

where Ψ^t is often chosen to be the advantage function $A^{\pi}(s_t, a_t)$, which provides a relative measure of value of each action a_t at a given state s_t . There is an active line of research [23] on designing an advantage function that provides both low-variance and low-bias gradient estimates. As this is not the focus of our work, we simply follow the asynchronous advantage actor critic (A3C) method [18] and define the advantage function as the k -step returns with function approximation,

$$A^{\pi}(s_t, a_t) = \sum_{i=0}^{k-1} (\gamma^i r(s_{t+i}, a_{t+i}) + \gamma^k V_{\phi}^{\pi}(s_{t+k})) - V_{\phi}^{\pi}(s_t),$$

where $V_{\phi}^{\pi}(s_t)$ is the value network, which provides an estimate of the expected sum of rewards from the given state following policy π , $V_{\phi}^{\pi}(s_t) = \mathbb{E}_{\pi}[R_t]$. To train the parameters of the value network, we again follow [18] by performing temporal difference updates, so as to minimize the squared difference between the bootstrapped k -step returns $\hat{R}_t$ and the prediction value $\frac{1}{2} \|\hat{R}_t - V_{\phi}^{\pi}(s_t)\|^2$.

2.2 Natural gradient using Kronecker-factored approximation

To minimize a nonconvex function $\mathcal{J}(\theta)$, the method of steepest descent calculates the update $\Delta\theta$ that minimizes $\mathcal{J}(\theta + \Delta\theta)$, subject to the constraint that $\|\Delta\theta\|_B < 1$, where $\|\cdot\|_B$ is the norm defined by $\|x\|_B = (x^T B x)^{\frac{1}{2}}$, and B is a positive semidefinite matrix. The solution to the constraint optimization problem has the form $\Delta\theta \propto -B^{-1} \nabla_{\theta} \mathcal{J}$, where $\nabla_{\theta} \mathcal{J}$ is the standard gradient. When the norm is Euclidean, i.e., $B = I$, this becomes the commonly used method of gradient descent. However, the Euclidean norm of the change depends on the parameterization θ . This is not favorable because the parameterization of the model is an arbitrary choice, and it should not affect the optimization trajectory. The method of natural gradient constructs the norm using the Fisher information matrix F , a local quadratic approximation to the KL divergence. This norm is independent of the model parameterization θ on the class of probability distributions, providing a more stable and effective update. However, since modern neural networks may contain millions of parameters, computing and storing the exact Fisher matrix and its inverse is impractical, so we have to resort to approximations.

A recently proposed technique called Kronecker-factored approximate curvature (K-FAC) [16] uses a Kronecker-factored approximation to the Fisher matrix to perform efficient approximate natural gradient updates. We let $p(y|x)$ denote the output distribution of a neural network, and $L = \log p(y|x)$ denote the log-likelihood. Let $W \in \mathbb{R}^{C_{out} \times C_{in}}$ be the weight matrix in the ℓ^{th} layer, where C_{out} and C_{in} are the number of output/input neurons of the layer. Denote the input activation vector to the layer as $a \in \mathbb{R}^{C_{in}}$, and the pre-activation vector for the next layer as $s = Wa$. Note that the weight gradient is given by $\nabla_W L = (\nabla_s L) a^{\top}$. K-FAC utilizes this fact and further approximates the block F_{ℓ} corresponding to layer ℓ as $\hat{F}_{\ell}$,

$$\begin{aligned} F_{\ell} &= \mathbb{E}[\text{vec}\{\nabla_W L\} \text{vec}\{\nabla_W L\}^{\top}] = \mathbb{E}[aa^{\top} \otimes \nabla_s L (\nabla_s L)^{\top}] \\ &\approx \mathbb{E}[aa^{\top}] \otimes \mathbb{E}[\nabla_s L (\nabla_s L)^{\top}] := A \otimes S := \hat{F}_{\ell}, \end{aligned}$$

where A denotes $\mathbb{E}[aa^{\top}]$ and S denotes $\mathbb{E}[\nabla_s L (\nabla_s L)^{\top}]$. This approximation can be interpreted as making the assumption that the second-order statistics of the activations and the backpropagated derivatives are uncorrelated. With this approximation, the natural gradient update can be efficiently computed by exploiting the basic identities $(P \otimes Q)^{-1} = P^{-1} \otimes Q^{-1}$ and $(P \otimes Q) \text{vec}(T) = PTQ^{\top}$:

$$\text{vec}(\Delta W) = \hat{F}_{\ell}^{-1} \text{vec}\{\nabla_W \mathcal{J}\} = \text{vec}(A^{-1} \nabla_W \mathcal{J} S^{-1}).$$

From the above equation we see that the K-FAC approximate natural gradient update only requires computations on matrices comparable in size to W . Grosse and Martens [7] have recently extended the K-FAC algorithm to handle convolutional networks. Ba et al. [2] later developed a distributed version of the method where most of the overhead is mitigated through asynchronous computation. Distributed K-FAC achieved 2- to 3-times speed-ups in training large modern classification convolutional networks.

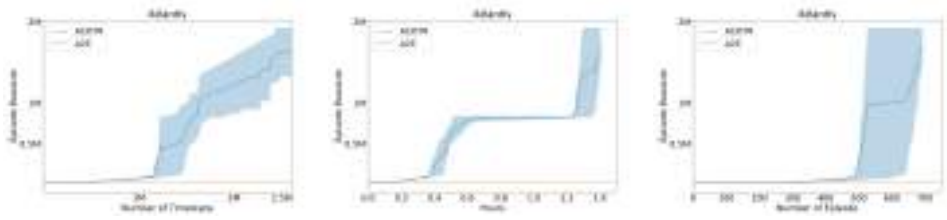


Figure 2: In the Atari game of Atlantis, our agent (ACKTR) quickly learns to obtain rewards of 2 million in 1.3 hours, 600 episodes of games, 2.5 million timesteps. The same result is achieved by advantage actor critic (A2C) in 10 hours, 6000 episodes, 25 million timesteps. ACKTR is 10 times more sample efficient than A2C on this game.

3 Methods

3.1 Natural gradient in actor-critic

Natural gradient was proposed to apply to the policy gradient method more than a decade ago by Kakade [11]. But there still doesn't exist a scalable, sample-efficient, and general-purpose instantiation of the natural policy gradient. In this section, we introduce the first scalable and sample-efficient natural gradient algorithm for actor-critic methods: the actor-critic using Kronecker-factored trust region (ACKTR) method. We use Kronecker-factored approximation to compute the natural gradient update, and apply the natural gradient update to both the actor and the critic.

To define the Fisher metric for reinforcement learning objectives, one natural choice is to use the policy function which defines a distribution over the action given the current state, and take the expectation over the trajectory distribution:

$$F = \mathbb{E}_{p(\tau)} [\nabla_{\theta} \log \pi(a_t | s_t) (\nabla_{\theta} \log \pi(a_t | s_t))^{\top}],$$

where $p(\tau)$ is the distribution of trajectories, given by $p(s_0) \prod_{t=0}^T \pi(a_t | s_t) p(s_{t+1} | s_t, a_t)$. In practice, one approximates the intractable expectation over trajectories collected during training.

We now describe one way to apply natural gradient to optimize the critic. Learning the critic can be thought of as a least-squares function approximation problem, albeit one with a moving target. In the setting of least-squares function approximation, the second-order algorithm of choice is commonly Gauss-Newton, which approximates the curvature as the Gauss-Newton matrix $G := \mathbb{E}[J^T J]$, where J is the Jacobian of the mapping from parameters to outputs [19]. The Gauss-Newton matrix is equivalent to the Fisher matrix for a Gaussian observation model [15]; this equivalence allows us to apply K-FAC to the critic as well. Specifically, we assume the output of the critic v is defined to be a Gaussian distribution $p(v | s_t) \sim \mathcal{N}(v; V(s_t), \sigma^2)$. The Fisher matrix for the critic is defined with respect to this Gaussian output distribution. In practice, we can simply set σ to 1, which is equivalent to the vanilla Gauss-Newton method.

If the actor and critic are disjoint, one can separately apply K-FAC updates to each using the metrics defined above. But to avoid instability in training, it is often beneficial to use an architecture where the two networks share lower-layer representations but have distinct output layers [18, 28]. In this case, we can define the joint distribution of the policy and the value distribution by assuming independence of the two output distributions, i.e., $p(a, v | s) = \pi(a | s) p(v | s)$, and construct the Fisher metric with respect to $p(a, v | s)$, which is no different than the standard K-FAC except that we need to sample the networks' outputs independently. We can then apply K-FAC to approximate the Fisher matrix $\mathbb{E}_{p(\tau)} [\nabla \log p(a, v | s) \nabla \log p(a, v | s)^{\top}]$ to perform updates simultaneously.

In addition, we use the factorized Tikhonov damping approach described by [16]. We also follow [2] and perform the asynchronous computation of second-order statistics and inverses required by the Kronecker approximation to reduce computation time.

3.2 Step-size Selection and trust-region optimization

Traditionally, natural gradient is performed with SGD-like updates, $\theta \leftarrow \theta - \eta F^{-1} \nabla_{\theta} L$. But in the context of deep RL, Schulman et al. [22] observed that such an update rule can result in large updates to the policy, causing the algorithm to prematurely converge to a near-deterministic policy. They advocate instead using a trust region approach, whereby the update is scaled down to modify the policy distribution (in terms of KL divergence) by at most a specified amount. Therefore, we adopt the trust region formulation of K-FAC introduced by [2], choosing the effective step size η to be $\min(\eta_{\max}, \sqrt{\frac{2\delta}{\Delta\theta^{\top} \bar{F} \Delta\theta}})$, where the learning rate $\eta_{\max}$ and trust region radius δ are hyperparameters. If the actor and the critic are disjoint, then we need to tune a different set of $\eta_{\max}$ and δ separately for both. The variance parameter for the critic output distribution can be absorbed into the learning rate parameter for vanilla Gauss-Newton. On the other hand, if they share representations, we need to tune one set of $\eta_{\max}$, δ , and also the weighting parameter of the training loss of the critic, with respect to that of the actor.

4 Related work

Natural gradient [1] was first applied to policy gradient methods by Kakade [11]. Bagnell and Schneider [3] further proved that the metric defined in [11] is a covariant metric induced by the path-distribution manifold. Peters and Schaal [20] then applied natural gradient to the actor-critic algorithm. They proposed performing natural policy gradient for the actor’s update and using a least-squares temporal difference (LSTD) method for the critic’s update. However, there are great computational challenges when applying natural gradient methods, mainly associated with efficiently storing the Fisher matrix as well as computing its inverse. For tractability, previous work restricted the method to using the compatible function approximator (a linear function approximator). To avoid the computational burden, Trust Region Policy Optimization (TRPO) [22] approximately solves the linear system using conjugate gradient with fast Fisher matrix-vector products, similar to the work of Martens [14]. This approach has two main shortcomings. First, it requires repeated computation of Fisher vector products, preventing it from scaling to the larger architectures typically used in experiments on learning from image observations in Atari and MuJoCo. Second, it requires a large batch of rollouts in order to accurately estimate curvature. K-FAC avoids both issues by using tractable Fisher matrix approximations and by keeping a running average of curvature statistics during training. Although TRPO shows better per-iteration progress than policy gradient methods trained with first-order optimizers such as Adam [12], it is generally less sample efficient.

Several methods were proposed to improve the computational efficiency of TRPO. To avoid repeated computation of Fisher-vector products, Wang et al. [28] solve the constrained optimization problem with a linear approximation of KL divergence between a running average of the policy network and the current policy network. Instead of the hard constraint imposed by the trust region optimizer, Heess et al. [9] and Schulman et al. [24] added a KL cost to the objective function as a soft constraint. Both papers show some improvement over vanilla policy gradient on continuous and discrete control tasks in terms of sample efficiency.

There are other recently introduced actor-critic models that improve sample efficiency by introducing experience replay [28], [8] or auxiliary objectives [10]. These approaches are orthogonal to our work, and could potentially be combined with ACKTR to further enhance sample efficiency.

5 Experiments

We conducted a series of experiments to investigate the following questions: (1) How does ACKTR compare with the state-of-the-art on-policy method and common second-order optimizer baseline in terms of sample efficiency and computational efficiency? (2) What makes a better norm for optimization of the critic? (3) How does the performance of ACKTR scale with batch size compared to the first-order method?

We evaluated our proposed method, ACKTR, on two standard benchmark platforms. We first evaluated it on the discrete control tasks defined in OpenAI Gym [5], simulated by Arcade Learning Environment [4], a simulator for Atari 2600 games which is commonly used as a deep reinforcement learning benchmark for discrete control. We then evaluated it on a variety of continuous control

Domain	Human level	ACKTR		A2C		TRPO (10 M)	
		Rewards	Episode	Rewards	Episode	Rewards	Episode
Beamrider	5775.0	13581.4	3279	8148.1	8930	670.0	N/A
Breakout	31.8	735.7	4094	581.6	14464	14.7	N/A
Pong	9.3	20.9	904	19.9	4768	-1.2	N/A
Q-bert	13455.0	21500.3	6422	15967.4	19168	971.8	N/A
Seaquest	20182.0	1776.0	N/A	1754.0	N/A	810.4	N/A
Space Invaders	1652.0	19723.0	14696	1757.2	N/A	465.1	N/A

Table 1: ACKTR and A2C results showing the last 100 average episode rewards attained after 50 million timesteps, and TRPO results after 10 million timesteps. The table also shows the episode N , where N denotes the first episode for which the mean episode reward over the N^{th} game to the $(N + 100)^{th}$ game crosses the human performance level [17], averaged over 2 random seeds.

benchmark tasks defined in OpenAI Gym [5], simulated by the MuJoCo [27] physics engine. Our baselines are (a) a synchronous and batched version of the asynchronous advantage actor critic model (A3C) [18], henceforth called A2C (advantage actor critic), and (b) TRPO [22]. ACKTR and the baselines use the same model architecture except for the TRPO baseline on Atari games, with which we are limited to using a smaller architecture because of the computing burden of running a conjugate gradient inner-loop. See the appendix for other experiment details.

5.1 Discrete control

We first present results on the standard six Atari 2600 games to measure the performance improvement obtained by ACKTR. The results on the six Atari games trained for 10 million timesteps are shown in Figure 1, with comparison to A2C and TRPO². ACKTR significantly outperformed A2C in terms of sample efficiency (i.e., speed of convergence per number of timesteps) by a significant margin in all games. We found that TRPO could only learn two games, Seaquest and Pong, in 10 million timesteps, and performed worse than A2C in terms of sample efficiency.

In Table 1 we present the mean of rewards of the last 100 episodes in training for 50 million timesteps, as well as the number of episodes required to achieve human performance [17]. Notably, on the games Beamrider, Breakout, Pong, and Q-bert, A2C required respectively 2.7, 3.5, 5.3, and 3.0 times more episodes than ACKTR to achieve human performance. In addition, one of the runs by A2C in Space Invaders failed to match human performance, whereas ACKTR achieved 19723 on average, 12 times better than human performance (1652). On the games Breakout, Q-bert and Beamrider, ACKTR achieved 26%, 35%, and 67% larger episode rewards than A2C.

We also evaluated ACKTR on the rest of the Atari games; see Appendix B for full results. We compared ACKTR with Q-learning methods, and we found that in 36 out of 44 benchmarks, ACKTR is on par with Q-learning methods in terms of sample efficiency, and consumed a lot less computation time. Remarkably, in the game of Atlantis, ACKTR quickly learned to obtain rewards of 2 million in 1.3 hours (600 episodes), as shown in Figure 2. It took A2C 10 hours (6000 episodes) to reach the same performance level.

5.2 Continuous control

We ran experiments on the standard benchmark of continuous control tasks defined in OpenAI Gym [5] simulated in MuJoCo [27], both from low-dimensional state-space representation and directly from pixels. In contrast to Atari, the continuous control tasks are sometimes more challenging due to high-dimensional action spaces and exploration. The results of eight MuJoCo environments trained for 1 million timesteps are shown in Figure 3. Our model significantly outperformed baselines on six out of eight MuJoCo tasks and performed competitively with A2C on the other two tasks (Walker2d and Swimmer).

We further evaluated ACKTR for 30 million timesteps on eight MuJoCo tasks and in Table 2 we present mean rewards of the top 10 consecutive episodes in training, as well as the number of

²The A2C and TRPO Atari baseline results are provided to us by the OpenAI team, <https://github.com/openai/baselines-results>.

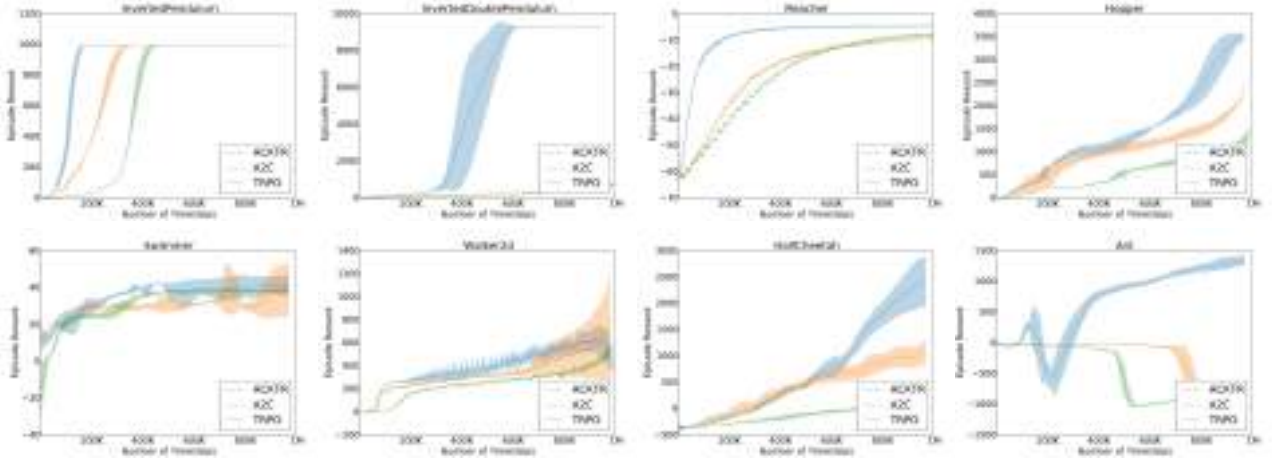


Figure 3: Performance comparisons on eight MuJoCo environments trained for 1 million timesteps (1 timestep equals 4 frames). The shaded region denotes the standard deviation over 3 random seeds.

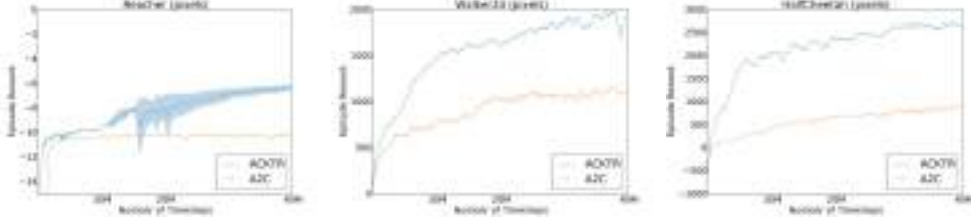


Figure 4: Performance comparisons on 3 MuJoCo environments from image observations trained for 40 million timesteps (1 timestep equals 4 frames).

episodes to reach a certain threshold defined in [8]. As shown in Table 2, ACKTR reaches the specified threshold faster on all tasks, except for Swimmer where TRPO achieves 4.1 times better sample efficiency. A particularly notable case is Ant, where ACKTR is 16.4 times more sample efficient than TRPO. As for the mean reward score, all three models achieve results comparable with each other with the exception of TRPO, which in the Walker2d environment achieves a 10% better reward score.

We also attempted to learn continuous control policies directly from pixels, without providing low-dimensional state space as an input. Learning continuous control policies from pixels is much more challenging than learning from the state space, partially due to the slower rendering time compared to Atari (0.5 seconds in MuJoCo vs 0.002 seconds in Atari). The state-of-the-art actor-critic method A3C [18] only reported results from pixels on relatively simple tasks, such as Pendulum, Pointmass2D, and Gripper. As shown in Figure 4 we can see that our model significantly outperforms A2C in terms of final episode reward after training for 40 million timesteps. More specifically, on Reacher, HalfCheetah, and Walker2d our model achieved a 1.6, 2.8, and 1.7 times greater final reward compared to A2C. The videos of trained policies from pixels can be found at <https://www.youtube.com/watch?v=gtM87w1xGoM>. Pretrained model weights are available at <https://github.com/emansim/acktr>.

5.3 A better norm for critic optimization?

The previous natural policy gradient method applied a natural gradient update only to the actor. In our work, we propose also applying a natural gradient update to the critic. The difference lies in the norm with which we choose to perform steepest descent on the critic; that is, the norm $\|\cdot\|_B$ defined in section 2.2. In this section, we applied ACKTR to the actor, and compared using a first-order method (i.e., Euclidean norm) with using ACKTR (i.e., the norm defined by Gauss-Newton) for critic optimization. Figures 5 (a) and (b) show the results on the continuous control task HalfCheetah and the Atari game Breakout. We observe that regardless of which norm we use to optimize the critic, there are improvements brought by applying ACKTR to the actor compared to the baseline A2C.

Domain	Threshold	ACKTR		A2C		TRPO (10 M)	
		Rewards	Episodes	Rewards	Episodes	Rewards	Episodes
Ant	3500 (6000)	4621.6	3660	4870.5	106186	5095.0	60156
HalfCheetah	4700 (4800)	5586.3	12980	5343.7	21152	5704.7	21033
Hopper	2000 (3800)	3915.9	17033	3915.3	33481	3755.0	39426
InvertedPendulum	950 (950)	1000.0	6831	1000.0	10982	1000.0	29267
InvertedDoublePendulum	9100 (9100)	9356.0	41996	9356.1	82694	9320.0	78519
Reacher	-7 (-3.75)	-1.5	3325	-1.7	20591	-2.0	14940
Swimmer	90 (360)	138.0	6475	140.7	11516	136.4	1571
Walker2d	3000 (N/A)	6198.8	15043	5874.9	26828	6874.1	27720

Table 2: ACKTR, A2C, and TRPO results, showing the top 10 average episode rewards attained within 30 million timesteps, averaged over the 3 best performing random seeds out of 8 random seeds. “Episode” denotes the smallest N for which the mean episode reward over the N^{th} to the $(N + 10)^{th}$ game crosses a certain threshold. The thresholds for all environments except for InvertedPendulum and InvertedDoublePendulum were chosen according to Gu et al. [8], and in brackets we show the reward threshold needed to solve the environment according to the OpenAI Gym website [5].

However, the improvements brought by using the Gauss-Newton norm for optimizing the critic are more substantial in terms of sample efficiency and episode rewards at the end of training. In addition, the Gauss-Newton norm also helps stabilize the training, as we observe larger variance in the results over random seeds with the Euclidean norm.

Recall that the Fisher matrix for the critic is constructed using the output distribution of the critic, a Gaussian distribution with variance σ . In vanilla Gauss-Newton, σ is set to 1. We experimented with estimating σ using the variance of the Bellman error, which resembles estimating the variance of the noise in regression analysis. We call this method adaptive Gauss-Newton. However, we find adaptive Gauss-Newton doesn’t provide any significant improvement over vanilla Gauss-Newton. (See detailed comparisons on the choices of σ in Appendix D).

5.4 How does ACKTR compare with A2C in wall-clock time?

We compared ACKTR to the baselines A2C and TRPO in terms of wall-clock time. Table 3 shows the average timesteps per second over six Atari games and eight MuJoCo (from state space) environments. The result is obtained with the same experiment setup as previous experiments. Note that in MuJoCo tasks episodes are processed sequentially, whereas in the Atari environment episodes are processed in parallel; hence more frames are processed in Atari environments. From the table we see that ACKTR only increases computing time by at most 25% per timestep, demonstrating its practicality with large optimization benefits.

(Timesteps/Second)	Atari			MuJoCo		
batch size	80	160	640	1000	2500	25000
ACKTR	712	753	852	519	551	582
A2C	1010	1038	1162	624	650	651
TRPO	160	161	177	593	619	637

Table 3: Comparison of computational cost. The average timesteps per second over six Atari games and eight MuJoCo tasks during training for each algorithms. ACKTR only increases computing time at most 25% over A2C.

5.5 How do ACKTR and A2C perform with different batch sizes?

In a large-scale distributed learning setting, large batch size is used in optimization. Therefore, in such a setting, it is preferable to use a method that can scale well with batch size. In this section, we compare how ACKTR and the baseline A2C perform with respect to different batch sizes. We experimented with batch sizes of 160 and 640. Figure 5 (c) shows the rewards in number of timesteps. We found that ACKTR with a larger batch size performed as well as that with a smaller batch size. However, with a larger batch size, A2C experienced significant degradation in terms of sample efficiency. This corresponds to the observation in Figure 5 (d), where we plotted the training curve

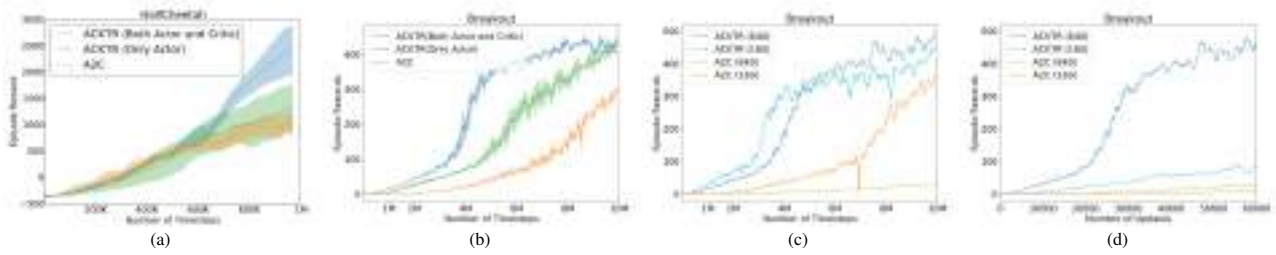


Figure 5: (a) and (b) compare optimizing the critic (value network) with a Gauss-Newton norm (ACKTR) against a Euclidean norm (first order). (c) and (d) compare ACKTR and A2C with different batch sizes.

in terms of number of updates. We see that the benefit increases substantially when using a larger batch size with ACKTR compared to with A2C. This suggests there is potential for large speed-ups with ACKTR in a distributed setting, where one needs to use large mini-batches; this matches the observation in [2].

6 Conclusion

In this work we proposed a sample-efficient and computationally inexpensive trust-region-optimization method for deep reinforcement learning. We used a recently proposed technique called K-FAC to approximate the natural gradient update for actor-critic methods, with trust region optimization for stability. To the best of our knowledge, we are the first to propose optimizing both the actor and the critic using natural gradient updates. We tested our method on Atari games as well as the MuJoCo environments, and we observed 2- to 3-fold improvements in sample efficiency on average compared with a first-order gradient method (A2C) and an iterative second-order method (TRPO). Because of the scalability of our algorithm, we are also the first to train several non-trivial tasks in continuous control directly from raw pixel observation space. This suggests that extending Kronecker-factored natural gradient approximations to other algorithms in reinforcement learning is a promising research direction.

Acknowledgements

We would like to thank the OpenAI team for their generous support in providing baseline results and Atari environment preprocessing codes. We also want to thank John Schulman for helpful discussions.

References

- [1] S. I. Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.
- [2] J. Ba, R. Grosse, and J. Martens. Distributed second-order optimization using Kronecker-factored approximations. In *ICLR*, 2017.
- [3] J. A. Bagnell and J. G. Schneider. Covariant policy search. In *IJCAI*, 2003.
- [4] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- [5] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [6] D. A. Clevert, T. Unterthiner, and S. Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *ICLR*, 2016.
- [7] R. Grosse and J. Martens. A Kronecker-factored approximate Fisher matrix for convolutional layers. In *ICML*, 2016.

- [28] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas. Sample efficient actor-critic with experience replay. In *ICLR*, 2016.
- [29] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [30] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

A Experimental details

A.1 Discrete control

For experiments on the Atari Environment, we adopted the same input preprocessing procedure as in [17], with a slight modification to the architecture. Specifically, we used a shared network to parameterize the policy and value function: The first convolutional layer is of 32 filters of size 8×8 with stride 4 followed by another convolutional layer with 64 filters of size 4×4 and stride 2, followed by a final convolutional layer with 32 filters of size 3×3 with stride 1, followed by a fully connected layer of size 512, followed by one softmax output layer that parameterizes the policy and a linear output layer that predicts the value. We used 32 filters in the third convolutional layer because we found that it saved time for computing the Fisher matrix inverse without any degradation in performance. (One alternative would be to use the doubly factored approximation [2] with all 64 filters.) For the baseline A2C, we used the same architecture as in [17]. For TRPO, because of its high per-iteration expense, we used a smaller architecture, with 2 convolutional layers followed by a fully connected layer with 128 units. The first convolutional layer had 8 filters of size 8×8 with stride 4, followed by another convolutional layer with 16 filters of size 4×4 with stride 2.

We tuned the maximum learning rate $\eta_{\max}$ using a grid search over $\{0.7, 0.2, 0.07, 0.02\}$ on the game of Breakout, with the trust region radius δ set to 0.001. We used the same hyperparameters for all Atari experiments. Both the baseline (A2C) and our method used a linear schedule for the learning rate over the course of training, and entropy regularization with weight 0.01. Following [17], the agent is trained on each game using 50 million time steps or 200 million frames. Unless otherwise stated, we used a batch size of 640 for ACKTR, 80 for A2C, and 512 for TRPO. The batch sizes were chosen to achieve better sample efficiency.

A.2 Continuous control

For experiments with low-dimensional state space as an input we used two separate neural networks with 64 hidden units per layer in a two-layer network. We used Tanh and ELU [6] nonlinearities for the policy network and value network, respectively, for all layers except the output layer, which didn’t have any nonlinearity. The log standard deviation of a Gaussian policy was parameterized as a bias in a final layer of policy network that didn’t depend on input state. For all experiments, we used ACKTR and A2C trained with batch sizes of 2500 and TRPO trained with a batch size of 25000. The batch sizes were chosen to be consistent with the experimental design of the results provided by the OpenAI team.

For experiments using pixels as an input we passed in a 42×42 RGB image along with the previous frame to a convolutional neural network. The two convolutional layers contained 32 filters of size 3×3 with stride of 2 followed by a fully connected layer of 256 hidden units. In contrast to our Atari experiments, we found that separating the policy network and value function into two separate networks resulted in better empirical performance in both ACKTR and A2C. We used ReLU nonlinearity for the policy network and ELU [6] nonlinearity for the value function. We also found that it is important to use orthogonal initialization for both networks, otherwise the A2C baseline failed to improve its episode reward. All models were trained with batch size of 8000. We tuned the maximum learning rate $\eta_{\max}$ using a grid search over $\{0.3, 0.03, 0.003\}$ on the tasks of Reacher and Hopper, with the trust region radius δ set to 0.001. We fixed hyperparameters for all MuJoCo experiments.

Table 4: Raw scores across all games, starting with 30 no-op actions. Other scores from [29].

GAMES	HUMAN	DQN	DDQN	DUEL	PRIOR.	PRIOR. DUEL.	ACKTR
Alien	7,127.7	1,620.0	3,747.7	4,461.4	4,203.8	3,941.0	3197.1
Amidar	1,719.5	978.0	1,793.3	2,354.5	1,838.9	2,296.8	1059.4
Assault	742.0	4,280.4	5,393.2	4,621.0	7,672.1	11,477.0	10,777.7
Asterix	8,503.3	4,359.0	17,356.5	28,188.0	31,527.0	375,080.0	31,583.0
Asteroids	47,388.7	1,364.5	734.7	2,837.7	2,654.3	1,192.7	34,171.6
Atlantis	29,028.1	279,987.0	106,056.0	382,572.0	357,324.0	395,762.0	3,433,182.0
Bank Heist	753.1	455.0	1,030.6	1,611.9	1,054.6	1,503.1	1,289.7
Battle Zone	37,187.5	29,900.0	31,700.0	37,150.0	31,530.0	35,520.0	8910.0
Beamrider	16,926.5	8,627.5	13,772.8	12,164.0	23,384.2	30,276.5	13,581.4
Berzerk	2,630.4	585.6	1,225.4	1,472.6	1,305.6	3,409.0	927.2
Bowling	160.7	50.4	68.1	65.5	47.9	46.7	24.3
Boxing	12.1	88.0	91.6	99.4	95.6	98.9	1.45
Breakout	30.5	385.5	418.5	345.3	373.9	366.0	735.7
Centipede	12,017.0	4,657.7	5,409.4	7,561.4	4,463.2	7,687.5	7,125.28
Crazy Climber	35,829.4	110,763.0	117,282.0	143,570.0	141,161.0	162,224.0	150,444.0
Demon Attack	1,971.0	12,149.4	58,044.2	60,813.3	71,846.4	72,878.6	274,176.7
Double Dunk	-16.4	-6.6	-5.5	0.1	18.5	-12.5	-0.54
Enduro	860.5	729.0	1,211.8	2,258.2	2,093.0	2,306.4	0.0
Fishing Derby	-38.7	-4.9	15.5	46.4	39.5	41.3	33.73
Freeway	29.6	30.8	33.3	0.0	33.7	33.0	0.0
Gopher	2,412.5	8,777.4	14,840.8	15,718.4	32,487.2	104,368.2	47,730.8
Ice Hockey	0.9	-1.9	-2.7	0.5	1.3	-0.4	-4.2
James Bond	302.8	768.5	1,358.0	1,312.5	5,148.0	812.0	490.0
Kangaroo	3,035.0	7,259.0	12,992.0	14,854.0	16,200.0	1,792.0	3,150.0
Krull	2,665.5	8,422.3	7,920.5	11,451.9	9,728.0	10,374.4	9,686.9
Kung-Fu Master	22,736.3	26,059.0	29,710.0	34,294.0	39,581.0	48,375.0	34,954.0
Phoenix	7,242.6	8,485.2	12,252.5	23,092.2	18,992.7	70,324.3	133,433.7
Pitfall!	6,463.7	-286.1	-29.9	0.0	-356.5	0.0	-1.1
Pong	14.6	19.5	20.9	21.0	20.6	20.9	20.9
Q-bert	13,455.0	13,117.3	15,088.5	19,220.3	16,256.5	18,760.3	23,151.5
River Raid	17,118.0	7,377.6	14,884.5	21,162.6	14,522.3	20,607.6	17,762.8
Road Runner	7,845.0	39,544.0	44,127.0	69,524.0	57,608.0	62,151.0	53,446.0
Robotank	11.9	63.9	65.1	65.3	62.6	27.5	16.5
Seaquest	42,054.7	5,860.6	16,452.7	50,254.2	26,357.8	931.6	1,776.0
Solaris	12,326.7	3,482.8	3,067.8	2,250.8	4,309.0	133.4	2,368.6
Space Invaders	1,668.7	1,692.3	2,525.5	6,427.3	2,865.8	15,311.5	19,723.0
Star Gunner	10,250.0	54,282.0	60,142.0	89,238.0	63,302.0	125,117.0	82,920.0
Time Pilot	5,229.2	4,870.0	8,339.0	11,666.0	9,197.0	7,553.0	22,286.0
Tutankham	167.6	68.1	218.4	211.4	204.6	245.9	314.3
Up and Down	11,693.2	9,989.9	22,972.2	44,939.6	16,154.1	33,879.1	436,665.8
Video Pinball	17,667.9	196,760.4	309,941.9	98,209.5	282,007.3	479,197.0	100,496.6
Wizard Of Wor	4,756.5	2,704.0	7,492.0	7,855.0	4,802.0	12,352.0	702.0
Yars' Revenge	54,576.9	18,098.9	11,712.6	49,622.1	11,357.0	69,618.1	125,169.0
Zaxxon	9,173.3	5,363.0	10,163.0	12,944.0	10,469.0	13,886.0	17,448.0

B Results on the remaining Atari games

In this section we present results on the rest of the Atari games in Table 4. The score reported for our method is the mean of the last 100 episode rewards after 50 million time steps. Each episode is started with 30 no-op actions. We find that there is no result reported in A3C [18] or A2C using the same metric. Hence we compare our results with other Q-learning methods obtained from [29]. Due to limited computational resources, we were only able to evaluate ACKTR on a subset of the games. Our results are obtained with a single random seed and we have not tuned any hyperparameters. Although we use only one random seed, our results are on par with Q-learning methods, which use off-policy techniques such as experience replay. Q-learning methods usually take days to finish one training, whereas our method takes only 16 hours on a modern GPU.

C MuJoCo results with comparisons to OpenAI baselines

We compared ACKTR with the results of A2C and TRPO sent to us by the OpenAI team (<https://github.com/openai/baselines-results>). We followed their experimental protocol as closely as possible. Like our baselines, A2C and TRPO were trained with the same two-layer architecture with 64 hidden units in each layer on batch sizes of 2500 and 25000 respectively. However, in contrast to our baselines, the value function used Tanh nonlinearities and was “softly” updated by calculating the weighted average of the value function before and after the update.

Compared to our implementation of the A2C baseline, A2C implemented by OpenAI performed better on the Hopper, InvertedPendulum, Swimmer, and Walker2d tasks while performing worse on the Reacher and HalfCheetah tasks. TRPO by OpenAI performed worse than the TRPO trained by us on Hopper while achieving the same performance on the rest of the tasks. Results are shown in Figure 6.

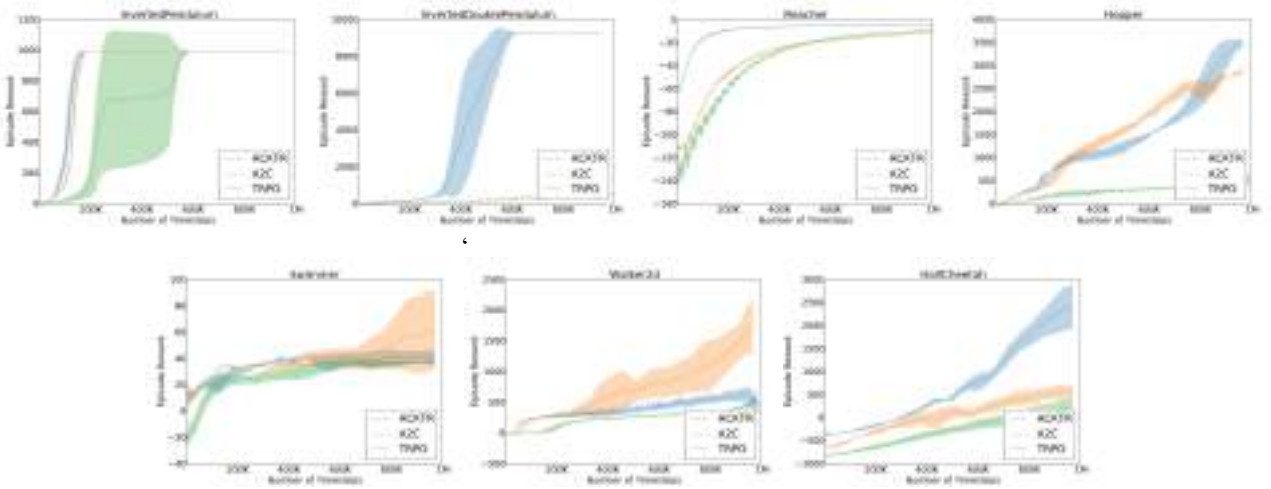


Figure 6: Performance comparisons on seven MuJoCo environments trained for 1 million timesteps (1 timestep equals 4 frames). The numbers for A2C and TRPO were provided to us by the OpenAI team. The shaded region denotes the standard deviation over 3 random seeds.

D Adaptive Gauss-Newton?

In this section we investigate whether training the critic using adaptive Gauss-Newton (i.e., keeping an estimate of the standard deviation of the Bellman error as the standard deviation of the critic output distribution) provides any improvement over vanilla Gauss-Newton (both defined in Section 3.1). We ran adaptive Gauss-Newton on all six standard Atari games and eight MuJoCo tasks. The results are shown in Figure 7 and Figure 8. We see that in Atari games, adaptive Gauss-Newton hurts the performance in terms of sample efficiency in Beamrider, Q-bert and Seaquest, and shows only a slight improvement in the game of Pong. In MuJoCo tasks, adaptive Gauss-Newton gives a slight improvement on the tasks of InvertedDoublePendulum, Swimmer, Walker2d, and Ant while performing on par on the tasks of InvertedPendulum and Reacher, and working considerably worse on the HalfCheetah task compared to vanilla Gauss-Newton.

E How well does the Kronecker-factored quadratic approximation match the exact KL?

We indirectly tested how accurate the Kronecker-factored approximation to the curvature is by measuring the exact KL changes during training, while performing trust region optimization using a Kronecker-factored quadratic model. We tested this in two Mujoco environments, HalfCheetah and Reacher. The values of approximated KL and exact KL are shown in Figure 9. From the plot we see

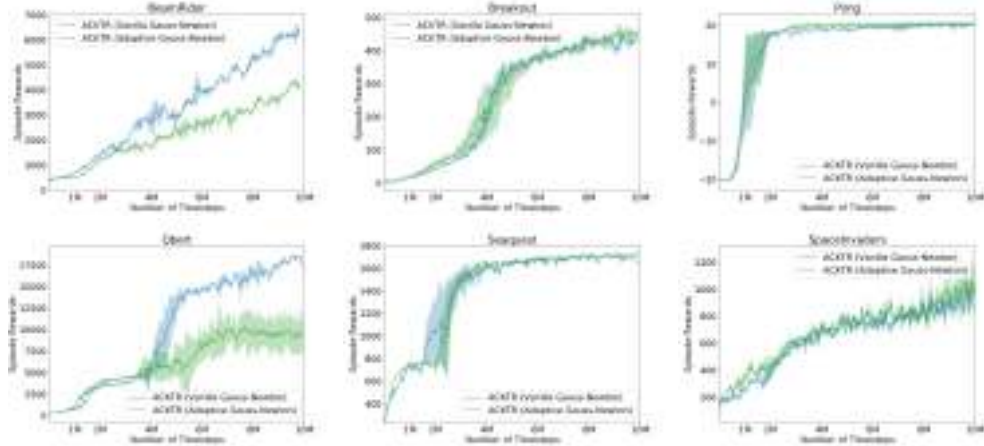


Figure 7: Performance comparisons of critic trained with adaptive Gauss-Newton and vanilla Gauss-Newton on six Atari environments trained for 10 million timesteps (1 timestep equals 4 frames). The shaded region denotes the standard deviation over 2 random seeds.

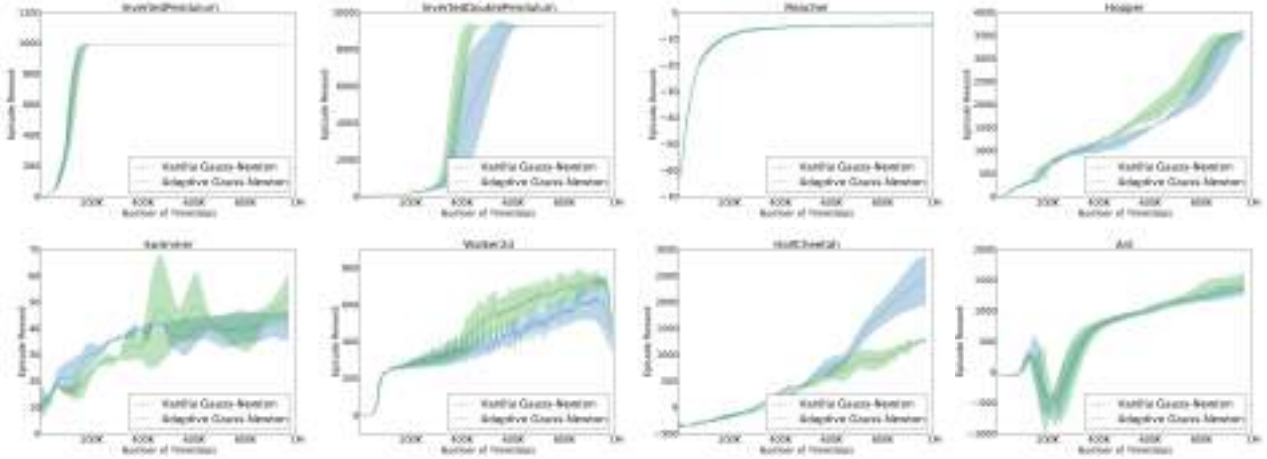


Figure 8: Performance comparisons of critic trained with adaptive Gauss-Newton and vanilla Gauss-Newton on eight MuJoCo environments trained for 1 million timesteps (1 timestep equals 4 frames). The shaded region denotes the standard deviation over 3 random seeds.

that exact KL is close to the trust region radius, showing the effectiveness of trust region optimization via Kronecker-factored approximation.

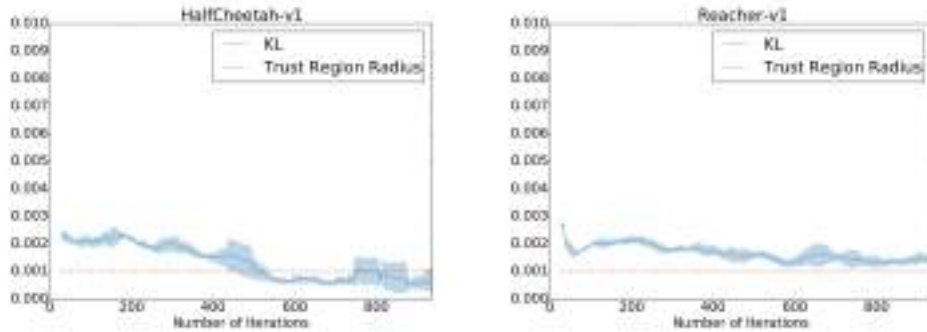


Figure 9: The plot shows the exact KL changes during training with trust region optimization using ACKTR. The actual KL is close to the trust region radius, showing the effectiveness of trust region optimization via Kronecker-factored approximation.

LEARNING TO REINFORCEMENT LEARN

JX Wang¹, Z Kurth-Nelson¹, D Tirumala¹, H Soyer¹, JZ Leibo¹,
R Munos¹, C Blundell¹, D Kumaran^{1,3}, M Botvinick^{1,2}

¹DeepMind, London, UK

²Gatsby Computational Neuroscience Unit, UCL, London, UK

³Institute of Cognitive Neuroscience, UCL, London, UK

{wangjane, zebk, dhruvat, soyer, jzl, munos, cblundell,
dkumaran, botvinick}@google.com

ABSTRACT

In recent years deep reinforcement learning (RL) systems have attained superhuman performance in a number of challenging task domains. However, a major limitation of such applications is their demand for massive amounts of training data. A critical present objective is thus to develop deep RL methods that can adapt rapidly to new tasks. In the present work we introduce a novel approach to this challenge, which we refer to as deep meta-reinforcement learning. Previous work has shown that recurrent networks can support meta-learning in a fully supervised context. We extend this approach to the RL setting. What emerges is a system that is trained using one RL algorithm, but whose recurrent dynamics implement a second, quite separate RL procedure. This second, learned RL algorithm can differ from the original one in arbitrary ways. Importantly, because it is learned, it is configured to exploit structure in the training domain. We unpack these points in a series of seven proof-of-concept experiments, each of which examines a key aspect of deep meta-RL. We consider prospects for extending and scaling up the approach, and also point out some potentially important implications for neuroscience.

1 INTRODUCTION

Recent advances have allowed long-standing methods for reinforcement learning (RL) to be newly extended to such complex and large-scale task environments as Atari (Mnih et al., 2015) and Go (Silver et al., 2016). The key enabling breakthrough has been the development of techniques allowing the stable integration of RL with non-linear function approximation through deep learning (LeCun et al., 2015; Mnih et al., 2015). The resulting deep RL methods are attaining human- and often superhuman-level performance in an expanding list of domains (Jaderberg et al., 2016; Mnih et al., 2015; Silver et al., 2016). However, there are at least two aspects of human performance that they starkly lack. First, deep RL typically requires a massive volume of training data, whereas human learners can attain reasonable performance on any of a wide range of tasks with comparatively little experience. Second, deep RL systems typically specialize on one restricted task domain, whereas human learners can flexibly adapt to changing task conditions. Recent critiques (e.g., Lake et al., 2016) have invoked these differences as posing a direct challenge to current deep RL research.

In the present work, we outline a framework for meeting these challenges, which we refer to as *deep meta-reinforcement learning*, a label that is intended to both link it with and distinguish it from previous work employing the term “meta-reinforcement learning” (e.g. Schmidhuber et al., 1996; Schweighofer and Doya, 2003, discussed later). The key concept is to use standard deep RL techniques to train a recurrent neural network in such a way that the recurrent network comes to implement its own, free-standing RL procedure. As we shall illustrate, under the right circumstances, the secondary learned RL procedure can display an adaptiveness and sample efficiency that the original RL procedure lacks.

The following sections review previous work employing recurrent neural networks in the context of meta-learning and describe the general approach for extending such methods to the RL setting. We

then present seven proof-of-concept experiments, each of which highlights an important ramification of the deep meta-RL setup by characterizing agent performance in light of this framework. We close with a discussion of key challenges for next-step research, as well as some potential implications for neuroscience.

2 METHODS

2.1 BACKGROUND: META-LEARNING IN RECURRENT NEURAL NETWORKS

Flexible, data-efficient learning naturally requires the operation of prior biases. In general terms, such biases can derive from two sources; they can either be engineered into the learning system (as, for example, in convolutional networks), or they can themselves be acquired through learning. The second case has been explored in the machine learning literature under the rubric of *meta-learning* (Schmidhuber et al., 1996; Thrun and Pratt, 1998).

In one standard setup, the learning agent is confronted with a series of tasks that differ from one another but also share some underlying set of regularities. Meta-learning is then defined as an effect whereby the agent improves its performance in each new task more rapidly, on average, than in past tasks (Thrun and Pratt, 1998). At an architectural level, meta-learning has generally been conceptualized as involving two learning systems: one lower-level system that learns relatively quickly, and which is primarily responsible for adapting to each new task; and a slower higher-level system that works across tasks to tune and improve the lower-level system.

A variety of methods have been pursued to implement this basic meta-learning setup, both within the deep learning community and beyond (Thrun and Pratt, 1998). Of particular relevance here is an approach introduced by Hochreiter and colleagues (Hochreiter et al., 2001), in which a recurrent neural network is trained on a series of interrelated tasks using standard backpropagation. A critical aspect of their setup is that the network receives, on each step within a task, an auxiliary input indicating the target output for the preceding step. For example, in a regression task, on each step the network receives as input an x value for which it is desired to output the corresponding y , but the network also receives an input disclosing the target y value for the preceding step (see Hochreiter et al., 2001; Santoro et al., 2016). In this scenario, a different function is used to generate the data in each training episode, but if the functions are all drawn from a single parametric family, then the system gradually tunes into this consistent structure, converging on accurate outputs more and more rapidly across episodes.

One interesting aspect of Hochreiter’s method is that the process that underlies learning within each new task inheres entirely in the dynamics of the recurrent network, rather than in the backpropagation procedure used to tune that network’s weights. Indeed, after an initial training period, the network can improve its performance on new tasks even if the weights are held constant (see also Cotter and Conwell, 1990; Prokhorov et al., 2002; Younger et al., 1999). A second important aspect of the approach is that the learning procedure implemented in the recurrent network is fit to the structure that spans the family of tasks on which the network is trained, embedding biases that allow it to learn efficiently when dealing with tasks from that family.

2.2 DEEP META-RL: DEFINITION AND KEY FEATURES

Importantly, Hochreiter’s original work (Hochreiter et al., 2001), as well as its subsequent extensions (Cotter and Conwell, 1990; Prokhorov et al., 2002; Santoro et al., 2016; Younger et al., 1999) only addressed supervised learning (i.e. the auxiliary input provided on each step explicitly indicated the target output on the previous step, and the network was trained using explicit targets). In the present work we consider the implications of applying the same approach in the context of reinforcement learning. Here, the tasks that make up the training series are interrelated RL problems, for example, a series of bandit problems varying only in their parameterization. Rather than presenting target outputs as auxiliary inputs, the agent receives inputs indicating the action output on the previous step and, critically, the quantity of reward resulting from that action. The same reward information is fed in parallel to a deep RL procedure, which tunes the weights of the recurrent network.

It is this setup, as well as its result, that we refer to as deep meta-RL (although from here on, for brevity, we will often simply call it meta-RL, with apologies to authors who have used that term

previously). As in the supervised case, when the approach is successful, the dynamics of the recurrent network come to implement a learning algorithm entirely separate from the one used to train the network weights. Once again, after sufficient training, learning can occur within each task even if the weights are held constant. However, here the procedure the recurrent network implements is itself a full-fledged reinforcement learning algorithm, which negotiates the exploration-exploitation tradeoff and improves the agent’s policy based on reward outcomes. A key point, which we will emphasize in what follows, is that this learned RL procedure can differ starkly from the algorithm used to train the network’s weights. In particular, its policy update procedure (including features such as the effective learning rate of that procedure), can differ dramatically from those involved in tuning the network weights, and the learned RL procedure can implement its own approach to exploration. Critically, as in the supervised case, the learned RL procedure will be fit to the statistics spanning the multi-task environment, allowing it to adapt rapidly to new task instances.

2.3 FORMALISM

Let us write as $\mathcal{D}$ a distribution (the prior) over Markov Decision Processes (MDPs). We want to demonstrate that meta-RL is able to learn a prior-dependent RL algorithm, in the sense that it will perform well on average on MDPs drawn from $\mathcal{D}$ or slight modifications of $\mathcal{D}$. An appropriately structured agent, embedding a recurrent neural network, is trained by interacting with a sequence of MDP environments (also called tasks) through episodes. At the start of a new episode, a new MDP task $m \sim \mathcal{D}$ and an initial state for this task are sampled, and the internal state of the agent (i.e., the pattern of activation over its recurrent units) is reset. The agent then executes its action-selection strategy in this environment for a certain number of discrete time-steps. At each step t an action $a_t \in A$ is executed as a function of the whole history $\mathcal{H}_t = \{x_0, a_0, r_0, \dots, x_{t-1}, a_{t-1}, r_{t-1}, x_t\}$ of the agent interacting in the MDP m during the current episode (set of states $\{x_s\}_{0 \leq s \leq t}$, actions $\{a_s\}_{0 \leq s < t}$, and rewards $\{r_s\}_{0 \leq s < t}$ observed since the beginning of the episode, when the recurrent unit was reset). The network weights are trained to maximize the sum of observed rewards over all steps and episodes.

After training, the agent’s policy is fixed (i.e. the weights are frozen, but the activations are changing due to input from the environment and the hidden state of the recurrent layer), and it is evaluated on a set of MDPs that are drawn either from the same distribution $\mathcal{D}$ or slight modifications of that distribution (to test the generalization capacity of the agent). The internal state is reset at the beginning of the evaluation of any new episode. Since the policy learned by the agent is history-dependent (as it makes uses of a recurrent network), when exposed to any new MDP environment, it is able to adapt and deploy a strategy that optimizes rewards for that task.

3 EXPERIMENTS

In order to evaluate the approach to learning that we have just described, we conducted a series of six proof-of-concept experiments, which we present here along with a seventh experiment originally reported in a related paper (Mirowski et al., 2016). One particular point of interest in these experiments was to see whether meta-RL could be used to learn an adaptive balance between exploration and exploitation, as demanded of any fully-fledged RL procedure. A second and still more important focus was on the question of whether meta-RL can give rise to learning that gains efficiency by capitalizing on task structure.

In order to examine these questions, we performed four experiments focusing on bandit tasks and two additional experiments focusing on Markov decision problems. All of our experiments (as well as the additional experiment we report) employ a common set of methods, with minor implementational variations. In all experiments, the agent architecture centers on a recurrent neural network (LSTM; Hochreiter and Schmidhuber, 1997) feeding into a soft-max output representing discrete actions. As detailed below, the parameters of this network core, as well as some other architectural details, varied across experiments (see Figure 1 and Table 1). However, it is important to emphasize that comparisons between specific architectures are outside the scope of this paper. Our main aim is to illustrate and validate the meta-RL framework in a more general way. To this end, all experiments used the high-level task setup previously described: Both training and testing were organized into fixed-length episodes, each involving a task randomly sampled from a predetermined task distribution, with the LSTM hidden state initialized at the beginning of each episode. Task-specific inputs and

Parameter	Exps. 1 & 2	Exp. 3	Exp. 4	Exp. 5	Exp. 6
No. threads	1	1	1	1	32
No. LSTMs	1	1	1	1	2
No. hiddens	48	48	48	48	256/64
Steps unrolled	100	5	150	20	100
β_e	<i>annealed</i>	<i>annealed</i>	<i>annealed</i>	0.05	0.001
β_v	0.05	0.05	0.05	0.05	0.4
Learning rate	<i>tuned</i>	0.001	0.001	<i>tuned</i>	<i>tuned</i>
Discount factor	<i>tuned</i>	0.8	0.8	<i>tuned</i>	<i>tuned</i>
Input	a, r, t	a, r, t	a, r, t	a, r, t, x	a, r, x
Observation	n/a	n/a	n/a	1-hot	RGB (84x84)
No. trials/episode	100	5	150	10	10
Episode length	100	5	150	20	≤ 3600

Table 1: List of hyperparameters. β_e = coefficient of entropy regularization loss; in Exps. 1-4, β_e is annealed from 1.0 to 0.0 over the course of training. β_v = coefficient of value function loss (Mirowski et al., 2016). r = reward, a = last action, t = current time step, x = current observation. Exp. 1: Bandits with independent arms (Section 3.1.1); Exp. 2: Bandits with dependent arms I (Section 3.1.2); Exp. 3: Bandits with dependent arms II (Section 3.1.3); Exp. 4: Restless bandits (Section 3.1.4); Exp. 5: The “Two-Step Task” (Section 3.2.1); Exp. 6: Learning abstract task structure (Section 3.2.2).

action outputs are described in conjunction with individual experiments. In all experiments except where specified, the input included a scalar indicating the reward received on the preceding time-step as well as a one-hot representation of the action sampled on that time-step.

All reinforcement learning was conducted using the Advantage Actor-Critic algorithm, as detailed in Mnih et al. (2016) and Mirowski et al. (2016) (see also Figure 1). Details of training, including the use of entropy regularization and a combined policy and value estimate loss, closely follow the methods detailed in Mirowski et al. (2016), with the exception that our experiments used a single thread unless otherwise noted. For a full listing of parameters refer to Table 1.

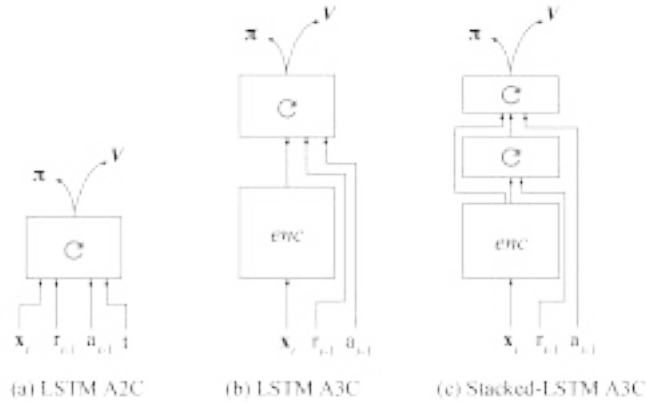


Figure 1: Advantage actor-critic with recurrence. In all architectures, reward and last action are additional inputs to the LSTM. For non-bandit environments, observation is also fed into the LSTM either as a one-hot or passed through an encoder model [3-layer encoder: two convolutional layers (first layer: 16 8x8 filters applied with stride 4, second layer: 32 4x4 filters with stride 2) followed by a fully connected layer with 256 units and then a ReLU non-linearity. See for details Mirowski et al., (2016)]. For bandit experiments, current time step is also fed in as input. π = policy; v = value function. A3C is the distributed multi-threaded asynchronous version of the advantage actor-critic algorithm (Mnih et al., 2016); A2C is single threaded. (a) Architecture used in experiments 1-5. (b) Convolutional-LSTM architecture used in experiment 6. (c) Stacked LSTM architecture with convolutional encoder used in experiments 6 and 7.

3.1 BANDIT PROBLEMS

As an initial setting for evaluating meta-RL, we studied a series of bandit problems. Except for a very limited set of bandit environments, it is intractable to compute the (prior-dependent) Bayesian-optimal strategy. Here we demonstrate that a recurrent system trained on a set of bandit environments drawn i.i.d. from a given distribution of environments produces a bandit algorithm which performs well on problems drawn from that distribution, and to a certain extent generalizes to related distributions. Thus, meta-RL learns a prior-dependent bandit algorithm.

The specific bandit instantiation of the general meta-RL procedure described in Section 2.3 is defined as follows. Let $\mathcal{D}$ be a *training distribution* over bandit environments. The meta-RL system is trained on a sequence of bandit environments through episodes. At the start of a new episode, its LSTM state is reset and a bandit task $b \sim \mathcal{D}$ is sampled. A bandit task is defined as a set of distributions – one for each arm – from which rewards are sampled. The agent plays in this bandit environment for a certain number of trials and is trained to maximize observed rewards. After training, the agent’s policy is evaluated on a set of bandit tasks that are drawn from a *test distribution* $\mathcal{D}'$, which can either be the same as $\mathcal{D}$ or a slight modification of it.

We evaluate the resulting performance of the learned bandit algorithm by the cumulative regret, a measure of the loss (in expected rewards) suffered when playing sub-optimal arms. Writing $\mu_a(b)$ the expected reward of arm a in bandit environment b , and $\mu^*(b) = \max_a \mu_a(b) = \mu_{a^*(b)}(b)$ (where $a^*(b)$ is one optimal arm) the optimal expected reward, we define the cumulative regret (in environment b) as $R_T(b) = \sum_{t=1}^T \mu^*(b) - \mu_{a_t}(b)$, where a_t is the arm (action) chosen at time t . In experiment 4 (Restless bandits; Section 3.1.4), μ^* also depends on t . We report the performance (average over bandit environments drawn from the test distribution) either in terms of the cumulative regret: $\mathbb{E}_{b \sim \mathcal{D}'} [R_T(b)]$ or in terms of number of sub-optimal pulls: $\mathbb{E}_{b \sim \mathcal{D}'} [\sum_{t=1}^T \mathbb{I}\{a_t \neq a^*(b)\}]$.

3.1.1 BANDITS WITH INDEPENDENT ARMS

We first consider a simple two-armed bandit task to examine the behavior of meta-RL under conditions where theoretical guarantees exist and general purpose algorithms apply. The arm distributions are independent Bernoulli distributions (rewards are 1 with probability p and 0 with probability $1 - p$), where the parameters of each arm (p_1 and p_2) are sampled independently and uniformly over $[0, 1]$. We denote by $\mathcal{D}_i$ the corresponding distribution over these independent bandit environments (where the subscript i stands for independent arms).

At the beginning of each episode, a new bandit task is sampled and held constant for 100 trials. Training lasted for 20,000 episodes. The network is given as input the last reward, last action taken, and the trial number t , subsequently producing the action for the next trial $t + 1$ (Figure 1). After training, we evaluated on 300 new episodes with the learning rate set to zero (the learned policy is fixed).

Across model instances, we randomly sampled learning rate and discount, following Mnih et al. (2016). For all figures, we plotted the average of the top 5 runs of 100 randomly sampled hyperparameter settings, where the top agents were selected from the first half of the 300 evaluation episodes and performance was plotted for the second half. We measured the cumulative expected regret across the episode, comparing with several algorithms tailored for this independent bandit setting: Gittins indices (Gittins, 1979) (which is Bayesian optimal in the finite-horizon case), UCB (Auer et al., 2002) (which comes with theoretical finite-time regret guarantees), and Thompson sampling (Thompson, 1933) (which is asymptotically optimal in this setting: see Kaufmann et al., 2012b). Model simulations were conducted with the PymaBandits toolbox from (Kaufmann et al., 2012a) and custom Matlab scripts.

As shown in Figure 2a (green line; “Independent”), meta-RL outperforms both Thompson sampling (gray dashed line) and UCB (light gray dashed line), although it performs less well compared to Gittins (black dashed line). To verify the critical importance of providing reward information to the LSTM, we removed this input, leaving all other inputs as before. As expected, performance was at chance levels on all bandit tasks.

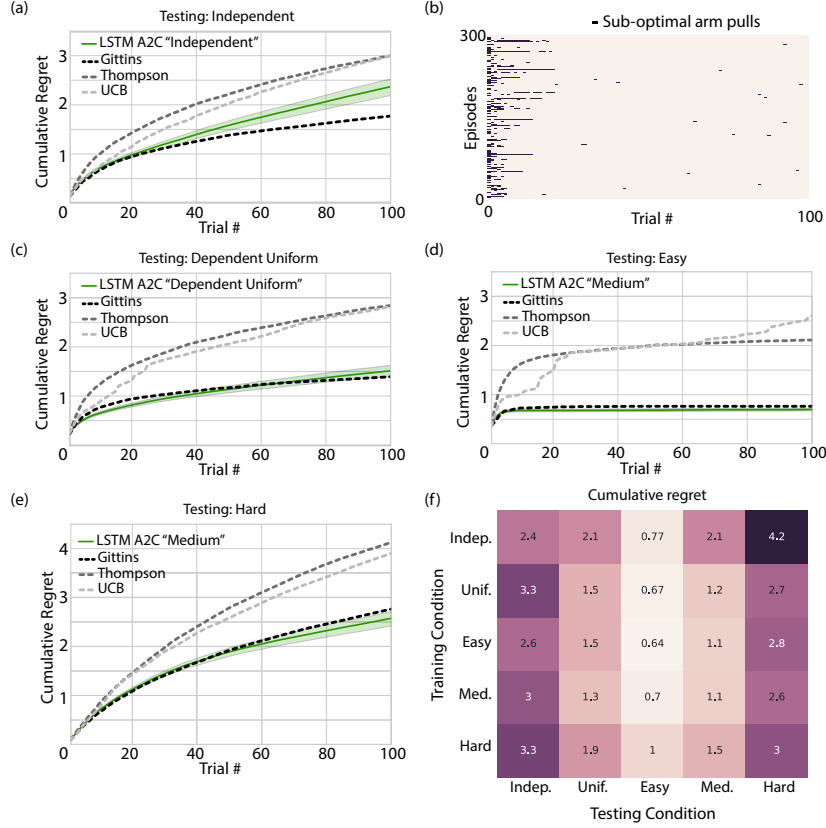


Figure 2: Performance on independent- and correlated-arm bandits. We report performance as the cumulative expected regret R_T for 150 test episodes, averaged over the top 5 hyperparameters for each agent-task configuration, where the top 5 was determined based on performance on a separate set of 150 test episodes. (a) LSTM A2C trained and evaluated on bandits with independent arms (distribution $\mathcal{D}_i$; see text), and compared with theoretically optimal models. (b) A single agent playing the medium difficulty task with distribution $\mathcal{D}_m$. Suboptimal arm pulls over trials are depicted for 300 episodes. (c) LSTM A2C trained and evaluated on bandits with dependent uniform arms (distribution $\mathcal{D}_u$), (d) trained on medium bandit tasks ($\mathcal{D}_m$) and tested on easy ($\mathcal{D}_e$), and (e) trained on medium ($\mathcal{D}_m$) and tested on hard task ($\mathcal{D}_h$). (f) Cumulative regret for all possible combinations of training and testing environments ($\mathcal{D}_i, \mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m, \mathcal{D}_h$).

3.1.2 BANDITS WITH DEPENDENT ARMS (I)

As we have emphasized, a key property of meta-RL is that it gives rise to a learned RL algorithm that exploits consistent structure in the training distribution. In order to garner empirical evidence for this point, we tested the agent from our first distribution in a more structured bandit task. Specifically, we trained the system on two-arm bandits in which arm reward distributions are correlated. In this setting, unlike the one studied in the previous section, experience with either arm provides information about the other. Standard bandit algorithms, including UCB and Thompson sampling, perform suboptimally in this setting, as they are not designed to exploit such correlations. In some cases it is possible to tailor algorithms for specific arm structures (see for example [Lattimore and Munos, 2014](#)), but extensive problem-specific analysis is typically required. Our approach aims to learn a structure-dependent bandit algorithm directly from experience with the target bandit domain.

We consider Bernoulli distributions where the parameters (p_1, p_2) of the two arms are correlated in the sense that $p_1 = 1 - p_2$. We consider several training and test distributions. The *uniform* means that $p_1 \sim \mathcal{U}([0, 1])$ (uniform distribution over the unit interval). The *easy* means that $p_1 \sim \mathcal{U}(\{0.1, 0.9\})$ (uniform distribution over those two possible values), and similarly we call *medium* when $p_1 \sim \mathcal{U}(\{0.25, 0.75\})$ and *hard* when $p_1 \sim \mathcal{U}(\{0.4, 0.6\})$. We denote by $\mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m$, and $\mathcal{D}_h$ the corresponding induced distributions over bandit environments. In addition

we also considered the independent uniform distribution (as in the previous section, $\mathcal{D}_i$) where $p_1, p_2 \sim \mathcal{U}([0, 1])$ independently. Agents were both trained and tested on those five distributions over bandit environments (among which four correspond to correlated distributions: $\mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m$ and $\mathcal{D}_h$; and one to the independent case: $\mathcal{D}_i$). As a validation of the names given to the task distributions ($\mathcal{D}_e, \mathcal{D}_m, \mathcal{D}_h$), results show that the easy task is easier to learn than the medium which itself is easier than the hard one (Figure 2f). This is compatible with the general notion that the hardness of a bandit problem is inversely proportional to the difference between the expected reward of the optimal and sub-optimal arms. We again note that withholding the reward input to the LSTM resulted in chance performance on even the easiest bandit task, as should be expected.

Figure 2f reports the results of all possible training-testing regimes. From observing the cumulative expected regrets, we make the following observations: i) agents trained in structured environments ($\mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m$, and $\mathcal{D}_h$) develop prior knowledge that can be used effectively when tested on structured distributions – performing comparably to Gittins (Figure 2c-f), and superiorly compared to agents trained on independent arms ($\mathcal{D}_i$) in all structured tasks at test (Figure 2f). This is because an agent trained on independent rewards ($\mathcal{D}_i$) has not learned to exploit the reward correlations that are useful in those structured tasks. ii) Conversely, previous training on any structured distribution ($\mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m$, or $\mathcal{D}_h$) hurts performance when agents are tested on an independent distribution ($\mathcal{D}_i$; Figure 2f). This makes sense, as training on correlated arms may produce a policy that relies on specific reward structure, thereby impacting performance in problems where no such structure exists. iii) Whilst the previous results emphasize the point that meta-RL gives rise to a separate learnt RL algorithm that implements prior-dependent bandit strategies, results also provide evidence that there is some generalization beyond the exact training distribution encountered (Figure 2f). For example, agents trained on the distributions $\mathcal{D}_e$ and $\mathcal{D}_m$ perform well when tested over a much wider structured distribution (i.e. $\mathcal{D}_u$). Further, our evidence suggests that there is generalization from training on the easier tasks ($\mathcal{D}_e, \mathcal{D}_m$) to testing on the hardest task ($\mathcal{D}_h$; Figure 2e), with similar or even marginally superior performance as compared to training on the hard distribution $\mathcal{D}_h$ itself (Figure 2f). In contrast, training on the hard distribution $\mathcal{D}_h$ results in relatively poor generalization to other structured distributions ($\mathcal{D}_u, \mathcal{D}_e, \mathcal{D}_m$), suggesting that training purely on hard instances may result in a learned RL algorithm that is more constrained by prior knowledge, perhaps due to the difficulty of solving the original problem.

3.1.3 BANDITS WITH DEPENDENT ARMS (II)

In the previous experiment, the agent could outperform standard bandit algorithms by making use of learned dependencies between arms. However, it could do this while always choosing what it believes to be the highest-paying arm. We next examine a problem where information can be gained by paying a short-term reward cost. Similar problems have been examined before as providing a challenge to standard bandit algorithms (see e.g. [Russo and Van Roy, 2014](#)). In contrast, humans and animals make decisions that sacrifice immediate reward for information gain (e.g. [Bromberg-Martin and Hikosaka, 2009](#)).

In this experiment, the agent was trained on 11-armed bandits with strong dependencies between arms. All arms had deterministic payouts. Nine “non-target” arms had reward = 1, and one “target” arm had reward = 5. Meanwhile, arm a_{11} was always “informative”, in that the target arm was indexed by 10 times a_{11} ’s reward (e.g. a reward of 0.2 on a_{11} indicated that a_2 was the target arm). Thus, a_{11} ’s payouts ranged from 0.1 to 1. In each episode, the index of the target arm was randomly assigned. On the first trial of each episode, the agent could not know which arm was the target, so the informative arm returned expected reward 0.55 and every target arm returned expected reward 1.4. Choosing the informative arm thus meant foregoing immediate reward, but with the compensation of valuable information. Episodes were five steps long. Again, the reward on the previous trial was provided as an additional observation to the agent. To facilitate learning, this was encoded in 1-hot format.

Results are shown in Figure 3. The agent learned the optimal long-run strategy of sampling the informative arm once, despite the short-term cost, and then using the resulting information to exploit the high-value target arm. Thompson sampling, if supplied the true prior, searched potential target arms and exploited the target if found. UCB performed worse because it sampled every arm once even if the target arm was found early.

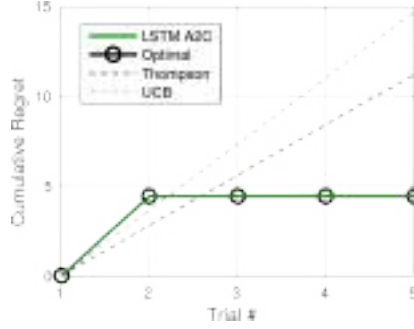


Figure 3: Learned RL procedure pays immediate cost to gain information to improve long-run returns. In this task, one arm is lower-paying but provides perfect information about which of the other ten arms is highest-paying. The remaining nine arms are intermediate in reward. The index of the informative arm is fixed between episodes, but the index of the highest-paying arm is randomized between episodes. On the first trial, the trained agent samples the informative arm. On subsequent trials, the agent uses the information it gained to deterministically exploit the highest-paying arm. Thompson sampling and UCB are not able to take advantage of the dependencies between arms.

3.1.4 RESTLESS BANDITS

In previous experiments we considered stationary problems where the agent’s actions yielded information about task parameters that remained fixed throughout each episode. Next, we consider a bandit problem in which reward probabilities change over the course of an episode, with different rates of change (volatilities) in different episodes. To perform well, the agent must not only track the best arm, but also infer the volatility of the episode and adjust its own learning rate accordingly. In such an environment, learning rates should be higher when the environment is changing rapidly, because past information becomes irrelevant more quickly (Behrens et al., 2007; Sutton and Barto, 1998).

We tested whether meta-RL would learn such a flexible RL policy using a two-armed Bernoulli bandit task with reward probabilities p_1 and $1-p_1$. The value of p_1 changed slowly in “low vol” episodes and quickly in “high vol” episodes. The agent had no way of knowing which type of episode it was in, except for its reward history within the episode. Figure 4a shows example “low vol” and “high vol” episodes. Reward magnitude was fixed at 1, and episodes were 100 steps long. UCB and Thompson sampling were again implemented for comparison. The confidence bound term $\frac{\chi \log n}{n_i}$ in UCB had parameter χ which was set to 1, selected empirically for good performance on our data set. Thompson sampling’s posterior update included knowledge of the Gaussian random walk, but with a fixed volatility for all episodes.

As in the previous experiment, meta-RL achieved lower regret in test than Thompson sampling, UCB, or the Rescorla-Wagner (R-W) learning rule (Figure 4b; Rescorla et al., 1972) with the best fixed learning rate ($\alpha=0.5$). To test whether the agent adjusted its effective learning rate to match environments with different volatility levels, we fit R-W models to the agent’s behavior, concatenating episodes into blocks of 10, where each block consisted of only “low vol” or only “high vol” episodes. We considered four different models encompassing different combinations of three parameters: learning rate α , softmax inverse temperature β , and a lapse rate ϵ to account for unexplained choice variance not related to estimated value (Economides et al., 2015). Model “b” included only β , “ab” included α and β , “be” included β and ϵ , and “abe” included all three. All parameters were estimated separately on each block of 10 episodes. In models where ϵ and α were not free, they were fixed to 0 and 0.5, respectively. Model comparison by Bayesian Information Criterion (BIC) indicated that meta-RL’s behavior was better described by a model with different learning rates for each block than a model with a fixed learning rate across blocks. As a control, we performed the same model comparison on the behavior produced by the best R-W agent, finding no benefit of allowing different learning rates across episodes (models “abe” and “ab” vs “be” and “b”; Figure 4c-d). In these models, the parameter estimates for meta-RL’s behavior were strongly related to the volatility of the episodes, indicating that meta-RL adjusted its learning rate to the volatility of the episode, whereas model fitting the R-W behavior simply recovered the fixed parameters (Figure 4e-f).

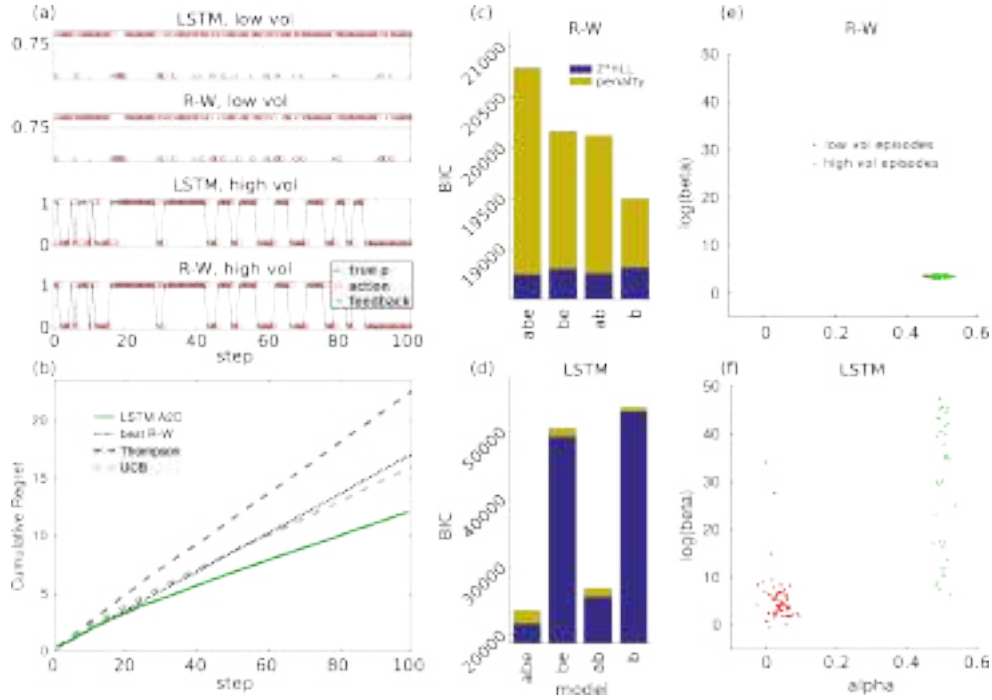


Figure 4: Learned RL procedure adapts its own learning rate to the environment. (a) Agents were trained on two-armed bandits with perfectly anti-correlated Bernoulli reward probabilities, p_1 and $1-p_1$. Two example episodes are shown. p_1 changed within an episode (solid black line), with a fast Poisson jump rate in “high vol” episodes and a slow rate in “low vol” episodes. (b) The trained LSTM agent outperformed UCB, Thompson sampling, and a Rescorla-Wagner (R-W) learner with fixed learning rate $\alpha=0.5$ (selected for being optimal on average in this distribution of environments). (c,d) We fit R-W models by maximum likelihood both to the behavior of R-W (as a control) and to the behavior of LSTM. Models including a learning rate that could vary between episodes (“ab” and “abe”) outperformed models without these free parameters on LSTM’s data, but not on R-W’s data. Addition of a lapse parameter further improved model fits on LSTM’s data (“be” and “abe”), suggesting that the algorithm implemented by LSTM is not exactly Rescorla-Wagner. (e,f) The LSTM’s, but not R-W’s, estimated learning rate was higher in volatile episodes. Small jitter added to visualize overlapping points.

3.2 MARKOV DECISION PROBLEMS

The foregoing experiments focused on bandit tasks in which actions do not affect the task’s underlying state. We turn now to MDPs where actions do influence state. We begin with a task derived from the neuroscience literature and then turn to a task, originally studied in the context of animal learning, which requires learning of abstract task structure. As in the previous experiments, our focus is on examining how meta-RL adapts to invariances in task structure. We wrap up by reviewing an experiment recently reported in a related paper (Mirowski et al., 2016), which demonstrates how meta-RL can scale to large-scale navigation tasks with rich visual inputs.

3.2.1 THE “TWO-STEP TASK”

Here we examine meta-RL in a setting that has been widely used in the neuroscience literature to distinguish the contribution of different systems viewed to support decision making (Daw et al., 2005). Specifically, this paradigm – known as the “two-step task” (Daw et al., 2011) – was developed to dissociate a model-free system that caches values of actions in states (e.g. TD(1) Q-learning; see Sutton and Barto, 1998), from a model-based system which learns an internal model of the environment and evaluates the value of actions at the time of decision-making through look-ahead planning (Daw et al., 2005). Our interest was in whether meta-RL would give rise to behavior emulating a model-based strategy, despite the use of a model-free algorithm (in this case A2C) to train the system weights.

We used a modified version of the two-step task, designed to bolster the utility of model-based over model-free control (see [Kool et al., 2016](#)). The task’s structure is diagrammed in Figure [5a](#). From the first-stage state S_1 , action a_1 leads to second-stage states S_2 and S_3 with probability 0.75 and 0.25, respectively, while action a_2 leads to S_2 and S_3 with probabilities 0.25 and 0.75. One second-stage state yielded a reward of 1.0 with probability 0.9 (and otherwise zero); the other yielded the same reward with probability 0.1. The identity of the higher-valued state was assigned randomly for each episode. Thus, the expected values for the two first-stage actions were either $r_a = 0.9$ and $r_b = 0.1$, or $r_a = 0.1$ and $r_b = 0.9$. All three states were represented by one-hot vectors, with the transition model held constant across episodes: i.e. only the expected value of the second stage states changed from episode to episode.

We applied the conventional analysis used in the neuroscience literature to dissociate model-free from model-based control ([Daw et al., 2011](#)). This focuses on the “stay probability,” that is, the probability with which a first-stage action is selected at trial $t + 1$ following a second-stage reward at trial t , as a function of whether trial t involved a common transition (e.g. action a_1 at state S_1 led to S_2) or rare transition (action a_2 at state S_1 led to S_3). Under the standard interpretation (see [Daw et al., 2011](#)), model-free control – *à la* TD(1) – predicts that there should be a main effect of reward: First-stage actions will tend to be repeated if followed by reward, regardless of transition type, and such actions will tend not to be repeated (choice switch) if followed by non-reward (Figure [5b](#)). In contrast, model-based control predicts an interaction between the reward and transition type, reflecting a more goal-directed strategy, which takes the transition structure into account. Intuitively, if you receive a second-stage reward (e.g. at S_2) following a rare transition (i.e. having taken action a_2 at state S_1), to maximize your chances of getting to this reward on the next trial based on your knowledge of the transition structure, the optimal first stage action is a_1 (i.e. switch).

The results of the stay-probability analysis performed on the agent’s choices show a pattern conventionally interpreted as implying the operation of model-based control (Figure [5c](#)). As in previous experiments, when reward information was withheld at the level of network input, performance was at chance levels.

If interpreted following standard practice in neuroscience, the behavior of the model in this experiment reflects a surprising effect: training with model-free RL gives rise to behavior reflecting model-based control. We hasten to note that different interpretations of the observed pattern of behavior are available ([Akam et al., 2015](#)), a point to which we will return below. However, notwithstanding this caveat, the results of the present experiment provide a further illustration of the point that the learning procedure that emerges from meta-RL can differ starkly from the original RL algorithm used to train the network weights, and takes a form that exploits consistent task structure.

3.2.2 LEARNING ABSTRACT TASK STRUCTURE

In the final experiment we conducted, we took a step towards examining the scalability of meta-RL, by studying a task that involves rich visual inputs, longer time horizons and sparse rewards. Additionally, in this experiment we studied a meta-learning task that requires the system to tune into an abstract task structure, in which a series of objects play defined roles which the system must infer.

The task was adapted from a classic study of animal behavior, conducted by [Harlow \(1949\)](#). On each trial in the original task, Harlow presented a monkey with two visually contrasting objects. One of these covered a small well containing a morsel of food; the other covered an empty well. The animal chose freely between the two objects and could retrieve the food reward if present. The stage was then hidden and the left-right positions of the objects were randomly reset. A new trial then began, with the animal again choosing freely. This process continued for a set number of trials using the same two objects. At completion of this set of trials, two entirely new and unfamiliar objects were substituted for the original two, and the process began again. Importantly, within each block of trials, one object was chosen to be consistently rewarded (regardless of its left-right position), with the other being consistently unrewarded. What Harlow ([Harlow, 1949](#)) observed was that, after substantial practice, monkeys displayed behavior that reflected an understanding of the task’s rules. When two new objects were presented, the monkey’s first choice between them was necessarily arbitrary. But after observing the outcome of this first choice, the monkey was at ceiling thereafter, always choosing the rewarded object.

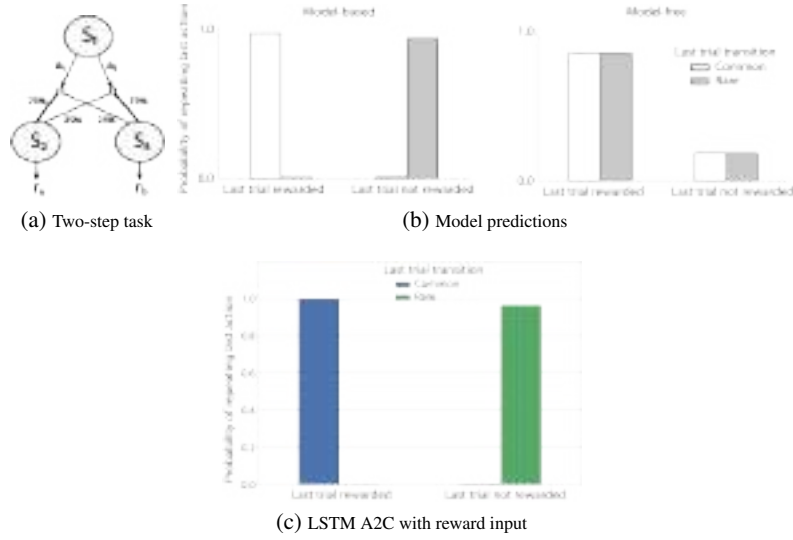


Figure 5: Three-state MDP modeled after the “two-step task” from Daw et al. (2011). (a) MDP with 3 states and 2 actions. All trials start in state S_1 , with transition probabilities after taking actions a_1 or a_2 depicted in the graph. S_2 and S_3 result in expected rewards r_a and r_b (see text). (b) Predictions of choice probabilities given either a model-based strategy or a model-free strategy (Daw et al. 2011). Specifically, model-based strategies take into account transition probabilities and would predict an interaction between the amount of reward received on the last trial and the transition (common or uncommon) observed. (c) Agent displays a perfectly model-based profile when given the reward as input.

We anticipated that meta-RL should give rise to the same pattern of abstract one-shot learning. In order to test this, we adapted Harlow’s paradigm into a visual fixation task, as follows. A 84x84 pixel input represented a simulated computer screen (see Figure 6a-c). At the beginning of each trial, this display was blank except for a small central fixation cross (red crosshairs). The agent selected discrete left-right actions which shifted its view approximately 4.4 degrees in the corresponding direction, with a small momentum effect (alternatively, a no-op action could be selected). The completion of a trial required performing two tasks: saccading to the central fixation cross, followed by saccading to the correct image. If the agent held the fixation cross in the center of the field of view (within a tolerance of 3.5 degrees visual angle) for a minimum of four time steps, it received a reward of 0.2. The fixation cross then disappeared and two images – drawn randomly from the ImageNet dataset (Deng et al., 2009) and resized to 34x34 – appeared on the left and right side of the display (Figure 6b). The agent’s task was then to “select” one of the images by rotating until the center of the image aligned with the center of the visual field of view (within a tolerance of 7 degrees visual angle). Once one of the images was selected, both images disappeared and, after an intertrial interval of 10 time-steps, the fixation cross reappeared, initiating the next trial. Each episode contained a maximum of 10 trials or 3600 steps. Following Mirowski et al. (2016), we implemented an action repeat of 4, meaning that selecting an image took a minimum of three independent decisions (twelve primitive actions) after having completed the fixation. It should be noted, however, that the rotational position of the agent was not limited; that is, 360 degree rotations could occur, while the simulated computer screen only subtended 65 degrees.

Although new ImageNet images were chosen at the beginning of each episode (sampled with replacement from a set of 1000 images), the same images were re-used across all trials within an episode, though in randomly varying left-right placement, similar to the objects in Harlow’s experiment. And as in that experiment, one image was arbitrarily chosen to be the “rewarded” image throughout the episode. Selection of this image yielded a reward of 1.0, while the other image yielded a reward of -1.0. During test, the A3C learning rate was set to zero and ImageNet images were drawn from a separate held-out set of 1000, never presented during training.

A grid search was conducted for optimal hyperparameters. At perfect performance, agents can complete one trial per 20-30 steps and achieve a maximum expected reward of 9 per 10 trials. Given

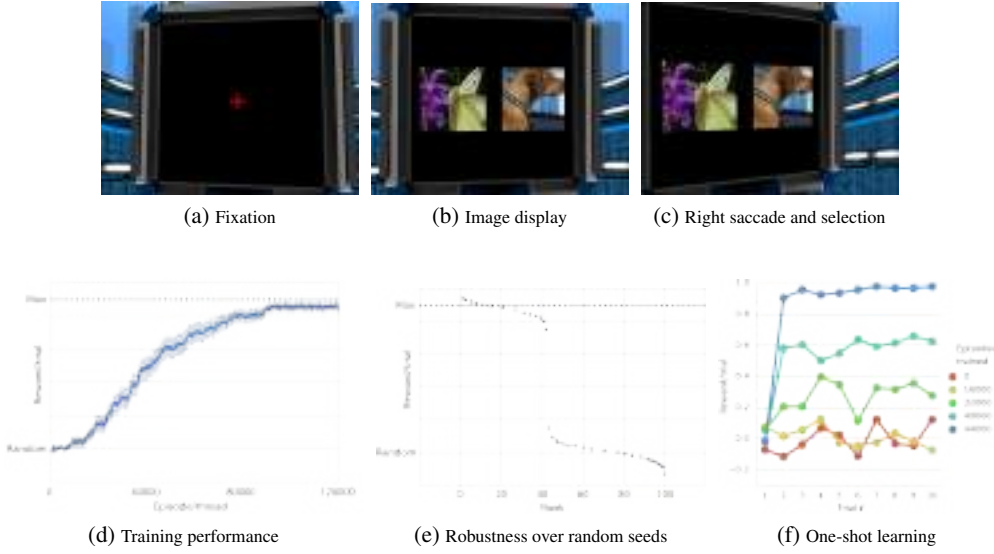


Figure 6: Learning abstract task structure in visually rich 3D environment. a-c) Example of a single trial, beginning with a central fixation, followed by two images with random left-right placement. d) Average performance (measured in average reward per trial) of top 40 out of 100 seeds during training. Maximum expected performance is indicated with black dashed line. e) Performance at episode 100,000 for 100 random seeds, in decreasing order of performance. f) Probability of selecting the rewarded image, as a function of trial number for a single A3C stacked LSTM agent for a range of training durations (episodes per thread, 32 threads).

the nature of the task – which requires one-shot image-reward memory together with maintenance of this information over a relatively long timescale (i.e. over fixation-cross selections and across trials) – we assessed the performance of not only a convolutional-LSTM architecture which receives reward and action as additional input (see Figure 1b and Table 1), but also a convolutional-stacked LSTM architecture used in a navigation task discussed below (see Figure 1c).

Agent performance is illustrated in Figure 6d-f. Whilst the single LSTM agent was relatively successful at solving the task, the stacked-LSTM variant exhibited much better robustness. That is, 43% of random seeds of the best hyperparameter set performed at ceiling (Figure 6e), compared to 26% of the single LSTM.

Like the monkeys in Harlow’s experiment (Harlow, 1949), the networks converge on an optimal policy: Not only does the agent successfully fixate to begin each trial, but starting on the second trial of each episode it invariably selects the rewarded image, regardless of which image it selected on the first trial (Figure 6f). This reflects an impressive form of one-shot learning, which reflects an implicit understanding of the task structure: After observing one trial outcome, the agent binds a complex, unfamiliar image to a specific task role.

Further experiments, reported elsewhere (Wang et al., 2017), confirmed that the same recurrent A3C system is also able to solve a substantially more difficult version of the task. In this task, only one image – which was randomly designated to be either the rewarding item to be selected, or the unrewarding item to be avoided – was presented on every trial during an episode, with the other image presented being novel on every trial.

3.2.3 ONE-SHOT NAVIGATION

The experiments using the Harlow task demonstrate the capacity of meta-RL to operate effectively within a visually rich environment, with relatively long time horizons. Here we consider related experiments recently reported within the navigation domain (Mirowski et al., 2016) (see also Jaderberg et al., 2016), and discuss how these can be recast as examples of meta-RL – attesting to the scalability of this principle to more typical MDP settings that pose challenging RL problems due to dynamically changing sparse rewards.

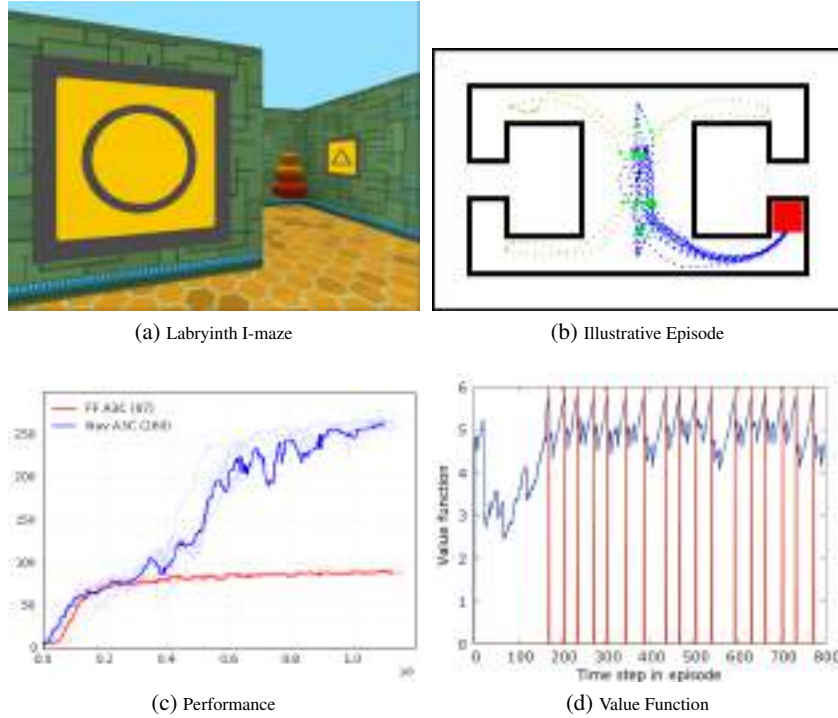


Figure 7: a) view of I-maze showing goal object in one of the 4 alcoves b) following initial exploration (light trajectories), agent repeatedly goes to goal (blue trajectories) c) Performance of stacked LSTM (termed “Nav A3C”) and feedforward (“FF A3C”) architectures, per episode (goal = 10 points) averaged across top 5 hyperparameters. e) following initial goal discovery (goal hits marked in red), value function occurs well in advance of the agent seeing the goal which is hidden in an alcove. Figure used with permission from [Mirowski et al. \(2016\)](#).

Specifically, we consider a setting where the environment layout is fixed but the goal changes location randomly each episode (Figure 7; [Mirowski et al., 2016](#)). Although the layout is relatively simple, the Labyrinth environment (see for details [Mirowski et al., 2016](#)) is richer and more finely discretized (cf VizDoom), resulting in long time horizons; a trained agent takes approximately 100 steps (10 seconds) to reach the goal for the first time in a given episode. Results show that a stacked LSTM architecture (Figure 1c), that receives reward and action as additional inputs equivalent to that used in our Harlow experiment achieves near-optimal behavior – showing one-shot memory for the goal location after an initial exploratory period, followed by repeated exploitation (see Figure 7c). This is evidenced by a substantial decrease in latency to reach the goal for the first time (~100 timesteps) compared to subsequent visits (~30 timesteps). Notably, a feedforward network (see Figure 7c), that receives only a single image as observation, is unable to solve the task (i.e. no decrease in latency between successive goal rewards). Whilst not interpreted as such in [Mirowski et al. \(2016\)](#), this provides a clear demonstration of the effectiveness of meta-RL: a separate RL algorithm with the capability of one-shot learning emerges through training with a fixed and more incremental RL algorithm (i.e. policy gradient). Meta-RL can be viewed as allowing the agent to infer the optimal value function following initial exploration (see Figure 7d) – with the additional LSTM providing information about the currently relevant goal location to the LSTM that outputs the policy over the extended timeframe of the episode. Taken together, meta-RL allows a base model-free RL algorithm to solve a challenging RL problem that might otherwise require fundamentally different approaches (e.g. based on successor representations or fully model-based RL).

4 RELATED WORK

We have already touched on the relationship between deep meta-RL and pioneering work by [Hochreiter et al. \(2001\)](#) using recurrent networks to perform meta-learning in the setting of full supervision

(see also [Cotter and Conwell, 1990](#); [Prokhorov et al., 2002](#); [Younger et al., 1999](#)). That approach was recently extended in [Santoro et al., 2016](#), which demonstrated the utility of leveraging an external memory structure. The idea of crossing meta-learning with reinforcement learning has been previously discussed by [Schmidhuber et al., 1996](#). That work, which appears to have introduced the term “meta-RL,” differs from ours in that it did not involve a neural network implementation. More recently, however, there has been a surge of interest in using neural networks to learn optimization procedures, using a range of innovative meta-learning techniques ([Andrychowicz et al., 2016](#); [Chen et al., 2016](#); [Li and Malik, 2016](#); [Zoph and Le, 2016](#)). Recent work by [Chen et al., 2016](#) is particularly close in spirit to the work we have presented here, and can be viewed as treating the case of “infinite bandits” using a meta-learning strategy broadly analogous to the one we have pursued.

The present research also bears a close relationship with a different body of recent work that has not been framed in terms of meta-learning. A number of studies have used deep RL to train recurrent neural networks on navigation tasks, where the structure of the task (e.g., goal location or maze configuration) varies across episodes ([Jaderberg et al., 2016](#); [Mirowski et al., 2016](#)). The final experiment that we presented above, drawn from ([Mirowski et al., 2016](#)), is one example. To the extent that such experiments involve the key ingredients of deep meta-RL – a neural network with memory, trained through RL on a series of interrelated tasks – they are almost certain to involve the kind of meta-learning we have described in the present work. This related work provides an indication that meta-RL can be fruitfully applied to larger scale problems than the ones we have studied in our own experiments. Importantly, it indicates that a key ingredient in scaling the approach may be to incorporate memory mechanisms beyond those inherent in unstructured recurrent neural networks (see [Graves et al., 2016](#); [Mirowski et al., 2016](#); [Santoro et al., 2016](#); [Weston et al., 2014](#)). Our work, for its part, suggests that there is untapped potential in deep recurrent RL agents to meta-learn quite abstract aspects of task structure, and to discover strategies that exploit such structure toward rapid, flexible adaptation.

During completion of the present research, closely related work was reported by [Duan et al., 2016](#). Like us, Duan and colleagues use deep RL to train a recurrent network on a series of interrelated tasks, with the result that the network dynamics learn a second RL procedure which operates on a faster time-scale than the original algorithm. They compare the performance of these learned procedures against conventional RL algorithms in a number of domains, including bandits and navigation. An important difference between this parallel work and our own is the former’s primary focus on relatively unstructured task distributions (e.g., uniformly distributed bandit problems and random MDPs); our main interest, in contrast, has been in structured task distributions (e.g., dependent bandits and the task introduced by [Harlow, 1949](#)), because it is in this setting where the system can learn a biased – and therefore efficient – RL procedure that exploits regular task structure. The two perspectives are, in this regard, nicely complementary.

5 CONCLUSION

A current challenge in artificial intelligence is to design agents that can adapt rapidly to new tasks by leveraging knowledge acquired through previous experience with related activities. In the present work we have reported initial explorations of what we believe is one promising avenue toward this goal. Deep meta-RL involves a combination of three ingredients: (1) Use of a deep RL algorithm to train a recurrent neural network, (2) a training set that includes a series of interrelated tasks, (3) network input that includes the action selected and reward received in the previous time interval. The key result, which emerges naturally from the setup rather than being specially engineered, is that the recurrent network dynamics learn to implement a second RL procedure, independent from and potentially very different from the algorithm used to train the network weights. Critically, this learned RL algorithm is tuned to the shared structure of the training tasks. In this sense, the learned algorithm builds in domain-appropriate biases, which can allow it to operate with greater efficiency than a general-purpose algorithm. This bias effect was particularly evident in the results of our experiments involving dependent bandits (sections 3.1.2 and 3.1.3), where the system learned to take advantage of the task’s covariance structure; and in our study of Harlow’s animal learning task (section 3.2.2), where the recurrent network learned to exploit the task’s structure in order to display one-shot learning with complex novel stimuli.

One of our experiments (section 3.2.1) illustrated the point that a system trained using a model-free RL algorithm can develop behavior that emulates model-based control. A few further comments on this result are warranted. As noted in our presentation of the simulation results, the pattern of choice behavior displayed by the network has been considered in the cognitive and neuroscience literatures as reflecting model-based control or tree search. However, as has been remarked in very recent work, the same pattern can arise from a model-free system with an appropriate state representation (Akam et al., 2015). Indeed, we suspect this may be how our network in fact operates. However, other findings suggest that a more explicitly model-based control mechanism can emerge when a similar system is trained on a more diverse set of tasks. In particular, Ilin et al. (2007) showed that recurrent networks trained on random mazes can approximate dynamic programming procedures (see also Silver et al., 2017; Tamar et al., 2016). At the same time, as we have stressed, we consider it an important aspect of deep meta-RL that it yields a learned RL algorithm that capitalizes on invariances in task structure. As a result, when faced with widely varying but still structured environments, deep meta-RL seems likely to generate RL procedures that occupy a grey area between model-free and model-based RL.

The two-step decision problem studied in Section 3.2.1 was derived from neuroscience, and we believe deep meta-RL may have important implications in that arena (Wang et al., 2017). The notion of meta-RL has been discussed previously in neuroscience but only in a narrow sense, according to which meta-learning adjusts scalar hyperparameters such as the learning rate or softmax inverse temperature (Khamassi et al., 2011, 2013; Kobayashi et al., 2009; Lee and Wang, 2009; Schweighofer and Doya, 2003; Soltani et al., 2006). In recent work (Wang et al., 2017) we have shown that deep meta-RL can account for a wider range of experimental observations, providing an integrative framework for understanding the respective roles of dopamine and the prefrontal cortex in biological reinforcement learning.

ACKNOWLEDGEMENTS

We would like to thank the following colleagues for useful discussion and feedback: Nando de Freitas, David Silver, Koray Kavukcuoglu, Daan Wierstra, Demis Hassabis, Matt Hoffman, Piotr Mirowski, Andrea Banino, Sam Ritter, Neil Rabinowitz, Peter Dayan, Peter Battaglia, Alex Lerchner, Tim Lillicrap and Greg Wayne.

REFERENCES

- Thomas Akam, Rui Costa, and Peter Dayan. Simple plans or sophisticated habits? state, transition and learning interactions in the two-step task. *PLoS Comput Biol*, 11(12):e1004648, 2015.
- Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. *arXiv preprint arXiv:1606.04474*, 2016.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2-3):235–256, 2002.
- Timothy EJ Behrens, Mark W Woolrich, Mark E Walton, and Matthew FS Rushworth. Learning the value of information in an uncertain world. *Nature neuroscience*, 10(9):1214–1221, 2007.
- Ethan S Bromberg-Martin and Okihide Hikosaka. Midbrain dopamine neurons signal preference for advance information about upcoming rewards. *Neuron*, 63(1):119–126, 2009.
- Yutian Chen, Matthew W Hoffman, Sergio Gomez, Misha Denil, Timothy P Lillicrap, and Nando de Freitas. Learning to learn for global optimization of black box functions. *arXiv preprint arXiv:1611.03824*, 2016.
- NE Cotter and PR Conwell. Fixed-weight networks can learn. In *1990 IJCNN International Joint Conference on Neural Networks*, pages 553–559, 1990.
- Nathaniel D Daw, Yael Niv, and Peter Dayan. Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. *Nature neuroscience*, 8(12):1704–1711, 2005.
- Nathaniel D Daw, Samuel J Gershman, Ben Seymour, Peter Dayan, and Raymond J Dolan. Model-based influences on humans’ choices and striatal prediction errors. *Neuron*, 69(6):1204–1215, 2011.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, pages 248–255. IEEE, 2009.
- Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL2: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016. URL <http://arxiv.org/abs/1611.02779>.

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI

{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a “surrogate” objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q -learning [Mni+15], “vanilla” policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q -learning (with function approximation) fails on many simple problems¹ and is poorly understood, vanilla policy gradient methods have poor data efficiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains the data efficiency and reliable performance of TRPO, while using only first-order optimization. We propose a novel objective with clipped probability ratios, which forms a pessimistic estimate (i.e., lower bound) of the performance of the policy. To optimize policies, we alternate between sampling data from the policy and performing several epochs of optimization on the sampled data.

Our experiments compare the performance of various different versions of the surrogate objective, and find that the version with the clipped probability ratios performs best. We also compare PPO to several previous algorithms from the literature. On continuous control tasks, it performs better than the algorithms we compare against. On Atari, it performs significantly better (in terms of sample complexity) than A2C and similarly to ACER though it is much simpler.

¹While DQN works well on game environments like the Arcade Learning Environment [Bel+15] with discrete action spaces, it has not been demonstrated to perform well on continuous control benchmarks such as those in OpenAI Gym [Bro+16] and described by Duan et al. [Dua+16].

2 Background: Policy Optimization

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (1)$$

where π_{θ} is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t . Here, the expectation $\hat{\mathbb{E}}_t[\dots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (2)$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the “no clipping or penalty” setting).

2.2 Trust Region Methods

In TRPO [Sch+15b], an objective function (the “surrogate” objective) is maximized subject to a constraint on the size of the policy update. Specifically,

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad (3)$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \quad (4)$$

Here, θ_{old} is the vector of policy parameters before the update. This problem can efficiently be approximately solved using the conjugate gradient algorithm, after making a linear approximation to the objective and a quadratic approximation to the constraint.

The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right] \quad (5)$$

for some coefficient β . This follows from the fact that a certain surrogate objective (which computes the max KL over states instead of the mean) forms a lower bound (i.e., a pessimistic bound) on the performance of the policy π . TRPO uses a hard constraint rather than a penalty because it is hard to choose a single value of β that performs well across different problems—or even within a single problem, where the characteristics change over the course of learning. Hence, to achieve our goal of a first-order algorithm that emulates the monotonic improvement of TRPO, experiments show that it is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation (5) with SGD; additional modifications are required.

3 Clipped Surrogate Objective

Let $r_t(\theta)$ denote the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$, so $r(\theta_{\text{old}}) = 1$. TRPO maximizes a “surrogate” objective

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] = \hat{\mathbb{E}}_t [r_t(\theta) \hat{A}_t]. \quad (6)$$

The superscript *CPI* refers to conservative policy iteration [KL02], where this objective was proposed. Without a constraint, maximization of L^{CPI} would lead to an excessively large policy update; hence, we now consider how to modify the objective, to penalize changes to the policy that move $r_t(\theta)$ away from 1.

The main objective we propose is the following:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (7)$$

where epsilon is a hyperparameter, say, $\epsilon = 0.2$. The motivation for this objective is as follows. The first term inside the min is L^{CPI} . The second term, $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t$, modifies the surrogate objective by clipping the probability ratio, which removes the incentive for moving r_t outside of the interval $[1 - \epsilon, 1 + \epsilon]$. Finally, we take the minimum of the clipped and unclipped objective, so the final objective is a lower bound (i.e., a pessimistic bound) on the unclipped objective. With this scheme, we only ignore the change in probability ratio when it would make the objective improve, and we include it when it makes the objective worse. Note that $L^{CLIP}(\theta) = L^{CPI}(\theta)$ to first order around θ_{old} (i.e., where $r = 1$), however, they become different as θ moves away from θ_{old} . Figure 1 plots a single term (i.e., a single t) in L^{CLIP} ; note that the probability ratio r is clipped at $1 - \epsilon$ or $1 + \epsilon$ depending on whether the advantage is positive or negative.

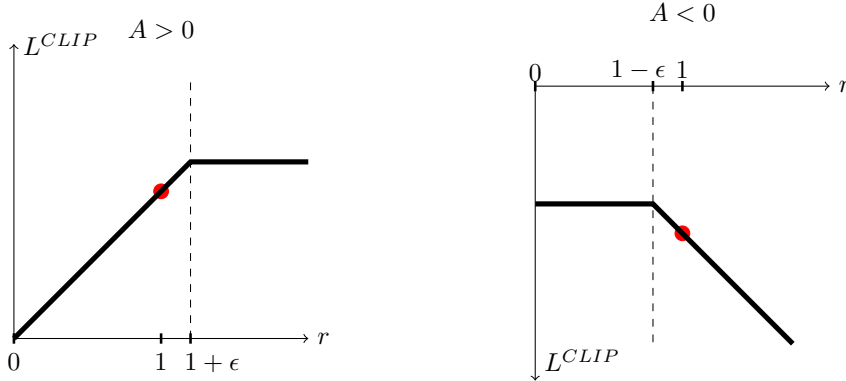


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r , for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

Figure 2 provides another source of intuition about the surrogate objective L^{CLIP} . It shows how several objectives vary as we interpolate along the policy update direction, obtained by proximal policy optimization (the algorithm we will introduce shortly) on a continuous control problem. We can see that L^{CLIP} is a lower bound on L^{CPI} , with a penalty for having too large of a policy update.

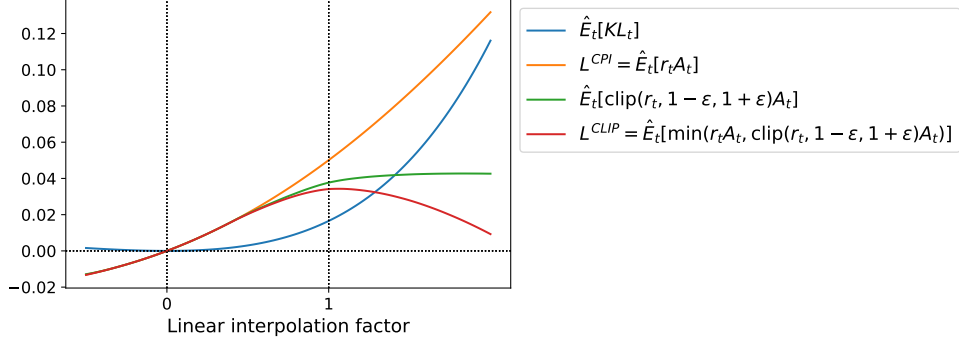


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old} , and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the Hopper-v1 problem, using hyperparameters provided in Section 6.1.

4 Adaptive KL Penalty Coefficient

Another approach, which can be used as an alternative to the clipped surrogate objective, or in addition to it, is to use a penalty on KL divergence, and to adapt the penalty coefficient so that we achieve some target value of the KL divergence d_{targ} each policy update. In our experiments, we found that the KL penalty performed worse than the clipped surrogate objective, however, we’ve included it here because it’s an important baseline.

In the simplest instantiation of this algorithm, we perform the following steps in each policy update:

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right] \quad (8)$$

- Compute $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)]]$
 - If $d < d_{\text{targ}}/1.5$, $\beta \leftarrow \beta/2$
 - If $d > d_{\text{targ}} \times 1.5$, $\beta \leftarrow \beta \times 2$

The updated β is used for the next policy update. With this scheme, we occasionally see policy updates where the KL divergence is significantly different from d_{targ} , however, these are rare, and β quickly adjusts. The parameters 1.5 and 2 above are chosen heuristically, but the algorithm is not very sensitive to them. The initial value of β is another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

5 Algorithm

The surrogate losses from the previous sections can be computed and differentiated with a minor change to a typical policy gradient implementation. For implementations that use automatic differentiation, one simply constructs the loss L^{CLIP} or $L^{KL PEN}$ instead of L^{PG} , and one performs multiple steps of stochastic gradient ascent on this objective.

Most techniques for computing variance-reduced advantage-function estimators make use a learned state-value function $V(s)$; for example, generalized advantage estimation [Sch+15a], or the

finite-horizon estimators in [Mni+16]. If using a neural network architecture that shares parameters between the policy and value function, we must use a loss function that combines the policy surrogate and a value function error term. This objective can further be augmented by adding an entropy bonus to ensure sufficient exploration, as suggested in past work [Wil92; Mni+16]. Combining these terms, we obtain the following objective, which is (approximately) maximized each iteration:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)], \quad (9)$$

where c_1, c_2 are coefficients, and S denotes an entropy bonus, and L_t^{VF} is a squared-error loss $(V_\theta(s_t) - V_t^{\text{targ}})^2$.

One style of policy gradient implementation, popularized in [Mni+16] and well-suited for use with recurrent neural networks, runs the policy for T timesteps (where T is much less than the episode length), and uses the collected samples for an update. This style requires an advantage estimator that does not look beyond timestep T . The estimator used by [Mni+16] is

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \quad (10)$$

where t specifies the time index in $[0, T]$, within a given length- T trajectory segment. Generalizing this choice, we can use a truncated version of generalized advantage estimation, which reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \quad (11)$$

$$\text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (12)$$

A proximal policy optimization (PPO) algorithm that uses fixed-length trajectory segments is shown below. Each iteration, each of N (parallel) actors collect T timesteps of data. Then we construct the surrogate loss on these NT timesteps of data, and optimize it with minibatch SGD (or usually for better performance, Adam [KB14]), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

```

for iteration=1, 2, ... do
  for actor=1, 2, ...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for

```

6 Experiments

6.1 Comparison of Surrogate Objectives

First, we compare several different surrogate objectives under different hyperparameters. Here, we compare the surrogate objective L^{CLIP} to several natural variations and ablated versions.

No clipping or penalty:	$L_t(\theta) = r_t(\theta)\hat{A}_t$
Clipping:	$L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta)), 1 - \epsilon, 1 + \epsilon)\hat{A}_t$
KL penalty (fixed or adaptive)	$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$

For the KL penalty, one can either use a fixed penalty coefficient β or an adaptive coefficient as described in Section 4 using target KL value d_{targ} . Note that we also tried clipping in log space, but found the performance to be no better.

Because we are searching over hyperparameters for each algorithm variant, we chose a computationally cheap benchmark to test the algorithms on. Namely, we used 7 simulated robotics tasks² implemented in OpenAI Gym [Bro+16], which use the MuJoCo [TET12] physics engine. We do one million timesteps of training on each one. Besides the hyperparameters used for clipping (ϵ) and the KL penalty (β, d_{targ}), which we search over, the other hyperparameters are provided in in Table 3.

To represent the policy, we used a fully-connected MLP with two hidden layers of 64 units, and tanh nonlinearities, outputting the mean of a Gaussian distribution, with variable standard deviations, following [Sch+15b; Dua+16]. We don’t share parameters between the policy and value function (so coefficient c_1 is irrelevant), and we don’t use an entropy bonus.

Each algorithm was run on all 7 environments, with 3 random seeds on each. We scored each run of the algorithm by computing the average total reward of the last 100 episodes. We shifted and scaled the scores for each environment so that the random policy gave a score of 0 and the best result was set to 1, and averaged over 21 runs to produce a single scalar for each algorithm setting.

The results are shown in Table 1. Note that the score is negative for the setting without clipping or penalties, because for one environment (half cheetah) it leads to a very negative score, which is worse than the initial random policy.

algorithm	avg. normalized score
No clipping or penalty	-0.39
Clipping, $\epsilon = 0.1$	0.76
Clipping, $\epsilon = 0.2$	0.82
Clipping, $\epsilon = 0.3$	0.70
Adaptive KL $d_{\text{targ}} = 0.003$	0.68
Adaptive KL $d_{\text{targ}} = 0.01$	0.74
Adaptive KL $d_{\text{targ}} = 0.03$	0.71
Fixed KL, $\beta = 0.3$	0.62
Fixed KL, $\beta = 1.$	0.71
Fixed KL, $\beta = 3.$	0.72
Fixed KL, $\beta = 10.$	0.69

Table 1: Results from continuous control benchmark. Average normalized scores (over 21 runs of the algorithm, on 7 environments) for each algorithm / hyperparameter setting . β was initialized at 1.

6.2 Comparison to Other Algorithms in the Continuous Domain

Next, we compare PPO (with the “clipped” surrogate objective from Section 3) to several other methods from the literature, which are considered to be effective for continuous problems. We compared against tuned implementations of the following algorithms: trust region policy optimization [Sch+15b], cross-entropy method (CEM) [SL06], vanilla policy gradient with adaptive stepsize³,

²HalfCheetah, Hopper, InvertedDoublePendulum, InvertedPendulum, Reacher, Swimmer, and Walker2d, all “-v1”

³After each batch of data, the Adam stepsize is adjusted based on the KL divergence of the original and updated policy, using a rule similar to the one shown in Section 4. An implementation is available at <https://github.com/berkeleydeeprlcourse/homework/tree/master/hw4>.

A2C [Mni+16], A2C with trust region [Wan+16]. A2C stands for advantage actor critic, and is a synchronous version of A3C, which we found to have the same or better performance than the asynchronous version. For PPO, we used the hyperparameters from the previous section, with $\epsilon = 0.2$. We see that PPO outperforms the previous methods on almost all the continuous control environments.

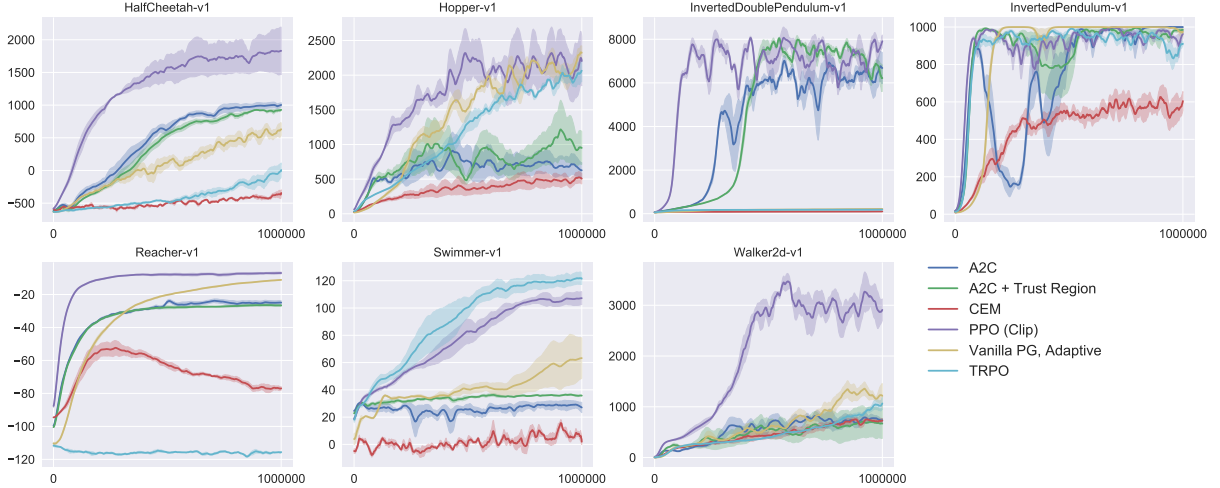


Figure 3: Comparison of several algorithms on several MuJoCo environments, training for one million timesteps.

6.3 Showcase in the Continuous Domain: Humanoid Running and Steering

To showcase the performance of PPO on high-dimensional continuous control problems, we train on a set of problems involving a 3D humanoid, where the robot must run, steer, and get up off the ground, possibly while being pelted by cubes. The three tasks we test on are (1) RoboschoolHumanoid: forward locomotion only, (2) RoboschoolHumanoidFlagrun: position of target is randomly varied every 200 timesteps or whenever the goal is reached, (3) RoboschoolHumanoidFlagrunHarder, where the robot is pelted by cubes and needs to get up off the ground. See Figure 5 for still frames of a learned policy, and Figure 4 for learning curves on the three tasks. Hyperparameters are provided in Table 4. In concurrent work, Heess et al. [Hee+17] used the adaptive KL variant of PPO (Section 4) to learn locomotion policies for 3D robots.

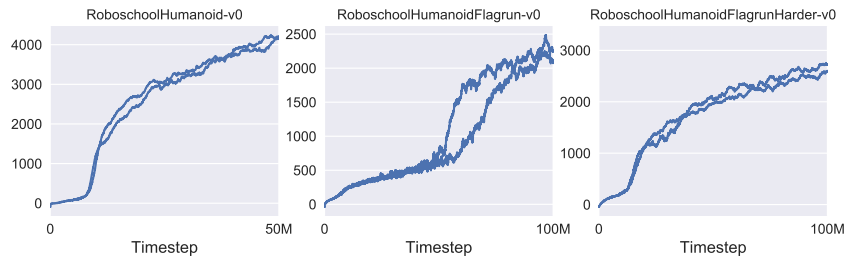


Figure 4: Learning curves from PPO on 3D humanoid control tasks, using Roboschool.

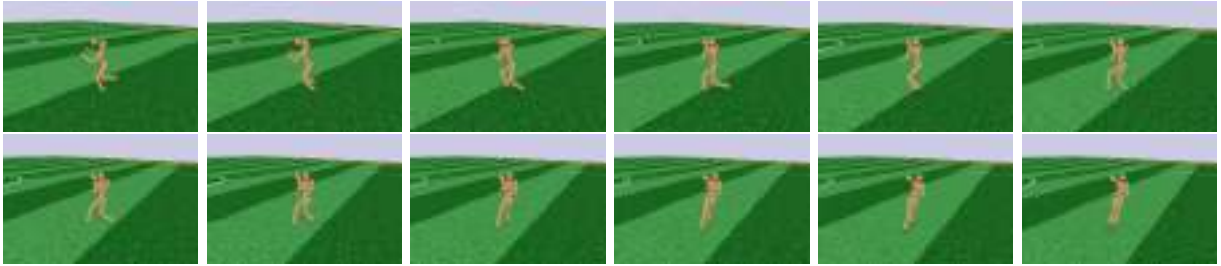


Figure 5: Still frames of the policy learned from RoboschoolHumanoidFlagrun. In the first six frames, the robot runs towards a target. Then the position is randomly changed, and the robot turns and runs toward the new target.

6.4 Comparison to Other Algorithms on the Atari Domain

We also ran PPO on the Arcade Learning Environment [Bel+15] benchmark and compared against well-tuned implementations of A2C [Mni+16] and ACER [Wan+16]. For all three algorithms, we used the same policy network architecture as used in [Mni+16]. The hyperparameters for PPO are provided in Table 5. For the other two algorithms, we used hyperparameters that were tuned to maximize performance on this benchmark.

A table of results and learning curves for all 49 games is provided in Appendix B. We consider the following two scoring metrics: (1) *average reward per episode over entire training period* (which favors fast learning), and (2) *average reward per episode over last 100 episodes of training* (which favors final performance). Table 2 shows the number of games “won” by each algorithm, where we compute the victor by averaging the scoring metric across three trials.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	30	0
(2) avg. episode reward over last 100 episodes	1	28	19	1

Table 2: Number of games “won” by each algorithm, where the scoring metric is averaged across three trials.

7 Conclusion

We have introduced proximal policy optimization, a family of policy optimization methods that use multiple epochs of stochastic gradient ascent to perform each policy update. These methods have the stability and reliability of trust-region methods but are much simpler to implement, requiring only few lines of code change to a vanilla policy gradient implementation, applicable in more general settings (for example, when using a joint architecture for the policy and value function), and have better overall performance.

8 Acknowledgements

Thanks to Rocky Duan, Peter Chen, and others at OpenAI for insightful comments.

A Hyperparameters

Hyperparameter	Value
Horizon (T)	2048
Adam stepsize	3×10^{-4}
Num. epochs	10
Minibatch size	64
Discount (γ)	0.99
GAE parameter (λ)	0.95

Table 3: PPO hyperparameters used for the Mujoco 1 million timestep benchmark.

Hyperparameter	Value
Horizon (T)	512
Adam stepsize	*
Num. epochs	15
Minibatch size	4096
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	32 (locomotion), 128 (flagrun)
Log stdev. of action distribution	LinearAnneal($-0.7, -1.6$)

Table 4: PPO hyperparameters used for the Roboschool experiments. Adam stepsize was adjusted based on the target value of the KL divergence.

Hyperparameter	Value
Horizon (T)	128
Adam stepsize	$2.5 \times 10^{-4} \times \alpha$
Num. epochs	3
Minibatch size	32×8
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	8
Clipping parameter ϵ	$0.1 \times \alpha$
VF coeff. c_1 (9)	1
Entropy coeff. c_2 (9)	0.01

Table 5: PPO hyperparameters used in Atari experiments. α is linearly annealed from 1 to 0 over the course of learning.

B Performance on More Atari Games

Here we include a comparison of PPO against A2C on a larger collection of 49 Atari games. Figure 6 shows the learning curves of each of three random seeds, while Table 6 shows the mean performance.

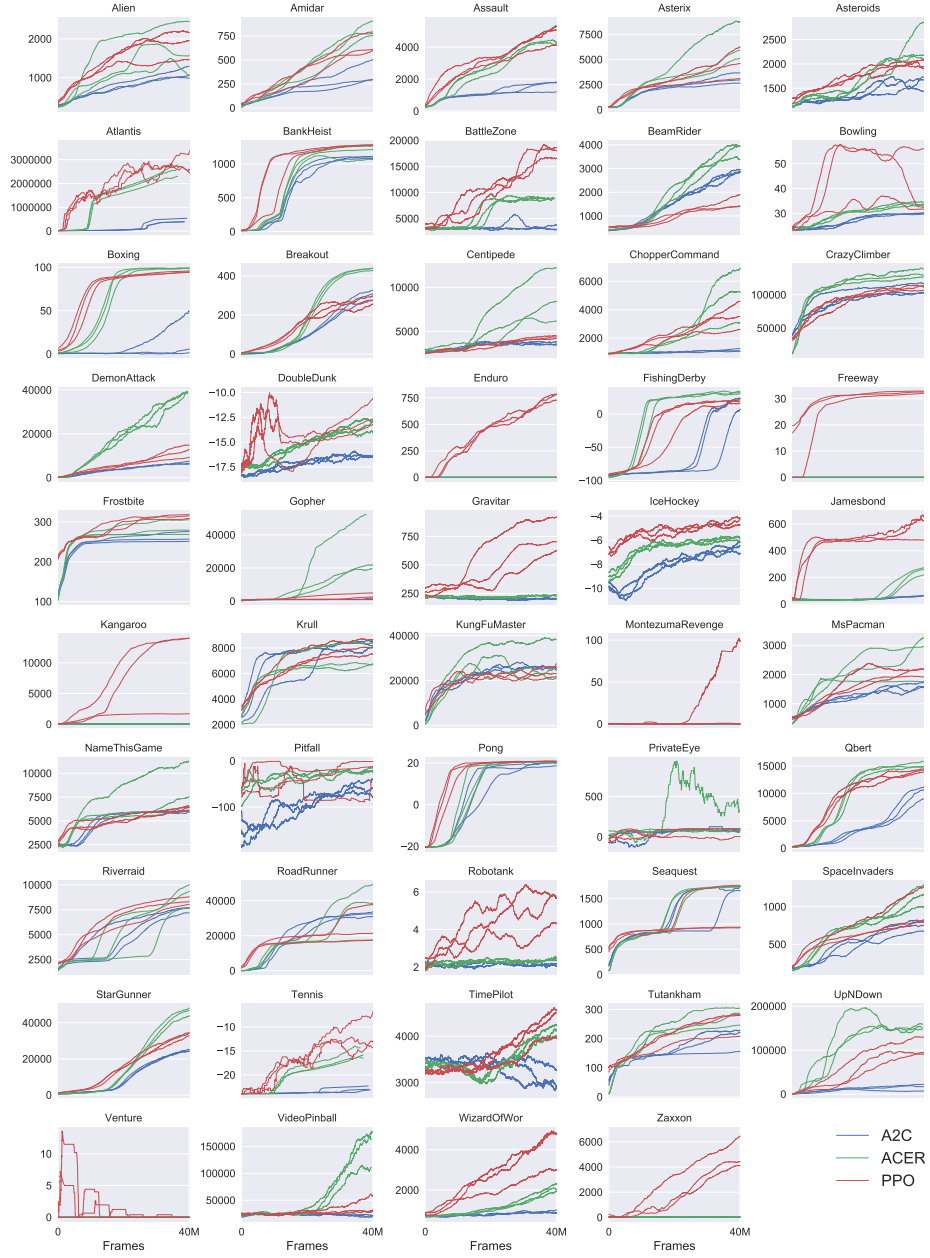


Figure 6: Comparison of PPO and A2C on all 49 ATARI games included in OpenAI Gym at the time of publication.

	A2C	ACER	PPO
Alien	1141.7	1655.4	1850.3
Amidar	380.8	827.6	674.6
Assault	1562.9	4653.8	4971.9
Asterix	3176.3	6801.2	4532.5
Asteroids	1653.3	2389.3	2097.5
Atlantis	729265.3	1841376.0	2311815.0
BankHeist	1095.3	1177.5	1280.6
BattleZone	3080.0	8983.3	17366.7
BeamRider	3031.7	3863.3	1590.0
Bowling	30.1	33.3	40.1
Boxing	17.7	98.9	94.6
Breakout	303.0	456.4	274.8
Centipede	3496.5	8904.8	4386.4
ChopperCommand	1171.7	5287.7	3516.3
CrazyClimber	107770.0	132461.0	110202.0
DemonAttack	6639.1	38808.3	11378.4
DoubleDunk	-16.2	-13.2	-14.9
Enduro	0.0	0.0	758.3
FishingDerby	20.6	34.7	17.8
Freeway	0.0	0.0	32.5
Frostbite	261.8	285.6	314.2
Gopher	1500.9	37802.3	2932.9
Gravitar	194.0	225.3	737.2
IceHockey	-6.4	-5.9	-4.2
Jamesbond	52.3	261.8	560.7
Kangaroo	45.3	50.0	9928.7
Krull	8367.4	7268.4	7942.3
KungFuMaster	24900.3	27599.3	23310.3
MontezumaRevenge	0.0	0.3	42.0
MsPacman	1626.9	2718.5	2096.5
NameThisGame	5961.2	8488.0	6254.9
Pitfall	-55.0	-16.9	-32.9
Pong	19.7	20.7	20.7
PrivateEye	91.3	182.0	69.5
Qbert	10065.7	15316.6	14293.3
Riverraid	7653.5	9125.1	8393.6
RoadRunner	32810.0	35466.0	25076.0
Robotank	2.2	2.5	5.5
Seaquest	1714.3	1739.5	1204.5
SpaceInvaders	744.5	1213.9	942.5
StarGunner	26204.0	49817.7	32689.0
Tennis	-22.2	-17.6	-14.8
TimePilot	2898.0	4175.7	4342.0
Tutankham	206.8	280.8	254.4
UpNDown	17369.8	145051.4	95445.0
Venture	0.0	0.0	0.0
VideoPinball	19735.9	156225.6	37389.0
WizardOfWor	859.0	2308.3	4185.3
Zaxxon	16.3	29.0	5008.7

Table 6: Mean final scores (last 100 episodes) of PPO and A2C on Atari games after 40M game frames (10M timesteps).

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja¹ Aurick Zhou¹ Pieter Abbeel¹ Sergey Levine¹

Abstract

Model-free deep reinforcement learning (RL) algorithms have been demonstrated on a range of challenging decision making and control tasks. However, these methods typically suffer from two major challenges: very high sample complexity and brittle convergence properties, which necessitate meticulous hyperparameter tuning. Both of these challenges severely limit the applicability of such methods to complex, real-world domains. In this paper, we propose soft actor-critic, an off-policy actor-critic deep RL algorithm based on the maximum entropy reinforcement learning framework. In this framework, the actor aims to maximize expected reward while also maximizing entropy. That is, to succeed at the task while acting as randomly as possible. Prior deep RL methods based on this framework have been formulated as Q-learning methods. By combining off-policy updates with a stable stochastic actor-critic formulation, our method achieves state-of-the-art performance on a range of continuous control benchmark tasks, outperforming prior on-policy and off-policy methods. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving very similar performance across different random seeds.

of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017b) or A3C (Mnih et al., 2016), require new samples to be collected for each gradient step. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al., 2009). This challenge is further exacerbated in continuous state and action spaces, where a separate actor network is often used to perform the maximization in Q-learning. A commonly used algorithm in such settings, deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), provides for sample-efficient learning but is notoriously challenging to use due to its extreme brittleness and hyperparameter sensitivity (Duan et al., 2016; Henderson et al., 2017).

We explore how to design an efficient and stable model-free deep RL algorithm for continuous state and action spaces. To that end, we draw on the maximum entropy reinforcement learning objective with an entropy maximization term (Ziebart et al., 2008; Toussaint, 2009; Rawlik et al.,

1. Introduction

Model-free deep reinforcement learning (RL) algorithms have been applied in a range of challenging domains, from games (Mnih et al., 2013; Silver et al., 2016) to robotic control (Schulman et al., 2015). The combination of RL and high-capacity function approximators such as neural networks holds the promise of automating a wide range of decision making and control tasks, but widespread adoption

¹Berkeley Artificial Intelligence Research, University of California, Berkeley, USA. Correspondence to: Tuomas Haarnoja <haarnoja@berkeley.edu>.

2012; Fox et al., 2016; Haarnoja et al., 2017). Maximum entropy reinforcement learning alters the RL objective, though the original objective can be recovered using a temperature parameter (Haarnoja et al., 2017). More importantly, the maximum entropy formulation provides a substantial improvement in exploration and robustness: as discussed by Ziebart (2010), maximum entropy policies are robust in the face of model and estimation errors, and as demonstrated by (Haarnoja et al., 2017), they improve exploration by acquiring diverse behaviors. Prior work has proposed model-free deep RL algorithms that perform on-policy learning with entropy maximization (O’Donoghue et al., 2016), as well as off-policy methods based on soft Q-learning and its variants (Schulman et al., 2017a; Nachum et al., 2017a; Haarnoja et al., 2017). However, the on-policy variants suffer from poor sample complexity for the reasons discussed above, while the off-policy variants require complex approximate inference procedures in continuous action spaces.

In this paper, we demonstrate that we can devise an off-policy maximum entropy actor-critic algorithm, which we call soft actor-critic (SAC), which provides for both sample-efficient learning and stability. This algorithm extends readily to very complex, high-dimensional tasks, such as the Humanoid benchmark (Duan et al., 2016) with 21 action dimensions, where off-policy methods such as DDPG typically struggle to obtain good results (Gu et al., 2016). SAC also avoids the complexity and potential instability associated with approximate inference in prior off-policy maximum entropy algorithms based on soft Q-learning (Haarnoja et al., 2017). We present a convergence proof for policy iteration in the maximum entropy framework, and then introduce a new algorithm based on an approximation to this procedure that can be practically implemented with deep neural networks, which we call soft actor-critic. We present empirical results that show that soft actor-critic attains a substantial improvement in both performance and sample efficiency over both off-policy and on-policy prior methods. We also compare to twin delayed deep deterministic (TD3) policy gradient algorithm (Fujimoto et al., 2018), which is a concurrent work that proposes a deterministic algorithm that substantially improves on DDPG.

2. Related Work

Our soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration. We review prior works that draw on some of these ideas in this section. Actor-critic algorithms are typically derived starting from policy iteration, which alternates between *policy evaluation*—computing the value function for a policy—

and *policy improvement*—using the value function to obtain a better policy (Barto et al., 1983; Sutton & Barto, 1998). In large-scale reinforcement learning problems, it is typically impractical to run either of these steps to convergence, and instead the value function and policy are optimized jointly. In this case, the policy is referred to as the actor, and the value function as the critic. Many actor-critic algorithms build on the standard, on-policy policy gradient formulation to update the actor (Peters & Schaal, 2008), and many of them also consider the entropy of the policy, but instead of maximizing the entropy, they use it as an regularizer (Schulman et al., 2017b; 2015; Mnih et al., 2016; Gruslys et al., 2017). On-policy training tends to improve stability but results in poor sample complexity.

There have been efforts to increase the sample efficiency while retaining robustness by incorporating off-policy samples and by using higher order variance reduction techniques (O’Donoghue et al., 2016; Gu et al., 2016). However, fully off-policy algorithms still attain better efficiency. A particularly popular off-policy actor-critic method, DDPG (Lillicrap et al., 2015), which is a deep variant of the deterministic policy gradient (Silver et al., 2014) algorithm, uses a Q-function estimator to enable off-policy learning, and a deterministic actor that maximizes this Q-function. As such, this method can be viewed both as a deterministic actor-critic algorithm and an approximate Q-learning algorithm. Unfortunately, the interplay between the deterministic actor network and the Q-function typically makes DDPG extremely difficult to stabilize and brittle to hyperparameter settings (Duan et al., 2016; Henderson et al., 2017). As a consequence, it is difficult to extend DDPG to complex, high-dimensional tasks, and on-policy policy gradient methods still tend to produce the best results in such settings (Gu et al., 2016). Our method instead combines off-policy actor-critic training with a *stochastic* actor, and further aims to maximize the entropy of this actor with an entropy maximization objective. We find that this actually results in a considerably more stable and scalable algorithm that, in practice, exceeds both the efficiency and final performance of DDPG. A similar method can be derived as a zero-step special case of stochastic value gradients (SVG(0)) (Heess et al., 2015). However, SVG(0) differs from our method in that it optimizes the standard maximum expected return objective, and it does not make use of a separate value network, which we found to make training more stable.

Maximum entropy reinforcement learning optimizes policies to maximize both the expected return and the expected entropy of the policy. This framework has been used in many contexts, from inverse reinforcement learning (Ziebart et al., 2008) to optimal control (Todorov, 2008; Toussaint, 2009; Rawlik et al., 2012). In guided policy search (Levine & Koltun, 2013; Levine et al., 2016), the maximum entropy distribution is used to guide policy learn-

ing towards high-reward regions. More recently, several papers have noted the connection between Q-learning and policy gradient methods in the framework of maximum entropy learning (O’Donoghue et al., 2016; Haarnoja et al., 2017; Nachum et al., 2017a; Schulman et al., 2017a). While most of the prior model-free works assume a discrete action space, Nachum et al. (2017b) approximate the maximum entropy distribution with a Gaussian and Haarnoja et al. (2017) with a sampling network trained to draw samples from the optimal policy. Although the soft Q-learning algorithm proposed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution. Hence, Haarnoja et al. (2017) motivates the actor network as an approximate sampler, rather than the actor in an actor-critic algorithm. Crucially, the convergence of this method hinges on how well this sampler approximates the true posterior. In contrast, we prove that our method converges to the optimal policy from a given policy class, regardless of the policy parameterization. Furthermore, these prior maximum entropy methods generally do not exceed the performance of state-of-the-art off-policy algorithms, such as DDPG, when learning from scratch, though they may have other benefits, such as improved exploration and ease of fine-tuning. In our experiments, we demonstrate that our soft actor-critic algorithm does in fact exceed the performance of prior state-of-the-art off-policy deep RL methods by a wide margin.

3. Preliminaries

We first introduce notation and summarize the standard and maximum entropy reinforcement learning frameworks.

3.1. Notation

We address policy learning in continuous action spaces. We consider an infinite-horizon Markov decision process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r)$, where the state space $\mathcal{S}$ and the action space $\mathcal{A}$ are continuous, and the unknown state transition probability $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, \infty)$ represents the probability density of the next state $\mathbf{s}_{t+1} \in \mathcal{S}$ given the current state $\mathbf{s}_t \in \mathcal{S}$ and action $\mathbf{a}_t \in \mathcal{A}$. The environment emits a bounded reward $r : \mathcal{S} \times \mathcal{A} \rightarrow [r_{\min}, r_{\max}]$ on each transition. We will use $\rho_\pi(\mathbf{s}_t)$ and $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ to denote the state and state-action marginals of the trajectory distribution induced by a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$.

3.2. Maximum Entropy Reinforcement Learning

Standard RL maximizes the expected sum of rewards $\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t)]$. We will consider a more general maximum entropy objective (see e.g. Ziebart (2010)), which favors stochastic policies by augmenting the objective

with the expected entropy of the policy over $\rho_\pi(\mathbf{s}_t)$:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot|\mathbf{s}_t))]. \quad (1)$$

The temperature parameter α determines the relative importance of the entropy term against the reward, and thus controls the stochasticity of the optimal policy. The maximum entropy objective differs from the standard maximum expected reward objective used in conventional reinforcement learning, though the conventional objective can be recovered in the limit as $\alpha \rightarrow 0$. For the rest of this paper, we will omit writing the temperature explicitly, as it can always be subsumed into the reward by scaling it by α^{-1} .

This objective has a number of conceptual and practical advantages. First, the policy is incentivized to explore more widely, while giving up on clearly unpromising avenues. Second, the policy can capture multiple modes of near-optimal behavior. In problem settings where multiple actions seem equally attractive, the policy will commit equal probability mass to those actions. Lastly, prior work has observed improved exploration with this objective (Haarnoja et al., 2017; Schulman et al., 2017a), and in our experiments, we observe that it considerably improves learning speed over state-of-the-art methods that optimize the conventional RL objective function. We can extend the objective to infinite horizon problems by introducing a discount factor γ to ensure that the sum of expected rewards and entropies is finite. Writing down the maximum entropy objective for the infinite horizon discounted case is more involved (Thomas, 2014) and is deferred to Appendix A.

Prior methods have proposed directly solving for the optimal Q-function, from which the optimal policy can be recovered (Ziebart et al., 2008; Fox et al., 2016; Haarnoja et al., 2017). We will discuss how we can devise a soft actor-critic algorithm through a policy iteration formulation, where we instead evaluate the Q-function of the current policy and update the policy through an *off-policy* gradient update. Though such algorithms have previously been proposed for conventional reinforcement learning, our method is, to our knowledge, the first off-policy actor-critic method in the maximum entropy reinforcement learning framework.

4. From Soft Policy Iteration to Soft Actor-Critic

Our off-policy soft actor-critic algorithm can be derived starting from a maximum entropy variant of the policy iteration method. We will first present this derivation, verify that the corresponding algorithm converges to the optimal policy from its density class, and then present a practical deep reinforcement learning algorithm based on this theory.

4.1. Derivation of Soft Policy Iteration

We will begin by deriving soft policy iteration, a general algorithm for learning optimal maximum entropy policies that alternates between policy evaluation and policy improvement in the maximum entropy framework. Our derivation is based on a tabular setting, to enable theoretical analysis and convergence guarantees, and we extend this method into the general continuous setting in the next section. We will show that soft policy iteration converges to the optimal policy within a set of policies which might correspond, for instance, to a set of parameterized densities.

In the policy evaluation step of soft policy iteration, we wish to compute the value of a policy π according to the maximum entropy objective in Equation 1. For a fixed policy, the soft Q-value can be computed iteratively, starting from any function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and repeatedly applying a modified Bellman backup operator $\mathcal{T}^\pi$ given by

$$\mathcal{T}^\pi Q(s_t, \mathbf{a}_t) \triangleq r(s_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V(\mathbf{s}_{t+1})], \quad (2)$$

where

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [Q(s_t, \mathbf{a}_t) - \log \pi(\mathbf{a}_t | s_t)] \quad (3)$$

is the soft state value function. We can obtain the soft value function for any policy π by repeatedly applying $\mathcal{T}^\pi$ as formalized below.

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

Proof. See Appendix B.1. $\square$

In the policy improvement step, we update the policy towards the exponential of the new Q-function. This particular choice of update can be guaranteed to result in an improved policy in terms of its soft value. Since in practice we prefer policies that are tractable, we will additionally restrict the policy to some set of policies Π , which can correspond, for example, to a parameterized family of distributions such as Gaussians. To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence. In the other words, in the policy improvement step, for each state, we update the policy according to

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | s_t) \left\| \frac{\exp(Q^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)} \right\| \right). \quad (4)$$

The partition function $Z^{\pi_{\text{old}}}(s_t)$ normalizes the distribution, and while it is intractable in general, it does not contribute to the gradient with respect to the new policy and can thus be ignored, as noted in the next section. For this projection, we can show that the new, projected policy has a higher value than the old policy with respect to the objective in Equation 1. We formalize this result in Lemma 2.

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(s_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(s_t, \mathbf{a}_t)$ for all $(s_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.2. $\square$

The full soft policy iteration algorithm alternates between the soft policy evaluation and the soft policy improvement steps, and it will provably converge to the optimal maximum entropy policy among the policies in Π (Theorem 1). Although this algorithm will provably find the optimal solution, we can perform it in its exact form only in the tabular case. Therefore, we will next approximate the algorithm for continuous domains, where we need to rely on a function approximator to represent the Q-values, and running the two steps until convergence would be computationally too expensive. The approximation gives rise to a new practical algorithm, called soft actor-critic.

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(s_t, \mathbf{a}_t) \geq Q^\pi(s_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(s_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.3. $\square$

4.2. Soft Actor-Critic

As discussed above, large continuous domains require us to derive a practical approximation to soft policy iteration. To that end, we will use function approximators for both the Q-function and the policy, and instead of running evaluation and improvement to convergence, alternate between optimizing both networks with stochastic gradient descent. We will consider a parameterized state value function $V_\psi(s_t)$, soft Q-function $Q_\theta(s_t, \mathbf{a}_t)$, and a tractable policy $\pi_\phi(\mathbf{a}_t | s_t)$. The parameters of these networks are ψ , θ , and ϕ . For example, the value functions can be modeled as expressive neural networks, and the policy as a Gaussian with mean and covariance given by neural networks. We will next derive update rules for these parameter vectors.

The state value function approximates the soft value. There is no need in principle to include a separate function approximator for the state value, since it is related to the Q-function and policy according to Equation 3. This quantity can be

estimated from a single action sample from the current policy without introducing a bias, but in practice, including a separate function approximator for the soft value can stabilize training and is convenient to train simultaneously with the other networks. The soft value function is trained to minimize the squared residual error

$$J_V(\psi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[\frac{1}{2} (V_\psi(\mathbf{s}_t) - \mathbb{E}_{\mathbf{a}_t \sim \pi_\phi} [Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)])^2 \right] \quad (5)$$

where $\mathcal{D}$ is the distribution of previously sampled states and actions, or a replay buffer. The gradient of Equation 5 can be estimated with an unbiased estimator

$$\hat{\nabla}_\psi J_V(\psi) = \nabla_\psi V_\psi(\mathbf{s}_t) (V_\psi(\mathbf{s}_t) - Q_\theta(\mathbf{s}_t, \mathbf{a}_t) + \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)), \quad (6)$$

where the actions are sampled according to the current policy, instead of the replay buffer. The soft Q-function parameters can be trained to minimize the soft Bellman residual

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \hat{Q}(\mathbf{s}_t, \mathbf{a}_t))^2 \right], \quad (7)$$

with

$$\hat{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V_{\bar{\psi}}(\mathbf{s}_{t+1})], \quad (8)$$

which again can be optimized with stochastic gradients

$$\hat{\nabla}_\theta J_Q(\theta) = \nabla_\theta Q_\theta(\mathbf{s}_t, \mathbf{a}_t) (Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - r(\mathbf{s}_t, \mathbf{a}_t) - \gamma V_{\bar{\psi}}(\mathbf{s}_{t+1})). \quad (9)$$

The update makes use of a target value network $V_{\bar{\psi}}$, where $\bar{\psi}$ can be an exponentially moving average of the value network weights, which has been shown to stabilize training (Mnih et al., 2015). Alternatively, we can update the target weights to match the current value function weights periodically (see Appendix E). Finally, the policy parameters can be learned by directly minimizing the expected KL-divergence in Equation 4:

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[\text{D}_{\text{KL}} \left(\pi_\phi(\cdot | \mathbf{s}_t) \left\| \frac{\exp(Q_\theta(\mathbf{s}_t, \cdot))}{Z_\theta(\mathbf{s}_t)} \right\| \right) \right]. \quad (10)$$

There are several options for minimizing J_π . A typical solution for policy gradient methods is to use the likelihood ratio gradient estimator (Williams, 1992), which does not require backpropagating the gradient through the policy and the target density networks. However, in our case, the target density is the Q-function, which is represented by a neural network and can be differentiated, and it is thus convenient to apply the reparameterization trick instead, resulting in a lower variance estimator. To that end, we reparameterize the policy using a neural network transformation

$$\mathbf{a}_t = f_\phi(\epsilon_t; \mathbf{s}_t), \quad (11)$$

Algorithm 1 Soft Actor-Critic

```

Initialize parameter vectors  $\psi, \bar{\psi}, \theta, \phi$ .
for each iteration do
  for each environment step do
     $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$ 
     $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ 
     $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ 
  end for
  for each gradient step do
     $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$ 
     $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$  for  $i \in \{1, 2\}$ 
     $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ 
     $\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$ 
  end for
end for

```

where ϵ_t is an input noise vector, sampled from some fixed distribution, such as a spherical Gaussian. We can now rewrite the objective in Equation 10 as

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} [\log \pi_\phi(f_\phi(\epsilon_t; \mathbf{s}_t) | \mathbf{s}_t) - Q_\theta(\mathbf{s}_t, f_\phi(\epsilon_t; \mathbf{s}_t))], \quad (12)$$

where π_ϕ is defined implicitly in terms of f_ϕ , and we have noted that the partition function is independent of ϕ and can thus be omitted. We can approximate the gradient of Equation 12 with

$$\begin{aligned} \hat{\nabla}_\phi J_\pi(\phi) &= \nabla_\phi \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) \\ &\quad + (\nabla_{\mathbf{a}_t} \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) - \nabla_{\mathbf{a}_t} Q(\mathbf{s}_t, \mathbf{a}_t)) \nabla_\phi f_\phi(\epsilon_t; \mathbf{s}_t), \end{aligned} \quad (13)$$

where $\mathbf{a}_t$ is evaluated at $f_\phi(\epsilon_t; \mathbf{s}_t)$. This unbiased gradient estimator extends the DDPG style policy gradients (Lillicrap et al., 2015) to any tractable stochastic policy.

Our algorithm also makes use of two Q-functions to mitigate positive bias in the policy improvement step that is known to degrade performance of value based methods (Hasselt, 2010; Fujimoto et al., 2018). In particular, we parameterize two Q-functions, with parameters θ_i , and train them independently to optimize $J_Q(\theta_i)$. We then use the minimum of the Q-functions for the value gradient in Equation 6 and policy gradient in Equation 13, as proposed by Fujimoto et al. (2018). Although our algorithm can learn challenging tasks, including a 21-dimensional Humanoid, using just a single Q-function, we found two Q-functions significantly speed up training, especially on harder tasks. The complete algorithm is described in Algorithm 1. The method alternates between collecting experience from the environment with the current policy and updating the function approximators using the stochastic gradients from batches sampled from a replay buffer. In practice, we take a single environment step followed by one or several gradient steps (see Appendix D

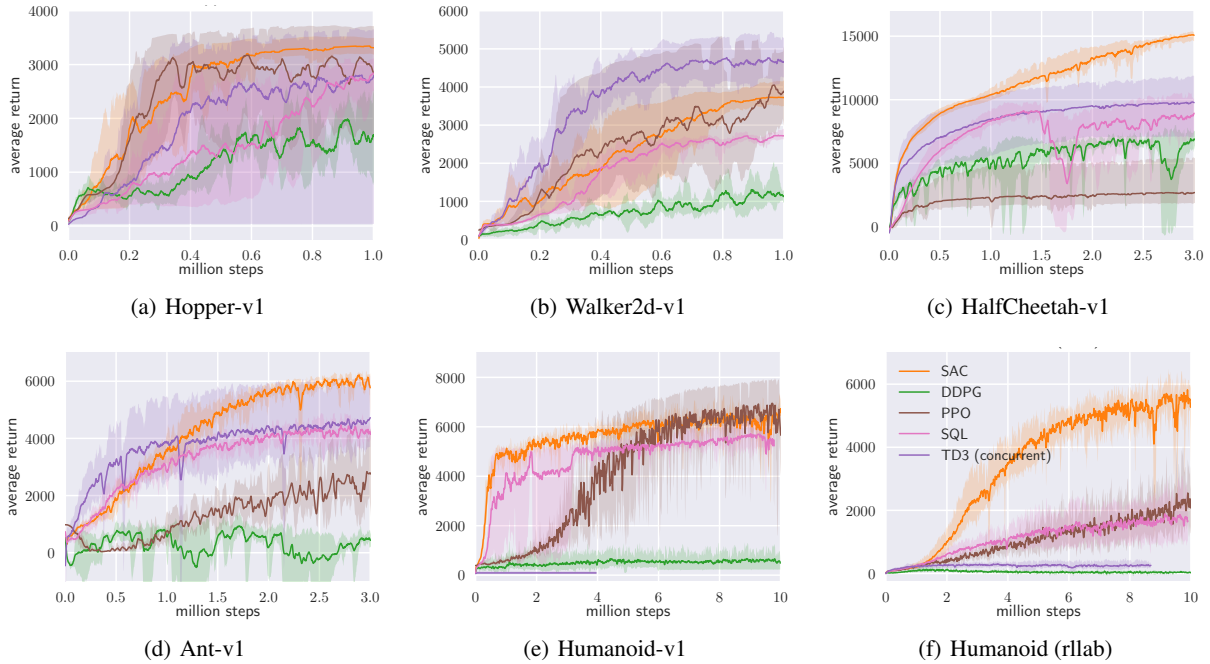


Figure 1. Training curves on continuous control benchmarks. Soft actor-critic (yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

for all hyperparameter). Using off-policy data from a replay buffer is feasible because both value estimators and the policy can be trained entirely on off-policy data. The algorithm is agnostic to the parameterization of the policy, as long as it can be evaluated for any arbitrary state-action tuple.

5. Experiments

The goal of our experimental evaluation is to understand how the sample complexity and stability of our method compares with prior off-policy and on-policy deep reinforcement learning algorithms. We compare our method to prior techniques on a range of challenging continuous control tasks from the OpenAI gym benchmark suite (Brockman et al., 2016) and also on the rllab implementation of the Humanoid task (Duan et al., 2016). Although the easier tasks can be solved by a wide range of different algorithms, the more complex benchmarks, such as the 21-dimensional Humanoid (rllab), are exceptionally difficult to solve with off-policy algorithms (Duan et al., 2016). The stability of the algorithm also plays a large role in performance: easier tasks make it more practical to tune hyperparameters to achieve good results, while the already narrow basins of effective hyperparameters become prohibitively small for the more sensitive algorithms on the hardest benchmarks, leading to poor performance (Gu et al., 2016).

We compare our method to deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), an algorithm that is regarded as one of the more efficient off-policy deep RL methods (Duan et al., 2016); proximal policy optimization (PPO) (Schulman et al., 2017b), a stable and effective on-policy policy gradient algorithm; and soft Q-learning (SQL) (Haarnoja et al., 2017), a recent off-policy algorithm for learning maximum entropy policies. Our SQL implementation also includes two Q-functions, which we found to improve its performance in most environments. We additionally compare to twin delayed deep deterministic policy gradient algorithm (TD3) (Fujimoto et al., 2018), using the author-provided implementation. This is an extension to DDPG, proposed concurrently to our method, that first applied the double Q-learning trick to continuous control along with other improvements. We have included trust region path consistency learning (Trust-PCL) (Nachum et al., 2017b) and two other variants of SAC in Appendix E. We turned off the exploration noise for evaluation for DDPG and PPO. For maximum entropy algorithms, which do not explicitly inject exploration noise, we either evaluated with the exploration noise (SQL) or use the mean action (SAC). The source code of our SAC implementation¹ and videos² are available online.

¹github.com/haarnoja/sac

²sites.google.com/view/soft-actor-critic

5.1. Comparative Evaluation

Figure 1 shows the total average return of evaluation rollouts during training for DDPG, PPO, and TD3. We train five different instances of each algorithm with different random seeds, with each performing one evaluation rollout every 1000 environment steps. The solid curves corresponds to the mean and the shaded region to the minimum and maximum returns over the five trials.

The results show that, overall, SAC performs comparably to the baseline methods on the easier tasks and outperforms them on the harder tasks with a large margin, both in terms of learning speed and the final performance. For example, DDPG fails to make any progress on Ant-v1, Humanoid-v1, and Humanoid (rllab), a result that is corroborated by prior work (Gu et al., 2016; Duan et al., 2016). SAC also learns considerably faster than PPO as a consequence of the large batch sizes PPO needs to learn stably on more high-dimensional and complex tasks. Another maximum entropy RL algorithm, SQL, can also learn all tasks, but it is slower than SAC and has worse asymptotic performance. The quantitative results attained by SAC in our experiments also compare very favorably to results reported by other methods in prior work (Duan et al., 2016; Gu et al., 2016; Henderson et al., 2017), indicating that both the sample efficiency and final performance of SAC on these benchmark tasks exceeds the state of the art. All hyperparameters used in this experiment for SAC are listed in Appendix D.

5.2. Ablation Study

The results in the previous section suggest that algorithms based on the maximum entropy principle can outperform conventional RL methods on challenging tasks such as the humanoid tasks. In this section, we further examine which particular components of SAC are important for good performance. We also examine how sensitive SAC is to some of the most important hyperparameters, namely reward scaling and target value update smoothing constant.

Stochastic vs. deterministic policy. Soft actor-critic learns stochastic policies via a maximum entropy objective. The entropy appears in both the policy and value function. In the policy, it prevents premature convergence of the policy variance (Equation 10). In the value function, it encourages exploration by increasing the value of regions of state space that lead to high-entropy behavior (Equation 5). To compare how the stochasticity of the policy and entropy maximization affects the performance, we compare to a deterministic variant of SAC that does not maximize the entropy and that closely resembles DDPG, with the exception of having two Q-functions, using hard target updates, not having a separate target actor, and using fixed rather than learned exploration noise. Figure 2 compares five individual runs with both variants, initialized with different random

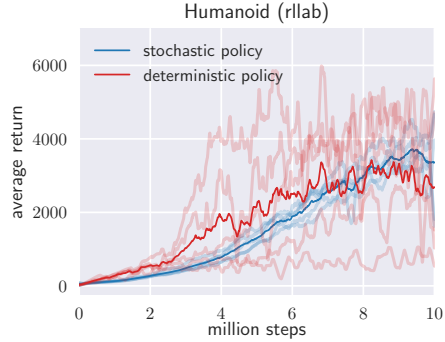


Figure 2. Comparison of SAC (blue) and a deterministic variant of SAC (red) in terms of the stability of individual random seeds on the Humanoid (rllab) benchmark. The comparison indicates that stochasticity can stabilize training as the variability between the seeds becomes much higher with a deterministic policy.

seeds. Soft actor-critic performs much more consistently, while the deterministic variant exhibits very high variability across seeds, indicating substantially worse stability. As evident from the figure, learning a stochastic policy with entropy maximization can drastically stabilize training. This becomes especially important with harder tasks, where tuning hyperparameters is challenging. In this comparison, we updated the target value network weights with hard updates, by periodically overwriting the target network parameters to match the current value network (see Appendix E for a comparison of average performance on all benchmark tasks).

Policy evaluation. Since SAC converges to stochastic policies, it is often beneficial to make the final policy deterministic at the end for best performance. For evaluation, we approximate the maximum a posteriori action by choosing the mean of the policy distribution. Figure 3(a) compares training returns to evaluation returns obtained with this strategy indicating that deterministic evaluation can yield better performance. It should be noted that all of the training curves depict the sum of rewards, which is different from the objective optimized by SAC and other maximum entropy RL algorithms, including SQL and Trust-PCL, which maximize also the entropy of the policy.

Reward scale. Soft actor-critic is particularly sensitive to the scaling of the reward signal, because it serves the role of the temperature of the energy-based optimal policy and thus controls its stochasticity. Larger reward magnitudes correspond to lower entropies. Figure 3(b) shows how learning performance changes when the reward scale is varied: For small reward magnitudes, the policy becomes nearly uniform, and consequently fails to exploit the reward signal, resulting in substantial degradation of performance. For large reward magnitudes, the model learns quickly at first,

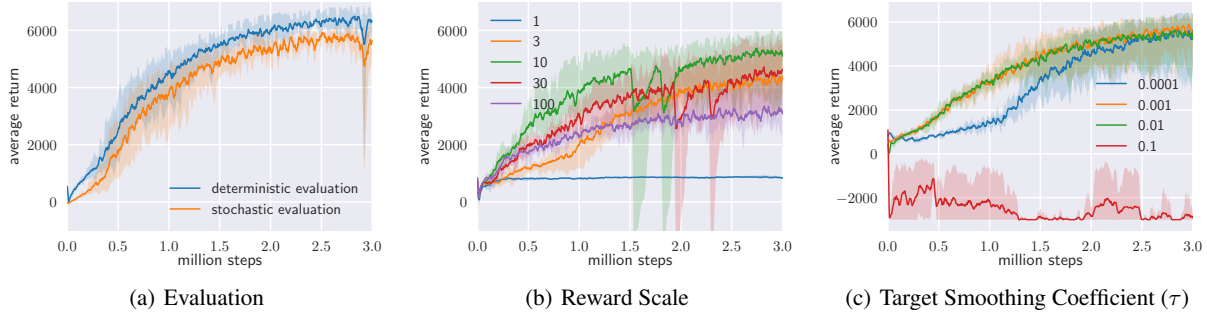


Figure 3. Sensitivity of soft actor-critic to selected hyperparameters on Ant-v1 task. (a) Evaluating the policy using the mean action generally results in a higher return. Note that the policy is trained to maximize also the entropy, and the mean action does not, in general, correspond the optimal action for the maximum return objective. (b) Soft actor-critic is sensitive to reward scaling since it is related to the temperature of the optimal policy. The optimal reward scale varies between environments, and should be tuned for each task separately. (c) Target value smoothing coefficient τ is used to stabilize training. Fast moving target (large τ) can result in instabilities (red), whereas slow moving target (small τ) makes training slower (blue).

but the policy then becomes nearly deterministic, leading to poor local minima due to lack of adequate exploration. With the right reward scaling, the model balances exploration and exploitation, leading to faster learning and better asymptotic performance. In practice, we found reward scale to be the only hyperparameter that requires tuning, and its natural interpretation as the inverse of the temperature in the maximum entropy framework provides good intuition for how to adjust this parameter.

Target network update. It is common to use a separate target value network that slowly tracks the actual value function to improve stability. We use an exponentially moving average, with a smoothing constant τ , to update the target value network weights as common in the prior work (Lillicrap et al., 2015; Mnih et al., 2015). A value of one corresponds to a hard update where the weights are copied directly at every iteration and zero to not updating the target at all. In Figure 3(c), we compare the performance of SAC when τ varies. Large τ can lead to instabilities while small τ can make training slower. However, we found the range of suitable values of τ to be relatively wide and we used the same value (0.005) across all of the tasks. In Figure 4 (Appendix E) we also compare to another variant of SAC, where instead of using exponentially moving average, we copy over the current network weights directly into the target network every 1000 gradient steps. We found this variant to benefit from taking more than one gradient step between the environment steps, which can improve performance but also increases the computational cost.

6. Conclusion

We present soft actor-critic (SAC), an off-policy maximum entropy deep reinforcement learning algorithm that provides sample-efficient learning while retaining the benefits of entropy maximization and stability. Our theoretical results derive soft policy iteration, which we show to converge to the optimal policy. From this result, we can formulate a soft actor-critic algorithm, and we empirically show that it outperforms state-of-the-art model-free deep RL methods, including the off-policy DDPG algorithm and the on-policy PPO algorithm. In fact, the sample efficiency of this approach actually exceeds that of DDPG by a substantial margin. Our results suggest that stochastic, entropy maximizing reinforcement learning algorithms can provide a promising avenue for improved robustness and stability, and further exploration of maximum entropy methods, including methods that incorporate second order information (e.g., trust regions (Schulman et al., 2015)) or more expressive policy classes is an exciting avenue for future work.

Acknowledgments

We would like to thank Vitchyr Pong for insightful discussions and help in implementing our algorithm as well as providing the DDPG baseline code; Ofir Nachum for offering support in running Trust-PCL experiments; and George Tucker for his valuable feedback on an early version of this paper. This work was supported by Siemens and Berkeley DeepDrive.

A. Maximum Entropy Objective

The exact definition of the discounted maximum entropy objective is complicated by the fact that, when using a discount factor for policy gradient methods, we typically do not discount the state distribution, only the rewards. In that sense, discounted policy gradients typically do not optimize the true discounted objective. Instead, they optimize average reward, with the discount serving to reduce variance, as discussed by [Thomas \(2014\)](#). However, we can define the objective that is optimized under a discount factor as

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} \left[\sum_{l=t}^{\infty} \gamma^{l-t} \mathbb{E}_{\mathbf{s}_l \sim p, \mathbf{a}_l \sim \pi} [r(\mathbf{s}_l, \mathbf{a}_l) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_l))] | \mathbf{s}_t, \mathbf{a}_t \right]. \quad (14)$$

This objective corresponds to maximizing the discounted expected reward and entropy for future states originating from every state-action tuple $(\mathbf{s}_t, \mathbf{a}_t)$ weighted by its probability ρ_{π} under the current policy.

B. Proofs

B.1. Lemma 1

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^{\pi}$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^{\pi} Q^k$. Then the sequence Q^k will converge to the soft Q -value of π as $k \rightarrow \infty$.*

Proof. Define the entropy augmented reward as $r_{\pi}(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [\mathcal{H}(\pi(\cdot | \mathbf{s}_{t+1}))]$ and rewrite the update rule as

$$Q(\mathbf{s}_t, \mathbf{a}_t) \leftarrow r_{\pi}(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p, \mathbf{a}_{t+1} \sim \pi} [Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})] \quad (15)$$

and apply the standard convergence results for policy evaluation ([Sutton & Barto, 1998](#)). The assumption $|\mathcal{A}| < \infty$ is required to guarantee that the entropy augmented reward is bounded. $\square$

B.2. Lemma 2

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. Let $\pi_{\text{old}} \in \Pi$ and let $Q^{\pi_{\text{old}}}$ and $V^{\pi_{\text{old}}}$ be the corresponding soft state-action value and soft state value, and let π_{new} be defined as

$$\begin{aligned} \pi_{\text{new}}(\cdot | \mathbf{s}_t) &= \arg \min_{\pi' \in \Pi} D_{\text{KL}}(\pi'(\cdot | \mathbf{s}_t) \parallel \exp(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot) - \log Z^{\pi_{\text{old}}}(\mathbf{s}_t))) \\ &= \arg \min_{\pi' \in \Pi} J_{\pi_{\text{old}}}(\pi'(\cdot | \mathbf{s}_t)). \end{aligned} \quad (16)$$

It must be the case that $J_{\pi_{\text{old}}}(\pi_{\text{new}}(\cdot | \mathbf{s}_t)) \leq J_{\pi_{\text{old}}}(\pi_{\text{old}}(\cdot | \mathbf{s}_t))$, since we can always choose $\pi_{\text{new}} = \pi_{\text{old}} \in \Pi$. Hence

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} [\log \pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)] \leq \mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{old}}} [\log \pi_{\text{old}}(\mathbf{a}_t | \mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)], \quad (17)$$

and since partition function $Z^{\pi_{\text{old}}}$ depends only on the state, the inequality reduces to

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} [Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t)] \geq V^{\pi_{\text{old}}}(\mathbf{s}_t). \quad (18)$$

Next, consider the soft Bellman equation:

$$\begin{aligned} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V^{\pi_{\text{old}}}(\mathbf{s}_{t+1})] \\ &\leq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [\mathbb{E}_{\mathbf{a}_{t+1} \sim \pi_{\text{new}}} [Q^{\pi_{\text{old}}}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \log \pi_{\text{new}}(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})]] \\ &\vdots \\ &\leq Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t), \end{aligned} \quad (19)$$

where we have repeatedly expanded $Q^{\pi_{\text{old}}}$ on the RHS by applying the soft Bellman equation and the bound in Equation 18. Convergence to $Q^{\pi_{\text{new}}}$ follows from Lemma 1. $\square$

B.3. Theorem 1

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement to any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. Let π_i be the policy at iteration i . By Lemma 2, the sequence Q^{π_i} is monotonically increasing. Since Q^π is bounded above for $\pi \in \Pi$ (both the reward and entropy are bounded), the sequence converges to some π^* . We will still need to show that π^* is indeed optimal. At convergence, it must be case that $J_{\pi^*}(\pi^*(\cdot | \mathbf{s}_t)) < J_{\pi^*}(\pi(\cdot | \mathbf{s}_t))$ for all $\pi \in \Pi$, $\pi \neq \pi^*$. Using the same iterative argument as in the proof of Lemma 2, we get $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) > Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, that is, the soft value of any other policy in Π is lower than that of the converged policy. Hence π^* is optimal in Π . $\square$

C. Enforcing Action Bounds

We use an unbounded Gaussian as the action distribution. However, in practice, the actions needs to be bounded to a finite interval. To that end, we apply an invertible squashing function ($\tanh$) to the Gaussian samples, and employ the change of variables formula to compute the likelihoods of the bounded actions. In the other words, let $\mathbf{u} \in \mathbb{R}^D$ be a random variable and $\mu(\mathbf{u}|\mathbf{s})$ the corresponding density with infinite support. Then $\mathbf{a} = \tanh(\mathbf{u})$, where $\tanh$ is applied elementwise, is a random variable with support in $(-1, 1)$ with a density given by

$$\pi(\mathbf{a}|\mathbf{s}) = \mu(\mathbf{u}|\mathbf{s}) \left| \det \left(\frac{d\mathbf{a}}{d\mathbf{u}} \right) \right|^{-1}. \quad (20)$$

Since the Jacobian $d\mathbf{a}/d\mathbf{u} = \text{diag}(1 - \tanh^2(\mathbf{u}))$ is diagonal, the log-likelihood has a simple form

$$\log \pi(\mathbf{a}|\mathbf{s}) = \log \mu(\mathbf{u}|\mathbf{s}) - \sum_{i=1}^D \log (1 - \tanh^2(u_i)), \quad (21)$$

where u_i is the i^{th} element of $\mathbf{u}$.

D. Hyperparameters

Table 1 lists the common SAC parameters used in the comparative evaluation in Figure 1 and Figure 4. Table 2 lists the reward scale parameter that was tuned for each environment.

Table 1. SAC Hyperparameters

Parameter	Value
<i>Shared</i>	
optimizer	Adam (Kingma & Ba, 2015)
learning rate	$3 \cdot 10^{-4}$
discount (γ)	0.99
replay buffer size	10^6
number of hidden layers (all networks)	2
number of hidden units per layer	256
number of samples per minibatch	256
nonlinearity	ReLU
<i>SAC</i>	
target smoothing coefficient (τ)	0.005
target update interval	1
gradient steps	1
<i>SAC (hard target update)</i>	
target smoothing coefficient (τ)	1
target update interval	1000
gradient steps (except humanoids)	4
gradient steps (humanoids)	1

Table 2. SAC Environment Specific Parameters

Environment	Action Dimensions	Reward Scale
Hopper-v1	3	5
Walker2d-v1	6	5
HalfCheetah-v1	6	5
Ant-v1	8	5
Humanoid-v1	17	20
Humanoid (rllab)	21	10

E. Additional Baseline Results

Figure 4 compares SAC to Trust-PCL (Figure 4). Trust-PC fails to solve most of the task within the given number of environment steps, although it can eventually solve the easier tasks (Nachum et al., 2017b) if ran longer. The figure also includes two variants of SAC: a variant that periodically copies the target value network weights directly instead of using exponentially moving average, and a deterministic ablation which assumes a deterministic policy in the value update (Equation 6) and the policy update (Equation 13), and thus strongly resembles DDPG with the exception of having two Q-functions, using hard target updates, not having a separate target actor, and using fixed exploration noise rather than learned. Both of these methods can learn all of the tasks and they perform comparably to SAC on all but Humanoid (rllab) task, on which SAC is the fastest.

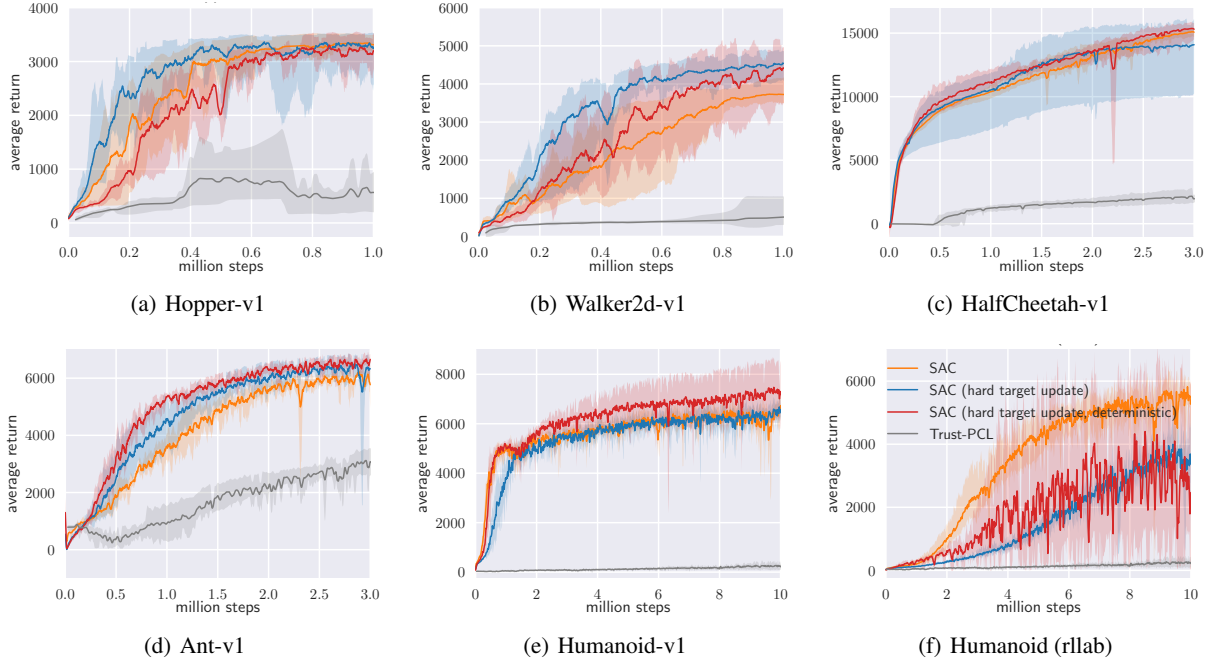


Figure 4. Training curves for additional baseline (Trust-PCL) and for two SAC variants. Soft actor-critic with hard target update (blue) differs from standard SAC in that it copies the value function network weights directly every 1000 iterations, instead of using exponentially smoothed average of the weights. The deterministic ablation (red) uses a deterministic policy with fixed Gaussian exploration noise, does not use a value function, drops the entropy terms in the actor and critic function updates, and uses hard target updates for the target Q-functions. It is equivalent to DDPG that uses two Q-functions, hard target updates, and removes the target actor.

Soft Actor-Critic Algorithms and Applications

Tuomas Haarnoja^{*†‡} Aurick Zhou^{*†} Kristian Hartikainen^{*†} George Tucker[‡]
 Sehoon Ha[‡] Jie Tan[‡] Vikash Kumar[‡] Henry Zhu[†] Abhishek Gupta[†]
 Pieter Abbeel[†] Sergey Levine^{†‡}

Abstract

Model-free deep reinforcement learning (RL) algorithms have been successfully applied to a range of challenging sequential decision making and control tasks. However, these methods typically suffer from two major challenges: high sample complexity and brittleness to hyperparameters. Both of these challenges limit the applicability of such methods to real-world domains. In this paper, we describe Soft Actor-Critic (SAC), our recently introduced off-policy actor-critic algorithm based on the maximum entropy RL framework. In this framework, the actor aims to simultaneously maximize expected return and entropy; that is, to succeed at the task while acting as randomly as possible. We extend SAC to incorporate a number of modifications that accelerate training and improve stability with respect to the hyperparameters, including a constrained formulation that automatically tunes the temperature hyperparameter. We systematically evaluate SAC on a range of benchmark tasks, as well as challenging real-world tasks such as locomotion for a quadrupedal robot and robotic manipulation with a dexterous hand. With these improvements, SAC achieves state-of-the-art performance, outperforming prior on-policy and off-policy methods in sample-efficiency and asymptotic performance. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving similar performance across different random seeds. These results suggest that SAC is a promising candidate for learning in real-world robotics tasks.

1 Introduction

Model-free deep reinforcement learning (RL) algorithms have been applied in a range of challenging domains, from games (Mnih et al., 2013; Silver et al., 2016) to robotic control (Gu et al., 2017; Haarnoja et al., 2018b). The combination of RL and high-capacity function approximators such as neural networks holds the promise of automating a wide range of decision making and control tasks, but widespread adoption of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al.,

[†]UC Berkeley, [‡]Google Brain, ^{*}Contributed equally



(2017b) or A3C (Mnih et al., 2016), require new samples to be collected for (nearly) every update to the policy. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al., 2009). This challenge is further exacerbated in continuous state and action spaces, where a separate actor network is often used to perform the maximization in Q-learning.

In (Haarnoja et al., 2018c), we introduced the Soft Actor-Critic (SAC) algorithm based on the maximum entropy framework (Ziebart et al., 2008; Toussaint, 2009; Rawlik et al., 2012; Fox et al., 2016; Haarnoja et al., 2017). In the first sections of this paper, we summarize the SAC algorithm, describe the reasoning behind the design choices, and present key theoretical results from (Haarnoja et al., 2018c). Unfortunately, SAC as presented in (Haarnoja et al., 2018c) can suffer from brittleness to the temperature hyperparameter. Unlike in conventional reinforcement learning, where the optimal policy is independent of scaling of the reward function, in maximum entropy reinforcement learning the scaling factor has to be compensated by the choice of a suitable temperature, and a sub-optimal temperature can drastically degrade performance (Haarnoja et al., 2018c). To resolve this issue, we devise an automatic gradient-based temperature tuning method that adjusts the expected entropy over the visited states to match a target value. Although this modification is technically simple, we find that in practice it largely eliminates the need for per-task hyperparameter tuning. Finally, we present empirical results that show that Soft Actor-Critic attains a substantial improvement in both performance and sample efficiency over prior off-policy and on-policy methods including the recently introduced twin delayed deep deterministic (TD3) policy gradient algorithm (Fujimoto et al., 2018). We also evaluate our method on real-world challenging tasks such as locomotion for a quadrupedal robot and robotic manipulation with a dexterous hand from image observations.

2 Related Work

Maximum entropy reinforcement learning generalizes the expected return RL objective, although the original objective can be recovered in the zero temperature limit (Haarnoja et al., 2017). More importantly, the maximum entropy formulation provides a substantial improvement in exploration and robustness: as discussed by Ziebart (2010), maximum entropy policies are robust in the face of model and estimation errors, and as demonstrated by (Haarnoja et al., 2017), they improve exploration by acquiring diverse behaviors. Prior work has proposed model-free deep RL algorithms that perform on-policy learning with entropy maximization (O’Donoghue et al., 2016), as well as off-policy methods based on soft Q-learning and its variants (Schulman et al., 2017a; Nachum et al., 2017a; Haarnoja et al., 2017). However, the on-policy variants suffer from poor sample complexity for the reasons discussed above, while the off-policy variants require complex approximate inference procedures in continuous action spaces.

Our soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to encourage stability and exploration. We review prior work that draw on some of these ideas in this section. Actor-critic algorithms are typically derived starting from policy iteration, which alternates between *policy evaluation*—computing the value function for a policy—and *policy improvement*—using the value function to obtain a better policy (Barto et al., 1983; Sutton & Barto, 1998). In large-scale reinforcement learning problems, it is typically impractical to run either of these steps to convergence, and instead the value function and policy are optimized jointly. In this case, the policy is referred to as the actor, and the value function as the critic. Many actor-critic algorithms build on the standard, on-policy policy gradient formulation to update the actor (Peters & Schaal, 2008), and many of them also consider the entropy of the policy, but instead of maximizing the entropy, they use it as a regularizer (Schulman et al., 2017b, 2015; Mnih et al., 2016; Gruslys et al., 2017). On-policy training tends to improve stability but results in poor sample complexity.

There have been efforts to increase the sample efficiency while retaining robustness by incorporating off-policy samples and by using higher order variance reduction techniques (O’Donoghue et al., 2016; Gu et al., 2016). However, fully off-policy algorithms still attain better efficiency. A particularly

popular off-policy actor-critic method, DDPG (Lillicrap et al., 2015), which is a deep variant of the deterministic policy gradient (Silver et al., 2014) algorithm, uses a Q-function estimator to enable off-policy learning, and a deterministic actor that maximizes this Q-function. As such, this method can be viewed both as a deterministic actor-critic algorithm and an approximate Q-learning algorithm. Unfortunately, the interplay between the deterministic actor network and the Q-function typically makes DDPG extremely difficult to stabilize and brittle to hyperparameter settings (Duan et al., 2016; Henderson et al., 2017). As a consequence, it is difficult to extend DDPG to complex, high-dimensional tasks, and on-policy policy gradient methods still tend to produce the best results in such settings (Gu et al., 2016). Our method instead combines off-policy actor-critic training with a stochastic actor, and further aims to maximize the entropy of this actor with an entropy maximization objective. We find that this actually results in a considerably more stable and scalable algorithm that, in practice, exceeds both the efficiency and final performance of DDPG. A similar method can be derived as a zero-step special case of stochastic value gradients (SVG(0)) (Heess et al., 2015). However, SVG(0) differs from our method in that it optimizes the standard maximum expected return objective.

Maximum entropy reinforcement learning optimizes policies to maximize both the expected return and the expected entropy of the policy. This framework has been used in many contexts, from inverse reinforcement learning (Ziebart et al., 2008) to optimal control (Todorov, 2008; Toussaint, 2009; Rawlik et al., 2012). Maximum a posteriori policy optimization (MPO) makes use of the probabilistic view and optimizes the standard RL objective via expectation maximization (Abdolmaleki et al., 2018). In guided policy search (Levine & Koltun, 2013; Levine et al., 2016), the maximum entropy distribution is used to guide policy learning towards high-reward regions. More recently, several papers have noted the connection between Q-learning and policy gradient methods in the framework of maximum entropy learning (O’Donoghue et al., 2016; Haarnoja et al., 2017; Nachum et al., 2017a; Schulman et al., 2017a). While most of the prior model-free works assume a discrete action space, Nachum et al. (2017b) approximate the maximum entropy distribution with a Gaussian, and Haarnoja et al. (2017) with a sampling network trained to draw samples from the optimal policy. Although the soft Q-learning algorithm proposed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution. Hence, Haarnoja et al. (2017) motivates the actor network as an approximate sampler, rather than the actor in an actor-critic algorithm. Crucially, the convergence of this method hinges on how well this sampler approximates the true posterior. In contrast, we prove that our method converges to the optimal policy from a given policy class, regardless of the policy parameterization. Furthermore, these prior maximum entropy methods generally do not exceed the performance of state-of-the-art off-policy algorithms, such as TD3 (Fujimoto et al., 2018) or MPO (Abdolmaleki et al., 2018), when learning from scratch, though they may have other benefits, such as improved exploration and ease of fine-tuning.

3 Preliminaries

We first introduce notation and summarize the standard and maximum entropy reinforcement learning frameworks.

3.1 Notation

We will address learning of maximum entropy policies in continuous action spaces. Our reinforcement learning problem can be defined as policy search in an a Markov decision process (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, p, r)$. The state space $\mathcal{S}$ and action space $\mathcal{A}$ are assumed to be continuous, and the state transition probability $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, \infty)$ represents the probability density of the next state $\mathbf{s}_{t+1} \in \mathcal{S}$ given the current state $\mathbf{s}_t \in \mathcal{S}$ and action $\mathbf{a}_t \in \mathcal{A}$. The environment emits a reward $r : \mathcal{S} \times \mathcal{A} \rightarrow [r_{\min}, r_{\max}]$ on each transition. We will also use $\rho_\pi(\mathbf{s}_t)$ and $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ to denote the state and state-action marginals of the trajectory distribution induced by a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$.

3.2 Maximum Entropy Reinforcement Learning

The standard reinforcement learning objective is the expected sum of rewards $\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t)]$ and our goal is to learn a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$ that maximizes that ob-

jective. The maximum entropy objective (see e.g. (Ziebart, 2010)) generalizes the standard objective by augmenting it with an entropy term, such that the optimal policy additionally aims to maximize its entropy at each visited state:

$$\pi^* = \arg \max_{\pi} \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))], \quad (1)$$

where α is the temperature parameter that determines the relative importance of the entropy term versus the reward, and thus controls the stochasticity of the optimal policy. Although the maximum entropy objective differs from the standard maximum expected return objective used in conventional reinforcement learning, the conventional objective can be recovered in the limit as $\alpha \rightarrow 0$. If we wish to extend either the conventional or the maximum entropy RL objective to infinite horizon problems, it is convenient to also introduce a discount factor γ to ensure that the sum of expected rewards (and entropies) is finite. In the context of policy search algorithms, the use of a discount factor is actually a somewhat nuanced choice, and writing down the precise objective that is optimized when using the discount factor is non-trivial (Thomas, 2014). We include the discounted, infinite-horizon objective in Appendix A, but we will use the discount γ in the following derivations and in our final algorithm.

The maximum entropy objective has a number of conceptual and practical advantages. First, the policy is incentivized to explore more widely, while giving up on clearly unpromising avenues. Second, the policy can capture multiple modes of near-optimal behavior. In problem settings where multiple actions seem equally attractive, the policy will commit equal probability mass to those actions. In practice, we observe improved exploration with this objective, as also has been reported in the prior work (Schulman et al., 2017a), and we observe that it considerably improves learning speed over state-of-art methods that optimize the conventional RL objective function.

Optimization problems of this type have been explored in a number of prior works (Kappen, 2005; Todorov, 2007; Ziebart et al., 2008). These prior methods have proposed directly solving for the optimal Q-function, from which the optimal policy can be recovered (Ziebart et al., 2008; Fox et al., 2016; Haarnoja et al., 2017). In the following section, we discuss how we can devise a soft actor-critic algorithm through a policy iteration formulation, where we instead evaluate the Q-function of the current policy and update the policy through an off-policy gradient update. Though such algorithms have previously been proposed for conventional reinforcement learning, our method is, to our knowledge, the first off-policy actor-critic method in the maximum entropy reinforcement learning framework.

4 From Soft Policy Iteration to Soft Actor-Critic

Our off-policy soft actor-critic algorithm can be derived starting from a maximum entropy variant of the policy iteration method. We will first present this derivation, verify that the corresponding algorithm converges to the optimal policy from its density class, and then present a practical deep reinforcement learning algorithm based on this theory. In this section, we treat the temperature as a constant, and later in Section 5 propose an extension to SAC that adjusts the temperature automatically to match an entropy target in expectation.

4.1 Soft Policy Iteration

We will begin by deriving soft policy iteration, a general algorithm for learning optimal maximum entropy policies that alternates between policy evaluation and policy improvement in the maximum entropy framework. Our derivation is based on a tabular setting, to enable theoretical analysis and convergence guarantees, and we extend this method into the general continuous setting in the next section. We will show that soft policy iteration converges to the optimal policy within a set of policies which might correspond, for instance, to a set of parameterized densities.

In the policy evaluation step of soft policy iteration, we wish to compute the value of a policy π according to the maximum entropy objective. For a fixed policy, the soft Q-value can be computed iteratively, starting from any function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and repeatedly applying a modified Bellman backup operator $\mathcal{T}^{\pi}$ given by

$$\mathcal{T}^{\pi} Q(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V(\mathbf{s}_{t+1})], \quad (2)$$

where

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [Q(\mathbf{s}_t, \mathbf{a}_t) - \alpha \log \pi(\mathbf{a}_t | \mathbf{s}_t)] \quad (3)$$

is the soft state value function. We can obtain the soft Q-function for any policy π by repeatedly applying $\mathcal{T}^\pi$ as formalized below.

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-function of π as $k \rightarrow \infty$.*

Proof. See Appendix B.1. □

In the policy improvement step, we update the policy towards the exponential of the new soft Q-function. This particular choice of update can be guaranteed to result in an improved policy in terms of its soft value. Since in practice we prefer policies that are tractable, we will additionally restrict the policy to some set of policies Π , which can correspond, for example, to a parameterized family of distributions such as Gaussians. To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence. In the other words, in the policy improvement step, for each state, we update the policy according to

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | \mathbf{s}_t) \left\| \frac{\exp \left(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot) \right)}{Z^{\pi_{\text{old}}}(\mathbf{s}_t)} \right) \right). \quad (4)$$

The partition function $Z^{\pi_{\text{old}}}(\mathbf{s}_t)$ normalizes the distribution, and while it is intractable in general, it does not contribute to the gradient with respect to the new policy and can thus be ignored. For this projection, we can show that the new, projected policy has a higher value than the old policy with respect to the maximum entropy objective. We formalize this result in Lemma 2.

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.2. □

The full soft policy iteration algorithm alternates between the soft policy evaluation and the soft policy improvement steps, and it will provably converge to the optimal maximum entropy policy among the policies in Π (Theorem 1). Although this algorithm will provably find the optimal solution, we can perform it in its exact form only in the tabular case. Therefore, we will next approximate the algorithm for continuous domains, where we need to rely on a function approximator to represent the Q-values, and running the two steps until convergence would be computationally too expensive. The approximation gives rise to a new practical algorithm, called soft actor-critic.

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.3. □

4.2 Soft Actor-Critic

As discussed above, large continuous domains require us to derive a practical approximation to soft policy iteration. To that end, we will use function approximators for both the soft Q-function and the policy, and instead of running evaluation and improvement to convergence, alternate between optimizing both networks with stochastic gradient descent. We will consider a parameterized soft Q-function $Q_\theta(\mathbf{s}_t, \mathbf{a}_t)$ and a tractable policy $\pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$. The parameters of these networks are θ and ϕ . For example, the soft Q-function can be modeled as expressive neural networks, and the policy as a Gaussian with mean and covariance given by neural networks. We will next derive update rules for these parameter vectors.

The soft Q-function parameters can be trained to minimize the soft Bellman residual

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V_{\bar{\theta}}(\mathbf{s}_{t+1})]))^2 \right], \quad (5)$$

where the value function is implicitly parameterized through the soft Q-function parameters via Equation 3 and it can be optimized with stochastic gradients¹

$$\nabla_{\theta} J_Q(\theta) = \nabla_{\theta} Q_{\theta}(\mathbf{a}_t, \mathbf{s}_t) (Q_{\theta}(\mathbf{s}_t, \mathbf{a}_t) - (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma (Q_{\bar{\theta}}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \alpha \log(\pi_{\phi}(\mathbf{a}_{t+1}|\mathbf{s}_{t+1}))))). \quad (6)$$

The update makes use of a target soft Q-function with parameters $\bar{\theta}$ that are obtained as an exponentially moving average of the soft Q-function weights, which has been shown to stabilize training (Mnih et al., 2015). Finally, the policy parameters can be learned by directly minimizing the expected KL-divergence in Equation 4 (multiplied by α and ignoring the constant log-partition function and by α):

$$J_{\pi}(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} [\mathbb{E}_{\mathbf{a}_t \sim \pi_{\phi}} [\alpha \log(\pi_{\phi}(\mathbf{a}_t|\mathbf{s}_t)) - Q_{\theta}(\mathbf{s}_t, \mathbf{s}_t)]] \quad (7)$$

There are several options for minimizing J_{π} . A typical solution for policy gradient methods is to use the likelihood ratio gradient estimator (Williams, 1992), which does not require backpropagating the gradient through the policy and the target density networks. However, in our case, the target density is the Q-function, which is represented by a neural network and can be differentiated, and it is thus convenient to apply the reparameterization trick instead, resulting in a lower variance estimator. To that end, we reparameterize the policy using a neural network transformation

$$\mathbf{a}_t = f_{\phi}(\epsilon_t; \mathbf{s}_t), \quad (8)$$

where ϵ_t is an input noise vector, sampled from some fixed distribution, such as a spherical Gaussian. We can now rewrite the objective in Equation 7 as

$$J_{\pi}(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} [\alpha \log \pi_{\phi}(f_{\phi}(\epsilon_t; \mathbf{s}_t)|\mathbf{s}_t) - Q_{\theta}(\mathbf{s}_t, f_{\phi}(\epsilon_t; \mathbf{s}_t))], \quad (9)$$

where π_{ϕ} is defined implicitly in terms of f_{ϕ} . We can approximate the gradient of Equation 9 with

$$\hat{\nabla}_{\phi} J_{\pi}(\phi) = \nabla_{\phi} \alpha \log(\pi_{\phi}(\mathbf{a}_t|\mathbf{s}_t)) + (\nabla_{\mathbf{a}_t} \alpha \log(\pi_{\phi}(\mathbf{a}_t|\mathbf{s}_t)) - \nabla_{\mathbf{a}_t} Q(\mathbf{s}_t, \mathbf{a}_t)) \nabla_{\phi} f_{\phi}(\epsilon_t; \mathbf{s}_t), \quad (10)$$

where $\mathbf{a}_t$ is evaluated at $f_{\phi}(\epsilon_t; \mathbf{s}_t)$. This unbiased gradient estimator extends the DDPG style policy gradients (Lillicrap et al., 2015) to any tractable stochastic policy.

5 Automating Entropy Adjustment for Maximum Entropy RL

In the previous section, we derived a practical off-policy algorithm for learning maximum entropy policies of a given temperature. Unfortunately, choosing the optimal temperature is non-trivial, and the temperature needs to be tuned for each task. Instead of requiring the user to set the temperature manually, we can automate this process by formulating a different maximum entropy reinforcement learning objective, where the entropy is treated as a constraint. The magnitude of the reward differs not only across tasks, but it also depends on the policy, which improves over time during training. Since the optimal entropy depends on this magnitude, this makes the temperature adjustment particularly difficult: the entropy can vary unpredictably both across tasks and during training as the policy becomes better. Simply forcing the entropy to a fixed value is a poor solution, since the policy should be free to explore more in regions where the optimal action is uncertain, but remain more deterministic in states with a clear distinction between good and bad actions. Instead, we formulate a constrained optimization problem where the average entropy of the policy is constrained, while the entropy at different states can vary. Similar approach was taken in (Abdolmaleki et al., 2018), where the policy was constrained to remain close to the previous policy. We show that the dual to this constrained optimization leads to the soft actor-critic updates, along with an additional update for the dual variable, which plays the role of the temperature. Our formulation also makes it possible to learn the entropy with more expressive policies that can model multi-modal distributions, such as policies based on normalizing flows (Haarnoja et al., 2018a) for which no closed form expression for

¹In (Haarnoja et al., 2018c) we introduced an additional function approximator for the value function, but later found it to be unnecessary.

the entropy exists. We will derive the update for finite horizon case, and then derive an approximation for stationary policies by dropping the time dependencies from the policy, soft Q-function, and the temperature.

Our aim is to find a stochastic policy with maximal expected return that satisfies a minimum expected entropy constraint. Formally, we want to solve the constrained optimization problem

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho_\pi} \left[\sum_{t=0}^T r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [-\log(\pi_t(\mathbf{a}_t|\mathbf{s}_t))] \geq \mathcal{H} \quad \forall t \quad (11)$$

where $\mathcal{H}$ is a desired minimum expected entropy. Note that, for fully observed MDPs, the policy that optimizes the expected return is deterministic, so we expect this constraint to usually be tight and do not need to impose an upper bound on the entropy.

Since the policy at time t can only affect the future objective value, we can employ an (approximate) dynamic programming approach, solving for the policy backward through time. We rewrite the objective as an iterated maximization

$$\max_{\pi_0} \left(\mathbb{E}[r(\mathbf{s}_0, \mathbf{a}_0)] + \max_{\pi_1} \left(\mathbb{E}[\dots] + \max_{\pi_T} \mathbb{E}[r(\mathbf{s}_T, \mathbf{a}_T)] \right) \right), \quad (12)$$

subject to the constraint on entropy. Starting from the last time step, we change the constrained maximization to the dual problem. Subject to $\mathbb{E}_{(\mathbf{s}_T, \mathbf{a}_T) \sim \rho_\pi} [-\log(\pi_T(\mathbf{s}_T|\mathbf{s}_T))] \geq \mathcal{H}$,

$$\max_{\pi_T} \mathbb{E}_{(\mathbf{s}_T, \mathbf{a}_T) \sim \rho_\pi} [r(\mathbf{s}_T, \mathbf{a}_T)] = \min_{\alpha_T \geq 0} \max_{\pi_T} \mathbb{E} [r(\mathbf{s}_T, \mathbf{a}_T) - \alpha_T \log \pi(\mathbf{a}_T|\mathbf{s}_T)] - \alpha_T \mathcal{H}, \quad (13)$$

where α_T is the dual variable. We have also used strong duality, which holds since the objective is linear and the constraint (entropy) is convex function in π_T . This dual objective is closely related to the maximum entropy objective with respect to the policy, and the optimal policy is the maximum entropy policy corresponding to temperature α_T : $\pi_T^*(\mathbf{a}_T|\mathbf{s}_T; \alpha_T)$. We can solve for the optimal dual variable α_T^* as

$$\arg \min_{\alpha_T} \mathbb{E}_{\mathbf{s}_T, \mathbf{a}_T \sim \pi_T^*} [-\alpha_T \log \pi_T^*(\mathbf{a}_T|\mathbf{s}_T; \alpha_T) - \alpha_T \mathcal{H}]. \quad (14)$$

To simplify notation, we make use of the recursive definition of the soft Q-function

$$Q_t^*(\mathbf{s}_t, \mathbf{a}_t; \pi_{t+1:T}^*, \alpha_{t+1:T}^*) = \mathbb{E}[r(\mathbf{s}_t, \mathbf{a}_t)] + \mathbb{E}_{\rho_\pi} [Q_{t+1}^*(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \alpha_{t+1}^* \log \pi_{t+1}^*(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})], \quad (15)$$

with $Q_T^*(\mathbf{s}_T, \mathbf{a}_T) = \mathbb{E}[r(\mathbf{s}_T, \mathbf{a}_T)]$. Now, subject to the entropy constraints and again using the dual problem, we have

$$\begin{aligned} & \max_{\pi_{T-1}} \left(\mathbb{E}[r(\mathbf{s}_{T-1}, \mathbf{a}_{T-1})] + \max_{\pi_T} \mathbb{E}[r(\mathbf{s}_T, \mathbf{a}_T)] \right) \\ &= \max_{\pi_{T-1}} (Q_{T-1}^*(\mathbf{s}_{T-1}, \mathbf{a}_{T-1}) - \alpha_T \mathcal{H}) \\ &= \min_{\alpha_{T-1} \geq 0} \max_{\pi_{T-1}} \left(\mathbb{E}[Q_{T-1}^*(\mathbf{s}_{T-1}, \mathbf{a}_{T-1})] - \mathbb{E}[\alpha_{T-1} \log \pi(\mathbf{a}_{T-1}|\mathbf{s}_{T-1})] - \alpha_{T-1} \mathcal{H} \right) + \alpha_T^* \mathcal{H}. \end{aligned} \quad (16)$$

In this way, we can proceed backwards in time and recursively optimize [Equation 11](#). Note that the optimal policy at time t is a function of the dual variable α_t . Similarly, we can solve the optimal dual variable α_t^* after solving for Q_t^* and π_t^* :

$$\alpha_t^* = \arg \min_{\alpha_t} \mathbb{E}_{\mathbf{a}_t \sim \pi_t^*} [-\alpha_t \log \pi_t^*(\mathbf{a}_t|\mathbf{s}_t; \alpha_t) - \alpha_t \mathcal{H}]. \quad (17)$$

The solution in [Equation 17](#) along with the policy and soft Q-function updates described in [Section 4](#) constitute the core of our algorithm, and in theory, exactly solving them recursively optimize the optimal entropy-constrained maximum expected return objective in [Equation 11](#), but in practice, we will need to resort to [function](#) approximators and stochastic gradient descent.

Algorithm 1 Soft Actor-Critic

Input: θ_1, θ_2, ϕ ▷ Initial parameters
 $\theta_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
 $\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
 for each environment step **do**
 $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$ ▷ Sample action from the policy
 $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
 end for
 for each gradient step **do**
 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
 $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
 $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$ ▷ Adjust temperature
 $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
 end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

6 Practical Algorithm

Our algorithm makes use of two soft Q-functions to mitigate positive bias in the policy improvement step that is known to degrade performance of value based methods (Hasselt, 2010; Fujimoto et al., 2018). In particular, we parameterize two soft Q-functions, with parameters θ_i , and train them independently to optimize $J_Q(\theta_i)$. We then use the minimum of the two soft Q-functions for the stochastic gradient in Equation 6 and policy gradient in Equation 10, as proposed by Fujimoto et al. (2018). Although our algorithm can learn challenging tasks, including a 21-dimensional Humanoid, using just a single Q-function, we found two soft Q-functions significantly speed up training, especially on harder tasks.

In addition to the soft Q-function and the policy, we also learn α by minimizing the dual objective in Equation 17. This can be done by approximating dual gradient descent (Boyd & Vandenberghe, 2004). Dual gradient descent alternates between optimizing the Lagrangian with respect to the primal variables to convergence, and then taking a gradient step on the dual variables. While optimizing with respect to the primal variables fully is impractical, a truncated version that performs incomplete optimization (even for a single gradient step) can be shown to converge under convexity assumptions (Boyd & Vandenberghe, 2004). While such assumptions do not apply to the case of nonlinear function approximators such as neural networks, we found this approach to still work in practice. Thus, we compute gradients for α with the following objective:

$$J(\alpha) = \mathbb{E}_{\mathbf{a}_t \sim \pi_t} [-\alpha \log \pi_t(\mathbf{a}_t | \mathbf{s}_t) - \alpha \tilde{\mathcal{H}}]. \quad (18)$$

The final algorithm is listed in Algorithm 1. The method alternates between collecting experience from the environment with the current policy and updating the function approximators using the stochastic gradients from batches sampled from a replay pool. Using off-policy data from a replay pool is feasible because both value estimators and the policy can be trained entirely on off-policy data. The algorithm is agnostic to the parameterization of the policy, as long as it can be evaluated for any arbitrary state-action tuple.

7 Experiments

The goal of our experimental evaluation is to understand how the sample complexity and stability of our method compares with prior off-policy and on-policy deep reinforcement learning algorithms. We compare our method to prior techniques on a range of challenging continuous control tasks from the OpenAI gym benchmark suite (Brockman et al., 2016) and also on the rllab implementation of the Humanoid task (Duan et al., 2016). Although the easier tasks can be solved by a wide range of different algorithms, the more complex benchmarks, such as the 21-dimensional Humanoid (rllab),

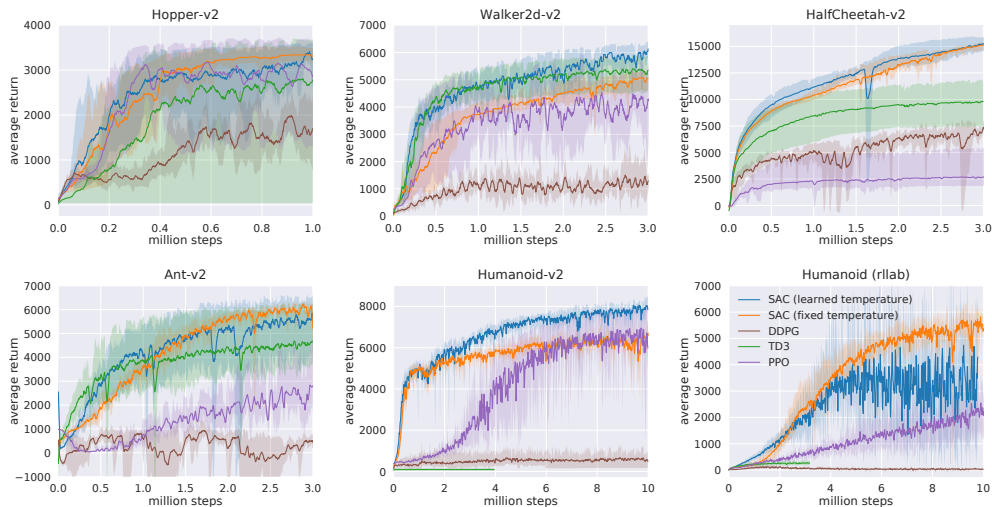


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

are exceptionally difficult to solve with off-policy algorithms (Duan et al., 2016). The stability of the algorithm also plays a large role in performance: easier tasks make it more practical to tune hyperparameters to achieve good results, while the already narrow basins of effective hyperparameters become prohibitively small for the more sensitive algorithms on the hardest benchmarks, leading to poor performance (Gu et al., 2016).

7.1 Simulated Benchmarks

We compare our method to deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), an algorithm that is regarded as one of the more efficient off-policy deep RL methods (Duan et al., 2016); proximal policy optimization (PPO) (Schulman et al., 2017b), a stable and effective on-policy policy gradient algorithm; and soft Q-learning (SQL) (Haarnoja et al., 2017), a recent off-policy algorithm for learning maximum entropy policies. Our SQL implementation also includes two Q-functions, which we found to improve its performance in most environments. We additionally compare to twin delayed deep deterministic policy gradient algorithm (TD3) (Fujimoto et al., 2018), using the author-provided implementation. This is an extension to DDPG, proposed concurrently to our method, that first applied the double Q-learning trick to continuous control along with other improvements. We turned off the exploration noise for evaluation for DDPG and PPO. For maximum entropy algorithms, which do not explicitly inject exploration noise, we either evaluated with the exploration noise (SQL) or use the mean action (SAC). The source code of our SAC implementation² is available online.

Figure 1 shows the total average return of evaluation rollouts during training for DDPG, PPO, and TD3. We train five different instances of each algorithm with different random seeds, with each performing one evaluation rollout every 1000 environment steps. The solid curves corresponds to the mean and the shaded region to the minimum and maximum returns over the five trials. For SAC, we include both a version, where the temperature parameter is fixed and treated as a hyperparameter and tuned for each environment separately (orange), and a version where the temperature is adjusted automatically (blue). The results show that, overall, SAC performs comparably to the baseline methods on the easier tasks and outperforms them on the harder tasks with a large margin, both in terms of learning speed and the final performance. For example, DDPG fails to make any progress on Ant-v1, Humanoid-v1, and Humanoid (rllab), a result that is corroborated by prior work (Gu et al., 2016; Duan et al., 2016). SAC also learns considerably faster than PPO as a consequence of the large batch sizes PPO needs to learn stably on more high-dimensional and complex tasks. Another

²<http://github.com/fail-berkeley/softlearning/>

maximum entropy RL algorithm, SQL, can also learn all tasks, but it is slower than SAC and has worse asymptotic performance. The quantitative results attained by SAC in our experiments also compare very favorably to results reported by other methods in prior work (Duan et al., 2016; Gu et al., 2016; Henderson et al., 2017), indicating that both the sample efficiency and final performance of SAC on these benchmark tasks exceeds the state of the art. The results also indicate that the automatic temperature tuning scheme works well across all the environments, and thus effectively eliminates the need for tuning the temperature. All hyperparameters used in this experiment for SAC are listed in Appendix D.

7.2 Quadrupedal Locomotion in the Real World

In this section, we describe an application of our method to learn walking gaits directly in the real world. We use the Minitaur robot, a small-scale quadruped with eight direct-drive actuators (Kenneally et al., 2016). Each leg is controlled by two actuators that allow it to move in the sagittal plane. The Minitaur is equipped with motor encoders that measure the motor angles and an IMU that measures orientation and angular velocity of Minitaur’s base. The action space are the swing angle and the extension of each leg, which are then mapped to desired motor positions and tracked with a PD controller (Tan et al., 2018). The observations include the motor angles as well as roll and pitch angles and angular velocities of the base. We exclude yaw since it is unreliable due to drift and irrelevant for the walking task. Note that latencies and contacts in the system make the dynamics non-Markovian, which can significantly degrade learning performance. We therefore construct the state out of the current and past five observations and actions. The reward function rewards large forward velocity, which is estimated using a motion capture system, and penalizes large angular accelerations, computed via finite differentiation from the last three actions. We also found it necessary to penalize for large pitch angles and for extending the front legs under the robot, which we found to be the most common failure cases that would require manual reset. We parameterize the policy and the value functions with feed-forward neural networks with two hidden-layers and 256 neurons per layer.

We have developed a semi-automatic robot training pipeline that consists of two components parallel jobs: training and data collection. These jobs run asynchronously on two different computers. The training process runs on a workstation, which updates the neural networks and periodically downloads the latest data from the robot and uploads the latest policy to the robot. On the robot, the on-board Nvidia Jetson TX2 runs the data collection job, which executes the policy, collects the trajectory and uploads these data to the workstation through Ethernet. Once the training is started, minimal human intervention is needed, except for the need to reset the robot state if it falls or drifts far from the initial state.

This learning task presents substantially challenges for real-world reinforcement learning. The robot is underactuated, and must therefore delicately balance contact forces on the legs to make forward progress. An untrained policy can lose balance and fall, and too many falls will eventually damage the robot, making sample-efficient learning essentially. Our method successfully learns to walk from 160k environment steps, or approximately 400 episodes with maximum length of 500 steps, which amount to approximately 2 hours of real-world training time.

However, in the real world, the utility of a locomotion policy hinges critically on its ability to generalize to different terrains and obstacles. Although we trained our policy only on flat terrain, as illustrated in Figure 2 (first row), we then tested it on varied terrains and obstacles (other rows). Because soft actor-critic learns robust policies, due to entropy maximization at training time, the policy can readily generalize to these perturbations without any additional learning. The robot is able to walk up and down a slope (first row), ram through an obstacle made of wooden blocks (second row), and step down stairs (third row) without difficulty, despite not being trained in these settings. To our knowledge, this experiment is the first example of a deep reinforcement learning algorithm learning underactuated quadrupedal locomotion directly in the real world without any simulation or pretraining. We have included videos of the the training process and evaluation on our project website.

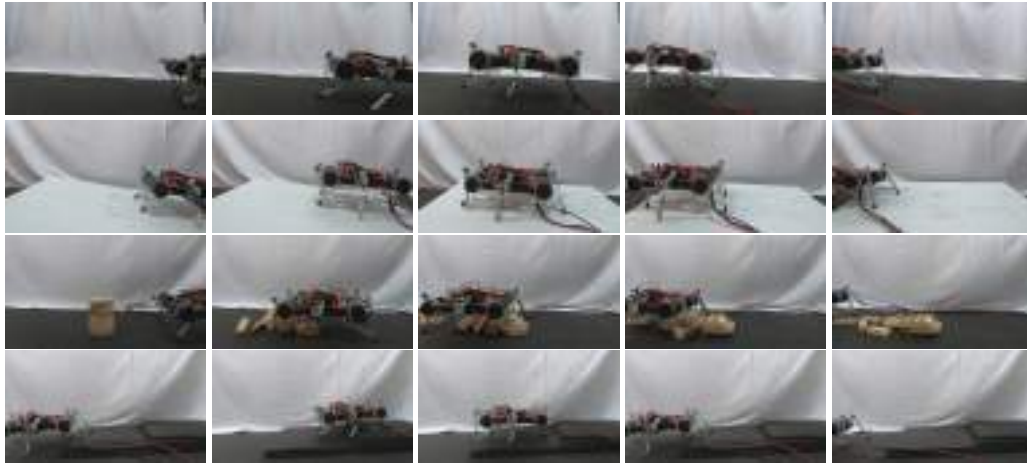


Figure 2: We trained the Minitaur robot to walk on flat terrain (first row) and tested how the learned gait generalizes to unseen situations (other rows)

7.3 Dexterous Hand Manipulation

Our second real-world robotic task involves training a 3-finger dexterous robotic hand to manipulate an object. The hand is based on the “dclaw” hand, discussed by (Zhu et al., 2018). This hand has 9 DoFs, each controlled by a Dynamixel servo-motor. The policy controls the hand by sending target joint angle positions for the on-board PID controller. The manipulation task requires the hand to rotate a “valve”-like object (resembling a sink faucet), as shown in Figure 3. In order to perceive the valve, the robot must use raw RGB images, which are illustrated in the second row of Figure 3. The images are processed in a neural network, consisting of two convolutional (four 3x3 filters) and max pool (3x3) layers, followed by two fully connected layers (256 units). The robot must rotate the valve into the correct position, with the colored part of the valve facing directly to the right, from any random starting position. The initial position of the valve is reset uniformly at random for each episode, forcing the policy to learn to use the raw RGB images to perceive the current valve orientation. A small motor is attached to the valve to automate resets and to provide the ground truth position for the determination of the reward function. The position of this motor is not provided to the policy.

This task is exceptionally challenging due to both the perception challenges and the physical difficulty of rotating the valve with such a complex robotic hand. As can be seen in the accompanying video on the project website³, rotating the valve requires a complex finger gait where the robot moves the fingers over the valve in a coordinated pattern, and stops precisely at the desired position.

Learning this task directly from raw RGB images requires 300k environment interaction steps, which is the equivalent of 20 hours of training, including all resets and neural network training time (Figure 4). To our knowledge, this task represents one of the most complex robotic manipulation tasks learned directly end-to-end from raw images in the real world with deep reinforcement learning, without any simulation or pretraining. We also learned the same task without images by feeding the valve position directly to the neural networks. In that case, learning takes 3 hours, which is substantially faster than what has been reported earlier on the same task using PPO (7.4 hours) (Zhu et al., 2018).

8 Conclusion

In this article, we presented soft actor-critic (SAC), an off-policy maximum entropy deep reinforcement learning algorithm that provides sample-efficient learning while retaining the benefits of entropy maximization and stability. Our theoretical results derive soft policy iteration, which we show to converge to the optimal policy. From this result, we can formulate a practical soft actor-critic algorithm

³<https://sites.google.com/view/sac-and-applications/>



Figure 3: Valve rotation task with Dynamixel Claw. Top row shows a high resolution image sequence of a rollout with the policy. Bottom row shows the 32x32 pixel image observations of the same situations. The policy is also provided the joint positions and velocities of the fingers, but it has to infer the valve position from the image.

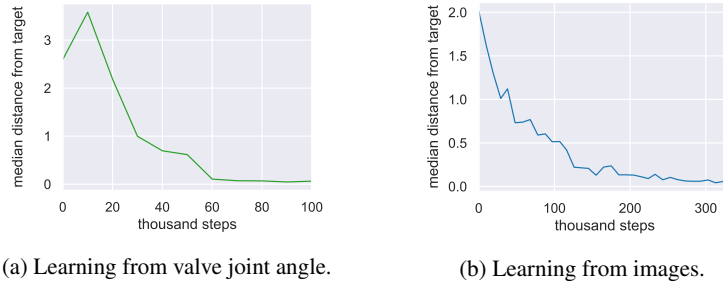


Figure 4: Learning curves for the valve rotation task. The curves correspond to the median distance (measured in radians) of the valve from the target angle during a rollout. (a) Learning without images takes about 3 hours. In this case, the valve is reset to point away from the target initially. (b) Learning from images takes about 20 hours. The initial valve position is sampled uniformly at random.

that can be used to train deep neural network policies, and we empirically show that it matches or exceeds the performance of state-of-the-art model-free deep RL methods, including the off-policy TD3 algorithm and the on-policy PPO algorithm without any environment specific hyperparameter tuning. Our real-world experiments indicate that soft actor-critic is robust and sample efficient enough for robotic tasks learned directly in the real world, such as locomotion and dexterous manipulation. To our knowledge, these results represent the first evaluation of deep reinforcement learning for real-world training of underactuated walking skills with a quadrupedal robot, as well as one of the most complex dexterous manipulation behaviors learned with deep reinforcement learning end-to-end from raw image observations.

Acknowledgments

We would like to thank Vitchyr Pong and Haoran Tang for constructive discussions during the development of soft actor-critic, Vincent Vanhoucke for his support towards the project at Google, and Amazon for providing computing support.

References

- Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., and Riedmiller, M. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.
- Barto, A. G., Sutton, R. S., and Anderson, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, pp. 834–846, 1983.

- Todorov, E. General duality between optimal control and estimation. In *IEEE Conference on Decision and Control (CDC)*, pp. 4286–4292. IEEE, 2008.
- Toussaint, M. Robot trajectory optimization using approximate inference. In *International Conference on Machine Learning (ICML)*, pp. 1049–1056. ACM, 2009.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.
- Zhu, H., Gupta, A., Rajeswaran, A., Levine, S., and Kumar, V. Dexterous manipulation with deep reinforcement learning: Efficient, general, and low-cost. *arXiv preprint arXiv:1810.06045*, 2018.
- Ziebart, B. D. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.
- Ziebart, B. D., Maas, A. L., Bagnell, J. A., and Dey, A. K. Maximum entropy inverse reinforcement learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438, 2008.

Appendix

A Infinite Horizon Discounted Maximum Entropy Objective

The exact definition of the discounted maximum entropy objective is complicated by the fact that, when using a discount factor for policy gradient methods, we typically do not discount the state distribution, only the rewards. In that sense, discounted policy gradients typically do not optimize the true discounted objective. Instead, they optimize average reward, with the discount serving to reduce variance, as discussed by Thomas (2014). However, we can define the objective that is optimized under a discount factor as

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} \left[\sum_{l=t}^{\infty} \gamma^{l-t} \mathbb{E}_{\mathbf{s}_l \sim p, \mathbf{a}_l \sim \pi} [r(\mathbf{s}_l, \mathbf{a}_l) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_l)) | \mathbf{s}_l, \mathbf{a}_l] \right]. \quad (19)$$

This objective corresponds to maximizing the discounted expected reward and entropy for future states originating from every state-action tuple $(\mathbf{s}_t, \mathbf{a}_t)$ weighted by its probability ρ_{π} under the current policy.

B Proofs

B.1 Lemma 1

Lemma 1 (Soft Policy Evaluation). Consider the soft Bellman backup operator $\mathcal{T}^{\pi}$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^{\pi} Q^k$. Then the sequence Q^k will converge to the soft Q -value of π as $k \rightarrow \infty$.

Proof. Define the entropy augmented reward as $r_{\pi}(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [\mathcal{H}(\pi(\cdot | \mathbf{s}_{t+1}))]$ and rewrite the update rule as

$$Q(\mathbf{s}_t, \mathbf{a}_t) \leftarrow r_{\pi}(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p, \mathbf{a}_{t+1} \sim \pi} [Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})] \quad (20)$$

and apply the standard convergence results for policy evaluation (Sutton & Barto, 1998). The assumption $|\mathcal{A}| < \infty$ is required to guarantee that the entropy augmented reward is bounded. $\square$

B.2 Lemma 2

Lemma 2 (Soft Policy Improvement). Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.

Proof. Let $\pi_{\text{old}} \in \Pi$ and let $Q^{\pi_{\text{old}}}$ and $V^{\pi_{\text{old}}}$ be the corresponding soft state-action value and soft state value, and let π_{new} be defined as

$$\begin{aligned}\pi_{\text{new}}(\cdot | \mathbf{s}_t) &= \arg \min_{\pi' \in \Pi} D_{\text{KL}}(\pi'(\cdot | \mathbf{s}_t) \parallel \exp(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot) - \log Z^{\pi_{\text{old}}}(\mathbf{s}_t))) \\ &= \arg \min_{\pi' \in \Pi} J_{\pi_{\text{old}}}(\pi'(\cdot | \mathbf{s}_t)).\end{aligned}\quad (21)$$

It must be the case that $J_{\pi_{\text{old}}}(\pi_{\text{new}}(\cdot | \mathbf{s}_t)) \leq J_{\pi_{\text{old}}}(\pi_{\text{old}}(\cdot | \mathbf{s}_t))$, since we can always choose $\pi_{\text{new}} = \pi_{\text{old}} \in \Pi$. Hence

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} [\log \pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)] \leq \mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{old}}} [\log \pi_{\text{old}}(\mathbf{a}_t | \mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)], \quad (22)$$

and since partition function $Z^{\pi_{\text{old}}}$ depends only on the state, the inequality reduces to

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} [Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_{\text{new}}(\mathbf{a}_t | \mathbf{s}_t)] \geq V^{\pi_{\text{old}}}(\mathbf{s}_t). \quad (23)$$

Next, consider the soft Bellman equation:

$$\begin{aligned}Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V^{\pi_{\text{old}}}(\mathbf{s}_{t+1})] \\ &\leq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [\mathbb{E}_{\mathbf{a}_{t+1} \sim \pi_{\text{new}}} [Q^{\pi_{\text{old}}}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \log \pi_{\text{new}}(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})]] \\ &\vdots \\ &\leq Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t),\end{aligned}\quad (24)$$

where we have repeatedly expanded $Q^{\pi_{\text{old}}}$ on the RHS by applying the soft Bellman equation and the bound in Equation 23. Convergence to $Q^{\pi_{\text{new}}}$ follows from Lemma 1. $\square$

B.3 Theorem 1

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement to any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. Let π_i be the policy at iteration i . By Lemma 2 the sequence Q^{π_i} is monotonically increasing. Since Q^π is bounded above for $\pi \in \Pi$ (both the reward and entropy are bounded), the sequence converges to some π^* . We will still need to show that π^* is indeed optimal. At convergence, it must be case that $J_{\pi^*}(\pi^*(\cdot | \mathbf{s}_t)) \leq J_{\pi^*}(\pi(\cdot | \mathbf{s}_t))$ for all $\pi \in \Pi$, $\pi \neq \pi^*$. Using the same iterative argument as in the proof of Lemma 2, we get $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) > Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, that is, the soft value of any other policy in Π is lower than that of the converged policy. Hence π^* is optimal in Π . $\square$

C Enforcing Action Bounds

We use an unbounded Gaussian as the action distribution. However, in practice, the actions needs to be bounded to a finite interval. To that end, we apply an invertible squashing function ($\tanh$) to the Gaussian samples, and employ the change of variables formula to compute the likelihoods of the bounded actions. In the other words, let $\mathbf{u} \in \mathbb{R}^D$ be a random variable and $\mu(\mathbf{u} | \mathbf{s})$ the corresponding density with infinite support. Then $\mathbf{a} = \tanh(\mathbf{u})$, where $\tanh$ is applied elementwise, is a random variable with support in $(-1, 1)$ with a density given by

$$\pi(\mathbf{a} | \mathbf{s}) = \mu(\mathbf{u} | \mathbf{s}) \left| \det \left(\frac{d\mathbf{a}}{d\mathbf{u}} \right) \right|^{-1}. \quad (25)$$

Since the Jacobian $d\mathbf{a}/d\mathbf{u} = \text{diag}(1 - \tanh^2(\mathbf{u}))$ is diagonal, the log-likelihood has a simple form

$$\log \pi(\mathbf{a} | \mathbf{s}) = \log \mu(\mathbf{u} | \mathbf{s}) - \sum_{i=1}^D \log(1 - \tanh^2(u_i)), \quad (26)$$

where u_i is the i^{th} element of $\mathbf{u}$.

D Hyperparameters

Table 1 lists the common SAC parameters used in the comparative evaluation in Figure 1.

Table 1: SAC Hyperparameters	
Parameter	Value
optimizer	Adam (Kingma & Ba, 2015)
learning rate	$3 \cdot 10^{-4}$
discount (γ)	0.99
replay buffer size	10^6
number of hidden layers (all networks)	2
number of hidden units per layer	256
number of samples per minibatch	256
entropy target	$-\dim(\mathcal{A})$ (e.g., -6 for HalfCheetah-v1)
nonlinearity	ReLU
target smoothing coefficient (τ)	0.005
target update interval	1
gradient steps	1

CROSSQ: BATCH NORMALIZATION IN DEEP REINFORCEMENT LEARNING FOR GREATER SAMPLE EFFICIENCY AND SIMPLICITY

Aditya Bhatt^{*1,4}Daniel Palenicek^{*1,2}Boris Belousov^{1,4}Max Argus³Artemij Amiranashvili³Thomas Brox³Jan Peters^{1,2,4,5}^{*}Equal contribution ¹Intelligent Autonomous Systems, TU Darmstadt ²Hessian.AI ³University of Freiburg⁴German Research Center for AI (DFKI) ⁵Centre for Cognitive Science, TU Darmstadt

aditya.bhatt@dfki.de, daniel.palenicek@tu-darmstadt.de

ABSTRACT

Sample efficiency is a crucial problem in deep reinforcement learning. Recent algorithms, such as REDQ and DroQ, found a way to improve the sample efficiency by increasing the update-to-data (UTD) ratio to 20 gradient update steps on the critic per environment sample. However, this comes at the expense of a greatly increased computational cost. To reduce this computational burden, we introduce CrossQ: A lightweight algorithm for continuous control tasks that makes careful use of Batch Normalization and removes target networks to surpass the current state-of-the-art in sample efficiency while maintaining a low UTD ratio of 1. Notably, CrossQ does not rely on advanced bias-reduction schemes used in current methods. CrossQ’s contributions are threefold: (1) it matches or surpasses current state-of-the-art methods in terms of sample efficiency, (2) it substantially reduces the computational cost compared to REDQ and DroQ, (3) it is easy to implement, requiring just a few lines of code on top of SAC.

1 INTRODUCTION

Sample efficiency is a crucial concern when applying Deep Reinforcement Learning (Deep RL) methods on real physical systems. One of the first successful applications of Deep RL to a challenging problem of quadruped locomotion was achieved using Soft Actor-Critic (SAC, Haarnoja et al. (2018a)), allowing a robot dog to learn to walk within 2h of experience (Haarnoja et al., 2018b). Subsequently, it was noted that the critic in SAC may be underfitted, as only a single gradient update step on the network parameters is performed for each environment step. Therefore, Randomized Ensembled Double Q-Learning (REDQ, Chen et al. (2021)) was proposed, which increased this number of gradient steps, termed update-to-data (UTD) ratio. In addition, Dropout Q functions (DroQ, Hiraoka et al. (2021)) improved the computational efficiency of REDQ while maintaining the same sample efficiency by replacing its ensemble of critics with dropout. This enabled learning quadruped locomotion in a mere 20min (Smith et al., 2022). Thus, REDQ and DroQ represent the state-of-the-art in terms of sample efficiency in Deep RL for continuous control.

Importantly, both REDQ and DroQ showed that naively increasing the UTD ratio of SAC does not perform well due to the critic networks’ Q value estimation bias. Therefore, ensembling techniques were introduced for bias reduction (explicit ensemble in REDQ and implicit

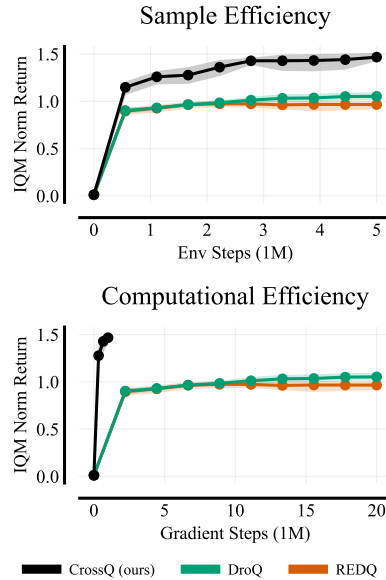


Figure 1: **CrossQ training performance aggregated over environments.** CrossQ is more sample efficient (top) while being significantly more computationally efficient (bottom) in terms of the gradient steps, thanks to a low UTD = 1. Following Agarwal et al. (2021), we normalize performance by the maximum of REDQ in each environment.

ensemble via dropout in DroQ), which allowed increasing the UTD to 20 critic updates per environment step. Higher UTD ratios improve sample efficiency by paying the price of increased computational cost, which manifests in higher wallclock time and energy consumption. It is, therefore, desirable to seek alternative methods that achieve the same or better sample efficiency at a lower computational cost, e.g., by using lower UTDs.

It turns out that even $UTD = 1$ can perform surprisingly well if other algorithmic components are adjusted appropriately. In this paper, we introduce **CrossQ**, a lightweight algorithm that achieves superior performance by removing much of the algorithmic design complexity that was added over the years, culminating in the current state-of-the-art methods. First, it *removes target networks*, an ingredient widely believed to slow down training in exchange for stability (Mnih et al., 2015; Lillicrap et al., 2016; Kim et al., 2019; Fan et al., 2020). Second, we find that *Batch Normalization* variants (Ioffe & Szegedy (2015); Ioffe (2017)), when applied in a particular manner, effectively stabilize training and significantly improve sample efficiency. This contradicts others’ observations that it hurts the learning performance in Deep RL, e.g. Hiraoka et al. (2021). Third, CrossQ uses *wider critic layers*, motivated by prior research on the ease of optimization of wider networks (Ota et al., 2021). In addition to the first two improvements, wider networks enable even higher returns.

Contributions. (1) We present the CrossQ algorithm, which matches or surpasses the current state-of-the-art for model-free off-policy RL for continuous control environments with state observations in sample efficiency while being multiple times more computationally efficient; (2) By removing target networks, we are able to successfully accelerate off-policy Deep RL with Batch-Norm; (3) We provide empirical investigations and hypotheses for CrossQ’s success. CrossQ’s changes mainly pertain to the deep network architecture of SAC; therefore, our study is chiefly empirical: through a series of ablations, we isolate and study the contributions of each part. We find that CrossQ matches or surpasses the state-of-the-art algorithms in sample efficiency while being up to $4\times$ faster in terms of wallclock time without requiring critic ensembles, target networks, or high UTD ratios. We provide the CrossQ source code at github.com/adityab/CrossQ.

2 BACKGROUND

2.1 OFF-POLICY REINFORCEMENT LEARNING AND SOFT ACTOR-CRITIC

We consider a discrete-time Markov Decision Process (MDP, Puterman (2014)), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \rho, \gamma \rangle$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition probability $s_{t+1} \sim \mathcal{P}(\cdot | s_t, a_t)$, reward function $r_t = \mathcal{R}(s_t, a_t)$, initial state distribution $s_0 \sim \rho$ and discount factor $\gamma \in [0, 1)$. RL describes the problem of an agent learning an optimal policy π for a given MDP. At each time step t , the agent receives a state s_t and interacts with the environment according to its policy π . We focus on the Maximum Entropy RL setting (Ziebart et al., 2008), where the agent’s objective is to find the optimal policy π^* , which maximizes the expected cumulative reward while keeping the entropy $\mathcal{H}$ high; $\arg \max_{\pi} \mathbb{E}_{s_0 \sim \rho} [\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \mathcal{H}(\pi(\cdot | s_t)))]$. The action-value function is defined by $Q(s, a) = \mathbb{E}_{\pi, \mathcal{P}} [\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \log \pi(a_t | s_t)) | s_0 = s, a_0 = a]$ and describes the expected reward when taking action a in state s . Soft Actor-Critic (SAC, (Haarnoja et al., 2018a)) is a popular algorithm that solves the MaxEnt RL problem. SAC parametrizes the Q function and policy as neural networks and trains two independent versions of the Q function, using the minimum of their estimates to compute the regression targets for Temporal Difference (TD) learning. This *clipped double-Q* trick, originally proposed by Fujimoto et al. (2018) in TD3, helps in reducing the potentially destabilizing overestimation bias inherent in approximate Q-learning (Hasselt, 2010).

2.2 HIGH UPDATE-TO-DATA RATIOS, REDQ, AND DROQ

Despite its popularity among practitioners and as a foundation for other more complex algorithms, SAC leaves much room for improvement in terms of sample efficiency. Notably, SAC performs exactly one gradient-based optimization step per environment interaction. SAC’s $UTD = 1$ setting is analogous to simply training for fewer epochs in supervised learning. Therefore, in recent years, gains in sample efficiency within RL have been achieved through increasing the UTD ratio (Janner et al., 2019; Chen et al., 2021; Hiraoka et al., 2021; Nikishin et al., 2022). Different algorithms, however, substantially vary in their approaches to achieving high UTD ratios. Janner et al. (2019)

```

1 def critic_loss(Q_params, policy_params, obs, acts, rews, next_obs):
2     next_acts, next_logpi = policy.apply(policy_params, obs)
3
4     # Concatenated forward pass
5     all_q, new_Q_params = Q.apply(Q_params,
6         jnp.concatenate([obs, next_obs]),
7         jnp.concatenate([acts, next_acts])
8     )
9     # Split all_q predictions and stop gradient on next_q
10    q, next_q = jnp.split(all_q, 2)
11    next_q = jnp.min(next_q, axis=0) # min over double Q function
12    next_q = jax.lax.stop_gradient(next_q - alpha * next_logpi)
13    return jnp.mean((q - (rews + gamma * next_q))**2), new_Q_params

```

Figure 2: **CrossQ critic loss in JAX.** The CrossQ critic loss is easy to implement on top of an existing SAC implementation. One just adds the batch normalization layers into the critic network and removes the target network. As we are now left with only the critic network, one can simply concatenate observations and next observations, as well as actions and next actions along the batch dimension, perform a joint forward pass, and split up the batches afterward. Combining two forward passes into one grants a small speed-up thanks to requiring only one CUDA call instead of two.

uses a model to generate synthetic data, which allows for more overall gradient steps. [Nikishin et al. \(2022\)](#) adopt a simpler approach: they increase the number of gradient steps while periodically resetting the policy and critic networks to fight premature convergence to local minima. We now briefly outline the two high-UTD methods to which we compare CrossQ.

REDQ. [Chen et al. \(2021\)](#) find that merely raising SAC’s UTD ratio hurts performance. They attribute this to the accumulation of the learned Q functions’ estimation bias over multiple update steps—despite the clipped double-Q trick—which destabilizes learning. To remedy this bias more strongly, they increase the number of Q networks from two to an ensemble of 10. Their method, called REDQ, permits stable training at high UTD ratios up to 20.

DroQ. [Hiraoka et al. \(2021\)](#) note that REDQ’s ensemble size, along with its high UTD ratio, makes training computationally expensive. They instead propose using a smaller ensemble of Q functions equipped with Dropout ([Srivastava et al., 2014](#)), along with Layer Normalization ([Ba et al., 2016](#)) to stabilize training in response to the noise introduced by Dropout. Called DroQ, their method is computationally cheaper than REDQ, yet still expensive due to its UTD ratio of 20.

3 THE CROSSQ ALGORITHM

In this paper, we challenge this current trend of high UTD ratios and demonstrate that we can achieve competitive sample efficiency at a much lower computational cost with a UTD = 1 method. CrossQ is our new state-of-the-art off-policy actor-critic algorithm. Based on SAC, it uses purely network-architectural engineering insights from deep learning to accelerate training. As a result, it ~~crosses out~~ much of the algorithmic design complexity that was added over the years and which led to the current state-of-the-art methods. In doing so, we present a much simpler yet more efficient algorithm. In the following paragraphs, we introduce the three design choices that constitute CrossQ.

3.1 DESIGN CHOICE 1: REMOVING TARGET NETWORKS

[Mnih et al. \(2015\)](#) originally introduced target networks to stabilize the training of value-based off-policy RL methods, and today, most algorithms require them ([Lillicrap et al., 2016](#); [Fujimoto et al., 2018](#); [Haarnoja et al., 2018a](#)). SAC updates the critics’ target networks with Polyak Averaging

$$\theta^\circ \leftarrow (1 - \tau)\theta^\circ + \tau\theta, \quad (1)$$

where θ° are the target network parameters, and θ are those of the trained critic. Here τ is the *target network smoothing coefficient*; with a high $\tau = 1$ (equivalent to cutting out the target network), SAC training can diverge, leading to explosive growth in θ and the Q predictions. Target networks stabilize training by explicitly delaying value function updates, arguably slowing down online learning ([Plappert et al., 2018](#); [Kim et al., 2019](#); [Morales, 2020](#)).

SAC: $Q_\theta(S_t, A_t) = q_t$ $Q_{\theta^\circ}(S_{t+1}, A_{t+1}) = q_{t+1}^\circ$ $\mathcal{L}_\theta = (q_t - r_t - \gamma q_{t+1}^\circ)^2$	CrossQ (Ours): $Q_\theta \left(\begin{bmatrix} S_t \\ S_{t+1} \end{bmatrix}, \begin{bmatrix} A_t \\ A_{t+1} \end{bmatrix} \right) = \begin{bmatrix} q_t \\ q_{t+1} \end{bmatrix}$ $\mathcal{L}_\theta = (q_t - r_t - \gamma q_{t+1} _{\text{sg}})^2$
---	---

Figure 3: SAC **without BatchNorm in the critic Q_θ** (left) requires **target Q values q_{t+1}°** to stabilize learning. CrossQ **with BatchNorm in the critic Q_θ** (right) removes the need for target networks and allows for a joint forward pass of both current and future values. Batches are sampled from the replay buffer $\mathcal{B}$: $S_t, A_t, r_t, S_{t+1} \sim \mathcal{B}$ and $A_{t+1} \sim \pi_\phi(S_{t+1})$ from the current policy. $|\cdot|_{\text{sg}}$ denotes the stop-gradient operation.

Recently, Yang et al. (2021) found that critics with Random Fourier Features can be trained without target networks, suggesting that the choice of layer activations affects the stability of training. Our experiments in Section 4.4 uncover an even simpler possibility: using bounded activation functions or feature normalizers is sufficient to prevent critic divergence in the absence of target networks, whereas the common choice of `relu` without normalization diverges. While others have used normalizers in Deep RL before, we are the first to identify that they make target networks redundant. Our next design choice exploits this insight to obtain an even greater boost.

3.2 DESIGN CHOICE 2: USING BATCH NORMALIZATION

BatchNorm has not yet seen wide adoption in value-based off-policy RL methods, despite its success and widespread use in supervised learning (He et al., 2016; Santurkar et al., 2018), attempts at doing so have fared poorly. Lillicrap et al. (2016) use BatchNorm layers on the state-only representation layers in the DDPG critic but find that it does not help significantly. Others use BatchNorm in decoupled feature extractors for Deep RL networks (Ota et al., 2020; 2021), but not in critic networks. Hiraoka et al. (2021) report that using BatchNorm in critics causes training to fail in DroQ.

We find using BatchNorm *carefully*, when *additionally* removing target networks, performs surprisingly well, trains stably, and is, in fact, algorithmically simpler than current methods.

First, we explain why BatchNorm needs to be used *carefully*. Within the critic loss $[Q_\theta(S, A) - (r + \gamma Q_{\theta^\circ}(S', A'))]^2$, predictions are made for two differently distributed batches of state-action pairs; (S, A) and (S', A') , where $A' \sim \pi_\phi(S')$ is sampled from the *current* policy, while A originates from old behavior policies.

Just like the target network, the BatchNorm parameters are updated by Polyak Averaging from the live network (Equation 1). The BatchNorm running statistics of the live network, which were estimated from batches of (s, a) pairs, will clearly not have *seen* samples $(s', \pi_\phi(s'))$ and will further not match their statistics. In other words, the state-action inputs evaluated by the target network will be out-of-distribution, given its mismatched BatchNorm running statistics. It is well known that the prediction quality of BatchNorm-equipped networks degrades in the face of such test-time distribution shifts (Pham et al., 2022; Lim et al., 2023).

Removing the target network provides an *elegant* solution. With the target network removed, we can concatenate both batches and feed them through the Q network in a single forward pass, as illustrated in Figure 3 and shown in code in Figure 2. This simple trick ensures that BatchNorm’s normalization moments arise from the union of both batches, corresponding to a 50/50 mixture of their respective distributions. Such normalization layers *do not* perceive the $(s', \pi_\phi(s'))$ batch as being out-of-distribution. This small change to SAC allows the safe use of BatchNorm and greatly accelerates training. We are not the only ones to identify this way of using BatchNorm to tackle the distribution mismatch; other works in supervised learning, e.g., Test-Time Adaptation (Lim et al.,

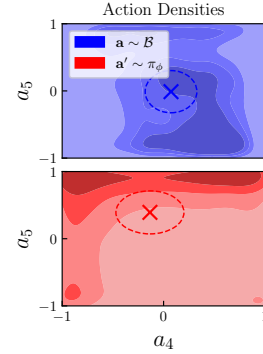


Figure 4: **Replay buffer and current policy actions are distributed differently.** Darker colors denote higher density. Estimated from a batch of 10^4 transitions $(a, s') \sim \mathcal{B}$; $a' \sim \pi_\phi(s')$, after 3×10^5 training steps on Walker2d; a_4 and a_5 are random action dimensions.

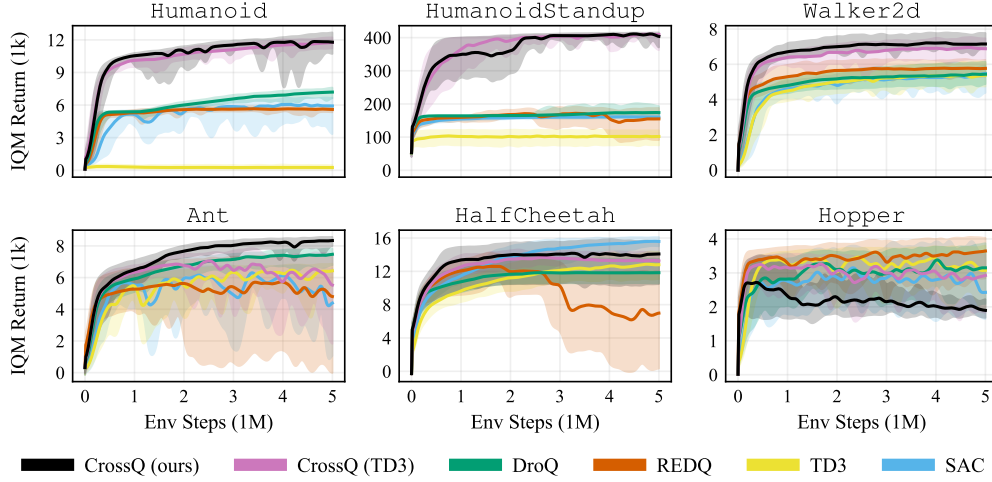


Figure 5: **CrossQ sample efficiency.** Compared to REDQ and DroQ (UTD = 20) CrossQ (UTD = 1) performs either comparably, better, or—for the more challenging Humanoid tasks—substantially better. These results directly transfer to TD3 as the base algorithm in CrossQ (TD3). We plot *interquartile mean* (IQM) and 70% quantile interval of the episodic returns over 10 seeds.

2023), EvalNorm (Singh & Shrivastava, 2019), and *Four Things Everyone Should Know to Improve Batch Normalization* (Summers & Dinneen, 2020) also use mixed moments to bridge this gap.

In practice, CrossQ’s actor and critic networks use Batch Renormalization (BRN, Ioffe (2017)), an improved version of the original BN (Ioffe & Szegedy, 2015) that is robust to long-term training instabilities originating from minibatch noise. BRN performs batch normalization using the less noisy *running statistics* after a warm-up period, instead of noisy minibatch estimates as in BN. In the rest of this paper, all discussions with “BatchNorm” apply equally to both versions unless explicitly disambiguated by BN or BRN.

3.3 DESIGN CHOICE 3: WIDER CRITIC NETWORKS

Following Ota et al. (2021), we find that wider critic network layers in CrossQ lead to even faster learning. As we show in our ablations in Section 4.4, most performance gains originate from the first two design choices; however, wider critic networks further boost the performance, helping to match or outperform REDQ and DroQ sample efficiency.

We want to stress again that **CrossQ**, a UTD = 1 method, *does not use bias-reducing ensembles, high UTD ratios or target networks*. Despite this, it achieves its competitive sample efficiency at a fraction of the compute cost of REDQ and DroQ (see Figures 5 and 6). Note that our proposed changes can just as well be combined with other off-policy TD-learning methods, such as TD3, as shown in our experiments in Section 4.1.

4 EXPERIMENTS AND ANALYSIS

We conduct experiments to provide empirical evidence for CrossQ’s performance, and investigate:

1. Sample efficiency of CrossQ compared to REDQ and DroQ;
2. Computational efficiency in terms of wallclock time and performed gradient step;
3. Effects of the proposed design choices on the performance via Q function bias evaluations;

And conduct further ablation studies for the above design choices. We evaluate across a wide range of continuous-control MuJoCo (Todorov et al., 2012) environments, with 10 random seeds each. Following Janner et al. (2019); Chen et al. (2021) and Hiraoka et al. (2021), we evaluate on the same four Hopper, Walker2d, Ant, and Humanoid tasks, as well as two additional tasks: HalfCheetah and the more challenging HumanoidStandup from Gymnasium (Towers et al., 2023). We adapted the JAX version of stable-baselines (Raffin et al., 2021) for our experiments.

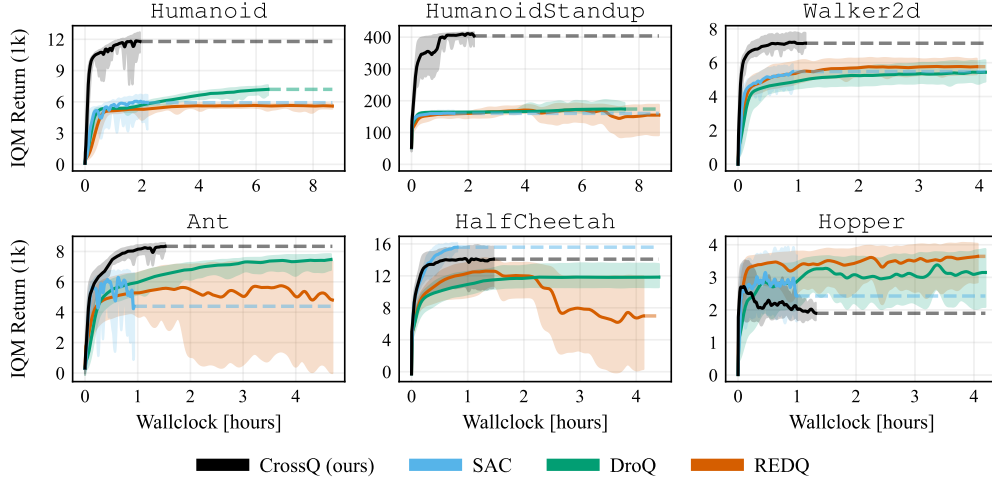


Figure 6: **Computational efficiency.** Cross Q trains an order of magnitude faster, taking only 5% of the gradient steps, substantially saving on wallclock time. The dashed horizontal lines are visual aids to better compare the final performance after training for 5×10^6 environment steps. We plot IQM and 70% quantile interval over 10 seeds. Appendix A.3 provides a table of wallclock times.

4.1 SAMPLE EFFICIENCY OF CROSS Q

Figure 5 compares our proposed Cross Q algorithm with REDQ, DroQ, SAC and TD3 in terms of their sample efficiency, i.e., average episode return at a given number of environment interactions. As a proof of concept, we also present Cross Q (TD3), a version of Cross Q which uses TD3 instead of SAC as the base algorithm. We perform periodic evaluations during training to obtain the episodic reward. From these, we report the mean and standard deviations over 10 random seeds. All subsequent experiments in this paper follow the same protocol.

This experiment shows that Cross Q matches or outperforms the best baseline in all the presented environments except on Ant, where REDQ performs better in the early training stage, but Cross Q eventually matches it. On Hopper, Walker, and HalfCheetah, the learning curves of Cross Q and REDQ overlap, and there is no significant difference. On the harder Humanoid and HumanoidStandup tasks, Cross Q and Cross Q (TD3) both substantially surpass all baselines.

4.2 COMPUTATIONAL EFFICIENCY OF CROSS Q

Figure 6 compares the computational efficiency of Cross Q to the baselines. This metric is where Cross Q makes the biggest leap forward. Cross Q requires $20\times$ fewer gradient steps than REDQ and DroQ, which results in roughly $4\times$ faster wallclock speeds (Table 2). Especially on the more challenging Humanoid and HumanoidStandup tasks the speedup is the most pronounced. In our view, this is a noteworthy feature. On the one hand, it opens the possibility of training agents in a truly online and data-efficient manner, such as in real-time robot learning. On the other hand, with large computing budgets Cross Q can allow the training of even larger models for longer than what is currently feasible, because of its computational efficiency stemming from its low UTD = 1.

4.3 EVALUATING Q FUNCTION ESTIMATION BIAS

All methods we consider in this paper are based on SAC and, thus, include the clipped double-Q trick to reduce Q function overestimation bias (Fujimoto et al., 2018). Chen et al. (2021) and Hiraoka et al. (2021) stress the importance of keeping this bias even lower to achieve their high performances and intentionally design REDQ and DroQ to additionally reduce bias with explicit and implicit ensembling. In contrast, Cross Q outperforms both baselines without any ensembling. Could Cross Q ’s high performance be attributed to implicitly reducing the bias as a side effect of our design choices? Using the same evaluation protocol as Chen et al. (2021), we compare the

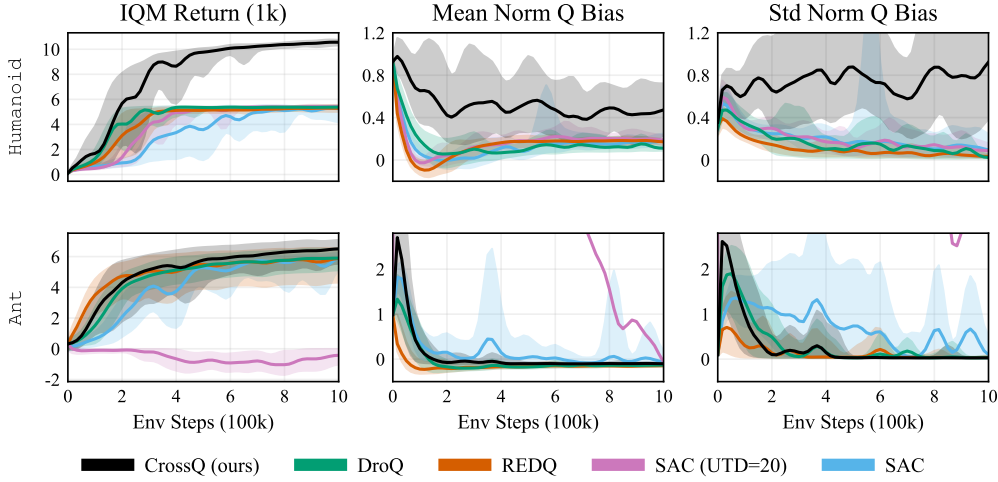


Figure 7: **Q estimation bias does not reliably influence learning performance.** Following the analysis of Chen et al. (2021), we plot the IQM and 70% quantile interval of the normalized Q function bias. REDQ generally has the least bias over 10 seeds. Cross Q matches or outperforms Dro Q , RED Q and SAC while showing more Q function bias in all environments. The full set of environments is shown in Fig. 17 in the Appendix.

normalized Q prediction biases in Figure 4.3. Due to space constraints, here we show Hopper and Ant and place the rest of the environments in Figure 17 in the Appendix.

We find that RED Q and Dro Q indeed have lower bias than SAC and significantly lower bias than SAC with $UTD = 20$. The results for Cross Q are mixed: while its bias trend exhibits a lower mean and variance than SAC, in some environments, its bias is higher than Dro Q , and in others, it is lower or comparable. RED Q achieves comparable or worse returns than Cross Q while maintaining the least bias. As Cross Q performs better *despite* having—perhaps paradoxically—generally higher Q estimation bias, we conclude that the relationship between performance and estimation bias is complex, and one does not seem to have clear implications on the other.

4.4 ABLATIONS

We conduct ablation studies to better understand the impact of different design choices in Cross Q .

4.4.1 DISENTANGLING THE EFFECTS OF TARGET NETWORKS AND BATCHNORM

Cross Q changes SAC in three ways; of these, two explicitly aim to accelerate optimization: the removal of target networks, and the introduction of BatchNorm. Unfortunately, SAC without target networks diverges; therefore, to study the contribution of the first change, we need a way to compare SAC—divergence-free—with and without target networks. Fortunately, we find that such a way exists: according to our supplementary experiments in Appendix A.6, simply using bounded activation functions in the critic appears to prevent divergence. This is a purely empirical observation and an in-depth study regarding the influence of activations and normalizers on the stability of Deep RL is beyond the scope of this paper. In this specific ablation, we use $\tanh$ activations instead of relu , solely as a tool to make the intended comparison possible.

Figure 8 shows the results of our experiment. The performance of SAC without target networks supports the common intuition that target networks indeed slow down learning to a small extent. We find that the combination of BatchNorm and Target Networks performs inconsistently, failing to learn anything in half of the environments. Lastly, the configuration of BatchNorm without target networks—and the closest to Cross Q —achieves the best aggregate performance, with the boost being significantly bigger than that from removing target networks alone. In summary, even though removing target networks may slightly improve performance in some environments, it is the combination of removing target networks and adding BatchNorm that accelerates learning the most.

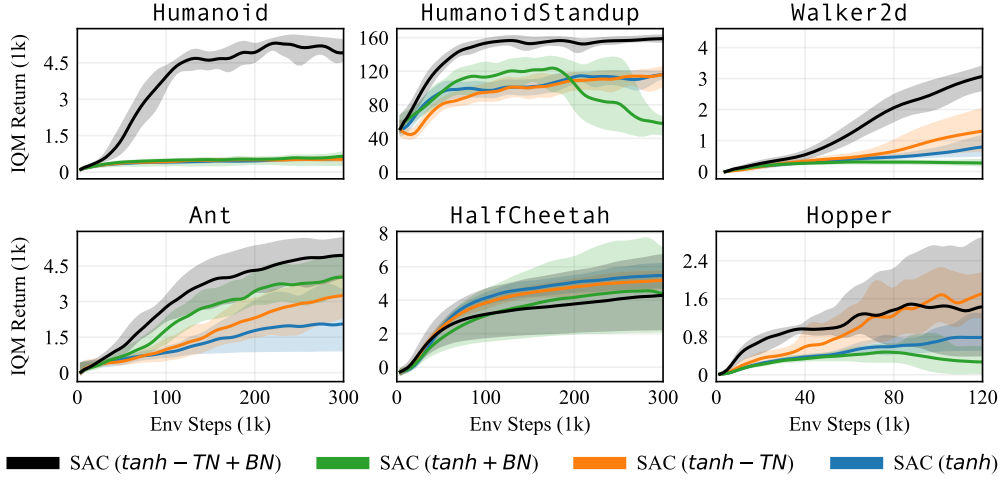


Figure 8: **The effects of target networks and BatchNorm on sample efficiency.** All SAC variants in this experiment use critics with tanh activations, since they allow divergence-free training without target networks, enabling this comparison. This ablation uses the original BatchNorm (BN, Ioffe & Szegedy (2015)). Removing target networks ($-TN$) provides only small improvements over the SAC baseline with target nets. BatchNorm with target nets ($+BN$, green) is unstable. Using BatchNorm after removing target nets ($-TN+BN$)—the configuration most similar to CrossQ—performs best. We plot IQM return and 70% quantile intervals over 10 seeds.

4.4.2 ABLATING THE DIFFERENT DESIGN CHOICES AND HYPERPARAMETERS

In this subsection, we examine the contributions of the different CrossQ design choices to show their importance. Figure 9 shows aggregated ablations of these components and various hyperparameters, while Figure 10 ablates the BatchNorm layer itself.

Hyperparameters. CrossQ uses the best hyperparameters obtained from a series of grid searches. Of these, only three are different from SAC’s default values. First, we find that **reducing the β_1 momentum** for the Adam optimizer (Kingma & Ba, 2015) from 0.9 to 0.5 as well as the **policy delay** of 3 have the smallest impact on the performance. However, since fewer actor gradient steps reduce compute, this setting is favorable. Second, **reducing the critic network’s width to 256**—the same small size as SAC—reduces performance and yet still significantly outperforms SAC. This suggests that practitioners may be able to make use of a larger compute budget, i.e., train efficiently across a range of different network sizes, by scaling up layer widths according to the available hardware resources. Third, as expected, **removing the BRN layers** proves to be detrimental and results in the worst overall performance. A natural question that comes to mind is whether other normalization strategies in the critic, such as Layer Normalization (LayerNorm, Ba et al. (2016)), would also give the same results. However, in our ablation, we find that **replacing BatchNorm with LayerNorm** degrades CrossQ’s performance significantly, roughly to the level of the SAC baseline. Lastly, SAC does not benefit from simply **widening critic layers to 2048**. And **naively adding BRN to SAC while keeping the target networks** proves detrimental. This finding is in line with our diagnosis of mismatched statistics being detrimental to the training.

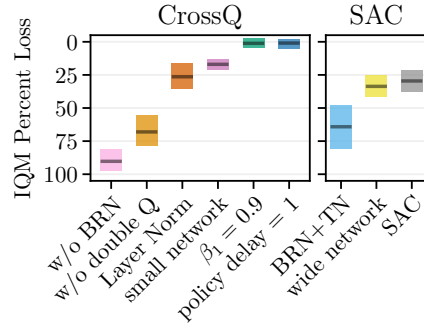


Figure 9: **Ablations on CrossQ and SAC.** Loss in IQM return in percent—relative to CrossQ—at 1M environment interactions. Aggregated over all environments and six seeds each, with 95% bootstrapped confidence intervals (Agarwal et al., 2021). Left shows CrossQ ablations; Right shows effects of adding parts on top of SAC. Figure 13 in Appendix shows individual training curves.

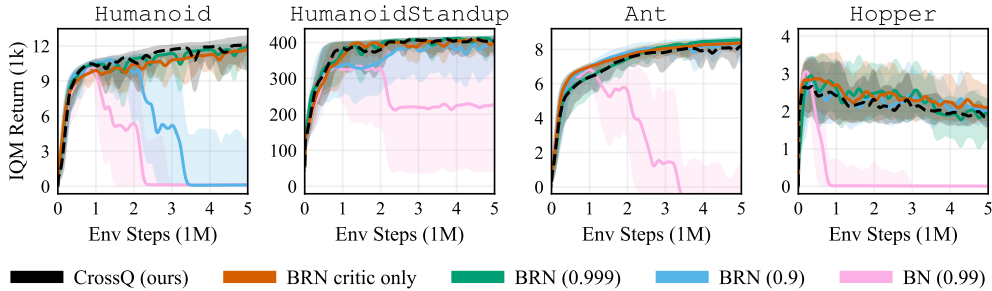


Figure 10: **Comparing BatchNorm hyperparameters.** All variants have comparably strong and stable curves early in the training. Omitting normalization in the actor (BRN critic only) does not significantly affect CrossQ. Using the original Batch Normalization (BN, with moving-average momentum 0.99) is prone to sudden performance collapses during longer training runs. Using BRN permits stabler training, which improves with higher momentums; CrossQ’s default 0.99 (black) and higher show no collapses. We plot IQM return and 70% quantile intervals over five seeds.

Batch Normalization Layers. In Figure 10, we ablate the BatchNorm versions (BN (Ioffe & Szegedy, 2015) and BRN (Ioffe, 2017)) and their internal moving-average momentums. Compared to CrossQ’s optimal combination—BRN with momentum 0.99—all variants have similar sample efficiency in the early stages of training (1M steps). When using BN, we sometimes observe sudden performance collapses later in training; we attribute these to BN’s unique approach of using noisy *minibatch estimates* of normalization moments. BRN’s improved approach of using the less noisy *moving-averages* makes these collapses less likely; further noise-reduction via higher momentums eliminates these collapses entirely. Additionally, we find that using BatchNorm only in the critic (instead of both the actor and the critic) is sufficient to drive the strong performance of CrossQ; however, including it in both networks performs slightly better.

5 CONCLUSION & FUTURE WORK

We introduced CrossQ, a new off-policy RL algorithm that matches or exceeds the performance of REDQ and DroQ—the current state-of-the-art on continuous control environments with state observations—in terms of sample efficiency while being multiple times more computationally efficient. To the best of our knowledge, CrossQ is the first method to successfully use BatchNorm to greatly accelerate off-policy actor-critic RL. Through benchmarks and ablations, we confirmed that target networks do indeed slow down training and showed a way to remove them without sacrificing training stability. We also showed that BatchNorm has the same accelerating effect on training in Deep RL as it does in supervised deep learning. The combined effect of removing target networks and adding BatchNorm is what makes CrossQ so efficient. We investigated the relationship between the Q estimation bias and the learning performance of CrossQ, but did not identify a straightforward dependence. This indicates that the relationship between the Q estimation bias and the agent performance is more complex than previously thought.

In future work, it would be interesting to analyze the Q estimation bias more extensively, similar to Li et al. (2022). Furthermore, a deeper theoretical analysis of the used BatchNorm approach in the context of RL would be valuable, akin to the works in supervised learning, e.g., Summers & Dinneen (2020). Although the wider critic networks do provide an additional performance boost, they increase the computation cost, which could potentially be reduced. Finally, while our work focuses on the standard continuous control benchmarking environments, a logical extension would be applying CrossQ to a real robot system and using visual observations in addition to the robot state. Techniques from image-based RL, such as state augmentation (Laskin et al., 2020; Yarats et al., 2021) and auxiliary losses (Schwarzer et al., 2021; He et al., 2022), also aim to learn efficiently from limited data. We believe some of these ideas could potentially be applied to CrossQ.

A APPENDIX

A.1 DEEPMIND CONTROL SUITE EXPERIMENTS

Figure 11 presents an additional set of experiments performed on the DeepMind Control Suite (Tassa et al., 2018). The experiments shown here are an extension to the experiments shown in Figure 5 in the main paper and have been moved to the Appendix due to space constraints. For the presented tasks, we lowered the learning rate to 8×10^{-4} for all algorithms, and set the CrossQ policy delay to 1. All other hyperparameters remained the same as for the main paper.

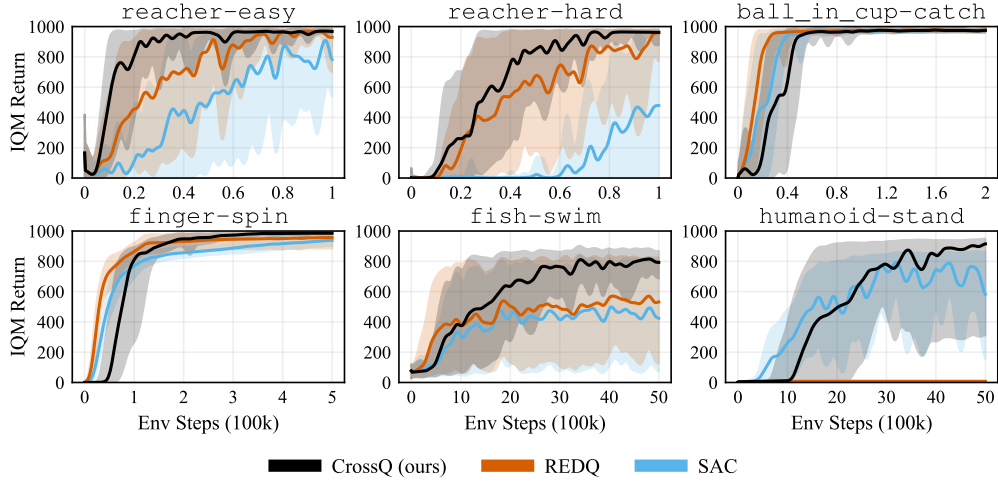


Figure 11: **Sample efficiency of CrossQ on DeepMind Control.** The experiments here were each performed on 5 different random seeds. CrossQ’s good sample efficiency transfers well to the presented tasks from the DeepMind Control Suite.

A.2 HYPERPARAMETERS

Experiment hyperparameters, used in the main paper. We adapted most hyperparameters that are commonly used in other works (Haarnoja et al., 2018b; Chen et al., 2021; Hiraoka et al., 2021). The Moving-Average *Momentum* corresponds to 1 minus the Moving-Average *Update Rate* as defined in both BatchNorm papers (Ioffe & Szegedy, 2015; Ioffe, 2017).

Table 1: Learning Hyperparameters

Parameter	SAC	REDQ	DroQ	CrossQ (ours)
Discount Factor (γ)			0.99	
Learning Rate (Actor & Critic)			0.001	
Replay Buffer Size			10^6	
Batch Size			256	
Activation Function			relu	
Layer Normalization	No	Yes	No	
Dropout Rate	N/A	0.01	N/A	
BatchNorm / Version	N/A		BRN	
BatchNorm / Moving-Average Momentum	N/A		0.99	
BatchNorm / BRN Warm-up Steps	N/A		10^5	
Critic Width		256		2048
Target Update Rate (τ)		0.005		N/A
Adam β_1		0.9		0.5
Update-To-Data ratio (UTD)	1	20		1
Policy Delay	1	20		3
Number of Critics	2	10		2

A.3 WALLCLOCK TIME MEASUREMENT

Wallclock times were measured by timing and averaging over four seeds each and represent *pure training times*, without the overhead of synchronous evaluation and logging, until reaching 5×10^6 environment steps. The times are recorded on an Nvidia RTX 3090 Turbo with an AMD EPYC 7453 CPU.

Table 2: **Wallclock times.** Evaluated for CrossQ and baselines across environments in hours and recorded on an RTX 3090, the details of the measurement procedure are described in Appendix 4.2. Comparing CrossQ with CrossQ (Small) and SAC, it is apparent that using wider critic networks does come with a performance penalty. However, compared to REDQ and DroQ, one clearly sees the substantial improvement in Wallclock time of CrossQ over those baselines.

	Wallclock Time [hours]				
	SAC	CrossQ (small)	CrossQ (ours)	REDQ	DroQ
HumanoidStandup-v4	1.5	2.1	2.2	8.7	7.5
Walker2d-v4	0.9	0.9	1.1	4.0	4.1
Ant-v4	0.9	1.2	1.5	4.7	4.7
HalfCheetah-v4	0.8	1.2	1.5	4.1	4.4
Hopper-v4	1.0	1.1	1.3	4.1	4.2

A.4 EVOLVING ACTION DISTRIBUTIONS

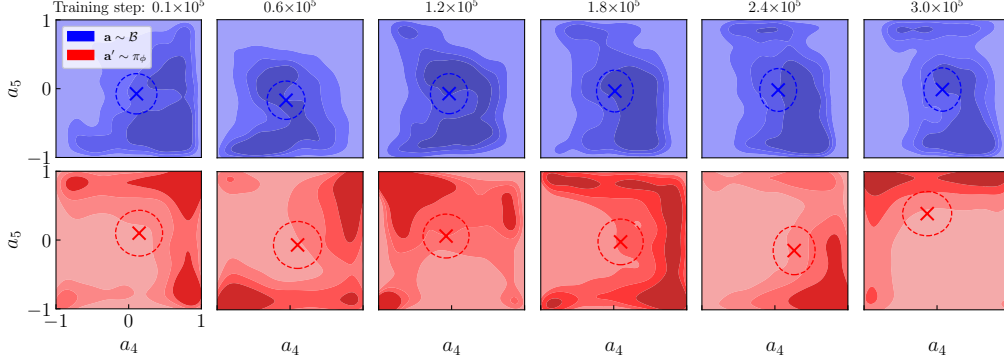


Figure 12: **Replay and policy action distributions are different, and evolve during training.** We train an agent for 300,000 steps on `Walker2d`. We take snapshots of the replay buffer $\mathcal{B}$ and policy π_ϕ every 60,000 steps. For each snapshot (one column), we sample a large batch of 10,000 transitions $(s, \mathbf{a}, s', \mathbf{a}' = \pi_\phi(s'))$ and use this to compute a visually interpretable 2D kernel density estimate of the distributions of $\mathbf{a}$ (blue) and $\mathbf{a}'$ (red), as seen through the action-space dimensions 4 and 5. The cross denotes the mean, and the dashed ellipse is one standard deviation wide for each of the two dimensions. We observe that the distributions as well as the means and standard deviations of the off-policy and on-policy actions are visibly and persistently different throughout the training run, and keep drifting as the training progresses. This discrepancy implies that BatchNorm must be used with care in off-policy TD learning.

A.4.1 ABLATING THE DIFFERENT DESIGN CHOICES AND HYPERPARAMETERS

Figure 13 depicts in detail the CrossQ and SAC ablations, previously shown in aggregate form by Figure 9.

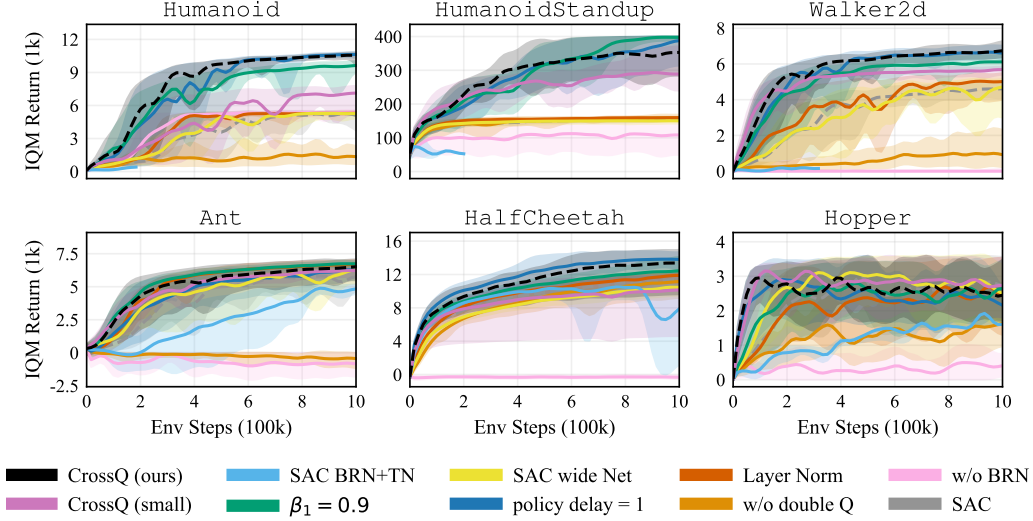


Figure 13: **CrossQ ablation study.** We ablate across different hyperparameter settings and architectural configurations. Using the same network width as SAC, **CrossQ (small)** shows weaker performance, yet is still competitive with CrossQ in four out of six environments. At the same time, **SAC with a wider critic** does not work better. Using the default Adam momentum $\beta_1 = 0.9$ instead of 0.5 degrades performance in some environments. Using a **policy delay of 1** instead of 3 has a very small effect, except on Ant. Using **LayerNorm** instead of BatchNorm results in slower learning; it also trains stably without target networks. **Removing BatchNorm** results in failure of training due to divergence. **Adding BatchNorm to SAC** and reusing the live critic’s normalization moments in the target network fails to train. Training **without double Q** networks (single critic) harms performance.

A.5 REDQ AND DROQ ABLATIONS

Figures 14 and 15 show REDQ and DroQ ablations on 5 seeds each. They show both baselines with the CrossQ hyperparameters: wider critic networks as well as $\beta_1 = 0.5$. Neither baseline benefits from the added changes. In most cases, the performance is unchanged, while in some cases, it deteriorates. The dashed black line shows CrossQ as a reference.

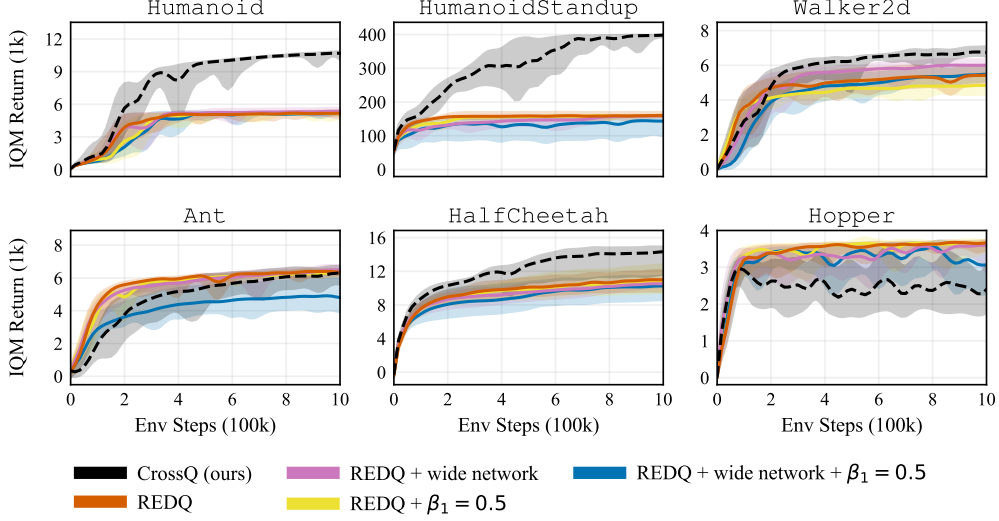


Figure 14: **REDQ ablation.** Showing performance for different combinations of the CrossQ hyperparameters. The changes in hyperparameters do not help REDQ to get better performance. In fact, in some cases, they even hurt the performance.

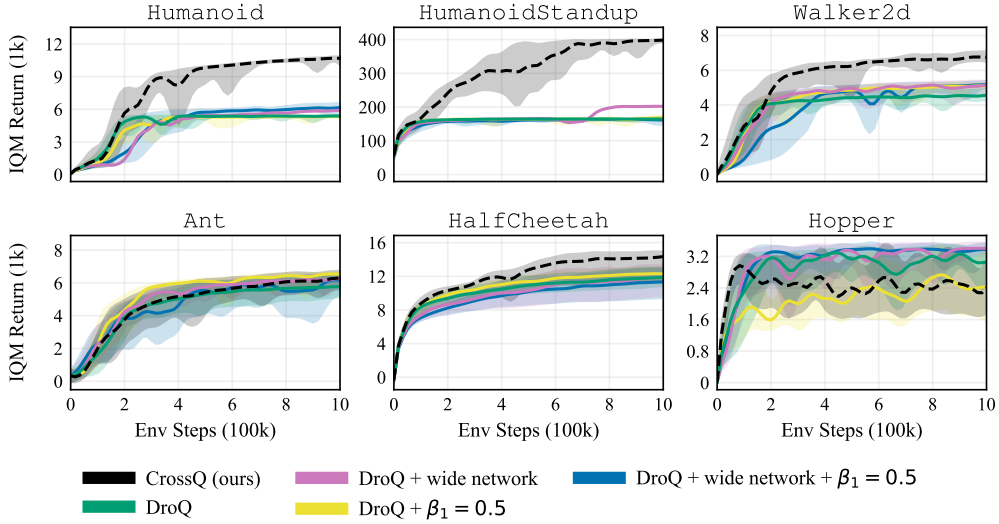


Figure 15: **DroQ ablation.** The changes in hyperparameters do not help DroQ to get better performance overall. In Hopper and Ant, performance rises to the CrossQ performance, however, on the Humanoid, it hurts performance.

A.6 EFFECT OF ACTIVATIONS AND NORMALIZERS ON LEARNING STABILITY

Figure 8 depicts a small exploratory experiment in which we remove target networks from SAC, and train it with different activation functions and feature normalizers. We do this only to explore whether the boundedness of activations has an influence on training stability. We learn from this experiment that SAC with tanh activations trains without divergence, allowing us to conduct the study in Section 4.4.1. We also observe that at least two feature normalization schemes (on top of the unbounded relu activations) permit divergence-free optimization.

For vectors $\mathbf{x}$, `relu_over_max( $\mathbf{x}$ )` denotes a simple normalization scheme using an underlying unbounded activation: $\text{relu}(\mathbf{x})/\max(\mathbf{x})$, with the maximum computed over the entire feature vector. `layernormed_relu` simply denotes LayerNorm applied *after* the relu activations. Both of these schemes prevent divergence. Using LayerNorm *before* the relu activations also prevent divergence, and is already explored in the ablations in Figure 13. None of these normalizers perform as strongly as BatchNorm.

A thorough theoretical or experimental study of how activations and normalizers affect the stability of Deep RL is beyond the scope of this paper. We hope, however, that our observations help inform future research directions for those interested in this topic.

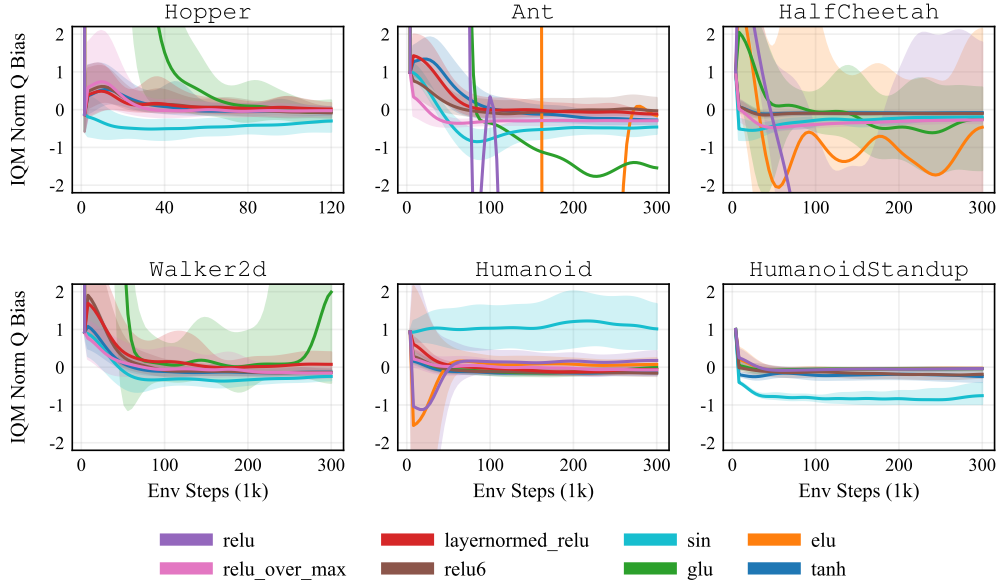


Figure 16: **(In)stability of SAC without target networks.** Observed through the Q estimation bias. In this small-scale experiment, we run SAC with unbounded (`relu`, `glu`, `elu`) and bounded (`tanh`, `relu6`, `sin`) activation functions, as well as “indirectly” bounded activations through the use of two custom normalizers other than BatchNorm (`relu_over_max`, `layernormed_relu`). SAC variants with unbounded activations appear highly unstable in most environments, whereas the variants with bounded activations (as well as the normalizers) do not diverge, maintaining relatively low bias.

A.7 NORMALIZED Q BIAS PLOTS

Figure 17 shows the results of the Q function bias analysis for all environments.

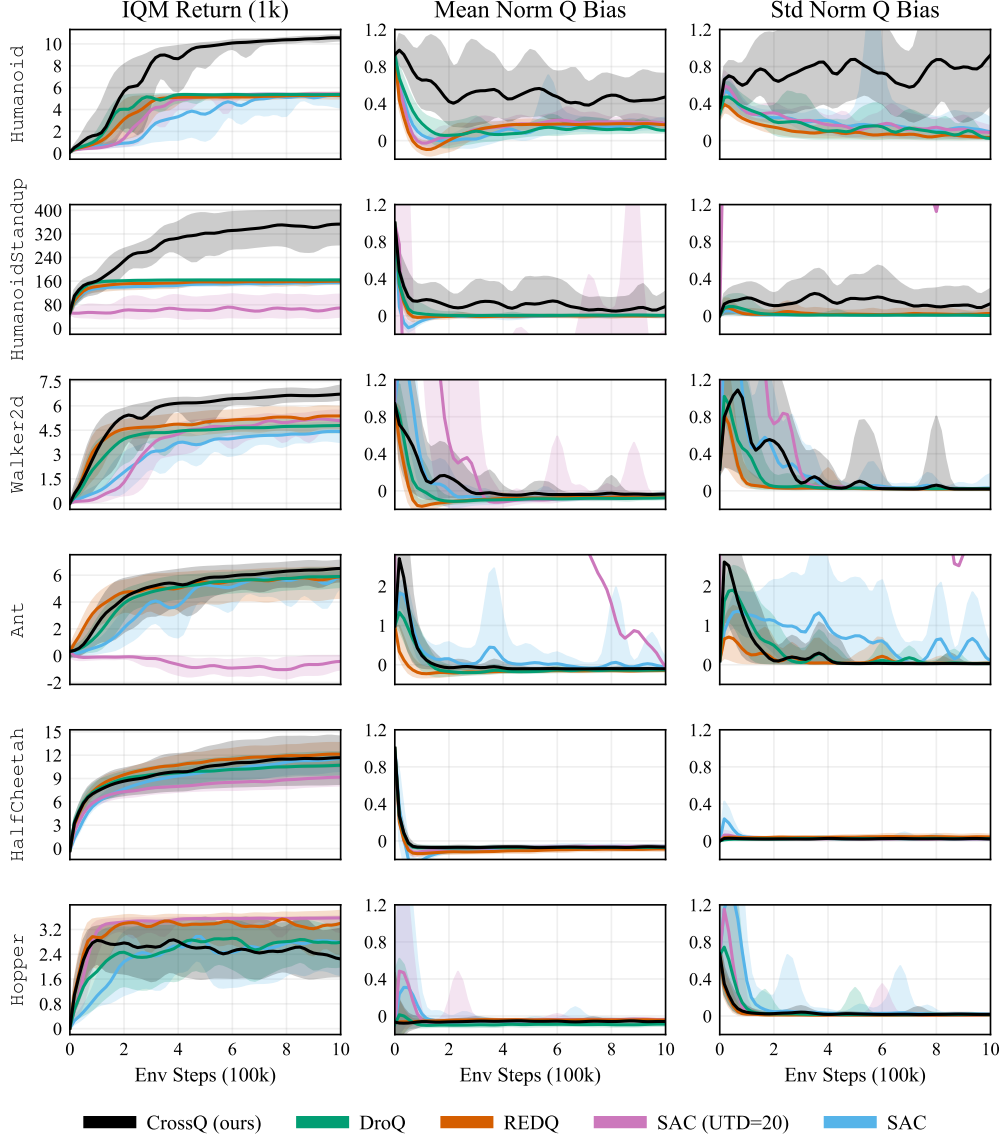


Figure 17: **Q estimation bias.** Mean and standard deviation of the normalized Q function bias, computed as described by [Chen et al. \(2021\)](#). As in the main paper, we do not find a straightforward connection between normalized Q function bias and learning performance. Cross Q generally shows the same or larger Q estimation bias compared to REDQ but matches or outperforms REDQ in learning speed, especially on the challenging Humanoid tasks.

Stable-Baselines3: Reliable Reinforcement Learning Implementations

Antonin Raffin¹

Ashley Hill²

Adam Gleave³

Anssi Kanervisto⁴

Maximilian Ernestus⁵

Noah Dormann¹

ANTONIN.RAFFIN@DLR.DE

ASHLEY.HILL@CEA.FR

GLEAVE@BERKELEY.EDU

ANSSK@CS.UEF.FI

MAXIMILIAN@ERNESTUS.COM

NOAH.DORMANN@DLR.DE

¹ *Robotics and Mechatronics Center (RMC), German Aerospace Center (DLR), Weßling, Germany*

² *Interactive Robotics Laboratory, University Paris-Saclay, CEA, Palaiseau, France*

³ *Electrical Engineering and Computer Science, University of California, Berkeley, CA, USA*

⁴ *School of Computing, University of Eastern Finland, Joensuu, Finland*

⁵ *Kiteswarms GmbH, Freiburg, Germany*

Editor: Andreas Mueller

Abstract

STABLE-BASELINES3 provides open-source implementations of deep reinforcement learning (RL) algorithms in Python. The implementations have been benchmarked against reference codebases, and automated unit tests cover 95% of the code. The algorithms follow a consistent interface and are accompanied by extensive documentation, making it simple to train and compare different RL algorithms. Our documentation, examples, and source-code are available at <https://github.com/DLR-RM/stable-baselines3>.

Keywords: Reinforcement Learning, Baselines, Software, Open-Source, Python, PyTorch

1. Introduction

Deep reinforcement learning (RL) research has grown rapidly in recent years, yet results are often difficult to reproduce (Henderson et al., 2018). A major challenge is that small implementation details can have a substantial effect on performance – often greater than the difference between algorithms (Engstrom et al., 2020). It is particularly important that implementations used as experimental *baselines* are reliable; otherwise, novel algorithms compared to weak baselines lead to inflated estimates of performance improvements.

To address this challenge, we propose STABLE-BASELINES3 (SB3), an open-source framework implementing seven commonly used model-free deep RL algorithms (see Section 2). We take great care to adhere to software engineering best practices to achieve high-quality implementations that match prior results. Each algorithm has been benchmarked on common environments (Raffin and Stulp, 2020) and compared to prior implementations. Our test suite covers 95% of the code and, together with our active user base¹ scrutinizing changes, ensures that any implementation errors are minimized.

1. At the time of writing, SB3 had 800+ stars on GitHub, 100+ closed issues and 80+ merged pull requests

```

import gym
from stable_baselines3 import SAC
# Train an agent using Soft Actor-Critic on Pendulum-v0
env = gym.make("Pendulum-v0")
model = SAC("MlpPolicy", env).learn(total_timesteps=20000)
# Save the model
model.save("sac_pendulum")
# Load the trained model
model = SAC.load("sac_pendulum")
# Start a new episode
obs = env.reset()
# What action to take in state `obs`?
action, _ = model.predict(obs, deterministic=True)

```

Figure 1: Using STABLE-BASELINES3 to train, save, load, and infer an action from a policy.

STABLE-BASELINES3 builds on the experience gained from maintaining our previous implementation, STABLE-BASELINES2 (SB2; Hill et al., 2018)², that was forked from OpenAI Baselines (Dhariwal et al., 2017) and uses TensorFlow (Abadi et al., 2016). SB3 is a complete rewrite of the codebase implemented in PyTorch (Paszke et al., 2019), the framework preferred by a majority of our users in a survey (Raffin, 2020a). SB3 maintains a similar API, allowing a seamless upgrade pathway from SB2.³

Our main goal is to provide a user-friendly and reliable RL library. To keep SB3 simple to use and maintain, we focus on model-free, single-agent RL algorithms, and rely on external projects to extend the scope to imitation (Wang et al., 2020) and offline learning (Seno, 2020). We prioritize maintaining *stable* implementations over adding new features or algorithms, and avoid making breaking changes. We provide a consistent, clean and fully documented API, inspired by the `scikit-learn` API (Pedregosa et al., 2011). Our code is easily modifiable by users as we favour readability and simplicity over modularity, although we make use of object-oriented programming to reduce code duplication.

2. Features

Simple API. Figure 1 shows that training agents in STABLE-BASELINES3 takes just a few lines of code, after which the agent can be queried for actions. This allows researchers to easily use the baseline algorithms and components in their experiments (e.g. Klink et al. (2020); Nair et al. (2019); Gleave et al. (2020)), as well as apply RL to novel tasks and environments, like continual learning when attacking WiFi networks (Margaritelli, 2020) or dampening bridge vibrations (Berkowitz, 2019).

Documentation. SB3 comes with extensive documentation of the code API.⁴ We also include a user guide, covering both basic and more advanced usage with a collection of concrete examples. Moreover, we have developed a Colab notebook based RL tutorial,⁵ enabling users to demo the library directly in the browser. Additionally, we include common

2. SB2 has 650+ closed issues, 220+ merged pull requests and 200+ citations on Google Scholar.

3. Upgrade guide: <https://stable-baselines3.readthedocs.io/en/master/guide/migration.html>.

4. <https://stable-baselines3.readthedocs.io/en/master/>

5. <https://github.com/araffin/rl-tutorial-jnrr19>

tips for running RL experiments and a developer guide. We also pay close attention to questions and uncertainties from SB3 users, updating the documentation to address these.

High-Quality Implementations. Algorithms are verified against published results by comparing the agent learning curves⁶. Moreover, all functions are typed (parameter and return types) and documented with a consistent style, and most functions are covered by unit tests. Continuous integration checks that all changes pass unit tests and type check, as well as validating the code style and documentation.

Comprehensive. STABLE-BASELINES3 contains the following state-of-the-art on- and off-policy algorithms, commonly used as experimental baselines: A2C (Mnih et al., 2016), PPO (Schulman et al., 2017), DDPG (Lillicrap et al., 2016), SAC (Haarnoja et al., 2018), TD3 (Fujimoto et al., 2018), HER (Andrychowicz et al., 2017) and DQN (Mnih et al., 2015).

Moreover, SB3 provides various algorithm-independent features. We support logging to CSV files and TensorBoard. Users can log custom metrics and modify training via user-provided callbacks. To speed up training, we support parallel (or “vectorized”) environments. To simplify training, we implement common environment wrappers, like preprocessing Atari observations to match the original DQN experiments (Mnih et al., 2015).

Experimental Framework. RL Baselines Zoo (Raffin, 2018, 2020b) provides scripts to train and evaluate agents, tune hyperparameters, record videos, store experiment setup and visualize results. We also include a collection of pre-trained reinforcement learning agents together with tuned hyperparameters for simple control tasks, PyBullet environments (Coumans and Bai, 2016–2019) and Atari games, optimized using Optuna (Akiba et al., 2019). We follow best practices for training and evaluation (Henderson et al., 2018), such as evaluating in a separate environment, using deterministic evaluation where required (SAC) and storing all hyperparameters necessary to replicate the experiment.

Stable-Baselines3 Contrib. We implement experimental features in a separate contrib repository (Raffin et al., 2020). This allows SB3 to maintain a stable and compact core, while still providing the latest features, like Truncated Quantile Critics (Kuznetsov et al., 2020). Implementations in contrib need not be tightly integrated with the main SB3 codebase, but we maintain the same stringent review requirements to ensure users can trust the contrib implementations. Implementations from contrib that have stood the test of time may be integrated into the main repository.

3. Comparison to Related Software

Most libraries are targeted at experienced RL researchers, requiring expert knowledge to use (Weng et al., 2020; Hoffman et al., 2020; Fujita et al., 2021; Castro et al., 2018; Guadarrama et al., 2018; Gauci et al., 2018; Stooke and Abbeel, 2019; Kolesnikov, 2018). Only a few RL libraries offer more than a brief API documentation (garage contributors, 2019; Liang et al., 2018; Kuinnie et al., 2017; Guadarrama et al., 2018), and some are notoriously hard to understand.⁷ By contrast, STABLE-BASELINES3 is designed to be easy to use and comes with extensive documentation and tutorials.

6. For example, issue #48 or issue #49.

7. OpenAI Baselines (Dhariwal et al., 2017), see <https://www.reddit.com/r/MachineLearning/comments/95ft1j/>. This was a major starting point for STABLE-BASELINES2 (Hill et al., 2018)

The previous version of STABLE-BASELINES3, STABLE-BASELINES2, was created as a fork of OpenAI Baselines (Dhariwal et al., 2017) but the two codebases quickly diverged (see PR #481). SB3 is a complete rewrite of STABLE-BASELINES2 in PyTorch that keeps the major improvements and new algorithms from SB2 while going even further into improving code quality (e.g. cleaner codebase, better test coverage, type hints). More precisely, compared to Baselines, SB3 is fully documented, commented, tested, has 4 additional algorithms (SAC, TD3, QR-DQN, TQC⁸) and many additional features (e.g. dictionary observation support, callbacks, evaluation with multiple environments, environment checker). The only legacy features of OpenAI Baselines are the code structure (one folder per algorithm), the use of code-level optimizations and the environment tools which are greatly improved⁹ (additional features, bug fixes, comments, documentation and more testing).

Many libraries have a modular design (Caspi et al., 2017; Keng and Graesser, 2017; Hoffman et al., 2020; garage contributors, 2019). This allows them to quickly combine advances from different papers, but forces new users to understand the full code structure before being able to tweak the library. On the other extreme, educational implementations like Spinning Up (Achiam, 2018) are self-contained but hard to maintain due to code duplication. SB3 strives to strike a balance: factoring out widely used components like replay buffers, but minimizing the amount of code that needs to be understood to modify an algorithm.

As an exhaustive comparison to all RL libraries is not possible, in Table 1 we compare SB3 to a subset of other active or popular libraries, with a focus on quality of implementation and openness to new users.

RLlib (Liang et al., 2018) scores highly in the table, but is targeted at a different use-case from SB3. Whereas SB3 focuses on simplicity and reliability, RLlib (Liang et al., 2018) prioritizes scalability and support for distributed training. Additionally, RLlib includes both a PyTorch and TensorFlow backend, and includes support for multi-agent training. This versatility comes at a cost of a larger and more complex codebase.

Overall, we find SB3 compares favourably to other libraries in terms of documentation, testing and activity.

	SB3	OAI Baselines	PFRL	RLlib	Tianshou	Acme	Tensorforce
Backend	PyTorch	TF	PyTorch	PyTorch/TF	PyTorch	Jax/TF	TF
User Guide / Tutorials	✓/ ✓	✗/ —	—/ ✓	✓/ ✓	—/ ✓	—/ ✓	✓/ —
API Documentation	✓	✗	✓	✓	✓	✗	✓
Benchmark	✓	✓	✓	✓	—	—	—
Pretrained models	✓	✗	✓	✗	✗	✗	✗
Test Coverage	95%	49%	?	?	94%	74%	81%
Type Checking	✓	✗	✗	✓	✓	✓	✗
Issue / PR Template	✓	✗	✗	✓	✓	✗	✗
Last Commit (age)	< 1 week	> 6 months	< 1 month	< 1 week	< 1 month	< 1 week	< 1 month
Approved PRs (6 mo.)	75	0	13	222	85	5	7

Table 1: Comparison of SB3 to a representative subset of active or popular RL libraries. **Key:** — means that the feature is only partially present; OAI: OpenAI; TF: TensorFlow; PR: Pull Request.

8. QR-DQN and TQC are in the contrib repo.

9. As an example, one can compare “VecNormalize” in OAI Baselines vs SB3.